



smallyu

Hello World

??

??

????

2017080 (6)

2017070 (2)

2017060 (3)

2017050 (2)

2017040 (2)

2017030 (3)

2017020 (3)

2017010 (4)

20160120 (1)

Z-Blog插件：站点信息统计

□ smallyu □ 2017-08-31 □ Z-Blog

在系统原有的站点信息的基础上，增加了文章总字数和页面总字数统计；可自定义显示条目。

站点信息统计

描述	配置
文章总数:31	1
页面总数:2	1
分类总数:1	1
标签总数:0	1
评论总数:0	1
浏览总数:3	0
文章总字数:319	0
页面总字数:38	0
1为显示，0为不显示	

提交

后台配置页面，1为显示，0为不显示，提交后保存，属性同其它系统模块。

20160110 (2)
20160109 (3)
2016090 (1)
2016080 (6)
2016070 (1)
2016060 (4)
2016050 (4)

站点统计

- 文章总数:31
- 页面总数:2
- 分类总数:1
- 标签总数:0
- 评论总数:0
- 浏览总数:3
- 文章总字数:319
- 页面总字数:38

前台显示效果会受到主题样式影响。

附件：[SiteInfoCount.zip](#)

偶然想到要统计一下自己博客上文章的总字数，几经辗转，还是新建一个插件，开始看着官方文档，试图实现这个功能。

这是smallyu尝试开发的第一个zblog插件。

插件目录使用系统默认的结构，插件嵌入页`include.php`和插件管理页`main.php`。

经过一整天⁺的探索时间，`smallyu`渐渐明白了`zblog`插件的运行机制，最终实现了包括但不限于文章总字数统计的功能。

因为之前`smallyu`就受限于系统自带的站点信息不能定制化显示，于是这次在插件上添加了这样的功能。

也因为`smallyu`比较懒，可能代码的实现形式比较`low`，也没有考虑插件给系统性能带来的影响，如果有机会，也许`smallyu`会重构这个插件的代码。

同时因为`zblog`后台并不支持直接提交插件到官方审核，然后`smallyu`对这个插件也非常没有信心，不认为能够通过审核，就不掺和了。

在插件嵌入页`include.php`，一共挂载了5个接口，当这些接口事件发生时，模块内容就会被更新（但前台页面需要重新编译）。

```
function ActivePlugin_SiteInfoCount() {  
    Add_Filter_Plugin('Filter_Plugin_Zbp_BuildTemplate','updateSIC');           // 模  
    板重新编译接口  
    Add_Filter_Plugin('Filter_Plugin_PostArticle_Succeed','updateSIC');       // 文  
    章编辑成功接口  
    Add_Filter_Plugin('Filter_Plugin_DelArticle_Succeed','updateSIC');       // 文  
    章删除成功接口  
    Add_Filter_Plugin('Filter_Plugin_PostPage_Succeed','updateSIC');         // 页  
    面编辑成功接口  
    Add_Filter_Plugin('Filter_Plugin_DelPage_Succeed','updateSIC');          // 页  
    面删除成功接口
```

```
}
```

插件的ID定义为SiteInfoCount，简写为SIC。官方定义了插件接口的挂载函数ActivePlugin_ID，在该函数里可以使用Add_Filter_Plugin函数挂载系统接口。

当文章或页面被编辑，统计字数就会发生改变，所以在这些事件发生时，需要更新模块内容，这里定义更新模块内容的函数为updateSIC，也就是接管系统接口之后会被执行的函数。

```
function updateSIC() {  
    global $zbp;  
    $config = configInit();  
    if(isset($zbp->modulesbyfilename['SICMoudle'])) {  
        $zbp->modulesbyfilename['SICMoudle']->Content = cttBycof($config);  
    }  
    $zbp->AddBuildModule('SICMoudle', 'updateSIC');  
}
```

这是updateSIC函数的内容。先声明使用全局变量\$zbp，然后获取配置信息到变量\$config，其中configInit函数是自定义的函数。if语句判断模块是否存在，如果存在则更改模块内容。最后面的语句可能是用于执行更新，samllyu并没有完全了解zblog的函数。

理论上更新模块有两种方式，一种是if语句内的样子，直接更改模块内容，另一种是使用下面的AddBuildModule函数执行更新模块的操作，函数内的两个参数分别表示要更新的模块是执行更新的用户函数。

samllyu懒得测试模块到底是怎样更新的了，就把两句都写上，反正程序可以正常执行.....即使是更新了两次模块，也并不会给系统带来太大额外开销。

```
function configInit($config = array()){  
    global $zbp;
```



```

$configFileName = "config.sic";
// 文件不存在、没有参数
if(!is_file($configFileName) && count($config) == 0) {
    $config = array(1, 2, 3, 4, 5, 6, 7, 8);
    configInput($config);
}
// 文件存在
if(is_file($configFileName)) {
    // 有参数
    if(count($config) != 0) {
        configInput($config);
    } else {
        // 没有参数，读取文件
        $configText = file_get_contents($zbp->host."zb_users/plugin/SiteInfoC
ount/".$configFileName);
        $config = explode("|", $configText);
    }
}
return $config;
}

```

这是上面用到的 `configInit` 函数。函数接收一个数组类型的参数，用于处理插件管理页面提交并需要保存的配置。

没错，配置信息是写入到文件里的，读取分别使用 `explode` 和 `implode` 将字符串分割为数组。

至于为什么，`smallyu` 之前不知道 `zblog` 系统提供了选项机制，可以使用类似 `$zbp->Config("ID")->test` 的形式储存数据，然后就没有改。虽然代码量不大，但要把数据从文件储存全部换成使用选项机制储存，也相当于重构整个插件了。

文件储存也并不影响使用，就是看起来新手一点。

其它使用到的函数还有：

```
moodleInit();    // 初始化模块，也就是新建模块，函数内配置了模块的一些信息
configInput();   // 细化的配置写入函数，在postInit函数里调用
cttBycof();      // 根据config检出content，接收数组参数
postInit();      // 处理提交的配置信息
```

每个函数的代码量都不大。其中站点信息截取了ModuleBuilder类的Statistics方法，直接获取到了字符串形式的站点信息。

插件管理页面main.php则只是把数据填充过来，其中也用到了看起来不太好的for循环，还把循环次数写死了，像这样：

```
// 数组转存
foreach($config as $value) {
    $config[$value] = $value;
}
// 获得显示数组
for($i = 1; $i <=8; $i++) {
    $cofValue[$i] = $config[$i] == $i ? 1 : 0;
}
```

从代码上看，好像不明所以。是的，仅仅过了一个小时，smallyu现在也不明所以.....

刚才zblog社区有人说插件改0后会出错

应该是全部改0后，会写入配置文件空的内容，然后取配置的时候没能得到参数，程序就报错了，这个bug

之前好像也遇到过

全部改0意思就是不显示任何内容，那么.....也许可以试着直接把模块撤掉.....这样就可以不显示任何内容了

好吧，我懒.....得改

[前一篇](#) 我的世界，你做主



smallyu

Hello World

??

??

????

2017080 (6)

2017070 (2)

2017060 (3)

2017050 (2)

2017040 (2)

2017030 (3)

2017020 (3)

2017010 (4)

20160120 (1)

我的世界，你做主

□ smallyu □ 2017-08-29 □ 我的世界

我要带你去绿色生态园看风景

看高大的树木进幽谧的森林

坐在丛林里板凳上休息

我要带你去文化生态园看灯展

看炫丽的灯光拍好看的照片

站在熙攘的人群里说笑

我要带你去旅游

去任意的地方

只要和你在一起

20160110 (2)

20160109 (3)

20160108 (1)

20160107 (6)

20160106 (1)

20160105 (4)

20160104 (4)

去哪里都可以

我要和你一起参加活动

也许是好多人的聚会

也许是两个人的约会

虽然要去很多地方

但不是我开车

因为我不想开车

我要和你一起坐在车的后排

也许是别人的车

也许是自己的车

我想到过很多和你在一起的场景

比如一起奋斗

可能是准备一场考试

可能是寻找一份工作

可能是做一次失败的生意

比如一起购物

买两张周末的电影票

买两张回老家的车票

买新房子里装修用的瓷砖

比如一起无聊

你在房间里无所事事

我就看着你无所事事

和你在一起又怎么会无聊

好像我经历过的回忆中，都有你

最近天在下雨

变凉了

注意身体

也要注意安全

其实我没有想你

但我还是忍不住关心你

虽然我联系不到你

也没有你

////////////////////
欢迎来到我的世界。

入坑我的世界好多天，经历了各种生生死死，游戏一次又一次地安装卸载。

刚进入游戏，到了晚上会被僵尸杀死，被拿弓箭的白色僵尸射死，被蜘蛛咬死，被末影人打死，还遇见不知名的拿药水瓶的人，把你毒死。

自作孽的时候，被爬行者炸死，从高处掉到地上摔死，泡水里淹死，跑岩浆里被火烧死，饿死。

直到慢慢的，收集地上的泥土，垒起两格高的围墙，把自己围在中间，晚上就在里面等啊等，等到白天再继续活动。

随着圈起围墙的地盘越来越大，开始学会收集木块，然后知道了最最最关键的一个步骤：用木块合成木板，这是一切一切的开始。

有了木板，就能有木棍，就能有工作台，就能有木锹、木稿、木锄、木剑，用木锹挖土，用木稿挖圆石，用木锄种小麦，用木剑打怪兽。

有了木稿，就能有圆石，就能有熔炉，就能有木炭，就能有火把，火把用来把自己的地盘照亮，照亮的地方就不会有僵尸出生了。

然后学会挖矿，铁矿、银矿、金矿、钻石矿，还有至关重要的黑曜石，并且用黑曜石搭建地狱门，可以进入地狱。

.....

我的世界已近神作，自由度太高了，它是给真正体验游戏的玩家玩的游戏。

虽然时间不长。

我并没能完成在游戏里想要完成的操作，比如建一座城堡。



这个是我的圈地用的城墙，目前四个面只留了这么一个入口，一扇木门。原想在城堡的四周都开一个入口，这样每次回家都不用跑到这个面找这个门。



这个是改造中的城墙，原想把泥土搭的城墙全部换成圆石墙，地面全部换成圆石。可是我圈地比较大，没能来得及换完。



这只狗是在很远的森林里遇到，喂了两根骨头就从良的，一直跟在身边。遇到僵尸攻击的时候，小狗也会积极配合上前攻击。



这个长长的杵上天的柱子，是暂时用来当信号灯用的。如果离开城堡，到了旁边的森林，又不是特别远，有这根柱子，一眼就能看出城堡的方向。



这些是在城堡里圈养的动物，有羊、鸡、兔子、狗。游戏中只要喂这些动物食物，它们就会繁殖了，羊吃小麦，鸡吃小麦种子，兔子吃萝卜，狗吃肉。

在城堡里如果放一些水用来种小麦和萝卜，那这个城堡里的生态圈就可以无限循环啦！以前是有的，后来为了城堡里面的地面平坦一点，加上准备把泥土地面全换成圆石的，就先撤掉了。



这个东西，里面是岩浆，外面是一圈玻璃。仅仅只是为了好玩放在那儿，岩浆晚上会发光，也能起到一点照明的作用。



这些箱子就是用来放物品的地方。除了工作台和熔炉，有六个箱子。东西太多了，很难分类分清楚，有建筑石块、矿石、装饰块、食物、杂物、工具.....



这个，就是地狱之门了。用十块黑曜石搭成的 2×3 的门，图中用玻璃把紫色的传送门挡住了，因为地狱里面的僵尸猎人太厉害，每次进去都被秒杀，把门堵住，防止自己误进，也防止门里面的僵尸猎人出来，还有偶尔小狗也会自己跳进去。

游戏里黑曜石很难天然形成，必须得自己，把水倒在岩浆上，然后用钻石镐挖。感觉钻石很难找。



这个留空的没有城墙的角落，是用来钓鱼的，而且很安全，僵尸们上不来，因为下面只铺了踩着的那一层，再下面就是悬空的了。



最后是一张不算全景照的全景照。其实地方蛮大的，还有不少能想到的想做的工作没做完。截图是在晚上，没能白天，也算是遗憾吧。

没做完，以后接着做？

单机游戏，实在不愿花费过多时间。

尽管和已经弃坑的王者荣耀比，我的世界用的时间一点都不多。王者荣耀，永远也不会有我的世界这么精彩

的游戏内容。

而且要开学了，刚放假的时候，对学校还是很陌生的感觉。

转眼暑假过去了。

很陌生。

现在想想，当时依旧是浑浑噩噩不知所以。

现在也依旧是浑浑噩噩不知所以。

平凡才是常态？

“连自己都找不到，还怎么找另一半？”

值得思考的，思考不出来的，思考出来做不到的，能做到但又不想做的，种种

拿起酒杯敬自己

敬明天

敬未来

未知的路才要前行

上一篇: [轻舟虽过万重山，心到桥头自然沉](#)

下一篇: [Z-Blog插件：站点信息统计](#)



smallyu

Hello World

??

??

????

2017080 (6)

2017070 (2)

2017060 (3)

2017050 (2)

2017040 (2)

2017030 (3)

2017020 (3)

2017010 (4)

20160120 (1)

轻舟虽过万重山，心到桥头自然沉

□ smallyu □ 2017-08-22 □ CET4 消愁

欢迎来到欢乐场。

今天8月22，星期二，天气，阴，心情，糟

CET4是有及格线的，425，因为只有425以上，才有资格报考CET6，是一条约定俗成的及格线，虽然官方并没有这样的说法。

本来向从各个途径找一点点哪怕一点聊以自慰的理由，实在找不到。

CET4重要吗？太不重要了，因为没人过不了。

会在乎425，是因为，我考了420，幸好，不是424。

上一次考382的时候，也没怎么在意，并没有伤心，毕竟差得很远。

呃

20160110 (2)

20160100 (3)

20160900 (1)

20160800 (6)

20160700 (1)

20160600 (4)

20160500 (4)

smallyu 很菜对不对

上大学以来，CET4是smallyu唯一没有通过的考试，连续两次。

四次期末考试，GPA最低3.2，最高3.7，目前平均不到3.5，不过没关系。

计算机二级

计算机三级

很想说服自己，CET4证书没用，所以过不了也没关系。

计算机等级证书有用吗？虽然不愿相信这个事实，但确实没用，尤其对于找工作来说。

非计算机相关类工作，自然没用。计算机相关类工作，那点水平又远不够用。

那CET4证书有用吗？

英语相关类工作，那点水平同样远不够用。但非英语相关类工作，才是CET4发挥作用的地方。

过了，不能说明什么，过不了，说明你很差啊。就像企业招人都会招985、211，不是说人才一定都在那些院校，只是在那些院校招到的人，是人才的概率大。企业在节省招人的成本。

想要考NCRE，也只是随便考考，找点乐子.....

计算机相关考试，目前只有NCRE。

软考？PAT？嗯.....

下面引用的这段是逻辑很不清的随便写写，注释掉：

/**

为什么NCRE会沦为完全没用的这样一个考试？

可能NCRE和CET的性质不同。

对于英语，多会一句，少会一句，多懂一句，少懂一句，运气稍微好以下，运气稍微差一下，都会产生一些影响。

真正的工作场景中，除了英语相关专业，使用英语的场景，以及对英语的要求并不高。

那么这里的问题在于，你怀揣CET证书，要应聘的并不是英语相关类的工作，像英语教师、人工翻译、同声传译。

要想找英语专业相关工作，CET4也远远不够。

而NCRE没用的说法是对于计算机相关专业类的工作而言。

应聘程序猿、攻城狮，一张证书不能说明什么。

面试官不会在意你二级了解了C语言，三级学会了局域网组网，四级学过了操作系统原理。

Android软件会开发吗？eclipse用熟了吗？Spring会吗？多线程、高并发了解多少？

技术相关的问题，会就是会，不会就是不会。

*/

计算机技术的道路，可以走下去吗？

一方面，大量的计算机人，和大量冲着程序猿这个金字招牌，以及火热的人工智能、大数据，入行IT。

另一方面，大把的IT人呼叫着35岁危机，复制粘贴，增删改查，单身，加班。

现在的码农，劳苦程度不亚于几十年前坐在缝纫机上的裁缝阿姨，和工厂流水线上工作的工人大叔。

码农，还是在打工啊。

要想赚钱，得经商。

无奸不商啊。

那要不要考研？

考研大概有这么几种原因：

1. 爱学习，爱生活，为知识而拼搏

这样的人高考成绩一定炸，已经与我无关

2. 认为高学历就能够找到好工作

考研：

成本：3年的读研时间

1年以上复习、准备考研的时间，大量精力

回报：研究生学历

知识、技能成长

认识更多厉害的人，包括导师

就业：

成本：3年的工作时间

回报：工作经历

薪水

老梁的观点是，研究生无用，因为目前大多数基层工作岗位需要的技能，本科学历足够。

读研，企业需要付给你更高的薪水。但是，你能带来比本科学历的人更高的价值吗？

研究生三年，国内学校的尿性是人尽皆知的，尤其是实战操作能力，三年不一定能够学到太多东西，反而本科毕业有整整三年工作经历的人，工作能力上的提高，不见得比三年学院生活少。

卿未离，知乎上最近在关注的很厉害的一个人，他对读研的观点是，如果想要在一线城市北上广深从事技术性强的相关工作，就可以选择读研，否则，该干嘛干嘛。

也就是说，只要不在一线城市，无论从事什么工作，都不要读研。

当然，事业单位除外，什么公务员、老师，甚至从政。

张雪峰是考研神嘴，提倡考研，比较关键的一个观点是，考研，可以让你认识一个非常牛的人物，那就是你的导师，甚至如果你将来的就业可以用上那么一点他的人脉资源，那就赚大了。

这些观点，都很难取舍。

3. 想要逃避现实生活，再在学校呆3年

当然不可取

4. 随大流

也不可取

有的人的观点是，你考研先考上再说，再去问要不要读研。

这样的观点，有一定道理，但也站不住。

当你背上所有的梦与想。

上大学了，我要好好学习，因为努力就有回报，因为我仅用高三一年的时间，就可以达线，就可以考上大学。

有很多很多日夜不眠，拼死奋战的人，也就将达线，甚至没有达线。

我的努力是有成效的。

因为高三之前，实在荒废了太多时间，高中期间回家从没有写过作业。于是考上大学，高三一年学了高中三年的东西，再甚包括初中两年的。

这么容易！

吃到甜头，还想再吃。

于是，大学，上课第一排，虽然不爱写作业。做事积极尝试，虽然明知道自己不行，但也要尝试。试过了，才配说不行。

工作？是，工作了，一定要出人头地，光，宗，耀，祖，你们都赚不到的钱，我一定要赚到，你们不敢想的薪水，我一定要拿到。钱钱钱！你们都在乎钱，你们都没有钱，我要有！

赚钱？是本科就业好，还是读研再就业好？

对了，越来越喜欢教育行业，尤其是老师，工作量不大，铁饭碗，社会地位高，赚的不多，但稳定的一塌糊涂。越来越崇尚，越来越，懒惰？是啊，懒又怎么样，我可不想毕业就失业，也不想整天加班，增删改查做CMS，不想找不到女朋友，不想35岁就被开除。

家里就我一个孩子，可惜，我完全没有信心，做出为人杰的事情。

当你看到，各色的脸上，各色的妆。

他们都变了，我的小学同学们，女生学会画浓妆，学会打麻将，男生抽烟，当保安挣钱。

初中时候不懂事的她，追求你的她，陪在你身边的她，要结婚了？

高中时候单纯的她，你心疼的她，让你失望的她，曾决心一起度过一生的她，让你奋发图强，努力奋斗的她， 好吧，你已经不认识了。

大学同学们，来自全国各地，家庭背景不一样，社会背景不一样，成长环境不一样，经济条件不一样，未来的前景不一样。

大学同学，只不过是按照当初随便报的专业，按照学号，机缘巧合被分在一起，去完成四年的学业任务而已。

情？

宿舍舍友？坐火车的时候，如果是软卧，也不得不和，来自不同地方，有着不同目的地的人，在同一间屋子里，睡上几觉。

我说不出更多东西忘了。

没人会记得，你的模样。

我始终一个人。

拿起酒杯，

对自己说，

一杯敬你，另一杯还敬你。

人生苦短何必念念不忘？

说得容易。

清醒的人不荒唐，一点都不荒唐。

这首歌明明不好听，曲不如摄影艺术，词不如雅俗共赏，人不如VAE。

可是念念不忘。

你的大学是否也会遇见，让你心烦意乱的人？

第一次四级考不过，所以她不是你应该依靠的人。

第二次四级考不过，因为第一次的考试结果伤透了你的心。

第二次四级考不过，因为你还在等一个可以依靠的人。

第二次四级考不过，因为你还在等一个给你信念让你考过的人。

考研犹豫不决，是因为对她的态度还毫无所知。

但，我是外人，我管你有什么理由考试不过？我管你有什么理由停滞不前？

我是外人，你不比我强，我不比你差。

是，我比你差，没有你强。

你过了四级，人们会说，啊，他四级竟然过了！

我没过四级，人们会说，啊，你四级竟然没过！

smallyu 是不是很自恋。

smallyu 是不是有假想敌。

smallyu 是不是心理有问题。

生而为人，而已。

总在盼望，总在失望，日子还不都这样。

天亮之后，不要离场，不要潦草。

我送你。

[前一篇](#): [NCRE4 . 网络工程师《计算机技术》概要](#)

[后一篇](#): [我的世界，你做主](#)

Copyright Hower'Blog Rights Reserved. . Powered By [Z-BlogPHP](#). theme by [????](#)



smallyu

Hello World

??

??

????

2017080 (6)

2017070 (2)

2017060 (3)

2017050 (2)

2017040 (2)

2017030 (3)

2017020 (3)

2017010 (4)

20160120 (1)

NCRE4 · 网络工程师 《计算机技术》 概要

□ smallyu □ 2017-08-09 □ NCRE4

前言——2013年下计算机等级考试四级新标准

一、开考类别

科目名称	科目代码	考核课程代码
网络工程师	41	401、 403
数据库工程师	42	404、 405
软件测试工程师	43	401、 405
信息安全工程师	44	401、 403
嵌入式系统开发工程师	45	401、 402

二、专业核心课程

课程代码	课程名称
401	四级操作系统原理
402	四级计算机组成与接口

20160110(2)
20160100(3)
2016090(1)
2016080(6)
2016070(1)
2016060(4)
2016050(4)

- 403 四级计算机网络
- 404 四级数据库原理
- 405 四级软件工程

三、获证条件

通过四级科目考试，并已经（或同时）获得三级相关证书。

四、考试形式：

- 1. 考试环境：采用无纸化考试的形式；
- 2. 考试时间：两门专业核心课程总时间为90分钟；
- 3. 合格标准：满分100分，两门课程分别同时达到30分；
- 4. 题型及分值：单选题30分，多选题20分。

五、题型分析

1. 总体难度分析

由于两门课程合计考试时间为90分钟，总体难度不会太大。

2. 考核能力分析

题型为单选题与多选题，对课程大纲要求的知识点考核以理论基础的 understanding 与判断为主，设计到复杂计

算的内容较少。

3. 单选题应试方法分析与学习建议

单选题以概念理解为主。考前学习中注意知识的理解，尽量减少死记硬背的内容。

应试中建议利用排除法。类比法快速确定答案，所以在考前学习中注意概念的理解，和概念的对比记忆。

1) 单选题常见题型₁

基本网络技术名词、概念的考核。多个名词的功能与原理的区别。难易度不大。

2) 单选题常见题型₂

某个网络技术相关概念、名词的理解，从选项中判断正误。

学习中注意概念的理解，应试中善于利用排除法。

3) 单选题常见题型₃

某个网络技术相关参数的简单计算。

对于通过固定公式计算的知识点主要记牢计算公式；

对于网络中常见的数值进制转换、子网计算等在学习中掌握计算方法，提高计算时间与准确率。

4. 多选题应试方法分析与学习建议

多选题以概念理解为主。考前学习中注意知识的理解，加强知识点的归纳总结，和概念的对比记忆。

计算机网络课程（课程代码403）介绍

基本要求：

- 一、理解计算机网络的基本概念
- 二、掌握局域网的基本工作原理
- 三、掌握TCP/IP及相关协议
- 四、掌握Internet基本服务类型
- 五、掌握较为新型的网络技术应用
- 六、理解网络管理与网络安全原理

考试内容：

- 一、网络技术基础
 1. 计算机网络的形成与发展
 2. 计算机网络的基本概念
 3. 分组交换技术

4. 网络体系结构与网络协议

5. 互联网应用的发展

二、局域网技术

1. 局域网与城域网的基本概念

2. 共享式局域网的工作原理

3. 高速局域网的工作原理

4. 交换式局域网与虚拟局域网

5. 无线局域网

三、Internet 基础

1. Internet 的构成与接入

2. IP 协议与服务

3. IP 地址与 IP 数据报

4. 差错与控制报文

5. 路由与路由选择

6. 组播技术

7.IPv6协议

8.TCP协议和UDP协议

9.NAT的基本原理

四、Internet基本服务

1.应用进程通信模型

2.域名系统

3.远程登录服务

4.FTP服务

5.电子邮件系统

6.Web服务系统

五、新型网络应用

1.即时通信系统

2.文件共享系统

3.IPTV

4.VoIP

5. 网络搜索技术

6. 社交网络应用

六、网络管理与网络安全

1. 网络管理

2. 网络安全基础

3. 加密技术

4. 认证技术

5. 安全技术应用

6. 入侵检测与防火墙

7. 计算机病毒

总体分析：考察内容与原三级网络技术的内容大部分相同，提出部分章节的老旧内容，新增部分前沿知识。90%以上内容相同。

应试建议：

1. 学习上以教材为主，配合网课。

2. 习题上以原三级网络技术的历年真题为主，多做试题。

第1章 网络技术基础

学习目标：

1. 了解计算机网络的形成与发展
2. 掌握计算机网络的基本概念
3. 掌握分组交换技术与包交换技术的基本原理
4. 掌握网络体系结构与网络协议的基本概念
5. 了解互联网应用的发展

一、计算机网络的形成与发展

一）计算机网络的发展阶段

第一阶段：20世纪50年代，将独立的计算机技术与通信技术结合；

第二阶段：20世纪60年代，美国的ARPANET与分组交换技术开始；

第三阶段：20世纪70年代，ISO推动系统参考模型与网络协议；

第四阶段：20世纪90年代，互联网、高速通信网络、无线网络与网络安全技术的发展，基于P2P的网络应用成为互联网产业与信息服务业的新增长点。

二）计算机网络的形成

1. 有一台中央主机通过通信线路连接大量的地理上分散的终端，构成面向终端的通信网络，终端分时访问中心计算机的资源，中心计算机将处理结构返回终端。

2. 世纪60年代中期，出现了多天计算机通过通信系统互连的系统，开创了“计算机—计算机”通信时代，这样分布在不同地点且具有独立功能的计算机就可以通过通信线路，彼此之间交换数据、传递信息。

3. ARPANET的发展以及OS的规定，使各种不同的网络互联、互相通信变为现实，实现了更大的范围内的计算机资源共享。

4. Internet是覆盖全球的信息基础设施之一，用户可以利用Internet实现全球范围的信息传输、信息查询、电子邮件、语音与图像通信服务等功能。

三）网络体系结构与协议标准化

在计算机网络发展的第三阶段，出现了很多网络，导致网络之间的通信困难。迫切需要统一的网络体系结构和统一的网络协议。

ISO制定了OSI参考模型，作为国际认可的标准模型。

TCP/IP协议以及体系结构早于OSI参考模型，因此TCP/IP协议与体系结构也是业内公认的标准。

四）互联网应用与高速网络技术发展

1. 互联网高速发展

互联网不仅是一种资源共享、数据通信和信息查询的手段，主键成为了人们了解世界、讨论问题、休闲、学术研究、商贸、教育甚至军事活动等重要领域。

2. 信息高速公路

高速网络技术主要体现在：异步传输模式（ATM），宽带综合业务数字网（B-ISDN），高速局域网，交换局域网，虚拟网络与无线网络。

3. 基于WEB技术的互联应用的发展

Web技术的出现使网站的数量和网络的通信量呈指数增长。

4. 基于P2P技术的应用技术发展

区别于C/S结构，对等（P2P）网络淡化了服务提供者和服务使用者的界限，扩大了网络资源的范围和深度。

5. 网络安全技术的发展

计算机及网络犯罪，使得网络必须具备足够的安全机制，防止信息被非法窃取、破坏与泄露。

搜索引擎是一种运行在Web上的应用软件系统；

P2P以“非中心化”方式使更多的用户同时身兼客户机与服务器的双重身份，达到信息共享最大化的目标。

五）宽带城域网的发展

宽带城域网与传统的通信网络在概念和技术上发生了很大的变化，主要体现在以下几个方面：

传统局域网、城域网与广域网在技术上的界限模糊。

传统的电信传输技术与计算机网络技术的界限越来越模糊。

传统的电信服务与互联网应用的界限越来越模糊。

电信传输网。计算机网络与广播电视网络技术的界越来越模糊。

1. 宽带城域网应该包括核心交换网与接入网；

2. 目前，可作为用户接入网的主要有三类：计算机网络、电信通信网与广播电视网；

3. 从技术角度来看，互联网的用户接入方式主要分为五类：地面有线通信系统、无线通信和移动通信网、卫星通信网、有线电视网和地面广播电视网。

4. 三网融合：计算机网络、电信通信网与电视传输网。

六）无线网络的发展

1. 无线局域网（WLAN）

以微波、激光与红外等无线电波作为传输介质，IEEE 802.11工作组专门从事WLAN标准的研究。

无线局域网主要有四个应用领域：传统局域网的扩充、建筑物之间的互联、漫游访问与特殊网络。

无线局域网的传输技术：红外线、扩频、窄带微波。

2. 无线自组网（Ad hoc）

是一种自组织、对等式、多跳的无线移动网络，无线自组网技术的发展趋势：一是向军事和特定行业发展的无线传感器网络，二是向民用的接入网领域发展的无线网状网。

3. 无线传感器网（WSN）

将Ad hoc与传感器技术相结合，WSN的三个要素是传感器、感知对象和观察者。

4. 无线网状网（WMN）：

由无线路由器构成骨干网，用于大范围的信号与节点连接。

WMN作为WLAN、WiMax技术的补充，成为解决无线接入“最后一公里”问题的新技术方案。

IEEE 802.11S工作组专门为WMN制定了MAC与PHY层协议。

5. 蓝牙技术

IEEE 802.15工作组制定了蓝牙通信标准。

七）操作系统的发展

操作系统包括两部分：客户机操作系统与服务器操作系统。

带网络功能的操作系统的基本任务：用屏蔽本地资源与网络资源的差异性，为用户提供各种网络服务功能，并提供网络系统的安全服务。

1.Windows操作系统

2.Unix

原本为小型机环境开发的操作系统。

主要用C语言写成。

符合易移植操作系统环境（POSIX）标准。

UNIX特点：

UNIX系统是一种多用户、多任务的分时操作系统。

UNIX系统结构分为两大部分：操作系统内核和外壳。内核原语可以直接对硬件起作用，使得系统易读、易修改、易移植。

UNIX提供了强大的shell语言（外壳语言）

UNIX采用树状目录结构具有良好的安全性、保密性和可维护性。

UNIX提供多种通信机制，如管道、软中断通信等。

UNIX采用进程对换的内存管理机制和请求调页的存储管理方式。

常用的UNIX系统有SUN公司的solaris，IBM公司的AIX，HP公司的HP-UX等。

3.Linux

基于Intel硬件、在微型机上运行，类似于Unix的操作系统。

编写内核效仿Unix，兼容Unix工具与外壳。

符合POSIX标准，开放源代码。

的特点：

Linux

Linux操作系统是自由软件，具有开放性。

Linux操作系统支持多用户、多任务。

Linux操作系统能把CPU的性能发挥到极限，具有出色的高速度。

Linux操作系统具有良好地用户界面。

Linux操作系统具有丰富的网路功能。

Linux操作系统采用了许多安全措施，为网络多用户提供了安全保障。

具有可移植性。

具有标准的兼容性。

八) 我国互联网的发展

balabala...

二、计算机网络的基本概念

一) 计算机网络的定义

1. 三类定义观点

广义、资源共享、用户透明性

符合网络特征的是资源共享的观点。

2. 资源共享观点定义

“以能够相互共享资源的方式互联起来的自治计算机系统的集合”，主要表现在以下方面：

建立计算机网络的主要目的是实现计算机资源的共享、

互联计算机是分布在不同地理位置的多台独立的“自治计算机”。

联网计算机之间的通信必须遵循共同的网络协议。

二) 计算机网络的分类

按网络的覆盖范围可以分为：局域网、城域网、广域网、个人区域网。

局域网： balabala...

城域网： balabala...

广域网： balabala...

个人区域网： balabala...

三) 计算机网络的拓扑结构

1. 网络拓扑的定义

抽象

“点”：工作站、服务器、通信设备等。

“线”：通信介质。

网络拓扑结构：

通过网络中节点与通信线路之间的几何关系表示网络结构，反应出网络中各个实体间的结构关系。

2. 网络拓扑的分类

按通信子网中的通信信道类型分为：

广播信道通信子网的拓扑与点到点线路的通信子网。

广播信道通信子网的拓扑结构包括4种：

总线型、树型、环型、无线与卫星通信。

点到点线路的通信子网的拓扑结构包括4种：

星型、环型、树型、网状型。

1) 星型拓扑

结点通过点对点线路与中心结点连接，结构简单、易于实现、便于管理，中心结点是整个网络的可靠性瓶颈。

2) 环型拓扑

结点通过点对点线路连接成闭合环路，结构简单、传输延时确定，但环中每条通信线路都会成为网络可靠性的瓶颈。

3) 树形拓扑

结点按层次进行连接，是星型拓扑的扩展，适用于汇聚数据的应用需求。

4) 网状拓扑

结点之间的连接是任意的，主要优点是可靠性高，目前广域网主要用网状拓扑构型。

四) 描述计算机网络传输特性的参数

1. 数据传输速率

数据传输速率是每秒钟传输二进制比特数，单位比特/秒，计作bps

公式: $S=1/T$ (bps)

T 为发送每个比特所需的时间

数据传输速率与带宽

带宽: 某个信号具有的频率宽度，单位是赫兹 (Hz)

“带宽”和“速率”几乎成了同义词

通信信道最大传输速率与信道带宽之间存在着明确的关系

奈奎斯特准则

香农定理

1) 奈奎斯特准则

描述了有限带宽、无噪声信道的最大数据传输速率与信道带宽的关系。

二进制数据信号的最大数据传输速率 $R_{\max}$ 与通信信道带宽（ $B=f$ ，单位Hz）的关系为：

$$R_{\max}=2f$$

2) 香农定理

描述了有限带宽、有随机热噪声信道时，最大传输速率与信道带宽、信号噪声功率比之间的关系：

$$R_{\max} = B \log_2(1 + S/N)$$

其中 $R_{\max}$ ：数据最大传输速率。单位是bps

B ：信道带宽，单位是Hz

S/N ：信号与噪声功率比

香农定理给出了一个有限带宽、有热噪声信道的最大数据传输速率的极限值

2. 误码率的定义

误码率是指二进制码在数据传输系统中被传错的概率，它在数值上近似等于： $P_e=N_e/N$

其中 N 指传输二进制码元总数， N_e 为被传错的元数。

对于当前使用的电话线路，传输速率在 $300\sim 2400\text{ps}$ 之间时，平均误码率在 $10^{-4}\sim 10^{-6}$ 之间；

传输速率在 $4800\sim 9600\text{bps}$ 之间时，平均误码率在 $10^{-2}\sim 10^{-4}$ 之间；

计算机通信的平均误码率要求低于 10^{-9} 。

三、分组交换与包交换

早期的广域网通信子网中，数据交换分为两类：线性交换、存储转发交换。

一）线路交换的基本概念

线路交换的通信过程分为3个阶段：

1. 线路建立阶段：

两台主机要传输数据，首先通过子网建立两台主机之间的线路连接。

2. 数据传输阶段

线路连接后，可以实现实时、双向的交换数据。

3. 线路释放阶段

数据传输结束后，源点向目的主机发送释放请求，目的同意后逐步释放连接。

线路交换方式的优点是：通信实时性强，适用于交互式会话类通信。

线路交换方式的缺点是：对突发性通信不适应，系统效率低，同时不具备数据存储能力和差错控制能力，不能平滑通信量，也无法发现和纠正传输过程中发送的差错。

二) 存储转发交换的特点

1. 发送的数据中包含目的地址、源地址与控制信息，按一定的格式组成一个数据单元；
2. 通信子网中的节点是通信控制处理剂，负责完成数据单元的接收、差错校验、存储、路由选择和转发功能。

存储转发交换方式可以分为两类：报文交换和报文分组交换

报文交换中不控制数据的长度，在发送数据时，只把数据当作一个逻辑单元，在数据中加上目的地址、源地址与控制信息后，按一定格式打包后组成一个报文。

分组交换则限制数组的最大长度，源节点需要将一个长报文分成多个分组，由目的结点将多个分组按顺序重新组织成报文。

存储转发交换方式，它克服了电路交换方式的缺点，具有明显的优点。

通信子网的路由器可以存储分组，因此多个分组可以共享通信信道，线路利用率高。

路由器具有路由选择功能，可以动态选择分组通过通信子网的最佳路径，同时可以平滑通信量，提高系统的效率。

分组在通过通信子网的每个路由时，都需要进行差错检查和纠错处理，因此可以减少传输错误，提供系统的可靠性。

路由器可以对不同通信速率的线路进行速率转换，也可以对不同的数据代码格式进行转换。

三) 数据报方式与虚电路方式

1. 数据报方式

即分组存储转发的一种形式

主要特点：

同一报文的不同分组可以经过不同的传输路径通过通信子网；

同一报文的不同分组到达目的节点时可能出现乱序、重复与丢失现象；

每个报文在传输过程中都必须带有目的地址与源地址；

数据报方式的传输延迟较大，适用于突发性通信、不适合于长报文、会话式通信。

2. 虚电路方式

虚电路方式视图将数据报和电路交换方式结合起来，发挥其各自的优点，以达到最佳的数据交换效果。主要特点：

每次传输分组之前，在源节点与目的结点之间建立一条逻辑连接；

一次通信的所有分组都通过虚电路顺序传输，分组中不必携带目的地址、源地址等信息，不会出现乱序、重复和丢失现象

分组通过虚电路上的每个结点只需进行差错检测，不许进行路由选择

通信子网中的每个结点可以与任何结点建立多条虚电路连接

四、网络体系结构与网络协议

一) 网络体系结构的概念

网络协议的定义：为网络数据交换而定义的规则、约定与标准。

网络协议三个要素：

语法：用户数据与控制信息的结构和格式

语义：需要发出何种控制信息，以及完成的动作与做出的响应

时序：对事件实现顺序的详细说明

网络层次模型和各层协议的集合定义为网络体系结构

计算机网络体系结构采用层次结构，好处是：

各层之间是独立的

设计灵活

结构上可分割开

易于实现和维护

能促进标准化工作

第一个计算机网络体系结构：IBM公司的SNA

二) ISO/OSI参考模型

1.OSI参考模型的基本概念

由ISO定义开放系统互联（OSI）模型定义了网络互联的七层框架，OSI定义了各层提供的服务，服务和实现无关。

分层的体系结构、三级抽象：

第一级抽象：体系结构

第二级抽象：服务定义

第三级抽象：协议规格说明

OSI/RM并不是一个标准，而只是一个概念性的框架

2.ISO/OSI参考模型结构和各层功能

OSI参考模型结构的划分基本原则：

网络中各结点都具有相同的层次；

不同结点的同等层具有相同的功能；

同一结点内相邻层之间通过接口通信；

每层可以使用下层提供的服务，并向其上层提供服务；

不同结点的同等层通过协议来实现对等层之间的通信。

1) 物理层 (Physical layer)

利用物理传输介质为数据链路层提供物理连接

透明地传送比特流

2) 数据链路层 (Data link layer)

在通信的实体之间建立数据链路连接

传送以“帧”为单位的数据

采用差错控制、流量控制方法，使有差错的物理线路变成无差错的数据链路

3) 网络层 (Network layer)

通过路由算法，为分组通过通信子网选择一条最佳路径

网络层要实现路由选择、拥塞控制与网络互连等功能

4) 传输层 (Transport layer)

向用户提供可靠的端到端 (End-to-End) 服务，透明地传送报文

向高层屏蔽了下层数据通信的细节

计算机通信体系结构中最关键的一层

5) 会话层 (Session layer)

组织两个会话进程之间的通信，并管理数据的交换

建立传输错纠正与保证数据完整性的控制机制

6) 表示层 (Presentation layer)

处理在两个通信系统中交换信息的表示方式

数据格式变换、数据加密与解密、数据压缩与恢复等功能

7) 应用层 (Application layer)

为应用程序提供了网络服务

应用层需要识别并保证通信对方的可用性，使得协同工作的应用程序之间的同步

建立传输错误纠正与保证数据完整性的控制机制

三) TCP/IP参考模型

1. TCP/IP参考模型与协议的发展过程

TCP/IP—Transmission Control Protocol/Internet Protocol，传输控制协议/互联网协议

最早在ARPANET中使用

TCP/IP特点：

开放的协议标准，可以免费使用

独立于特定的网络硬件

统一的网络地址分配方案

标准化的高层协议

2.TCP/IP参考模型与层次结构

TCP/IP的主机 网络层实现了OSI模型中物理和链路层的功能

TCP/IP的互联层功能主要体现在3个方面：

处理来自传输层的分组发送请求

处理接受的数据报

处理互联的路径、流控与拥塞问题

主要协议：

IP、ICMP、IGMP、ARP、RARP

传输层实现应用进程间的端到端通信，具有两个协议：TCP和UDP协议

1) TCP: 是一种可靠面向连接的协议, 允许将一台主机的字节流无差错地传送到目的主机。

2) UDP: 不可靠的无连接协议。不要求分组顺序到达目的地。

应用层的主要协议有: 远程登录协议 (Telnet), 文件传输协议 (FTP), 简单邮件传输此而已 (SMTP), 域名服务 (DNS), 路由信息协议 (RIP), 网络文件协议 (NFS), 超文本传输协议 (HTTP) 等。

3. OSI与TCP/IP参考模型比较

1) 共同点:

层次结构的概念

传输层功能相似

2) OSI参考模型的主要缺陷

模型的层次数量与内容不是最佳的选择

把“服务”与“协议”的定义结合起来使得模型变得很复杂, 寻址、流量控制与差错控制在每一层重复出现, 效率低

受到通信思想的支配, 不适合计算机与软件的工作方式

3) TCP/IP参考模型主要缺陷:

在服务、接口与协议的区别上不清楚

在某些层次的划分上不够合理

五、互联网应用的发展

一) 基于WEB应用的发展

WEB技术的出现使互联网从最初的主要由计算机专家和大学生使用，变为一种广泛使用的信息交流工具，使得网站的数量和网路的通信量呈指数增长，已经广泛用于电子政务、电子商务、远程教育与信息服务等领域，并有继续扩大的趋势。

二) 搜索引擎技术发展

搜索引擎是运行在WEB上的应用系统软件，是对网络上大量资源建立索引并提供检索服务的应用软件。

三) 博客技术的应用

博客是基于互联网的**数字广播技术之一**。balabala...

四) 博客技术的应用

balabala...

五) 网络电视的应用

网络电视（IPTV）通过**宽带IP**网络传输，可以实现与用户的**互动点播**，同时能够方便地将传统电视与WWW、E-MAIL等互联网结合起来。

六) 技术的应用

P2P

1.P2P网络的特点：

是信息达到最大化为目的而设计的“非集中式”的网络结点之间的结构

不依赖与互联网的DNS，并具有使用网络拓扑动态变化的独立自制系统

可以与网络中已有的“集中式”共享网络资源的结构共存于互补

P2P网络中的每一台计算机既可以作为网络服务的使用者，又可以作为网络服务的提供者

2.典型的P2P应用：

balabala...

第2章 局域网技术

学习目标

一、了解以太网与城域网的基本概念

二、掌握共享式以太网技术特点

三、掌握告诉局域网的工作原理

四、掌握交换式局域网与虚拟局域网的技术特点与原理

五、掌握无线局域网工作原理

一、局域网与城域网的基本概念

一) 决定局域网与城域网的三要素:

网络拓扑、传输介质与介质访问

二) 局域网网络拓扑结构类型

局域网覆盖范围是“有限地址范围”，所以基本通信机制与广域网不同，从存储转发方式改变为共享介质交换方式

局域网网络拓扑主要为：总线型、环型、星型

局域网传输介质主要为：双绞线、同轴电缆与光纤

1. 总线型拓扑

总线型局域网的介质访问控制方法采用“共享介质”方式

主要特点:

所有结点都通过网卡连接到公共传输介质总线上

总线通常采用双绞线或同轴电缆作为传输介质

所有结点可以通过总线发送或接受数据，但在同一段时间内只允许一个结点通过总线发送数据

可能出现冲突而造成传输失败

优点：

结构简单、容易实现、易于扩展，可靠性较好

缺点：

不易管理，故障诊断和隔离比较困难

2. 环型拓扑

环型拓扑是共享介质局域网的主要拓扑结构之一，环型拓扑结构中，节点之间通过网卡利用点对点线路连接构成闭合的环型，数据在环中按一个方向绕环逐站传输，通常采用令牌环控制方式实现介质访问控制。

主要优点：

初始安装比较容易，传输线路较短

故障诊断定位比较准确

适用于光纤连接，例如FDDI

主要缺点：

可靠性较差

重新配置较为困难

三) 传输介质类型与介质访问控制方法

局域网的传输介质类型常用的有同轴电缆、双绞线、光缆、无线信道

IEEE802.2标准定义的共享介质局域网有三类:

带有冲突检测的载波侦听多路访问 (CSMA/CD) 的总线型局域网

令牌总线 (Token Bus) 方法的总线型局域网

令牌环 (Token Ring) 方法的环型局域网

四) IEEE 802参考模型

IEEE 802参考模型研究重点是OSI参考模型中的数据链路层与物理层

1. IEEE 802标准

IEEE 802参考模型将数据链路层划分为两个子层: 逻辑链路控制子层 (LLC) 与介质访问控制子层 (MAC), 不同局域网在MAC子层和物理层可以采用不同的协议, 但在LLC子层必须采用相同协议。

2. IEEE 802标准

IEEE 802标准主要有四个:

IEEE 802.3标准: 定义CSMA/CD总线介质访问MAC子层与物理层标准

IEEE 802.11标准: 定义无线局域网介质访问MAC子层与物理层标准

IEEE 802.15标准：定义近距离个人无线网络介质访问MAC子层与物理层标准

IEEE 802.16标准：定义快带无线局域网介质访问MAC子层与物理层标准

二、共享式以太网

一) Ethernet技术的发展

1976年7月，Bob在ALOHA网络的基础山，提出总线型局域网的设计思想，并提出冲突检测、载波侦听与随机后退延迟算法，将这种局域网命令为以太网（Ethernet）

以太网的核心技术是：介质访问控制方法CSMA/CD

这种方法解决了多结点共享公用总线的问题

早期以太网的传输介质是同轴电缆，后用双绞线，再后来用光纤

二) Ethernet工作过程

1. 发送流程

载波侦听过程

冲突检测方法

发现冲突、停止发送

随机延迟重发

载波侦听过程：

发送数据前侦听总线是否空闲，如果线上已有数据在传输，总线电平将按曼切斯特编码规律出现跳变，则可判断此时“线路忙”

冲突检测方法：

载波侦听不能完全消除冲突，在节点_A发送数据到结点_B时，数据传输时需要时间的，在此过程中如果结点_B也发送数据，则可能发生冲突

以太网标准规定冲突窗口长度为 $51.2\mu s$ ，以太网数据传输速率为 $10Mbps$ ，冲突窗口 $51.2\mu s$ 可以发送 $64B$ 的数据

发现冲突、停止发送

如果检测到冲突，发送结点要进停止发送数据、随机延迟后重发数据，随机延迟重发的第一步是发送“冲突加强信号”，目的是让网中的所有结点能检测出冲突。

随机延迟重发

最大重发次数为 16 次，如果超过 16 ，则认为线路故障，进入“冲突过多”的结束状态。

CSMA/CD 后退延迟算法：截止二进制指数后退延迟

(TBEB)： $r = 2^k * R * a$

其中 r 为重新发送所需的后退延迟时间：

为冲突窗口值

a

R为随机数

$K = \min[\text{重传次数}_n, 10]$

2.Ethernet接收流程分析

第一步：判断接收帧是否小于最小长度

第二步：判断目的地址是否自己可以接收

第三步：CRC校验

CRC校验正确，进行帧长度检测，如果长度不对，报告“帧长度错”，长度正确，数据交给LLC子层，报告“成功接收”

CRC校验错误，判断该帧是否8的整数倍。若是，则报告“帧校验错”，若否，则报告“帧比特错”

3.Ethernet帧结构

前导码与帧定界符：用于接收同步阶段，比特序列101010...11，不计入帧头长度。

目的地址和源地址：目的地址第一位为0表示单播，为1表示多播，全1为广播

类型：表示网络层使用的协议类型，如0x0800表示IP，0x8137表示IPX

数据字段：46-1500B，所以帧长度范围为64-1518B

帧校验：多项式为

$$G(x)=x^{32}+x^{26}+x^{23}+x^{22}+x^{16}+x^{12}+x^{11}+x^{10}+x^8+x^7+x^5+x^4+x^2+x^1+1$$

4.Ethernet实现方法

从现实的角度来看，构成以太网网络连接的设备包括网卡、收发器和收发器电缆

从功能角度来看，包括发送与接收信号的收发器、曼切斯特编码器与解码器、以太网数据链路控制、帧装配及主机的接口

从层次的角度来看，这些功能覆盖了IEEE 802.3标准的MAC子层与物理层

5.Ethernet物理地址

物理地址按照48位编码（EUI-48），计作12个十六进制，两两一组，“-”分割，前三组（字节）为公司表示，后三组生产商自行分配，例如006008为3com公司的一个机构唯一标识符（OUI），生产的某一网卡地址为00-A6-38，则网卡的物理地址为00-60-08-00-A6-38

三、高速局域网的工作原理

1.高速局域网的发展

局域网的规模不断的扩大，网络通信负荷加重时，冲突和重发现象将大量发生，网络效率急剧下降，网络传输延迟增长，网络服务质量下降

为了克服传统网络规模与性能之间的矛盾，高速局域网的解决方案：

1) 提高带宽

2) 将大型局域网划分成多个用网桥或路由器互联的子网

3) 将共享介质方式改为交换方式

2. Fast Ethernet - 802.3u 标准

有关传输介质的标准:

100BASE-TX: 使用2对5类非屏蔽双绞线或2对1类屏蔽双绞线, 1对用于发送, 1对用于接收, 支持全双工

100BASE-T4: 使用4对3类非屏蔽双绞线, 3对用于传输, 一对用于检测, 不支持全双工

100BASE-FX: 支持2芯的单模或多模光纤, 传输距离达2km, 支持全双工

3. Gigabit Ethernet - 802.3z 标准

有关传输介质的标准:

1000BASE-T: 使用5类非屏蔽双绞线, 长度可达100m

1000BASE-CX: 使用5类屏蔽双绞线, 长度可达25m

1000BASE-LX: 使用波长1300nm的单模光纤, 长度可达3000m

1000BASE-SX: 使用波长850nm的多模光纤, 长度可达300~550m

4.10 Gigabit Ethernet - 802.3ae 标准

主要特点：

与10M/100M/1Gbps以太网的帧格式相同

保持802.3对以太网最小和最大帧长度的规定

不再使用双绞线，而使用光纤作为传输介质

只有全双工工作方式，传输距离不受冲突检测的限制

物理层标准：

局域网物理层（LAN PHY）

广域网物理层（WAN PHY）

5.40/100 Gigabit Ethernet

40Gbps网络使用波分复用技术

100 Gigabit Ethernet物理接口类型：

10X10Gbps短距离互联的LAN接口技术

4X25Gbps中短距离互联的LAN接口技术

10m的铜缆接口和1m的系统背板互联技术

四、交换式局域网与虚拟局域网

一) 交换式局域网的基本结构

1. 交换机的基本概念

典型的交换式局域网是交换式以太网，它的核心设备是以太网交换机

共享端口 -> 专用端口

端口之间可有多个并发连接

2. 交换机的技术特点

低交换传输延迟

从传输延迟时间的量级来看，局域网交换机延迟为几十_{us}，网桥为几百_{us}，路由器为几千_{us}。

高传输带宽

对于10Mbps的端口，半双工端口带宽为10Mbps，而全双工端口带宽为20Mbps；

对于100Mbps的端口，半双工端口带宽为100Mbps，而全双工端口带宽为200Mbps。

允许不同速率的端口共存

采用了自动侦测（Autosenese）技术时，交换机端口可以同时支持10Mbps/100Mbps、全双工/半双工两种工作方式

支持虚拟局域网服务

二) 局域网交换机的工作原理

1. 交换机的工作原理

balabala...

2. 端口号/MAC地址映射表

交换机利用“端口/MAC地址映射表”进行数据交换

交换机的“地址学习”是通过读取帧的源地址并记录帧进入交换机的端口，如果端口与地址的对应关系不存在则加入映射表中，如果已经存在则更新表中对应关系

每次更新或新加入的映射表表项，会被赋予一个计时器（生命），计时器到达时没有得到该表项的对应关系，则删除

3. 交换机的帧转发方式

直接交换方式

交换机只要接受一个帧并检测到目的地址，就立即转发该帧。

优点是交换延迟短，缺点是缺乏差错检测能力你，不支持不同速率端口之间的帧转发。

存储转发交换方式

交换机完整接受一个帧并进行差错检测，然后转发

优点是具有差错检测能力，支持不同速率端口之间的帧转发，缺点是交换延迟增大

改进的直接交换方式

接收到一个帧的前64字节后，检测帧头字段是否正确，如果正确则转发。这种方法对于短的Ethernet帧，交换延迟与直接交换方式相近；对于长的Ethernet帧，由于它只检测的地址字段与控制字段，因此交换延迟将会减少。

三) 虚拟局域网 (VLAN) 的工作原理

1. 虚拟网络的概念：

虚拟网络建立在交换式技术的基础上，如果网络中的结点按工作性质与需要换分为若干个“逻辑工作组”，则一个逻辑工作组就是一个虚拟网络。

虚拟网络以软件方式实现逻辑工作组的划分与管理，工作组的结点不受物理位置的限制（相同工作组的结点不一定在相同的物理网段，只要能够通过交换机互联）。从一个工作组到另一个工作组时，只要通过软件设定，无需改变结点在网络中的物理位置。

2. 虚拟局域网的组网方法

1) 用交换机端口定义虚拟局域网：

虚拟局域网从逻辑上将交换机的端口划分为不同的虚拟子网，各个虚拟子网相对独立

不允许不同虚拟网包含相同的物理网段或端口

缺点：当结点转移到另一个端口时，管理者必须重新配置虚拟局域网的成员。

2) 用MAC地址定义虚拟局域网：

由于MAC地址是与硬件相关的地址，结点将自动保持原来的虚拟局域网成员划分，因此允许结点移动到其他物理网段，是一种基于用户的虚拟局域网。

建立局域网阶段需要将用户配置到虚拟局域网中。

缺点：在大规模的网络环境中，人工完成这么多用户的初始配置非常麻烦。

3) 用网络层地址定义虚拟局域网：

使用结点的网络层地址来定义虚拟局域网

优点：用户可以按照协议类型来组建虚拟局域网，有利于组建基于服务或应用的虚拟局域网；用户可以随意移动结点，而无需重新配置网络地址，对基于TCP/IP的用户特别有利。

缺点：性能比较差，因为检查网络层地址比检查MAC地址更加费时。

4) 基于广播组的虚拟局域网：

当有广播包要发送到多个目的结点时，就需要动态建立虚拟局域网的代理，代理通过广播信息通知各个结点，如果结点响应信息就可以加入广播组，并成为基于广播组的虚拟局域网成员。

优点：可以根据服务灵活地组建，并可以跨越与广域网互联。

3. 虚拟局域网的优点

1) 方便网络用户管理，减少网络管理开销

2) 提供更好的安全性

3) 改善网路服务质量

五、无线局域网

一) 无线局域网的应用

1. 传统局域网的扩充
2. 建筑物之间的互联
3. 漫游访问
4. 特殊无线网络结构（如无线自组网）

二) 无线局域网的分类

1. 红外无线局域网（IR）

按视距方式传输，发送点与接收点中间必须无阻挡

红外无线局域网的数据传输技术：

定向光束红外传输

全方位红外传输——需要在天花板安装一个基站

漫反射红外传输

优点：通信安全性好，抗干扰性强，系统安装简单，易于管理，但是传输距离受一定限制

2. 扩频无线局域网

扩频通信是军事电子对抗中经常使用的一种方法，它是将数据基带信号频谱扩展几倍或几十倍，以牺牲通信的频带宽度为代价，来提高系统的抗干扰性与安全性。

扩频技术主要有2种：

1) 跳频扩频通信

将利用的频带分为多个子频带，子频带称为信道。

每个信道带宽相同，中心频率有伪随机数发生器的随机数确定，变化频率叫跳跃系列。

发送端和接收端采用相同的跳跃系列。

2) 直接序列扩频通信

将发送数据经过伪随机数发生器产生的伪随机码进行异或操作，再将异或操作的结果调制后发送

所有接受结点使用相同频段

发送端与接收端使用相同的伪随机码

3. 窄带微波无线局域网

窄带微波是指使用微波无线电频带进行数据传输，其带宽刚好能容纳信号。

三) 无线局域网标准IEEE 802.11

1.IEEE 802.11 协议结构

该协议采用的是层次模型结构，其中物理层定义了红外、跳频扩频与直接序列扩频的数据传输标准

MAC层主要功能是对无线环境的访问控制，可以在多个接入点上提供漫游支持，同时提供数据验证与保密服务

MAC层支持无争用服务与争用服务两种访问方式

无争用服务：系统中存在中心控制结点。中心控制结点具有点协调功能（PCF）

争用服务：类似以太网的随机争用访问控制方式，成为分布协调功能（DCF）

MAC层采用冲突避免的方法是CSMA/CA（载波侦听多路访问/冲突避免）

2.IEEE 802.11的MAC层工作原理

CSMA/CA方法要求每个发送结点在发送帧之前先侦听信道。如果信道空闲，结点可以发送帧，在发送完一帧后，必须再等待一个短的时间间隔，检查接收站是否发回帧的确认ACK。如果接收到确认，则说明此次发送没有出现冲突，发送成功；如果在规定时间内没接收到ACK，表明出现冲突，发送失败，并重发该帧。

3.IEEE 802.11 协议的帧结构

balabala...

4.IEEE 802.11 网络的结构

端站STA

接入点AP

接入控制器AC

AAA服务器

5.IEEE 802.11协议族组成

标准名称 标准描述

802.11a 5G Hz 波段，速率54Mbps

802.11b 2.4G Hz 波段，速率11Mbps

802.11g 2.4G Hz 波段，速率54Mbps

802.11n 5G Hz 波段，速率100Mbps

802.11i 增强无线通信安全的规范

802.11e 服务质量QOS

第3章 Internet基础

学习目标

一、了解Internet的构成

二、掌握Internet不同接入方式的原理和特点

三、掌握IP服务的特点和IP层的工作原理

四、掌握IP地址、IP数据包的基本知识

五、掌握基本的路由选择算法并运用算法进行计算

六、掌握RIP和OSPF协议的基本原理

七、了解组播的概念和特点

八、掌握组播的相关协议

九、了解IPv6协议的基本特点

十、掌握IPv6地址表示和数据报格式

十一、掌握TCP和UDP协议的特点和提供的服务

十二、掌握TCP的可靠性和流量控制的实现

十三、掌握NAT技术的基本工作原理和技术特点

一、Internet的构成

从设计者角度看：Internet是计算机互联网的一个实例；从使用者角度看：Internet是一个信息资源网

一) Internet 的主要组成部分

1. 通信线路: Internet 的基础设施, 包括有线线路和无线线路
2. 路由器: 网络互联的桥梁。主要任务是数据从一个网络到另一个网络时, 路由器为它选择最佳路由
3. 主机: 是信息资源和服务的载体。所有连接 Internet 的计算机统称为主机
4. 信息资源: 信息资源是用户最关注的问题之一。用户方便、快捷获取资源一直是 Internet 的研究方向。

二) Internet 的接入方式

1. 通过电话网接入
2. 利用 ADSL (非对称数字用户线路) 接入

下行数据传输速率远远大于上行数据传输速率 (非对称)。

3. 使用 HFC (有线电视网) 接入

HFC 采用非对称数据传输速率。

4. 通过数据通信线路接入

种类有: DOM, ATM, 帧中继等, 用户可以租用。

二、IP 协议与 IP 层服务

一) 互联网的工作原理

IP

Internet是将提供不同服务的、使用不同技术的、具有不同功能的网络互联起来形成的。

IP协议精确定义了IP数组格式、并且对数据寻址和路由、数据报分片和重组、差错控制和处理等作出了具体规定。

工作原理：假设主机_A发送数据到主机_B

主机_A的应用层形成的数据传输层送互联层处理

互联层将数据封装成IP数据报，并决定发送给最近的路由器

主机_A把IP数据报利用以太网控制传送到路由器

经由路由器对数据报进行拆封和处理；如果仍需传输，再封装后利用互联层的广域网控制程序传输

经由通信子网传输到主机_B

二) IP服务

运行IP协议的互联层可为其高层用户提供的服务有三个特点：

1. 不可靠的数据投递服务
2. 面向无连接的传输服务
3. 尽最大努力投递服务

三) 互联网的特点

IP

- 1.IP互联网隐藏了底层物理网络细节，向上为用户提供通用的、一致的网络服务
- 2.IP互联网不制定网络互联的拓扑结构、也不要求网络之间全互联
- 3.IP互联网在物理网络之间转发数，信息可以跨网传输
- 4.IP互联网平等的对待互联网中的每一个网络

三、IP地址

一) IP地址的作用

以太网中利用MAC地址（物理地址）标识网络中的一个结点，两个以太网结点需要知道对方的MAC地址才能通信

以太网不是唯一的网络，各种网络技术互不相同，它们之间通信是需要解决的问题

统一物理地址的方法不现实。在互联网层将各种物理地址统一必须通过上层软件完成

屏蔽各种物理地址的差异，使用IP协议规定的地址（IP地址）。IP地址中管理机构统一管理和分配，保证在网络中的每台计算机不会发生冲突

IP地址的作用是标识网络连接

（严格地说，IP地址指定的不是一台计算机，而是计算机到一个网络的连接，例如一台计算机有块网卡，有两条连接，有两个IP地址；或多个IP地址绑定在一条物理连接上）

IP地址由32比特的二进制数表示

二) IP地址的层次结构

IP地址有两层：网络层和主机号

网络号：标识互联网中一个特定的网络；而主机号标识该网络中主机的一个特定连接

IP地址含有主机的信息和网络的地址信息，所以主机从一个网络移动到另一个网络时，IP地址必须重新分配，否则不能与其他计算机通信

三) IP地址的分类

A、B、C、D、E 五类

A类地址的前一个字节表示网络地址，且最前端的1个二进制位固定是0，表示的地址范围是从1.0.0.0~126.255.255.255

B类地址的前两个字节表示网络地址，且最前端的2个二进制位固定是10，表示的地址范围是从128.0.0.0~191.255.255.255

C类地址的前三个字节表示网络地址，且最前端的3个二进制位是110，表示的地址范围是从192.0.0.0~223.255.255.255

四) IP地址的直观表示法

点分十进制：32位的二进制，8个一组，形式4组，魅族二进制转换成十进制，计作 $w.x.y.z$ 的格式

五) 特殊的 地址形式

IP

1. 网络地址：网络位不变，主机位全0

2. 广播地址

直接广播：网络位不变，主机位全1

有限广播：网络位全1，主机位全1

3. 回送地址：127.0.0.1

4. 本地地址：即私有地址

10.0.0.0/8、172.16.0.0/12、192.168.0.0/16

六) 子网编址

1. 子网编址的方法

标准的IP地址分为网络号和主机号两部分，为了避免IP地址的浪费，子网编制将IP地址的主机号部分进一步划分成子网部分和主机部分

在一个IP网络中划分子网使我们能将一个逻辑上单一的大型网络分成若干个小小的网络，即允许将网络划分成许多个部分供内部使用，但是对于外部网络仍像一个网络一样。

子网编制将IPv4地址分为三部分：网络部分、子网部分和主机部分

2. 子网表示法

子网掩码采用了和IP地址一样的编码方式，用32位二进制数值。其二进制数值必须是连续的1+连续的0组成。其中1位数对应IP地址的网络号部分，0位数对应IP地址的主机号部分。

3. 子网编址思路：

1) 你所选的子网掩码将会产生多少个子网？

$2^x - 2$ (x 代表借位部分掩码位，即2进制为1的部分)

2) 每个子网能有多少主机？

$2^y - 2$ (y 代表主机位，即2进制为0的部分)

3) 每个子网的广播地址？

广播地址 = 下个子网号 - 1

4) 每个子网的有效主机分别是？

忽略子网内全为0和全为1的地址剩下的就是有效主机地址：

最后有效1个主机地址 = 下个子网号 - 2 (即广播地址 - 1)

4. 可变长子网掩码VLSM子网编址

VLSM (Variable Length Subnet Mask, 可变长度子网掩码)，是在标准掩码上面再划分的子网的网络号码，无类路由选择网络可以使用VLSM，而有类路由选择网络中不能使用VLSM。

5. CIDR (无类域间路由技术) 地址规划方法

1) CIDR的技术特点

表示为“网络前缀+主机号”的二级地址结构。

网络前缀相同的连续的IP地址形成了一个CIDR地址块，它是由块起始地址决定的

2) CIDR的应用

超网

支持路由聚合

七) 地址解析协议ARP

1. ARP协议的功能

互联网中高层软件通过IP地址来指定源地址和目的地址，而低层的物理网络通过物理地址发送和接收信息

ARP就是以太网中使用的将IP地址映射到物理地址的实现方法

2. ARP实现中采用的高速缓存技术

为了避免ARP请求与相应信息的频繁发送与接收对网络产生的影响，每一个主机都设有一个ARP高速缓存（ARP cache）。

3. 其他提高ARP效率的技术

balabala...

四、IP数据报

IP数据报是IP协议使用的数据单元

一) IP数据报的格式

IP数据报的格式可以分为报头区和数据区两大部分。其中数据区包括高层需要传输的数据，报头区包含了源IP地址、目的IP地址等控制信息

IP数据报各字段的功能：

1.版本与协议类型

2.长度

3.服务类型

4.生存周期

5.头部校验和

6.地址

二) IP封装、分片、重组

IP数据报在互联网上传输，可能跨越多个网络，IP数据报最终需要封装成帧进行传输

路由器只在内存中保留 数据报而没有附加帧头信息，只有 数据报通过一个物理网络时，才会被封

IP
装进帧中，接受路由器会解封装帧

IP

1.MTU与分片

根据网络使用的技术不同，每种网络都规定了一个帧最多能够携带的数据量，这一限制称为最大传输单元（MTU）

2.重组

在所有段的基础上重新产生原数据报的过程叫重组

3.分片控制

在IP数据报报头中，标识、标志和片偏移三个字段与控制分片和重组有关

1) 标识：发送方将一个惟一的标识放进每个输出数据报的标识域中，接收方就可利用收到的分片的标识数和IP源地址来确定该分片属于哪个数据报，以便于数据报重组

2) 标志：标识该数据是否已经分片，是否是最后一个分片

3) 片偏移：用来指出本片数据在初始IP数据区中的位置，以8字节为单位。目的是提供重组的分片顺序

三) IP数据报选项

IP数据报选项主要目的：控制和测试

数据报选项：选项码、长度、选项数据

IP

选项数据：

1. 源路由

IP数据报穿越互联网所经过的路径由源主机指定，分为严格源路由选项和松散路由选项

2. 记录路由

记录IP数据报经过路径上各个路由器的IP地址

3. 时间戳

经过每个路由器时的当地时间

五、差错与控制报文

IP层使用的控制与差错协议是ICMP（互联网控制报文协议）

ICMP报文是作为IP数据报的数据部分而传输的

ICMP协议的两种典型应用：ping和traceroute

一）ICMP差错控制

ICMP最基本的功能是提供差错报告

每一个ICMP报文的产生总是对应与一个数据报，要么这个数据报遇到了问题，要么这个数据报携带者一个请求报文

ICMP

一个路由器从输入的数据报的头部中取出源地址，然后放到携带ICMP报文的数据报的头部中的目的地域中

ICMP差错报文的特点：

1. 携带ICMP报文的数据报并没有什么特别优先权——它们像其他数据报一样转发
2. ICMP差错报告数据中包含故障IP数据报报头之外，还包含IP数据报数据区的前64比特数，目的是了解高层协议信息
3. ICMP差错报告时伴随着抛弃出错IP数据报而产生的，一旦发现传输错误，先把出错报文抛弃

ICMP出错报告种类：

1. 目的地不可达报告
2. 超时报告（存活周期TTL）
3. 参数出错报告

用ICMP报文测试可达性

ping程序可以测试一个给定的目的地是否可达

它发送一个包含ICMP回应请求的报文给目的地，然后等待一段很短的时间如果没有收到应答，则重新传送请求如果重传的仍没有收到应答（或收到一个ICMP目的不可达报文），ping声称该远程机器为不可达

用ICMP跟踪路由

二) ICMP控制报文

IP层控制主要包括：拥塞控制、路由控制

1. 拥塞控制

路由器的处理速度太慢，不能完成IP数据报排队等日常工作

路由器传入数据速率大于传出数据速率

最终没有足够的缓冲区存放大量涌入的IP数据报

解决方案：

利用ICMP源抑制报文抑制源主机发送IP数据报的速率

2. 拥塞控制与源抑制报文

ICMP源抑制报文的方式

如果路由器的某输出队列已满，那么缓冲区空出之前，该队列将抛弃新来的IP数据报，并向源主机发送ICMP源抑制报文

为路由器的输出队列设置一个阈值，当队列的数据报超过阈值后，如果再有新的数据报到来，向源主机发送ICMP源抑制报文

3. 路由控制与重定向报文

二) ICMP控制报文

ICMP控制报文包括源抑制报文和重定向报文

源抑制报文___用于拥塞控制

重定向报文___用于路径控制

三) ICMP请求/应答报文对

ICMP请求/应答报文对可以获取的信息： 回应请求/应答

1. 回应请求与应答

回应请求/应答ICMP报文对用于测试目的主机或路由器的可达性

2. 时戳请求与应答

同步时钟

3. 掩码请求与应答

六、路由器与路由选择

在IP互联网中，路由选择（routing）是指选择一条路径发送IP数据报的过程，而进行这种路由选择的设备叫路由器（Router）

一) 表驱动 选路

IP

在需要路由选择的设备中保存一张IP路由表

IP路由表存储着有关可能的目的地址及怎样到达目的地址的信息

在转发IP数据报时，查询IP路由表，决定把数据报发往何处

路由表中的目的地址如何表示？

- 1) 大型互联网（如因特网）中可能存在成千上万台主机
- 2) 路由表中不可能包括所有目的主机的地址信息

内存资源占用巨大

路由表搜索时间很长

- 3) 隐藏主机信息

IP地址：网络号（netid）和主机号（hostid）

IP路由表中保存相关的目的网络信息

1. 标准路由选择算法

- 1) 下一站选路的基本思想

路由表仅指定从该路由器到目的路径上的下一步，而该路由器并不知道到达目的完整路径

2) 标准的IP路由表包含许多 (N, R) 对序偶

N: 目的网络的IP地址 (使用目的主机IP地址的较少)

R: 到N路径上的“下一个”路由器

2. 子网选路——标准路由选择算法的扩充

3. 路由表中的特殊路由

1) 默认路由

2) 特定主机路由

4. 统一的路由选择算法

1) 特定主机路由表项

2) 默认路由表项

3) 标准网络路由表项

4) 子网路由表项

二) 路由表的建立与更新

1. 静态路由

什么是静态路由?

静态路由是由人工建立和管理的

静态路由不会发生变化

静态路由必须手工更新以后反应到互联网拓扑结构或连接方式

静态路由的特点：

优势

安全可靠、简单直观，避免了动态路由选择的开销

适用环境

不太复杂的互联网结构

劣势

不适用于复杂的互联网结构：建立和维护工作量大，容易出现路由环；互联网出现故障，静态路由不会自动做出更改

2. 动态路由

动态路由可以通过自身学习，自动修改和刷新路由表

动态路由要求路由器之间不断的交换路由信息

优势：更多的自主性和灵活性

适用环境：拓扑结构复杂、网络规模庞大的互联网

自动排除错误路径

自动选择性能更优的路径

适用动态路由的基本条件

路由器运行相同的路由选择协议，执行相同的路由选择算法

广泛采用的路由选择协议

路由信息协议RIP：利用向量-距离算法

开放式最短路径优先协议OSPF：利用链路-状态算法

路由收敛（convergence）

含义：互联网中的所有路由器都运行着相同的。精确的、足以反映当前互联网拓扑结构的路由信息。

快速收敛是路由选择协议最希望具有的特征

三）RIP协议与向量-距离算法

1. 向量-距离路由算法

路由器周期性地向其相邻路由器广播自己知道的路由信息，用于通知相邻路由器自己可以到达的网络以及到达该网络的距离

相邻路由器可以根据收到的路由信息修改和刷新自己的路由表更改

向量 距离路由算法执行过程

- 1) 路由器启动时初始化自己的路由表
- 2) 各路由器周期性地向其相邻的路由器广播自己的路由表信息
- 3) 路由器收到其他路由器广播的路由信息后，刷新自己的路由表

向量 距离路由算法更新过程

收到对方（发送路由表的路由器）的路由表后是否学习总体原则

比我原有的表目好的判断原则：

对方的距离₊₁ < 本身原有的距离

学习的计算：

距离₊₁ 路径为对方

向量 距离路由算法的特点

- 1) 优点

算法简单、易于实现

- 2) 缺点

慢收敛问题

需要交换的信息量巨大

3) 适用环境

路由变化不剧烈的中小型互联网

2. RIP 协议

1) RIP 协议是向量 距离选择算法在局域网上的直接实现

2) RIP 协议规定了路由器之间交换路由信息的时间、交换信息的格式、错误的处理等内容

相邻的路由器之间每 30s 交换一次路由信息

路由信息来源与本地路由表

路由器到达目的网络的距离以“跳数”计算

RIP 协议的实现问题：

相同开销路由：

先见为主

过时路由：

使用计时器超时时间一般为 180s，相当于 6 个 RIP 刷新周期

慢收敛问题与对策

慢收敛问题的产生原因

慢收敛问题的解决对策

1) 限制路径最大“距离”对策

RIP协议规定，当路由表目“距离”等于或大于16为不可达路由，从路由表中删除

2) 水平分割对策

路由器从某个接口发送路由刷新报文时，不能包含该接口原有获取的信息

3) 保持对策

RIP协议规定在得知目的网络不可能到达后的一定时间内（60秒），不接受关于此网路的任何可达信息。

4) 带触发刷新的毒性逆转对策

当某路径崩溃后，最早广播该路由的路由器将原路由保留在更新报文中，并距离置位无限长，同时使用触发刷新技术，立即广播，不等待更新周期到来

3.RIP协议与子网路由

1) RFC颁布第一个版本之前，RIP协议已经被写成各种程序并被广泛使用

主要原因：配置和部署简单

2) RIP Version 1

使用标准的IP地址，不支持子网路由

3) RIP Version 2

支持子网路由

支持身份验证

支持多播

四) OSPF协议与链路状态算法

链路状态 (L-S) 路由选择算法的基本思想:

互联网上的每个路由器周期性地向其他路由器广播自己与相邻路由器的连接关系

互联网上的每个路由器利用收到的路由信息画出一张互联网拓扑结构图

利用画出的拓扑图和最短路径优先算法，计算自己到达各个网络的最短路径

OSPF路由选择协议

OSPF路由选择协议以链路状态算法为基础

主要优势

收敛速度快

支持服务类型选路

提供负载均衡和身份认证

使用环境

规模庞大、环境复杂的互联网

主要缺陷

要求较高的路由器处理能力

一定的带宽需求

主要解决方法

分层（区域）...骨干区域为area0，区域标识按IP地址格式指派路由器

向量 距离算法与链路 状态算法的原理性差异

五）部署和选择路由协议

1. 静态路由

适用于小型、单路径、静态IP互联网环境

传输只能通过一条路径进行

2. RIP路由协议

3.OSPF路由协议

七、组播

一) 单播、广播与组播

1. 单播

单播的特点是一对一地传输数据，数据发送方发出的信息包只能传送到一个接收方

若发送方同时给多个接收方传输相同的数据包，发送方必须发送多份的相同数据包

当客户机数量大、单机流量包也大时，服务器将不堪重负，发生延迟长、网络拥塞现象

2. 广播

广播的特点是路由器和交换机将同一个信息包无条件地发往每一条分支路径，由接收方自行决定接下来还是将其丢弃

服务器不用向每个用户单独发送数据，流量负载低，如有线电视网

但这种方式对于路由器而言，有多少分支就要重复发送多少次，而不论这些主机是否乐于接受该数据包，无法针对每个用户的要求和时间及时提供个性化服务

3. 组播

组播是一种允许一个或者多个发送方发送单一数据包到多个接收方的网络传输方式，它采用组播地址，只向需要数据包的主机和网络发送数据包

采用这种方式，需要相同数据流的客户端可以加入相同的组，共享一条数据流，节省了服务器的负载，提高了网络传输速率

与单播协议相比，组播协议可根据接收者的需要对数据流进行复制转发，服务器端的服务总带宽不受客户机接入带宽的限制，单组播协议没有纠错机制，发生丢包、错包后难以弥补

IP组播的特点：

使用组地址

动态的组地址

底层硬件支持

4. 单播、广播、组播比较

1) 单播

通信模式：一对一

目的地址：单播地址（A、B、C类地址）

优点：

服务器及时响应客户机的请求

服务器针对每个客户不同的请求发送不同的数据

缺点：在客户数量大、每个客户机流量大的流媒体应用中服务器不堪重负

应用举例：网络浏览

2) 广播

通信模式：一对所有

目的地址：广播地址

3) 组播

通信模式：一对一组

目的地址：组播地址（D类地址）

二) 组播技术基础

1. 组播的相关协议

根据协议的作用范围，组播协议分为组播管理协议和组播路由协议

组播管理协议是主机和路由器之间的协议，又包括Internet组管理协议（IGMP）和Ciso专用的组管理协议（CGMP）

组管理协议

IGMP协议运行于主机和主机直接连接的组播路由器之间，其功能是：

一方面通过协议，主机通知本地路由器希望加入并接收某个特定组播组的消息

另一方面， ...

IGMP协议的发展经过了IGMPv1、IGMPv2、IGMPv3、IGMPv4,目前通用的是IGMPv3.

组播路由协议是路由器与路由器之间的协议，又分为域内组播路由协议以及域间组播路由协议

IP组播路由协议

组播路由协议是IP组播协议体系中最核心的功能

组播路由由源地址、组地址、入接口列表和出接口列表⁴部分组成

组播路由协议分为域内组播路由协议以及域间组播路由协议

域内组播路由协议又分为密集模型（适用于带宽充裕的网络）和稀疏模型（适用于带宽未必有充裕的网络）

网络上使用较多的域内组播协议有DVMRP、MOSPF协议和PIM协议

对于域间组播路由的协议主要有多协议便捷网关协议（MBGP）和组播源发现协议（MSDP）

各种组播协议都需要获得网络的单播拓扑结构

八、IPv6协议

一）IPv6的局限性

1. 地址空间的局限性：IP地址空间耗尽制约了互联网的发展，子网划分、无类域间路由（CIDR）等方法虽使IP地址得到有效的利用，但也仅仅是缓解了地址短缺的矛盾

2.IP协议的性能问题

3.IP协议的安全性问题

4.自动配置问题：IP协议涉及初没有考虑自动配置问题。后来最优DHCP（动态主机配置协议）在一定程度上解决了地址自动配置问题，但需要对DHCP服务器配置，比较麻烦。

5.服务质量（QoS）

二）IPv6地址

1.IPv6的地址表示：128位地址长度

基本表示法：按16位一段，每个位段转换为4位十六进制数，并用冒号隔开。

零压缩法：在有多个零出现时，用一位0标识一个位段。

如果几个连续的段位的值都为0，那么这些零可以简写为::，称为双冒号表示法

在使用零压缩法时，不能把一个位段内部的有效0也压缩掉

双冒号在一个地址中只能出现一次

2.IPv6前缀：与无类域间路由类似，IPv6采用前缀表示法形如“地址/前缀长度”。前缀是IPv6的一部分，用来表示路由或子网标识。

3.IPv6地址类型

单播地址

组播地址

任播地址

特殊地址：全0地址（地址不存在），回送地址（0::1），IPv4兼容的IPv6地址，映射到IPv4的IPv6等

三) IPv6数据报格式

IPv6的数据报由一个IPv6的基本头、多个扩展头和一个高层协议数据单元组成。基本头固定的40字节长度。一些可选的内容放在扩展头部分实现，此种设计方法可提高数据报的处理效率。IPv6数据报格式对IPv4不向下兼容。

1. IPv6数据报各域段含义：

版本：取值为6，意思是IPv6协议

通信类型：表示IPv6的数据报类型或优先级，以提供区分服务

流标记

载荷长度：有效载荷长度，包含扩展头和高层数据

下一个报头：存在扩展头，该域段的值指明下一个扩展头的类型；如果无扩展头，该域段的值指明高层数据的类型，如TCP、UDP或ICMPv6等

跳数限制

源地址：源IPv6

2.IPv6扩展头

IPv6的基本报头固定40字节长，一些可选头信息由IPv6扩展头实现。

三) IPv6数据报

扩展头包含的内容

逐跳选项头：类型为0

目的选项头：类型为60

路由头：类型43

分片头：类型44

认证头：类型51

封装安全有效载荷报头：类型52

四) IPv6地址自动配置

分为两类：有状态和无状态两种形式

无状态地址配置：

128的IPv6地址由64位前缀和64位网络接口标识符组成。如果主机与本地网络的主机通信可以直接通信，主机之间可以利用该地址直接通信；如果与其他网络互联时，主机需要从网络中的路由器中获得该

网络使用的网络前缀，然后与₆₄位网络接口标识符结合形成有效的IPv₆的地址。

有状态地址配置：

自动配置需要DHCPv₆服务器的支持，主机向DHCPv₆服务器发多播“DHCP请求信息”，DHCPv₆返回“DHCP应答消息”中将分配的地址返回请求主机，主机利用该地址作为自己的IPv₆进行配置。

九、TCP与UDP

可靠性是一个计算机系统的基础，在网络数据传输中用户希望互联网能够提供迅速、准确、可靠的通信功能，保证不发生丢失、重复、错婿等可靠性问题。

传输层是TCP/IP体系结构中至关重要的一层。

一) 端对端通信

TCP：传输控制协议；UDP：用户数据报协议

传输层需要提供从一台主机到另一远程主机的端对端通信控制。传输层利用互联层发送数据，每一传输层都需要封装在一个互联层的数据报中通过互联网，到达目的地后，互联层再将数据提交给传输层。

二) 传输控制协议TCP

保证可靠性是传输层的主要责任

1.TCP提供的服务

从TCP用户的角度看，TCP可以提供面向连接、可靠的、全双工的数据流传输服务，保证数据在数

据连接关闭之前被可靠地投递到目的地

面向连接

完全可靠性

流接口

连接的可靠性与优雅关闭

2.TCP的可靠性实现

可靠性问题即包括数据丢失后的恢复问题，又包括连接的可靠性建立问题

3.数据丢失与重发

发送方利用重发技术补偿数据报的丢失，需要通信双方协同解决。（确认机制）

TCP选择重发等待时间具有适应性（如利用KARN算法计算）

4.连接的可靠建立与优雅关闭

为确保连接建立和终止的可靠性，TCP使用了“三次握手”法。简答说在建立和终止过程中，通信双方需要交换3个报文。

5.TCP缓冲、流控与窗口

TCP使用窗口机制进行流量控制，当连接建立时，连接的每一端分配一块缓冲区存储接收的数据，并将缓冲区的尺寸（大小）发送另一端。当数据到达时，接收方发送确认，其中包含自己剩余的缓冲区尺寸

。将剩余的缓冲区空间的数量称为窗口。

窗口和窗口通告可以有效控制TCP的流量，使发送方的数据不会溢出接收方的缓冲空间。

6.TCP连接与端口

一个TCP连接的两端称为端口，端口用16位的二进制数表示

TCP可利用端口提供多路复用功能

三) 用户数据协议UDP

从用户角度看，UDP虽处于传输层，但UDP提供了面向非连接，不可靠的传输服务

UDP面向非连接，它可以将数据直接封装在IP数据报中进行发送。

优点：运行高效和实现简单

四) 端口号的分配

目的：区分不同的应用程序

三种端口组：

熟知（著名）端口：0~1023

注册端口：1024~49151

动态或私有端口：49152~65535

十、NAT的基本工作原理

一) 为何使用网络地址转换

NAT是为了在现阶段解决IP地址短缺和不足的问题而设计的。

根据NAT技术的分类，主要分为：静态NAT，动态NAT和网络地址端口转换NAPT

1. 静态NAT

2. 动态NAT

当NAT地址池中的全局地址被全部占用后，NAT设备将拒绝再来的地址转换申请

3. 网络地址端口转化NAPT

NAPT是目前最常使用的一种NAT类型

外部网络的主机不能主动访问内部网络中的主机

第4章 Internet基本服务

学习目标

...

一、应用进程通信模型

一) 客户机/服务器模型

1. 什么是客户机/服务器模式

客户机和服务器分别是指两个应用程序

服务器处于守候状态，并监视客户机的请求

当客户机向服务器发出服务请求时，服务器做出响应

2. 客户机与服务器的特性

1) 服务器程序特性

一种专门用途的、享有特权的应用程序

能够同时处理多个远程客户机的请求

在一台共享计算机上运行

被动的等待客户机的通信请求

2) 客户机程序特性

一个非常普通的应用程序

一个用户服务可以随即开始和停止

在用户计算机上本地运行

主动地与服务器程序进行联系

3) 实现中需要解决的主要问题

1> 标识一个特定的服务

互联网TCP或UDP通常使用端口号作为自己特定的标识

2> 响应并发请求

客户机发出的请求是随机的，可能很多客户机同时请求不同服务，因此服务器必须具备多个并发处理能力，服务器有两种实现方案：

重复服务器：服务器包含请求队列，按照先进先出原则处理

并发服务器：是一个守护进程，没有请求时，它处于等待状态；客户机每来请求，服务器立即创建一个子进程，然后回到等待；并发服务器称主服务器，而子进程称从服务器

3> 服务器程序的安全问题

二) 对等计算模型

1. 什么是对等计算模型

对等计算模型（P2P）通过直接交换来共享计算机资源和服务。

2. P2P网络的基本结构

1) 集中目录式结构

Napster是一种典型的代表软件

要求服务器要持续运转

2) 分布式非结构化P2P网络结构

采用随机图的组织方式形成一个松散的网络

有较好的容错能力，支持复杂查询（多关键词、模糊查询）

多采用洪泛方式查询和定位资源（网络负载重）

可扩展性差

典型代表：Gnuella，没有中心服务器，用TTL机制决定是否继续转发信息

3) 分布式结构化P2P网络

采用基于分布式散列表（DHT）的分布式发现资源路由算法

DHT是由广域范围的大量节点共同维护的巨大散列表

DHT的节点是动态的，且数量巨大，非中心化和自组织成为两个重要的指标

典型代表：pastry，tapestry，chord，CAN

4) 混合式结构P2P网络

在分布式模式的基础上，将用户结点按能力进行分类，包括用户结点、搜索结点和索引结点

典型代表：skype、pplive、BT

3.P2P网络的应用

分布式科学计算

文件共享

协同工作

分布式搜索引擎

流媒体直播

二、域名系统

一) 互联网的命名机制

互联网利用IP地址识别主机，不便于记忆，使用字符串记忆主机好读易记（域名系统诞生）

命名机制遵循的原则：

全局唯一性

名字便于管理

高效地进行映射

1. 无层次命名机制

在无层次命名机制中，主机名简单地由一个字符串组成

无层次名字的管理与映射很简单

但随着互联网中主机的增加，这种命名机制的缺点就明显得表现出来

目前，无层次命名机制已被层次型命名机制所取代

2. 层次型命名机制

名字中加入结构，而这种结构是层次型的

层次命名机制对名字的管理非常有利。类似树状结构

主机的名字就是从树叶到树根路径上各结点标识符的有序序列

3. TCP/IP 互联网域名

在TCP/IP 互联网中的层次型名字管理机制是由域名系统DNS（Domain Name System）来实现的

4. Internet 域名

DNS 将整个因特网划分成多个顶级域，并为每个顶级域规定了国际通用的域名

顶级域的划分采用了两种划分模式，即组织模式和地理模式

二）域名解析

域名解析

将域名翻译为对应IP地址的过程

域名解析器

请求域名解析服务的软件，运行在客户端

域名服务器

负责对域名的解析工作

域名体系具有层次型，因此域名服务器也有不同的级别

最靠近用户的是本地域名服务器，最顶层为根域名服务器

1. 域名解析方式和解析过程

域名解析采用自顶向下的算法

在域名解析时，首先是向本地域名服务器提出域名请求

两种方式：可一次性完成全部名字地址变换

反复解析：每次请求一个服务器，不行再请求别的服务器

2. 提高域名解析效率

解析从本地服务器开始

域名服务器的高速缓存技术

主机上的高速缓存技术

3. 对象类型与资源记录

对象类型与类别

资源记录

三、远程登录服务

一) 远程登录协议 (Telnet)

用户使用Telnet命令，使自己的计算机成为远程计算机的一台仿真终端

远程登录允许任意类型的计算机之间进行通信，具体实现的功能有：

本地用户与远程计算机上运行的程序交互

远程登录后，可以运行远程计算机上的任何应用程序（有权限），屏蔽不同型号计算机之间的差异

二) 远程登录的工作原理

Telnet采用客户机/服务器模式

通过Tcp连接

网络虚拟终端NVT格式将不同的用户本地终端格式统一起来，是的各个终端只与虚拟终端NVT打交道

三) 使用远程登录

四、FTP服务

一) FTP客户机/服务器模型

TCP建立双重连接：一个控制连接和一个数据连接

建立数据连接的方式有两种：

主动模式（默认模式）

被动模式

二) FTP命令与响应

FTP命令和应答采用7位ASCII码

FTP命令有30多种

三) FTP文件格式

FTP协议支持两种文件传输方式：文本文件传输和二进制文件传输

文本文件传输

二进制文件传输（图像文件类型）

四) 用户接口

传统FTP命令

浏览器

下载工具

五) FTP访问控制

FTP服务器利用用户账号来控制用户对服务器的访问权限

用户名: Anonymous

口令: guest

五、电子邮件系统

一) 电子邮件系统的基本知识

1. 电子邮件系统

电子邮件系统采用客户机/服务器工作模式

电子邮件应用程序的功能

2. TCP/IP 电子邮件的传输过程

利用SMTP（简单邮件传输协议）协议向邮件服务器发送邮件，使用POP3（邮局协议）或IMAP协议从邮件服务器邮箱中读取邮件。

3. 电子邮件地址

二) 电子邮件传递协议

1.SMTP协议

SMTP协议负责邮件从一个邮局传送到另一个邮局，分为3个阶段：

连接建立：连接建立后，客户机与服务器互通自己的域名，同时确认对方域名

邮件传递：将邮件源地址、目的地址和邮件内容传递到SMTP服务器，SMTP服务器进行相应的相应并接收邮件

连接关闭：SMTP客户机发出QUIT命令，服务器处理命令后相应，随后关闭TCP连接

2.POP3协议

指第3个版本。一旦建立TCP连接，POP3客户机向服务器发送命令，下载和删除邮件。分为3个阶段：

认证阶段：邮箱具有权限，客户机将用户名和密码传递给服务器，服务器判断是否合法

事务处理阶段

更新阶段

三）电子邮件的报文格式

1.RFC822

邮件头

邮件体

2.MIME协议

对RFC822作了补充

添加了一些邮件头字段，主要有： MIME-Version、 Content-Type、 Content-Transfer-Encoding等

六、Web服务

一) WWW服务

1.WWW服务系统

WWW（world wide web）或WEB服务，以超文本标记语言（HTML）和超文本传输协议（HTTP）为基础。

特点：

以超文本方式组织网络多媒体信息

可在世界范围内任意查找、检索、浏览及添加信息

提供生动直观、易于使用、统一的图形用户界面

服务器之间可相互连接

可访问图像、声音、影像和文本信息

2.WWW服务器

WWW服务器上以web页方式组织文档，还包含指向其他页面的指针。利用超链接将分布在网络中的服务器建立关联。

超文本可利用引用链接其他不同类型的文件，这些具有多媒体操作的超文本，称为超媒体

3.WWW浏览器

WWW客户机程序称为WWW浏览器，是用来浏览服务器中WEB页面的软件

过程：WWW浏览器负责接收用户的请求，利用HTTP协议将用户的请求传送给WWW服务器。

4.页面地址——URL

WWW服务器中的页面很多，通过URL（统一资源定位器）指定什么协议、哪台服务器和哪个文件等

URL由三部分组成：协议类型、主机名和路径及文件名

二) WWW系统的传输协议

超文本协议是建立在TCP连接基础上，面向对象的协议

HTTP将一次请求/服务的全过程定义为一个简单事务处理，它由一下四个步骤组成：

连接

请求

应答

关闭

三) WWW系统的页面标识方式

四) Web的安全性

1.浏览器的安全性

2.Web服务器的安全性

第5章 新型网络应用

学习目标

...

一、即时通信系统

一) 即时通信系统的概述

1.定义

即时通信 (IM) 是一种基于Internet的通信服务

2.功能

3. 应用软件

二) 即时通信的基础通信模式

1. P2P通信模式

2. 中转通信模式

三) 即时通信实例

1. QQ用户登录过程

2. QQ聊天通信过程

3. QQ文件传输过程

四) 即时通信系统的通信协议

1. SIP协议

用户代理

代理服务器

重定向服务器

注册服务器

2. SIMPLE

是SIP协议的扩展，支持即时通信服务

3.XMPP

二、文件共享

一) 网络文件系统NFS

二) Windows LAN 文件共享

1.NetBIOS协议

2.CIFS协议

三) P2P 文件共享

1.P2P 文件共享历史

2.P2P 文件共享的理论基础

3.P2P 文件共享系统示例——Maze

三、IPTV

一) IPTV 系统

1. 视频点播 (VOD)

2. 直播电视

3. 时移电视

二) IPTV系统关键技术

IPTV基本技术形体可概括为视频数字化、传输IP化、播放流媒体化

关键技术包括：

媒体内容分发技术

数字版权管理技术

IPTV运营支撑管理系统

四、VOIP

一) VOIP实现方法

PC-to-PC

PC-to-Phone

Phone-to-Phone

二) VOIP系统的组成

终端设备

网关

多点控制单元

网守

三) Skype

五、网络搜索技术

一) 网络搜索引擎

1. 定义

二) 搜索引擎的原理与组成

1. 全文搜索引擎功能模块的组成

搜索器

索引器

检索器

用户接口

2. 目录导航和网页搜索引擎组成

3. Google 和百度搜索引擎

六、社交网络

一) 社交网络的概念与历史

二) 社会网络的基本模型

三) 社会网络应用实例

第6章 网络管理与网路安全

一、网络管理

一) 网络管理的基本概念

1. 网路管理的定义

2. 网络管理对象：硬件资源和软件资源

二) 网络管理的功能

配置管理

故障管理

计费管理

性能管理

安全管理

三) 网络管理模型

1. 网络管理的基本模型

集中式管理

分布式管理

四) 网络管理协议

1. 发展简史

2. SNMP 协议

从被管理设备中收集数据有轮询和基于中断两种方法

3. CMIP 协议

二、信息安全技术概述

一) 安全信息的概念

主要目标:

真实性

保密性

可用性

不可抵赖性

可控制性

可审查性

二) 信息安全策略

信息安全的实现要靠先进的技术、严格的安全管理、法律约束与安全教育

三) 信息安全等级

网络安全性标准 (DoD5200.28-STD)，即可信任计算机标准评估准则。

A1、 B1、 B2、 B3、 C1、 C2、 D1

在我国，以《计算机信息系统安全保护等级划分准则》 (GB17859-1999) 为指导，将信息和信息系统的系统分为5个等级：

自主保护级

指导保护级

三、网络安全问题与安全策略

一) 网络安全的概念

安全的目的

1.

对网络系统而言主要有信息的存储安全和信息的传输安全

传输安全则需要预防：网上信息的监听、用户身份的认证、网络信息的篡改、发出信息的否认、信息重放

2. 安全措施

社会法律政策、企业规章以及网络安全教育

技术方面措施

审计与管理措施

二) OSI 安全框架 (X.800)

OSI 安全框架关注三个方面：安全攻击、安全机制和安全服务

1. 安全攻击

被动攻击

主动攻击

服务攻击和非服务攻击

2. 安全机制

分为两类：特定协议层实现的和不输于任何的协议层的安全服务

3. 安全服务

三) 网络安全模型

对发送信息的相关安全变换，如消息加密

双方共享某些秘密信息，并希望这些消息不为攻击者所知，如加密密钥

安全服务主要包含₄个方面：安全传输、信息保密、分配和共享秘密信息、通信协议

由程序引起的威胁有₂种：信息访问威胁和服务威胁

四、加密技术

一) 密码学基本术语

明文：原始的消息

密文：加密后的消息

加密：从明文到密文的变换过程

解密：从密文到明文的变换过程

密码编辑学：研究各种加密方案的学科

密码分析学：研究破译密码获得消息的学科

密码学

1. 密码编码学

转换明文为密文的运算类型

所用的秘钥数

处理明文的方法

2. 密码分析学

密码分析攻击、穷举攻击

3. 无条件安全与计算上的安全

4. 代换与置换技术

代换法：将明文字母替换成其他字母、数字或符号的方法

置换法：通过置换而形成新的队列

二) 对称密码

1. 对称密码的模型

明文：作为算法的输入

加密算法：对明文进行各种代换和置换

密钥：加密算法的输入，不同于明文

密文：算法的输出

解密算法：加密算法的逆

2. 数据加密标准

广泛使用的加密体制是数据加密标准（DES）采用64位明文初始置换重新排列后和64位密钥进行一系列变换得到64位输出密文

1977年被美国的NIST，即国家标准和技术研究所采纳为联邦信息处理标注46（FIPS PUB 46）

3. 其他常见的对称加密算法

三重DES

2001年NIST将Rijndael作为AES算法，它的密钥长度为128、192或256位，分组长度为128位

Blowfish算法

RC5

三）公钥密码

1. 公钥密码体制

2. 公钥密码体制的应用

加密 解密

/

数字签名

密钥交换

3. RSA 算法

RSA 既能用于加密，也能用于数字签名的算法

选取密钥的过程

解密过程

4. 其他的公钥加密算法

Elgama 算法

基于离散对数的公钥密码体制和椭圆曲线加密体系

背包加密算法

基本原理：背包问题

四) 密钥管理

1. 密钥的分发

2. 密钥的验证

五、认证技术

一) 消息认证

1. 消息认证的概念
2. 消息认证的方法
3. 消息认证的模式
4. 认证函数

二) 数字签名

1. 数字签名的需求
2. 数字签名的创建
3. 数字签名的验证

三) 身份认证

1. 口令认证
2. 持证认证
3. 生物识别

四) 常用的身份认证协议

目前满足条件的协议主要有：

一次一密机制

请求应答机制

询问应答方式

X.509认证协议

Kerberos认证协议

Kerberos基于对称密钥体制，一般采用DES

六、安全技术应用

一) 安全电子邮件

1.PGP

鉴别、机密性、压缩、电子邮件的兼容性和分段

数字签名：采用DSS/SHA或RSA/SHA

报文加密：CAST或DEA，或3DES或RSA

压缩：ZIP

兼容：64-BASE转换

分段：支持分段和重新分配

2.S/MIME

功能：

加密数据

签名的数据

透明签名的数据

签名并且加密的数据

二) 网络层安全：IPSEC

主要协议：身份认证头（_{AH}）协议和封装安全负载（_{ESP}）协议

1._{AH}协议（身份认证头协议）

_{AH}协议提供源身份认证和数据完整性，但没有提供秘密性

_{AH}头在原有IP数据报数据（例如TCP或UDP段）和IP头之间

2._{ESP}协议

_{ESP}协议提供了数据完整性、身份认证和秘密性

_{ESP}协议不但提供了源主机身份认证，而且提供了网络层秘密性

三) WEB安全

1.Web所面临的威胁

2.Web流量安全性方法

网络级、传输级、应用级

七、入侵检测技术与防火墙

一) 入侵者

1.假冒者

2.非法者

3.秘密用户

二) 入侵技术

统计异常检测

基于规则的检测

1.审计记录

原有的审计记录

专门用于检测的审计记录

2. 统计异常检测

阈值检测和基于轮廓的检测

3. 基于规则的入侵检测

基于规则的异常检测方法是基于对过去行为的观察

基于规则的渗透鉴别采用了基于专家系统技术的方法

4. 分布式入侵检测

分布式入侵检测是要保护居于网内或内部互联网络内所有的主机安全

三) 防火墙的特性

防火墙的定义

1. 防火墙的设计目标

所有从内到外和从外到内的通信量都必须经过防火墙

只有被授权的通信才能通过防火墙

防火墙对于渗透是免疫的

2. 用来控制访问和执行站点安全策略的4中常用技术

服务控制（确定可以访问的Internet服务类型）

方向控制（决定哪些特定方向上服务请求可以被发起并通过防火墙）

用户控制（根据哪个用户尝试访问服务来控制对一个服务的访问）

行为控制（控制怎样使用特定的服务）

3. 防火墙的功能

四）防火墙的分类

包过滤防火墙、应用级网关、电路级网关、堡垒主机

八、计算机病毒

一）计算机病毒

计算机病毒的定义

1. 病毒的生命周期

潜伏阶段

繁殖阶段

触发阶段

执行阶段

2. 病毒的结构

3. 病毒的种类

寄生病毒

存储器驻留病毒

引导区病毒

隐形病毒

多态病毒

4. 几种常见的病毒

宏病毒

电子邮件病毒

特洛伊木马

计算机蠕虫

二) 计算机病毒的防治策略

反病毒软件分为四代:

简单的扫描程序

启发式的扫描程序

行为陷阱

全方位的保护

[前一篇: 用你的魔法和勇气，与小伙伴们一起勇闯《西方世界》](#) [后一篇: 轻舟虽过万重山，心到桥头自然沉](#)

Copyright Hower'Blog Rights Reserved. . Powered By [Z-BlogPHP](#). theme by [???](#)



smallyu

Hello World

??

??

????

2017080 (6)

2017070 (2)

2017060 (3)

2017050 (2)

2017040 (2)

2017030 (3)

2017020 (3)

2017010 (4)

20160120 (1)

用你的魔法和勇气，与小伙伴们一起勇闯《西方世界》

□ smallyu □ 2017-08-08 □ 单机游戏



章节一 追逐父亲

一位父亲和他的四个女儿相约去某个神秘的地方，由于种种原因，章节一的女主来晚了，还没等见到父亲，父亲就已经出发了。

20160111 (2)

2016010 (3)

201609 (1)

201608 (6)

201607 (1)

201606 (4)

201605 (4)



女主在追逐父亲的途中，遇到了她的姐妹们，她的姐妹们也没能和父亲一起前去。但她们找到一个传送装置，可以通过传送装置直接到达父亲所在的神秘地方。



不过这个装置很危险，它并不一定能够准确到达目的地，而有可能传送到任何地方。女主一进入传送装置，装置就被一个不明所以傻傻分不清的姐妹启动了。然后女主来到了这个不知道什么地方的地方。



女主属性：

武器：魔法杖

技能：使用魔法杖短距离攻击，使用魔法球，魔法球可反弹

远程：瞬移一段距离，可以穿过薄栅栏

特技：能够驱动魔法井

章节二 一个孩子和他的铁铲

这小子被困在了地下，回不去地表。在这儿看到了这么一棵树，就推测，树是需要阳光的，阳光从正上方照射下来，所以这里一定有出口。



哦不，上面的内容是第一映像得到的剧情。真实剧情是，男主所在是一个挖矿工团伙。团伙中每个人都深信不疑他们现在是在月球上挖月岩。男主看到了地表穿透下来的阳光，怀疑这里根本不是月球是地球，然后把这一消息告诉了团伙老大。

然后被团伙老大赶了出来。



所有人都开始厌恶男主，认为男主是卑鄙的。男主不服，开始了证明他的猜疑之旅。

男主属性：

武器：铁锹、溜冰鞋、炸药

技能：使用铁锹短距离攻击，扔炸弹远距离攻击

远程：溜冰

特技：能够穿过低矮的洞穴，能够使用铁锹潜入土质松软的地下，使用溜冰鞋在水面上行走
能够

章节三 魅惑雇佣兵

女主奉老板之命到西部区域的一处圣地取一块价值不菲的宝石。



任务途中女主遇到了老巫师和章节二中的铁铲小子，还看到了具有神秘力量的石柱，石柱上画着四个人的脸，包括女主的和铁铲小子的。

老巫师告诉女主，石柱上的四人将结为同伴，共同做一些不可描述的事情，她已经在石柱旁等了很长很长时间，石柱上的四人相遇是先前预言家预言过的。石柱还拥有让人瞬移的能力，在地图多处都有这样的石柱。

女主不信，继续找宝石。

寻找宝石的过程中，女主偶尔遇到困难，和铁铲小子合作通过了难关。

终于在西部区域的圣地拿到了宝石。



女主回到老板处交付宝石，并索取报酬。没想到老板背叛了她，不但没有给付报酬，还将她关押在地牢。

女主属性：

武器：罩巾、催眠术士的面具

技能：使用罩巾中距离攻击，敌人被击中会眩晕。催眠术士的面具让女主可控制被眩晕到的敌人

远程：移动加速

特技：使用罩巾拉动远方木桩，将自己拉送木桩所在位置

章节四 呔嗨

呔嗨是一位探险家，驾着船只从远方而来。



探险途中遇到了老巫师，以及章节一寻找父亲会使用魔法的女主。

男主属性：

武器：拳套

技能：拳击

远程：公牛冲锋

特技：击碎石头

章节五 勋爵和白鼬

现在咕哖和卢米纳（会魔法的章节一女主）要合力前进了。

咕哖追寻的是一只可与自己匹敌的野兽，卢米纳想要的则是回家的方法。

咕哖打败了一直体型庞大的凶猛野兽，实现了愿望。



不过卢米纳却落空了。她要找的正是章节三女主拿走的宝石。卢米纳不得不去找她。

卢卡纳依旧落空了，宝石在老板那里。卢卡纳救出了被老板押禁的女主，然后一起乘飞行器前往沙漠，追寻老板和宝石。

章节六 追击

两个人来到了沙漠。



在一个下着雪的神秘洞穴，遇到了这一章节的BOSS



没错，宝石依旧不在这里

宝石不在沙漠，在哪里呢？故事还远没有结束。

游戏剧情还是比较简陋的，大多是没有开头的故事，看的时候会不禁想问为什么，毕竟游戏没有把主人公的背景交代清楚

不过游戏很好玩，冒险解谜类游戏，尤其是好几个主人公在同一张地图中利用不同的技能穿过特性，真心想如果这是一款多人合作游戏该多好

但就单机游戏，它可以让人体会不同的人物情节，人物能力，以及合作的乐趣

画面也非常漂亮

尤其是你用一个人物第一遍走过了地形，第二遍还是同样的地方，却不得不因为人物的特殊能力另寻方法的时候，会感觉到游戏设计的精妙

人物和怪物都是Q版

难度不高，也不低，有些地方可能需要重复不止10次才能通过

游戏过程会有能力的提升，使用魔法的小女孩在到达天空（之城）后会学会使用可反弹的魔法球，后来在打败沙漠里的女BOSS后，学会了瞬移到魔法球所在位置的能力，位移距离大大增

加，铁铲小子的炸药也是在后面的章节找到的

糙大叔之前只能爬矮墙，到后面高墙也能直接爬上去了

到之后相信那罩巾的和那铲子的也会学到新的能力

还会有更多的游戏内容

但是我不想再玩了

游戏整体稍（太）显（TM）幼（难）稚（了）

如果再年轻几岁，一定要死命玩通关

[前一篇](#): [NCRE4 . 网络工程师《操作系统原理》概要](#)

[后一篇](#): [NCRE4 . 网络工程师《计算机技术》概要](#)



smallyu

Hello World

??

??

????

2017080 (6)

2017070 (2)

2017060 (3)

2017050 (2)

2017040 (2)

2017030 (3)

2017020 (3)

2017010 (4)

20160120 (1)

NCRE4 · 网络工程师《操作系统原理》概要

□ smallyu □ 2017-08-02 □ NCRE4

前言

考试方式：

上机考试，总分50分，与四级其他一门课程合计考试时间长90分钟。

题型：

单选题 10道题 30分

多选题 10道题 20分

基本要求

1. 掌握操作系统的基本概念、基本结构及运行机制。
2. 深入理解进程线程模型，深入理解进程同步机制，深入理解死锁概念及解决方案。
3. 掌握存储管理基本概念，掌握分区存储管理方案，深入理解虚拟页式存储管理方案。
4. 深入理解文件系统的设计、实现，以及提高文件系统性能的各种方法。
5. 了解I/O设备管理的基本概念、I/O软件组成，掌握典型的I/O设备管理技术。

20160111(2)

20160101(3)

20160901(1)

20160801(6)

20160701(1)

20160601(4)

20160501(4)

6.了解操作系统的演化过程、新的设计思想和实现技术。

第一章 操作系统概论

考试内容：

- 1.操作系统基本的概念、特征、分类
- 2.操作系统主要功能
- 3.操作系统发展演化进程，典型操作系统
- 4.操作系统结构设计，典型的操作系统结构

操作系统基本的概念：

balabala...

操作系统的特征：

1.并发性：在多道程序环境下，并发性是指两个或多个时间在同一时间间隔内发生，即宏观上有多道程序同时执行，而微观上，在单处理机系统中每一个时刻仅能执行一道程序。

2.共享性：共享指系统中的资源可供多个并发执行的进程使用。

涉及资源：中央处理器、内存储器、外存储器、外部设备

共享方式：互斥共享、同时共享

3. 随机性：也称异步性，不确定性，是指多道程序环境下，允许多个进程并发执行，由于资源的限制，进程的执行不是“一气呵成”的，是“走走停停”的。

操作系统的功能：

1. 进程管理

进程管理、进程同步、进程间通信、调度

2. 存储管理

内存的分配与回收、存储保护、内存扩充

3. 文件管理

文件存储空间的管理、目录管理、文件系统的安全性

4. 设备管理

缓冲管理、设备分配、设备处理

5. 用户接口

命令接口、程序接口

操作系统分类：

按照用户界面的使用环境和功能特征，分为三种：批处理系统、分时系统、实时系统

随着计算机体系结构的发展，出现的多类型操作系统：个人操作系统、网络操作系统、分布式操作系统、嵌入式操作系统

一、批处理操作系统

1. 基本工作方法

用户将作业交给操作员，在受到一定数量的作业后，由操作员把这批作业输入到计算机，最后将结果交给用户

2. 特点与分类

成批处理作业，

缺点：用户不直接与计算机交互，不适合调试程序

优点：自动化较高，资源利用率高，作业吞吐量大，提高整个系统效率

分类：简单批处理系统、多道批处理系统

单道批处理系统：主要特征：自动性、顺序性、单道性

多道批处理系统：引入的好处：提高CPU的利用率；可提高内存和I/O设备的利用率；增加系统吞吐量

3. 设计思想

先把一个作业调入内存，等这个作业运行完毕，再调入下一个作业，如此反复，直到作业处理完毕

作业及作业的衔接都有监控程序自动控制，提高了作业运行效率

4. 作业控制说明书

由作业控制语言编写的一段程序，通常放在程序前面，执行时，由程序读入作业控制说明书，按照说明书中的语句执行。

STEP1 ASM A

STEP2 FTN B

STEP3 LINK A,B,C

STEP4 RUN C

5. 一般指令和特权指令

特权指令：输入输出指令、停机指令等。只有监控程序才能执行特权指令。

用户只能执行一般指令，如需执行特权指令，处理器会将控制权移交给监控程序。

6. 假脱机技术（SPOOLing）

6.1 假脱机的概念

假脱机技术（SPOOLing）是指在联机情况下实现的同时外围操作，也称假脱机输入输出操作，它是操作系统中的一项将独占设备改为共享设备的技术。

6.2 假脱机技术的组成

假脱机技术由输入井和输出井、输入缓冲区和输出缓冲区、输入进程和输出进程、请求打印队列组成。SPOOLing系统的组成如下图所示。

6.3 假脱机技术的特点

- (1) 提高了输入输出速度
- (2) 将独占设备改造为共享设备
- (3) 实现了虚拟设备的功能

二、分时系统

1. 基本工作方式

用户通过终端交互式地向系统提出命令，系统接受命令后，采用时间片轮转方式处理服务请求

2. 设计思想

将CPU的时间划分为若干个时间段、操作系统以时间片为单位，轮流为每个终端用户服务

3. 特点

多路性：多个用户在同时使用一台计算机

交互性：用户直接干预操作的每一步

独占性：用户感觉不到计算机为他人服务

及时性：系统能对用户提出的请求即使给予响应

4. 影响响应时间的因素

机器处理能力

请求服务的时间长短

系统中连接的终端数目

服务请求的分布

调度算法（时间片的选取）

三、实时系统

实时系统是指系统能及时响应外部事件的请求，在规定的时间内，完成对该事件的处理，并控制所有实时任务协调一致地运行。

分类：

第一类：实时过程控制

工业控制，军事控制，...

第二类：实时通信（信息）处理

电讯（自动交换），银行，飞机订票，股票行情

实时系统的特征：有多路性、独立性、及时性、交互性和可靠性。

四、嵌入式操作系统

在各种设备、装置或系统中，完成特定功能的软硬件系统

它们是一个大设备、装置或系统中的一部分，这个大设备、装置或系统可以不是“计算机”

通常工作在反应式或对处理时间有较严格要求环境中

由于它们被嵌入在各种设备、装置或系统中，因此称为嵌入式系统

嵌入式领域广泛使用的操作系统有：嵌入式Linux、Windows Embedded、VxWorks等，以及应用在智能手机和平板电脑的Android、iOS等。

特点：系统内核小、专用性强、系统精简、高实用性、多任务的操作系统

五、个人计算机操作系统

单用户多任务的操作系统，比如Windows

特点：界面友好、使用方便、用户无需具备专门的知识，也能熟练使用。

六、网络操作系统

网络操作系统用于管理网络中的各种资源，为用户提供各种资源。其主要功能有网络通信管理、网络资源管理、网络安全管理和网络服务等。

类型：客户/服务器模式（C/S），对等模式

七、分布式操作系统

分布式处理系统是指由多个分散的处理单元经互联网络的连接而形成的系统。在分布式系统上配置的操作系统成为分布式操作系统。

特点：

分布性

并行性

透明性

共享性

健壮性

八、智能卡操作系统

它一般是紧紧围绕着它所服务的智能卡的特点而开发的。由于不可避免地收到了智能卡内微处理器芯片的性能及内存容量的影响，因此，在很大程度上不同于我们通常所见到的微机上的操作系统（例如DOS、UNIX等）。

网络和分布式操作系统的区别：

1. 分布具有各个计算机间相互通讯，无主从关系；网络有主从关系、

2. 分布式系统资源为所有用户共享；而网络有限制地共享。
3. 分布式系统中若干个计算机可相互协作共同完成一项任务。

操作系统的发展（演化）

1. 手工操作系统
2. 监控程序（早期批处理）
3. 多道批处理
4. 分时系统
5. UNIX通用操作系统
6. 个人计算机操作系统
7. Android操作系统

操作系统结构

按照系统的功能和特性要求，选择合适的结构，使用相应的结构设计方法将系统逐步地分解、抽象和综合，使操作系统结构清晰、简单、可靠、易读、易修改，而且使用方便，适应性强。

整体式结构

层次式结构

微内核（客户机 服务器）结构

/

一、整体结构

模块接口法、无序模块法、模块组合法

特点：根据功能划分模块

优点：结构紧密，接口简单直接，模块之间转接的灵活性使系统效率高

缺点：由于模块之间可以任意相互调用，形成网络，各模块相互联系，独立性差，系统结构不清晰

结论：可适应性较差，使用与规模较小、使用环境比较稳定却要求效率较高的系统

二、层次结构

balabala...

三、微内核结构

balabala...

第二章 操作系统运行机制

考试内容：

1. 内核态与用户态

2. 中断与异常

3. 系统调用接口

4. 存储系统

5. I/O 系统

6. 时钟 (Clock)

一、CPU 的构成与基本工作方式

处理器由运算器、控制器、一系列的寄存器以及高速缓存构成

运算器实现指令中的算术和逻辑运算，是计算机计算的核心

控制器负责控制程序运行的流程，包括取指令维护 CPU 状态、CPU 与内存的交互等等

寄存器是指令在 CPU 内部做处理的过程中暂存数据、地址以及指令信息的存储设备

在计算机的存储系统中它具有最快的访问速度，高速缓存处于 CPU 和物理内存之间

利用程序局部性原理使得高速指令处理和低速内存访问得以匹配，从而提高 CPU 的效率

1. 处理器中的寄存器

寄存器提供了一定的存储能力

速度比主存快得多

造价高，容量一般都很小

两类寄存器：

用户可见寄存器，高级语言编译器通过算法分析并使用之，以减少程序访问主存次数

控制和状态寄存器，用于控制处理器的操作由OS的特权代码使用，以控制其它程序的执行

用户可见寄存器：机器语言直接引用

包括数据寄存器、地址寄存器以及条件码寄存器

数据寄存器（data register）又称通用寄存器，主要用于各种算术逻辑指令和访存指令

地址寄存器（address register）用于存储数据及指令的物理地址、线性地址或者有效地址，用于某种特定方式的寻址。如index register、segment pointer、stack pointer

条件码寄存器保存CPU操作结果的各种标记位，如算术运算产生的溢出、符号等等

控制和状态寄存器

用于控制处理器的操作

大部分对于用户是不可见的

一部分可以在某种特权模式（由OS使用）下访问

常见的控制和状态寄存器：

程序计数器（PC），记录将要取出的指令的地址

PC Program Counter

指令寄存器（IR：Instruction Register），包含最近取出的指令

程序状态字（PSW：Program Status Word），记录处理剂的运行模式信息等等

2. 指令执行的基本过程

两个步骤：

先从储存器中每次读取一条指令

然后执行这条指令

一个单条指令处理过程称为一个指令周期

程序的执行是由不断取和执行的指令周期组成

仅当关机、出错或有停机相关指令时，程序才停止

3. 特权指令和非特权指令

特权指令：只能由操作系统使用的指令

使用多道程序设计技术的计算机指令系统必须要区分为特权指令和非特权指令，特权指令一般引起处理器状态的切换。

处理器通过特殊的机制将处理器状态切换到操作系统运行的特权状态（管态）

处理器的状态

4.

根据运行程序对资源和机器指令的使用权限将处理器设置为不同状态

管态：操作系统管理程序运行的状态，较高特权级别，又成为特权态（特态）、系统态、核心态

目态：用户程序运行时的状态，较低的特权级别，又称为（普态）、用户态

有些系统将处理器状态分为核心状态，管理状态和用户程序状态（目标状态）三种。

管态和目态的差别：

全部指令（包括特权指令）可以执行

可使用所有资源

具有改变处理器状态的能力

目态->管态

其转换的唯一途径是通过中断

管态->目态

可用设置PSW（修改程序状态字）可实现

5. 程序状态字PSW

在PSW中专门设置一位，根据运行程序使用指令的权限而设置，PSW（Program Status Word）：

CPU的工作状态码——指明管态还是目态，用来说明当前在CPU上执行的是操作系统还是一般用户，从而决定其是否可以使用特权指令或拥有其它的特殊权利

条件码——反映指令执行后的结果特征

中断屏蔽吗——指出是否允许中断

二、存储体系

支持OS运行硬件环境的一个重要方面：

作业必须把它的程序和数据存放在内存中才能运行

多道程系统中，若干程序和相关的数​​据要放入主存储器

操作系统要管理、保护程序和数据，使它们不至于受到破坏

操作系统本身也要存放主存储器中并运行

1. 存储器的层次结构

存储系统设计三个问题：

容量、速度和成本

容量：需求无止境

速度：能匹配处理器的速度

成本问题：成本和其它部件相比应在合适范围之内

2. 存储保护

对主存中的信息加以严格的保护，是操作系统及其它程序不被破坏，是其正确运行的基本条件之一

多用户、多任务操作系统：

OS给每个运行进程分配一个存储区域

为了保证软件程序只影响程序的内部

硬件可提供如下功能：

界地址寄存器（界限寄存器）

实现方法：

在CPU中设置一对下限寄存器和上限寄存器存放用户作业在主存中的下限和上限地址

也可将一个寄存器作为基址寄存器，另一寄存器作为限长寄存器（指示存储区长度）

每当CPU要访问主存，硬件自动将被访问的主存地址与界限寄存器的内容进行比较，以判断是否越界

如果未越界，则按此地址访问主存，否则将产生程序中断——越界中断（存储保护中断）

存储键

三、中断与异常机制

中断对于操作系统的重要性就像机器中的驱动齿轮一样，所以有人把操作系统成为是由“中断驱动”或者“（中断）事件驱动”，它使得OS可以捕获用户程序发出的系统功能调用及时处理设备的中断请求，定时启动指定程序

中断的概念：

CPU对系统发生的某个事件作出的一种反应

CPU暂停正在执行的程序，保留现场后自动转去执行相应事件的处理程序，处理完成后返回断电，继续执行被打断的程序

特点：

1. 中断是随机的
2. 中断是可恢复的
3. 中断是自动处理的

中断/异常概念：

中断/异常： balabala...

中断的引入： balabala...

异常的引入： balabala...

早期的中断和异常并没有区分，都把它们叫做中断。随着它们的发生原因和处理方式的差别愈发明显，才有了以后的中断和异常

中断的分类：

广义中断：

中断（外中断）：

I/O中断

时钟中断

硬件故障中断

异常（内中断）例外：

系统调用

缺页异常

断点指令

其他程序性异常（如算术溢出等）

中断（狭义）与异常的区别：

中断：与正执行指令无关，可以屏蔽

异常：与正执行指令有关，不可屏蔽

中断系统是现代计算机系统的核心机制之一。

中断系统的两大组成部分：硬件中断装置和软件中断处理程序

中断寄存器：

有的计算机中，由于可能有很多中断源请求同时发生，为了区分和不丢失中断信号，对应每个中断源分别用一固定触发器寄存中断信号。

规定值为 1 时，表示有中断信号，为 0 时表示无

这些触发器的全体称为中断寄存器

每个触发器称为一个中断位

所以中断寄存器是由若干个中断位组成

处理器如何发现中断信号？

处理器的控制部件中设一个能检测中断的机构成为中断扫描机构。

在每条指令执行周期的最后时刻扫描中断寄存器，询问是否有中断信号。

若无中断信号，继续执行下一条指令。

若有中断，中断硬件将该中断触发器内容按规定编码送入PSW的相应位，称为中断码。

典型的中断处理

I/O 中断

由 I/O 设备的控制器或者通道发出

两类 I/O 中断：

I/O 操作正常结束

如果要继续 I/O 操作，需要在准备好以后重新启动 I/O

若请求 I/O 程序正处于等待 I/O 状态，则应将其唤醒

I/O 异常

设备故障或特殊情况

设备故障时需要重新执行失败的 I/O 操作

重试次数有上限，次数过大，系统将判定硬件故障

时钟中断

系统多道能力的重要推动力量，时钟中断处理程序通常做与系统运转、管理和维护相关的工作，包括：

维护软件时钟：系统有若干个软件时钟，控制定时任务以及进程的处理器时间配额，时钟中断需要维护、定时更新这些软件时钟

处理器时间调度：维护当前进程时间片软件时钟，并在当前进程时间片到时以后运行调度程序选择下一个被调度的进程

控制系统定时任务：通过软件时钟和调度程序定时激活一些系统任务，如检测死锁、系统记账、系统审计等

硬件故障中断

硬件故障中断处理程序一般需要做的工作：

保存现场，使用一定警告手段，提供些辅助诊断信息

在高可靠系统中，中断处理程序还要评估系统可用性，尽可能恢复系统

程序性中断

程序指令出错、指令越权或者指令寻址越界而引发

两类处理方法：

1. 只能由操作系统的相关扩展功能模块完成，多为程序试图做不能做的操作引起得系统保护
2. 可由系统自己完成，如一些算术运算错误

四、系统调用

1. 系统调用与一般过程调用的特点

- 1) 运行在不同的系统状态

2) 状态的转换

3) 返回问题

4) 嵌套调用

系统调用的处理步骤

balabala...

五、I/O 技术

主要的I/O控制方式

通道

DMA技术

缓冲技术

通道

独立于中央处理器，专门负责数据I/O传输的处理机，它对外设实现统一管理，代替CPU对I/O操作进行控制，使CPU和外设可以并行工作

通道又称为I/O处理机

引入通道的目的：

为了使CPU从I/O事务中解脱出来

同时为了提高CPU与设备、设备与设备之间的并行度

DMA技术

中断的引入大大地提高了处理器处理I/O的效率，当处理器和I/O间传送数据时，效率任旧不高

解决方法：

直接存储器访问（DMA：Direct Memory Access）

通过系统总线中一独立控制单元——DMA控制器自动控制成块数据在内存和I/O单元间的传送，大大提高处理I/O的效能

缓冲技术

缓冲区是硬件设备之间进行数据传输时，用来暂存数据的一个存储区域

缓冲技术的三种用途：

处理器与主存储器之间

处理器和其它外部设备之间

设备和设备之间的通信

目的：解决硬件之间速度不匹配的问题

单缓冲区：

设备向缓冲区输入数据直到装满后，必须等待CPU将其取完，才能继续向其中输入数据。

为了提高设备利用率，单缓冲区不够

多缓冲区（Cache）技术：

Cache：离CPU最近，使CPU快速访问常使用的数据

如果没有，CPU到二级Cache中找

如果没有，CPU到系统内存中找

六、时钟

操作系统时钟是微机上所有软件获取时间的主要来源，其位置至关重要。

操作系统时钟的时间是硬件提供的，如果危机硬件时钟有错，就会影响到操作系统。

时钟一般分为硬件时钟和软件时钟

时钟的用途可分为绝对时钟和相对时钟

第三章 进程线程模型

考试内容：

1. 并发环境与多道程序设计
2. 进程的基本概念，进程控制块（PCB）
3. 进程状态及状态转换
4. 进程控制：创建、撤销、阻塞、唤醒、fork()的使用
5. 线程基本概念，线程的实现机制
6. 处理机调度

一、并发环境与多道程序设计

1. 程序的顺序执行

程序：指令或语句序列，体现了某种算法，所有程序是顺序的

顺序环境：在计算机系统中只有一个程序在运行，这个程序独占系统中所有资源，其执行不收外界影响。

顺序执行的特征：

顺序性

封闭性

程序执行结果的确定性

程序结果的可再现性

2. 多道程序系统

计算机能够同时处理多个具有独立功能的程序，以增强系统的处理能力和提高机器的利用率

多道程序的特点：

独立性

随机性

资源共享性

3. 程序的并发执行

并发环境：

在一定时间内物理机器上有两个或两个以上的程序同处于开始运行但尚未结束的状态，并且次序不是事前确定的。

并发的特征：

1) 并发程序在执行期间具有相互制约关系

2) 程序与计算不在一一对应

3) 并发程序执行结果不可再现

二、进程模型

1. 进程的概念

进程是正在执行的程序，从操作系统角度可分为系统进程和用户进程。

进程和程序既有联系又有区别。

2. 进程的特性

并发性

动态性

独立性

交往性

异步性

3. 进程的基本状态及其转换

进程的基本状态：

不同系统设置的进程状态数目不同

三状态进程模型

五状态进程模型

七状态进程模型

进程的三种基本状态

1) 就绪状态

2) 运行状态

3) 等待状态

4) 创建状态

5) 结束状态

6) 挂起状态

7) 激活状态

进程状态间的转换

1) 新状态->就绪状态

2) 就绪状态->执行状态

3) 执行状态->阻塞状态

4) 执行状态->就绪状态

5) 阻塞状态->就绪状态

6) 执行状态->终止状态

4. 进程控制块 (PCB) 的内容:

分为调度信息和现场信息两大部分。

调度信息供进程调度时使用, 描述进程当前所处的状况。

现场信息刻画了进程的运行情况。

进程的组成:

程序、数据和进程控制块 (PCB)

5. PCB 表的组织方式

1) 线性方式

2) 索引方式

3) 链接方式

6. 进程控制

创建、撤销进程以及完成进程各状态之间的转换, 由具有特定功能的原语完成。

创建原语

撤销原语

阻塞原语

唤醒原语

Unix的fork()函数

父进程通过fork()创建子进程

特点：只被执行一次，返回两次结果，一次是在调用进程中，一次是在创建的子进程中，父进程中返回的是子进程的PID，子进程中fork()返回0。

程序

```
int main(){
    pid_t pid;
    int x = 1;
    pid = fork();
    if(pid==0){
        printf("child:x=%d\n", ++x);
    }
    printf("parent:x=%d\n", --x);
}
```

有两种可能的运行结果：

child:x=2;

parent:x=0

程序₁

```
int main(){
    fork();
    printf("hello!\n");
}
```

输出两个hello!

程序₂

```
int main(){
    fork();
    fork();
    printf("hello!\n");
}
```

输出四个hello!

进程的创建

创建一个PCB

赋予一个统一进程标识符

为进程映像分配空间

初始化进程控制块

设置相应的链接

进程的撤销

1) 引起进程撤销的事件有三类:

进程正常结束

在进程运行期间, 由于出现某些错误和故障而使进程被迫中止

进程应外界的请求而终止运行

2) 进程撤销的过程

balabala...

进程阻塞和进程唤醒

1) 引起进程阻塞的事件

请求系统服务

启动某种操作

新数据尚未到达

无新工作可做

2) 引起进程唤醒的事件

请求系统服务得到满足

启动某种操作完成

新数据已经到达

有新工作可做

7. 线程的控制

进程作为系统中的一个基本单位，具有两个属性：一是资源分配和拥有的基本单位。二是一个可以地理调度和执行的基本单位。

而在进程操作过程中，因为进程是一个资源的拥有者。所以，系统要不断地进行资源的分配与回收、现场的保存、恢复等工作。系统要为此付出较大的时间与空间的开销。此外，在系统中所设置的进程数目不能过多，进程切换的频度不能过高，这就限制了系统并发程度的进一步提高。

如何能使进程更好地并发执行，同时又能尽量减少系统开销呢？

将进程的两个属性分开，由操作系统分别处理，即只作为资源分配与拥有的单位，不再是调度和执行的基本单位，使之轻装前进，而对资源分配与拥有的基本单位，不进行频繁的切换处理，以减少系统开销。正是因为这种思想，产生了一个新的概念——线程。

8. 线程的概念

线程是进程中的一个实体，是被系统独立调度和执行的基本单位。

线程自己基本上不拥有系统资源，只拥有一点在运行中必不可少的资源（如程序计数器、一组寄存器和栈），但是它可以与同属于一个进程的其他线程共享进程所拥有的全部资源。

一个线程可以创建和撤销另一个进程，同一进程中的多个线程之间可以并发执行。线程之间也会相互

制约，使其在运行中呈现异步性。因此，线程同样具有就绪、执行、阻塞三种基本状态。

9. 线程与进程的比较

1) 调度

传统操作系统中，进程是作为资源分配和独立调度的基本单位。在引入线程的操作系统中，则把线程作为调度和分派的基本单位，把进程作为资源拥有的基本单位。

2) 并发性

引入线程的操作系统中，不仅进程之间可以并发执行，而且在一个进程中的多个线程之间也可以并发执行。

3) 拥有资源

无论哪种操作系统，进程都是拥有资源的一个独立单位，一般情况，线程自己不拥有系统资源，但线程可以访问属于进程的资源。

4) 系统开销

由于进程的创建和撤销都需要分配或回收资源，因此操作系统所付出的开销显著大于创建和撤销进程时的开销。

同样，进程切换的开销也远大于线程切换的开销。

简而言之，一个程序至少有一个进程，一个进程至少有一个线程。

线程的实现机制

10.

1) 用户级线程。用户级线程是由用户控制的，对于用户级线程的创建、撤销与切换，都与系统控制无关，完全由用户自己管理。简单来说就是系统并不知道有用户级线程的存在，在系统中各种控制仍然是基于进程的。

2) 内核级线程。内核级线程是依赖于内核，即无论是用户进程中的线程，还是系统进程中的线程，它们的创建、撤销与切换都是由内核实现的。在系统中保留了一张线程控制块，系统根据该线程控制块来赶回线程的存在，并对线程进行控制。

3) 混合实现方式。同时实现用户级和内核级线程。

11.Pthread 进程包

Pthread是一套用户级线程库，定义了线程标准，大部分的UNIX系统都支持该标准。

12. 进程调度的概述

调度分层次的，可分为高级调度、中级调度、低级调度。

高级调度：它用于确定把后备队列上的哪些作业调入内存。

中级调度：核心按一定的调度算法，将内存中处于等待状态的某些进程调至外存对换区，来腾空这部分内存。

低级调度（进程调度）：指在多道程序环境下，内核按一定的调度算法，从就绪队列中选出一进程，把处理机分配给它。

进程调度的时机：

- 1) 正在执行的进程运行完毕
- 2) 正在执行的进程调用阻塞原语将自己阻塞，进入等待状态
- 3) 调用激活原语激活等待资源的进程
- 4) 时间片用完
- 5) 就绪队列中的某个进程的优先级高于当前运行进程的优先级

13. 调度算法设计原则

- 1) 面向用户的原则。这是为满足用户的需求而遵循的一些准则。
- 2) 面向系统的原则。这是为了满足系统要求而遵循的一些准则。

14. 进程（线程调度算法）

1) 先来先服务调度算法（FCFS）

先来先服务调度算法的裁决模式是非抢占式的，优先权函数 ρ 为进程在系统中的实际时间，仲裁规则是随机的。这种调度算法有利于长进程，而不利于短进程。

2) 最短作业优先调度算法（SJF）

短进程优先调度算法的裁决模式是非抢占式的，优先权函数 ρ 为运行时间，仲裁规则是按照时间先后顺序或随机方式。这种调度算法照顾到了系统中的短进程，有目的地降低了进程的平均等待时间，提高了系

统的吞吐量。但是，这种算法对长进程不利。

3) 最短剩余时间优先调度算法 (SRT)

最短剩余时间优先调度算法的裁决模式是抢占式的，优先权函数是动态的，随着进程运行和完成时间的减少而增加，仲裁规则是按照时间先后顺序或随机方式。这种调度算法充分照顾到了剩余运行时间短的进程。

4) 最短剩余时间优先调度算法 (SRT)

时间片轮转调度算法的裁决模式是面向时间片的，所有就绪进程的优先权函数值相同，仲裁规则是轮转规则。这种调度算法适用于交互进程的调度。

5) 最高优先级调度算法

优先权调度算法的裁决模式是抢占式的或非抢占式的，优先权函数是用户或系统赋予给它的优先权，仲裁规则是随机的或先进先出的。

6) 多级反馈队列调度算法 (MLF)

多级反馈队列调度算法的裁决模式是抢占式的或非抢占式的，优先权函数是从最大值开始每执行一次递减₁，仲裁规则是轮转的或按照时间先后次序。

第四章 并发与同步

考试内容

1.进程的同步与互斥

2.信号量

3.管程

4.进程的通信

1.进程的相互作用

1.1相关进程和无关进程

1) 进程互斥

2) 进程同步

1.2与时间有关的错误

2.进程互斥

解决办法两种：

1) 由竞争双方平等协商

2) 引入进程管理者

资源共享的程度分为三个层次：互斥、死锁、饥饿

2.1临界资源的概念

临界资源的访问过程：

进入区、临界区、退出区、剩余区

2.2 进程同步机制应遵循的原则

- 1) 空闲让进
- 2) 忙则等待
- 3) 有限等待
- 4) 让权等待

进程互斥的软件方法

- 1) 单标志算法

```
while (turn != i);  
临界区  
turn = j;  
剩余区
```

- 2) 双标志、先检查算法

```
while (flag[j])  
flag[i] = true;  
临界区  
flag[i] = false;  
剩余区
```


3) 双标志、后检查算法

```
flag[i]=true;
while(flag[j]);
flag[i]=false;
    剩余区
```

4) 先修改、后检查、后修改者等待算法

```
flag[i]=true;
turn=j;
while(flag[j]&&turn==j);
    临界区
flag[i]=false;
    剩余区
```

进程互斥的硬件方法

1) TS指令

每个临界区设置一个公共变量`lock:true`表示正被占用，`false`表示空闲进入区利用TS进行检查和修改表示`lock`。

2) Swap指令

balabala...

3) 信号量

年，荷兰学者提出的信号量机制是一种很有效的进程同步工具，得到了广泛的使用。

1965

Dijkstra

这里介绍最简单的经常使用的信号量——整型信号量。

信号量是一种特殊变量，它用来表示系统中资源的使用情况。而整型信号量就是一个整型变量。当其值大于“0”时，表示系统中对应可用资源的数目；当其值小于“0”时，其绝对值表示因该资源被阻塞的进程的数目；当其值等于“0”时，表示系统中对应资源已经用完，并且没有因该资源而被阻塞的进程。

对信号量的操作：

对于整型信号量，仅能通过两个标准的原语操作来访问，这两个操作被称为P操作、V操作，也合称为PV操作。其中P操作在进入临界区前执行，V操作在退出临界区后执行。

P操作，用wait(s)来描述：

```
wait(s)
{
    --s.count;    // 表示申请一个资源
    if(s.count<0)    // 表示没有空闲资源
    {
        调用进程进入等待队列s.queue;
        阻塞调用进程;
    }
}
```

V操作，用signal(s)来描述：

```
signal(s)
{
    ++s.count;    // 表示申请一个资源
    if(s.count<0)    // 表示没有空闲资源
```

```
        {  
            从等待队列s.queue中取出头一个进程;  
            进程P进入就绪队列;  
        }  
    }
```

3. 经典的进程同步问题

简单生产者_消费者问题

生产者进程P:

```
while(true){  
  
    P(empty)  
  
    生产一个产品;  
  
    送到缓冲区;  
  
    V(full);  
  
}
```

消费者进程Q:

```
while(true){  
  
    P(full)
```

从缓冲器取产品;

V(full);

消费产品;

}

利用PV操作实现进程同步的方法

1) 使用PV操作的规则

1> 分清哪些是互斥问题，哪些是同步问题

2> 对于互斥问题要设置互斥信号量，互斥信号量的个数只与临界资源的种类有关。通常，有几类临界资源就设置几个互斥信号量，且初值为1，代表临界资源可用

2> 对于同步问题要设置同步信号量，通常同步信号量个数与可参与同步的进程种类有关。同步信号量表示该资源是否可以开始或者进程是否已经结束。

4> 在每个进程中用于实现互斥的PV操作必须成对出现；用于实现同步的PV操作也必须成对出现，但是，它们分别出现在不同的进程中；在某个进程中如果同时存在互斥与同步的P操作，则其顺序不能颠倒。必须先执行对同步信号量的P操作，再执行对互斥信号的P操作。但是，V操作的顺序没有严格要求。

2) 同步互斥问题的解题步骤：

1> 确定进程

确定同步互斥关系

2>

4. 管程

4.1 引入管程的原因

每个访问临界资源的进程都必须使用PV操作，是的大量的同步操作分散在各个进程中。这不仅给系统的管理带来了麻烦，还可能因同步操作使用不当而导致系统故障，如顺序不当、读写、漏写等。为此，又产生了一种新的进程同步管理工具——管程。

4.2 管程的组成

管程由四部分组成：管程名称、共享数据的说明、对该数据进行操作的一组过程、对共享数据设置初始值的语句。此外，每个管程都有唯一的名字来标示。管程类似于面向对象程序设计中的类。

4.3 利用管程实现经典的同步问题

进程或者线程的阻塞和唤醒，可以通过管程的wait和signal操作来巧妙地实现，所有管程中的过程是互斥的。 balabala...

1) wait原语

2) signal原语

为了区别阻塞的原因，需要设置一个条件变量。条件变量是一个整型变量，用condition说明， balabala...

管程具有的三个特征：

1) 模块化

2) 抽象数据类型

3) 信息隐蔽

5. 进程通信

进程的同步与互斥成为低级通信。

用户直接利用操作系统提供的一组通信命令，高效地传送大量数据，成为高级通信，又称管道通信。

balabala...

进程之间的通信方式：

1) 共享内存

2) 消息机制

3) 通过共享文件

6. 共享内存

在相互通信的进程之间设置一个公共的内存区，一组进程向该公共内存中写，另一组进程从内存中读，通过这种方式实现两个进程之间的信息交换。

7. 消息机制

在消息机制中，进程间的数据交换是以消息为单位进行的。用户直接利用系统中提供的一组通信命令（原语）进行通信。

1) 消息缓冲通信

balabala...

发送原语: `Send(Receiver,message);`

接收原语: `Receive(Send,message);`

2) 信箱通信方式

balabala...

1>信箱通信的操作

2>信箱的分类

3>进程间的关系

8.管道通信系统

balabala...

第五章 存储管理方案

考试内容

1. 存储管理基本概念， 存储管理基本任务

2. 分区存储管理方案

3. 覆盖技术与交换技术

4. 虚存概念与虚拟存储技术

5. 虚拟页式存储管理方案

1. 存储体系

2. 存储器管理的主要任务

存储器由外存和内存组成

内存空间分为两部分：系统区、用户区

1) 内存的分配和回收

2) 存储共享

3) 存储保护

4) 权限保护

2.1 内存的分配和回收

1) 记住每个存储区域的状态

2) 实施分配

3) 回收

实现方法：

位示图表示法

空闲页面表

空闲块表

内存分配的方式：

1) 静态分配

2) 动态分配

2.2 存储共享

指两个或多个进程公用内存中的相同区域，使多道程序动态地共享，提高内存利用率

共享内容：代码共享、数据共享

2.3 存储共享

存储保护内容

1) 地址越界保护

2) 权限保护

3) 存储键保护

2.4 扩充内存

用户在编程的时候，不应该受内存容量限制，要采用一定的技术“扩充”内存的容量。

采用虚拟存储技术或其他交互技术。

3. 地址转换

将用户程序的逻辑地址转换为运行时的物理地址的过程称为地址转换，也称为地址映射（即重定位）。

静态重定位：在程序装入时，把程序中的指令地址和数据地址全部转换成绝对地址。由于地址转换工作是在程序开始执行前集中完成的，所以在程序执行过程中就无需在进行地址转换工作。

动态重定位：在程序装入时，不进行地址转换，而是直接把程序装入到分配的内存区域中。在程序执行过程中，每当执行一条指令时由硬件的地址转换机构将指令中的逻辑地址转换成物理地址。

4. 固定分区

基本定理

固定分区存储管理方式是最早使用的一种可以运行多道程序的存储管理方式。它要求把作业全部装入主存，且装入一个连续的存储空间。

基本特点

- 1) 一个作业只能装入一个分区，不能装入两个或多个相邻分区。一个分区只能装入一个作业，当分区大小不能满足作业的要求时，该作业暂时不能装入。
- 2) 通过对“分区分配表”的改写，来实现对主存空间的分配与回收。作业在执行时，不会改变存储区域，所以采用静态地址重定位方式易于实现，且系统开销小。
- 3) 当分区较大作业较小时，仍然浪费许多主存空间，并且分区总数固定，限制了并发执行的作业数目。

主存空间的分配

在作业分配之前，根据主存分区的划分情况，在分区分配表中填入每个分区的起始地址（简称始址）、大小，在状态栏中一律填入“0”，表示该分区可用，当作业装入时填入作业名。

5. 可变分区

可变分区存储管理方式又称为动态分区存储管理方式。它是根据用户作业的大小，在作业要求装入主存时，动态地划分分区，使分区的大小正好适应作业的要求。采用这种存储管理方式，分区的大小是不定的，分区的数目也是不定的。

可变分区存储管理方式必须解决三个问题：

一是分区分配中所用的数据结构

二是分区的分配算法

三是分区的分配和回收

移动技术

可变分区中，内存经过一段时间的分配和回收后，会存在很小的空闲块，这些小块不足以满足程序进一步分配内存的要求，但其总和可以满足程序的分配要求，解决的办法就是“碎片整理”，这一技术成为“移动技术”。

移动技术注意的问题：

- 1) 移动会增加系统的开销
- 2) 移动是有条件的

5.1 采用的数据结构

为了实现可变分区分配，系统必须配置相应的数据结构，用来记录主存的使用情况，包括空闲分区的情况和已分分区的情况，为作业分配主存空间提供依据。为此，设置了两张表，即已分分区表和空闲分区表。

5.2 主存空间的分配

分配主存时，先分配小地址，再分配大地址。空闲分区表中记录的排列也是从小地址向大地址排列的。首次分配时，只有一个空闲区。

常用的主存分配算法

- 1) 最先适应分配算法 (FF)

它要求空闲分区表中的记录按地址递增的顺序排列。在每次分配主存时，总是从第1条开始顺序查找空闲分区表，找到第一个能满足作业长度要求的空闲区，分割这个空闲区。一部分分配作业，另一部分仍作为空闲区。

它的特点是分配算法简单，容易差生过多的主存碎片。主存碎片是指小的不能使用的主存空间；这种算法把大的空闲区分成了小的空闲区，当有大作业要求分配时，不能满足要求，降低了系统的效率。

2) 最优适应分配算法 (BF)

它是从所有的空闲分区中挑选一个能满足作业要求的最小的空闲区进行分配。这样可以保证不去分割一个更大的空闲区，使装入大作业时比较容易得到满足。为了实现这种算法，把空闲区按长度递增次序登记在空闲分区表中，分配时，顺序查找。

它的优点是解决了大作业的分配问题，不足是容易产生主存碎片，降低了主存空间的利用率。另外收回主存时，要按长度递增顺序插入到空闲分区表中，增加了系统开销（操作系统所占有的系统资源和所需要的处理器的时间成为“系统开销”）。

3) 最坏适应分配算法 (WF)

它每次分配主存时总是挑选一个最大的空闲区，分割一部分给作业使用，使剩下的部分不至于太小而成为主存碎片。为了实现这种算法，把空闲区按长度递减的次序登记在空闲分区表中，分配时，顺序查找。

它的优点是不会产生过多的碎片，不足是影响大作业的分配。另外收回主存时，要按长度递减的顺序插入到空闲分区表中，增加了系统的开销。

) 下次分配算法

当接到内存申请时，查找分区说明，从上一次分配的位置开始扫描内存，选择下一个大小足够的可用块。

5.3 主存空间的回收

- 1) 回收的分区前后没有相邻的空闲分区
- 2) 回收分区的前后有相邻的空闲区
- 3) 回收分区的后面有相邻的空闲分区
- 4) 回收分区的前后都有相邻的空闲分区

分区管理的优缺点：

- 1) 分区的长度不是预先固定的，而是按作业的实际需求来划分的。分区的个数也不是预先确定的，而是由装入的作业个数决定的。
- 2) 分区的大小由作业的大小决定，提高了主存的使用频率。
- 3) 在主存分配过程中，会产生许多主存碎片，造成主存空间的浪费。

6. 覆盖技术

覆盖技术是指一个程序的若干程序段或几个程序段的某个部分共享一个存储空间。覆盖技术的实现是把程序划分为若干个功能上相对独立的程序段，按照其自身的逻辑结构是那些不会同时执行的程序共享一块内存区域。未执行的程序段先保存在磁盘上，当有关程序段的前一部分执行结束后，把后续程序段写入内存，

覆盖前面的程序段。

7. 交换技术

交换技术是指把主存中暂时不能运行的进程，或暂时不用的程序和数据，换出到外存上，把已具备运行条件的进程，或进程所需要的程序或数据换入到主存的技术。

1) 换出进程的选择

2) 交换时机的确定

3) 交换空间的分配

4) 换入进程换回内存时位置的确定

8. 页式存储管理

在页式存储管理方式中，将用户作业的地址空间分成若干大小相同的区域，称为页面或页，并为每个页从“0”开始编号。

相应地，主存空间也分为与页大小相同的若干存储块，成为物理块或页框，并且采用同样的方式为它们进行编号，从0开始：0块，1块，...， $n-1$ 块。

程序的逻辑地址页号和页内地址组成，页号的长度决定了分页的多少，页内地址的长度决定了页面的大小。在为作业分配主存时，以块为单位将作业中的若干页分别装入到多个不相邻的块中。作业执行时根据逻辑地址中的页号找到它所在的块号，再确定当前指令要访问的物理地址。它的地址转换属于动态重定位，由于进程的最后一页经常装不满一块，而形成不可利用的碎片，称为“页内碎片”。

8.1 主存空间的分配与回收

1) 采用的数据结构

为了实现页式存储管理方式，系统设置了主存分配表、位示图和页表，记录主存空间的使用情况和每个作业的分配情况。

2) 主存空间的回收

当一个作业执行结束，则应收回该作业所占用的主存块。根据主存分配表中的记录，取出该作业的页表。从该作业的页表中取出每一个归还的块号，计算出该块在位示图中的位置，将占用标志位置“0”。最后，把归还的块数加入到空闲块数中，删除该作业的页表，并把分区分配表中该作业的记录删除。

9. 地址转换和快表

9.1 地址转换。两个与地址计算有关的方法。

一是，由逻辑地址计算出页号和页内地址的计算方法如下：

页号 = 逻辑地址 / 页长 (商)

页内地址 = 逻辑地址 % 页长 (余数)

二是，由块号计算物理地址的计算方法为：

物理地址 = 块号 * 块长 + 块内地址 + 用户区基址

页表：

1) 多级页表

2) 散列页表

3) 反置页表

页式管理特点:

1) 有效地解决了“碎片”多的问题。可以使程序和数据存放在不连续的主存空间,而不必像可变分区管理那样通过增加系统开销来解决主存碎片的问题。

2) 通过位示图和页表来记录主存的使用情况和每个作业的分配情况,当主存很大,并且作业也很大时,位示图和页表都有可能占用较大的存储空间。另外,它要求有相应的硬件支持,从而增加了系统成本,也增加了系统开销。例如需要地址变换机构、快表等。

3) 要求页的大小固定。

9.2 快表

因页表在主存中,CPU在存取数据时,要访问主存两次。第一次是访问主存中的页表,从中找出该页的物理块号,将此块号与页内地址拼接形成物理地址,第二次是根据上一步得到的物理地址,到主存中获取数据。这样就降低了计算机的处理速度。为了提高处理速度,可以使用快表和两级页表的方法对页式存储管理进行改进。

具有快表的地址变换

具有两级页表的地址转换

10. 虚拟存储管理方式

虚拟存储器的定义

虚拟存储器是指仅把作业的一部分装入主存便可以运行的存储器系统。

虚拟存储器的逻辑容量也称为最大容量，是由地址寄存器的位数决定的。如果计算机的地址寄存器是24位，地址按单字节编址，则虚拟存储器的最大（逻辑）容量是224B。

虚拟存储器的物理容量也称为实际容量。当最大容量大于等于主存与硬盘容量之和时，虚拟存储器实际容量为主存与硬盘容量之和；当最大值小于主存与硬盘容量之和时，虚拟存储器实际容量就是最大容量。虚拟存储器的运行速度接近于主存速度，而其成本却又接近于外存。可见，虚拟存储技术是一种性能非常优越的存储器管理技术，故被广泛的应用于各类计算机中。

10.1 页式虚拟存储管理基本原理

页式虚拟存储管理是建立在纯分页基础上的，增加了请求调页功能和页面置换功能所形成的页式虚拟存储管理系统。他把作业分成大小相等的若干页，把主存分成与页大小相等的若干块；对每个作业限定分给它的主存块数，先把作业的部分页装入主存的这些块中，在作业运行时再装入所需要的页。

10.2 采用的数据结构

虚拟存储器在实现上是有一定难度的，既需要一定的硬件支持，又需要较多的软件支持，但是，请求分页式存储管理方式相对容易，因为它换进、换出的基本单位是固定长度的页面。采用的数据结构如下：位示图、页表和主存分配表。

10.3 页面调度策略：

调入策略

1) 请求调页

2) 预调页

置页策略

置换策略

1) 固定分配局部置换

2) 可变分配全局置换

3) 可变分配全局置换

10.4 页面置换算法

1) 先进先出置换算法 (FIFO)

该算法淘汰最早进入主存的页面。因为，最早进入的页面，不再使用的可能性比最近调入的页面要大。先进先出置换算法实现简单，只要把各调入主存的页按其进入主存的先后顺序连成一个队列即可，总是淘汰队首的那一页。

2) 最近最久未使用算法 (LRU)

该算法选择在最近一段时间内最久没有使用过的页淘汰掉。它依据的是程序局部性原理。最近最久未使用算法是利用一个特殊的栈来保存当前使用的各个页的页号。每当访问某页时，考察栈内是否有于此相同的页号，若有则将该页的页号从栈中抽出，再将它压入栈顶。

3) 最近最不经常使用算法 (LFU)

该算法选择到当前时间为止被访问次数最少的页置换。其实现方法是为每页设置一个访问计数器，每当页面被访问时，该页面的访问计数器就加“1”；发生缺页时，淘汰计数器淘汰值最小的页，同时将所有计数器清“0”。

4) 理想页面置换算法

从主存中溢出永远不需要的页。若无这样的页，则移出最长时间不需要访问的页。该算法只有理论意义。

5) 最近未使用页面置换算法 (NRU)

第₀类：没有被访问，没有被修改

第₁类：没有被访问，已被修改

第₂类：已被访问，没有被修改

第₃类：已被访问，已被修改

6) 第二次机会页面置换算法

7) 时钟页面置换算法

缺页中断率

缺页中断率 = $\frac{\text{中断次数}}{\text{页面访问总次数}}$

$$= \quad /$$

影响缺页中断的因素：

1>分配给程序的内存块数

2>页面的大小

3>程序编制方法

4>页面置换算法

11. 段式存储管理方式

基本原理

在段式存储管理方式中，作业的地址空间被划分为若干段，每段定义了一组逻辑信息。例如，有主程序段 $MAIN$ 、子程序段 X 、数据段 D 及栈段 S 等。每个段都有自己的名字。为了实现简单，通常可用一个段号来代替段名，每个段都从0开始编址，并采用一段连续的地址空间。段的长度由相应的逻辑信息组的长度决定，因而各段长度不等。

整个作业的逻辑地址空间，由于分成多个段，因而是二维的，即其逻辑地址由段号和段内地址所组成。

12. 段页式存储管理方式

基本原理

段页式存储管理方式的基本原理是段式和页式系统工作原理的组合，即先把用户程序分成若干段，并

为每个段划分成大小相等的若干页，把主存分成与页大小相同的块。每段分配与其页数相同的主存块，主存块可以连续，也可以不连续。

第六章 文件管理

考试内容

1. 文件的基本概念、文件逻辑结构、文件的物理结构和存取方式
2. 文件目录的基本概念，文件目录的实现
3. 文件的操作，目的操作
4. 磁盘空间的管理
5. 文件系统的可靠性和安全性
6. 文件系统的性能问题
7. Windows 的文件系统 FAT，UNIX 的文件系统

1. 什么是文件？

文件是指存放在外存上的已命名的一组相关信息的集合，通常将程序和数据组织成文件。

文件中的基本访问单位是位、字节或记录。

文件的属性包括文件类型、文件长度、文件的物理位置、文件的存取控制、文件的建立时间。

文件是一种抽象机制，它提供了一种把信息保存在存储介质上而且便于以后存取的方法，用户不用关心文件的实现细节。

2. 文件系统的概念

操作系统中与文件和目录相关的子系统统称为文件系统，是操作系统中统一管理信息资源的一种软件。

信息的存储要求：

- 1) 能够存储大量的信息
- 2) 长期保存信息
- 3) 可以共享信息

操作系统为系统管理者提供了对文件的透明存取。

研究文件系统的两种角度：

- 1) 用户的角度

主要关心文件由上面组成，如何命名，如何保护，如何操作等

- 2) 操作系统的角度

主要关心文件目录怎样实现，怎样管理存储空间，文件存储位置，磁盘实际运作方式等

文件的分类

1) 按文件的用途分类:

系统文件: 操作系统和各种应用程序和数据所组成的文件。

库函数文件: 标准子程序及常用应用程序组成文件

用户文件: 用户委托文件系统保存的文件

2) 按文件的阻值形式分类:

普通文件: 文件的组织格式为文件系统中所规定的最一般格式的文件

目录文件: 由文件的目录构成的特殊文件

特殊文件: 形式上与普通文件相同, 也可以进行查找目录操作, 但有其特殊的性质, 比如, UNIX 系统中, 输入输出设备被看作是特殊文件。

3) 按文件中的保护方式分类: 只读文件、读写文件、可执行文件、无保护文件等。

4) 按信息的流向分类: 输入文件、输出文件和输入输出文件等

5) 按文件的存放时限分类: 临时文件、永久文件和档案文件等

6) 按文件使用的介质类型分类: 磁盘文件、磁带文件、卡片文件和打印文件等

) 按照文件的组织结构分类:

7

逻辑文件：流式文件和记录文件等

物理结构：顺序文件、链接文件和索引文件等

UNIX操作系统中的文件分类：

1. 普通文件
2. 目录文件
3. 特殊文件

目的：对不同的文件进行管理，提高系统效率，同时提高文件系统的用户界面友好性

文件系统的功能

- 1) 统一管理文件的存储空间，实施存储空间的分配和回收
- 2) 实现文件的按名存取
- 3) 实现文件信息的共享，并提供文件的保护和保密措施
- 4) 向用户提供一个方便使用的接口
- 5) 系统维护及向用户提供有关信息
- 6) 保持文件系统的执行效率

7) 提供与I/O的统一接口

3. 文件的结构

1) 文件逻辑结构的概念

文件的逻辑结构是用户组织文件时可见的结构，即用户所观察到的文件组织形式。文件的逻辑结构是用户的文件组织形式。文件的逻辑结构是用户可以直接处理的数据及其结构，它独立于物理特性，又称为文件组织。

选择文件的逻辑结构主要有一下原则：

1>查找快捷。根据给定的逻辑结构，应使文件系统在尽可能短的时间内找到所需要的记录或基本信息单位。

2>修改方便。便于在文件中增加、删除和修改一条或多条记录。

3>空间紧凑。应使文件的信息占据尽可能小的存储空间。

4>易于操作。

2) 文件逻辑结构的形式

文件的逻辑结构从形式上分为两类：有结构的记录式文件和无结构的流式文件

流式文件：文件中的数据是一串字符，没有结构

记录文件：由若干逻辑记录组成，每条逻辑记录又由相同的数据项组成

记录式文件可分为定长记录文件和不定长记录文件。

4. 文件物理结构的概念

1) 文件物理结构的概念

文件的物理结构，又称为文件的存储结构，它是指文件在外存上存储时的组织结构。文件的物理结构与存储介质的物理特性及用户对文件的访问方式有关。

文件的物理结构通常划分为大小相等的物理块。这些物理块也称为物理记录，它是文件分配及传输信息的基本单位。物理记录的大小与物理设备有关，与逻辑记录的大小无关。

2) 文件物理结构的形式

根据文件存储设备的特征以及用户对文件的访问方式，可以在文件存储器中使用以下三种：

1> 顺序结构。顺序结构是很简单的一种物理结构。顺序结构将一个在逻辑上连续的文件信息一次存放在外存连续的物理块中，即所谓的逻辑上的连续，物理上也连续。

顺序结构的特点：

一旦知道文件在存储设备上的其实块号和文件长度，能快速进行存取

顺序结构支持顺序存取和随机存取

顺序结构的缺点：

文件不能动态增长

文件分配内存空间比较慢

容易产生存储碎片

2> 链接结构。克服顺序文件缺点的办法之一是采用链接结构。链接结构将文件存放在外存的若干个物理块中，这些物理块不必连续，并且在每一个物理块中设一个指针，指向下一个物理块的位置，从而使得存放在同一个文件的物理块链接起来。

windows 的 FAT 文件系统采用的就是链接结构。

链接结构的优点：

存储碎片问题解决

有利于文件动态扩充，有利于文件插入、删除

提高磁盘利用率

链接结构缺点：

存取速度慢

不适合随机存取文件

磁盘磁头移动多，效率比较低

存放文件的可靠性较差

链接指针需要额外的空间

3>索引结构。索引文件克服了顺序文件和链接文件的缺点。索引结构将文件存放在外存的若干个物理块中，并将

索引文件的优点：

保持了链接结构的优点，又解决了其缺点。

索引文件既适合于顺序存储，也适合于随机存储。

索引文件缺点：

会引起较多的寻道次数和寻道时间，索引表增加存储空间的开销。

索引文件结构对空间占用比较严重，索引表是定长还是不定长，解决问题如下：

1) 索引表的链接模式。这种模式存取文件需要读所有的索引表，对于大文件需要读很多块。

2) 多级索引

4>索引结构的实例——i节点

i节点是一种多级索引结构，最早出现在UNIX系统中。i节点在一般多级索引结构文件基础上，进行了结构的变化。

i节点的思想，是给每个文件赋予一张成为i节点的小表，这张小表列出了文件属性和文件中每个块在磁盘上的地址。

5.文件的存储介质，主要是文件的外存储设备

5.1 顺序存储设备

顺序存储设备是按信息的物理位置进行定位和读/写操作的存储设备。在顺序存储设备中，只有前面的物理块被存取之后，才能存取其后的物理块。

磁带的存储特性如下：

- 1) 磁带是一种顺序存取的存储设备，总是从磁头的当前位置开始读写
- 2) 磁带上的块不由地址来标识，而由其在次带上的位置来识别
- 3) 块与块之间有间隙，磁带上的物理块就是通过间隙来区分的
- 4) 磁带的存取速度与信息密度、磁带带速和块间间隙有关。如果带速高，信息密度大，且所需块间隙小，则磁带存取速度高。
- 5) 磁带的容量大，采用顺序存取方式时存取速度高，采用随机存取方式效率较低

5.2 随机存储设备

允许文件系统直接存取对应存储介质上的任意物理块的存储设备。如磁盘就是典型的随机存储设备。

与磁盘有关的概念

- 1) 磁道。磁盘盘片上的一系列同心圆成为磁道；为了描述磁道，对磁道由外向内进行编号，成为磁道号（编号均从0开始）。即系统通过磁道号完成对磁道的操作。
- 2) 柱面。与盘片中心有相同距离的所有磁道组成一个柱面；当磁臂移动到某一位置时，所有的读写磁头 都在同一个柱面上，盘面上的磁道号即为柱面号；对于软盘，一个柱面仅包含两个磁道。

3) 扇区。磁道沿径向又分成大小相等的若干区域，每个区域成为一个扇区，每个扇区可以存放相等字节数（一般为512字节）的信息，按照与磁盘旋转相反的方向依次给扇区编号，成为扇区号。

4) 磁头号。所有的读写磁头由上至下进行编号成为磁头号。

磁盘空间的位置由三个因素决定：柱面号、磁头号、扇区号。

访问磁盘的时间由三部分组成，即寻道时间、延迟时间和传输时间。其中寻道时间是指将磁头从当前位置移动到指定磁道所经历的时间，也称为移臂时间；延迟时间是通过磁盘的旋转将所指定扇区移动到磁头下面的时间，也称为旋转时间；传输时间是指将扇区上的数据从磁盘读出或向磁盘写入数据所经历的时间。

磁盘每个物理块的位置可用柱面号、磁头号和扇区号表示：

1) 已知物理块号，则磁盘地址：

$$\text{柱面号} = [\text{物理块号} / (\text{磁头数} * \text{扇区数})]$$

$$\text{磁头号} = [(\text{物理块号} \% (\text{磁头数} * \text{扇区数})) / \text{扇区数}]$$

$$\text{扇区号} = [(\text{物理块号} \% (\text{磁头数} * \text{扇区数})) \% \text{扇区数}]$$

2) 已知磁盘地址：

$$\text{物理块号} = \text{柱面号} * (\text{磁头数} * \text{扇区数}) + \text{磁头号} * \text{扇区数} + \text{扇区号}$$

6. 文件的存取方式

随机存取方式

随机（直接）存取方式

文件目录

是指存放文件有关信息的一种数据结构。它包含多条记录，每条记录为一个文件的文件控制块（FCB）的有关信息。最简单的记录包含文件名和文件的物理地址，用以建立文件名和存储地址的对应关系。较复杂的记录包含文件控制块的全部内容，此时，文件目录就是文件控制块的集合。

文件目录是文件实现按名存取的重要手段，是文件符号到文件物理地址之间的一种映射机制。

目录项：构成文件目录的项目（目录项即使FCB）

目录文件：用户新建一个文件时，与文件有关的信息在该文件的文件控制块内，多个文件的文件控制块一起组成了文件的目录。为了实现对文件目录的管理通常将文件目录以文件的形式保存在外存。

文件目录的管理形式可以分为一级目录、二级目录、多级目录三种。

一级目录，也称为单级目录，是一种最简单、最原始的目录结构。它采用的方法是为外存的全部文件建立一张目录表。表中包括全部文件的文件名、索引表的始址以及文件的其他属性，如文件长度、文件类型等。每个文件占据表中的一条记录。该目录表存放在外存的某个固定区域，需要时系统将其全部或部分导入主存。

一级目录的特点：

1) 目录结构易于实现，管理简单，只需要建立一个文件目录，对文件的所有操作，都是通过该文件目录实现的。

- 2) 易发生重名问题
- 3) 当文件较多时, 查找时间较长
- 4) 不便于实现文件共享, 适用于PC机的单用户系统

为了克服单级目录结构所存在的缺点, 可以把单级目录扩充为二级目录。

在二级目录中, 有主文件目录和用户文件目录。在主文件目录中, 每个用户文件目录都占有一个目录项, 其中包括用户名和指向该用户目录文件的指针。用户文件的文件说明组成的目录文件成为用户文件目录, 不同的用户拥有不同的用户文件目录, 这些文件目录具有相似的结构, 由用户所有文件的文件控制块组成。

二级目录的特点:

- 1) 提高了检索目录的速度
- 2) 可以解决用户文件重名问题
- 3) 可以使不同用户共享同一个文件
- 4) 可以实现对文件的保护和保密
- 5) 二级文件目录虽然解决了不同用户之间文件同名的问题, 但是, 同一用户的文件不能同名。

当一个用户的文件很多时, 这个矛盾就比较突出了。

为了解决用户文件同名的问题, 可以把二级目录的层次关系加以推广, 就形成了树型目录。

在二级目录结构中，如果进一步允许用户创建自己的子目录并相应地组织自己的文件，即可以形成三级目录结构，依次类推，还可以进一步形成多级目录。通常把三级或三级以上的目录结构成为树型目录结构。

在树型目录结构中，除了最低一级外，其他每一级存放的都是下一级目录或文件的说明信息，最高层为根目录，最底层为文件。UNIX和DOS系统中都采用了树型目录结构。

树型目录的特点：

1) 层次清楚：不同性、不同用户的文件可以构成不同的子树，便于管理；不同用户的文件可以被赋予不同的权限，有利于文件的保护。

2) 解决了用户文件重名问题：只要在同一目录下的文件名不发生重复，就不会由文件重名引起混乱。

3) 搜索速度快：可为每类文件建立一个子目录，对于多级目录，每次只查找子目录，因此速度比一级和二级目录时快。

用户在访问文件时，需要进行目录检索，用户给出文件名，系统按名寻找目录项。

有两种根据路径检索的方法：全路径名和相对路径。

全路径名：需要从根目录开始，列出由根到用户的所有子目录，又称为“绝对路径名”。

缺点：每次都从根目录开始检索，很不方便，因为通常目录放在外存故影响访问速度，尤其当目录层次较多时，检索需要耗费很长时间。

相对路径：用于检索的路径只是从当前目录开始到所要访问文件的一段路径，即以当前目录作为路径

的相对参照点。这样检索速度比较快。

文件寻址：根据FCB中文件物理地址等信息，求出文件的任意记录或字符在存取介质上的地址，称为文件寻址。

文件目录改进

为加快目录检索可采用目录项分解法：把FCB分成两部分：

符号目录项（次部）

文件名，文件号

基本目录项（主部）

除文件名外的所有项目

分解之后的查找过程：

查找一个目录项就分成两步：首先访问符号目录文件，根据文件名查找相应的文件内部号；然后访问基本目录文件，根据文件内部号，可直接计算出相应基本目录项所在基本目录文件中的相对位置和物理位置，并将它直接读入内存。

目录操作：

以UNIX为例：

1.Creat

2.Delete

3.OpenDir

4.Closedir

7. 存储空间的分配和回收

1) 位图法

用一串二进制位反应磁盘空间中分配使用情况，每个物理块对应一位，分配物理块为1，否则为0

申请物理块时，可以在位示图中查找为0的位，返回对应物理块号

归还时，将对应位转置0

描述能力强，适合各种物理结构

计算公式：

已知字号 i ，位号 j

块号 $= i * \text{字长} + j$

已知块号：

字号 $= \lfloor \text{块号} / \text{字长} \rfloor$

位号 $= \text{块号} \% \text{字长}$

2) 空闲块表

将所有空闲块记录在一个表中，即空闲块表特别适合文件物理结构为顺序结构的文件系统

3) 空闲块链表

把所有空闲块链成一个链

4) 成组链接法

文件系统的实现

内存中多需的表目

1) 系统打开文件表（整个系统一张）

放在内存。用于保存已打开文件的FCB

此外，文件号，共享计数，修改标志

2) 用户打开文件表（每个进程一个）

文件描述符，打开方式，读写指针，系统打开文件进程的PCB中，记录了用户打开文件表的位置

3) 用户打开文件表与系统打开文件表之间的关系

用户打开文件表指向系统打开文件表

如果多个进程共享同一个文件，则多个用户打开文件表目对用系统打开文件表的同一入口

8. 记录

记录是一组相关数据项的集合，用于描述数据对象某方面的属性。它是文件中数据处理的基本单位，是组成文件的基本元素。

在一个由大量记录组成的文件中，为了能惟一地标识一条记录，可以在记录的各个数据项中，确定出一个或几个数据项，把它（或它们）成为关键字（key）。如在描述学生的数据项中，学号可以作为关键字。

记录的成组和分解

每个用户的文件是由用户按照自己的需要组织的，逻辑记录的大小是由文件的性质决定的。而存储介质上的分块是根据存储介质的特性划分的。所以，逻辑记录的大小往往与存储块的大小不一致。为了节省存储空间，提高主存的利用率，系统引入了记录的成组和分解。

记录成组

记录成组是指把若干条逻辑记录合并成一组存入一个物理块的过程。

记录分解是指从一条物理记录中把逻辑记录分离出来的过程。

记录成组存放后，当用户需要某一条记录时，必须把含有该条记录的整块信息独处，再从这一组逻辑记录中找出用户所需要的记录进行处理。记录分解也需要使用主存缓冲区。

9. 文件的操作

提供设置和修改对用户文件存取权限

提供建立、修改、改变、删除目录的服务

提供文件共享，设置访问路径的服务

提供创建、打开、读、写、关闭、撤销文件等服务

文件系统维护

文件系统的转储和恢复

10. 文件的保护

文件的共享

1) 定义

一个文件被多个用户或程序使用

共享形式：

被多个用户使用，由存取权限控制

被多个程序使用，但各用自己的读写指针

被多个程序使用，但共享读写指针

2) 目的

节省时间和存储空间，减少了用户工作量；

进程间通过文件交换信息

文件安全是指避免合法用户有意或无意的错误操作破坏文件，或非法用户访问文件。

影响文件安全性的主要因素有：

- 1) 人为因素
- 2) 系统因素
- 3) 自然因素

为了确保文件系统的安全性，可以采取以下措施：

- 1) 建立副本，通过“后备系统”来防止由自然因素所造成的不安全性
- 2) 定时转储，通过系统容错技术来防止系统部分的故障所造成的文件不安全性
- 3) 规定文件的存取权限，通过存取控制机制来防止由人为因素引起的文件不安全性

特点：建立副本的方法简单，但是系统开销大，且文件更新时，所有副本都必须更新。这种方法使用容量较小且极为重要的文件。定时转储的方法简单，但是较为费时，在转储过程中一般要停止文件系统的使用。这种方法适用于容量较大的文件。

文件的使用权限可以设为：只能读、可读可写、只能执行、不能删除等。对多用户共享的文件采用属性、目录结构，反得到某级目录权限的用户就可以得到该目录所属的全部目录和文件。

11. 文件的存取权限

- 1) 存取控制矩阵：系统以一个二维矩阵来实施文件的存取控制。其方法在概念上比较简单。

2) 二级存取控制：两个存取级别。第一级，把用户按照某种关系分为若干组，进行对访问者的识别，第二级，进行对操作权限的识别。

UNIX中的文件存取权限：

第一级：对访问者的识别

对用户分类：

文件主 (owner)

文件主的同组用户 (group)

其它用户 (other)

第二级：对操作权限的识别

对操作分类：

读操作 (r)

写操作 (w)

执行操作 (x)

不能执行任何操作 (-)

文件属性 `rwXr--Xr--X` 用二进制表示是 `111101101`，在UNIX中使用八进制，因此权限是 `755`

文件保密

12.

文件保密是指文件本身不得被未经授权的用户访问，防止他人窃取文件。实现文件保密采用的方法有：

- 1) 隐藏文件目录
- 2) 设置口令
- 3) 使用密码

13. 文件系统的优化

文件系统的物理基础是磁盘设备

常见的技术措施：

- 1) 块高速缓存
- 2) 合理分配磁盘空间
- 3) 磁盘的驱动调度
- 4) 信息的优化分布
- 5) RAID 技术

1> 块高速缓存

系统在内存中保存一些块。逻辑上它们属于磁盘

2>合理分配磁盘空间

3>磁盘的驱动调度

公平：一个I/O请求在有限时间内满足

高效：减少设备机械运动所带来的时间浪费

磁盘调度考虑的问题：

一次访盘时间=寻道时间+旋转延迟时间+传输时间

磁盘驱动调度由“移臂调度”和“旋转调度”组成

移臂调度目的是尽可能减少寻找磁道时间

旋转调度目的是尽可能减少寻找扇区时间

磁盘调度算法：

1) 先来先服务 (FCFS)：

按访问请求到达的先后次序服务

优点：简单，公平

缺点：效率不高，相邻两次请求可能会造成最内到最外的柱面寻道，使磁头反复移动，增加了服务时间，对机械也不利

) 最短寻道时间优先 ()：

优先选择距当前磁头最近的访问请求进行服务，主要考虑寻道优先

特点：该调度算法，虽然减少了磁臂的移动距离，但是，会经常改变磁臂的移动方向，花费时间多又影响机械部件，还会导致“饥饿”现象，即较远距离的鼓励的访问者可能很长时间不能获得访问磁盘的机会。

3) 扫描算法 (SCAN 电梯算法)：

克服了最短寻道有限的缺点，考虑了距离和方向

4) 循环扫描 (C-SCAN) 调度算法：

在扫描算法的基础上改进的。balabala...

4> 旋转调度算法

旋转调度：根据延迟时间来决定执行次序的调度

5> 信息的优化分布

记录在磁道上的排列方式也会影响磁盘的输入输出时间

6> RAID 技术

磁盘是机械设备，一方面速度慢，一方面也容易出现故障。

RAID 是一种把多块独立的硬盘（物理硬盘）按不同的方式组合起来形成一个硬盘组（逻辑硬盘

），从而提供比单个硬盘更高的存储性能和提供数据备份技术。

Windows的FAT文件系统

FAT文件系统是各种Windows操作系统都支持的一个简单的文件系统

FAT文件系统簇为单位进行分配

FAT有三个版本：FAT-12，FAT-16，FAT-32

FAT卷的结构包括：引导扇区、文件分配表、根目录

UNIX文件系统

UNIX目录中为每个文件保留了一项，每个目录项包含了两个域，文件名和节点号。

UNIX普通文件的物理结构是三级索引结构

第七章 I/O设备管理

考试内容

1.设备与设备分类

2.I/O硬件组成

3.I/O软件的特点及结构

4. 典型技术：通道技术，缓冲技术，SPOOLing

5. I/O 性能问题及解决方案

1. 设备与设备的分类

按设备的使用特性分类

1) 存储设备。存储设备是指用来存放信息的设备，如磁盘、磁带等。

2) I/O（输入输出）设备

按设备共享属性分类

1) 独占设备。独占设备是指在一段时间内只允许一个进程访问的设备。系统一旦把这种设备分配给一个进程后，便由该进程独占，直到用完释放，其他进程才能使用。多数低速设备都属于此类设备，如打印机。

2) 共享设备。共享设备是指在一段时间内允许多个进程访问的设备，如磁盘。

3) 虚拟设备。虚拟设备是指通过虚拟技术将一台独占设备变换为若干台逻辑设备，供若干个进程同时使用的设备，如虚拟打印机。

按设备的信息组织方式来分类

1) 块设备。块设备是指处理信息的基本单位是字符块。一般块的大小为512B~4KB，如磁盘、磁带等。

) 字符设备。字符设备是指处理信息的基本单位是字符，如键盘、显示器、打印机等。

2.I/O 硬件组成

2.1 计算机的I/O系统结构

从硬件的角度来看，一个典型的计算机结构，中央部分是CPU和主存，通过总线与第二层的接口部分连接，第三层各种外围设备控制器，最外层是外围设备。

2.2I/O设备数据传输控制方式

程序直接控制方式

程序直接控制方式也称为“忙__等待”方式，即在一个设备的操作没有完成时，控制程序一直检测设备的状态，直到该操作完成，才能进行下一个操作。

步骤为：

- 1) 当用户需要输入数据时，由处理器向设备控制器发出一条输入输出指令，启动设备进行输入。
。
- 2) 当用户进程需要像设备输出数据时，也必须发出启动命令启动设备输出，并等待输出操作完成。

特点：工作过程简单，CPU的利用率低。直接控制方式适用于早期的无中断的计算机系统。

程序中断控制方式

中断控制是指计算机在执行期间，系统内发生任何给寻常的或非预期的急需处理事件，使得

暂时中止当前执行的程序而转去执行相应的时间处理程序，待处理完毕后又返回原来被中止处继续执行或调度新的进程。

特点：中断控制方式比程序直接控制方式提高了CPU的利用率。每输入一个数据都会发生中断，传输一组数据需要多次中断，浪费了CPU的时间。中断控制方式应用与现代计算机系统中。

直接存储器存取控制方式（DMA）

直接存储器存取方式是指对输入输出设备的控制由DMA控制器完成，在DMA控制器的作用下，设备和主存之间可以成批地进行数据交换，而不用CPU的干涉。

特点：数据的传送方向、存放数据的主存始址及传送数据的长度等都由CPU控制，具体的数据传送由DMA控制器负责，每台设备需要配一个DMA控制器，这样输入输出传输速度快，CPU负担少。直接存储器存取控制方式适用于块设备的数据传输。

通信控制方式

I/O通信是一种特殊的处理机

具有执行I/O指令的能力

具有执行通道（I/O）程序控制I/O操作

指令类型单一，主要限于I/O操作有关的指令

没有自己的内存，通道程序放在主机内存中

特点：通道所需要的CPU干预更少，并可以实现CPU、通道和输入输出设备三者之间的并行操作，从而更有效地提高整个系统资源的利用率。通道控制方式适用于现代计算机系统的大量数据交换。

2.3 输入输出通道的分类

输入输出通道是用于控制外围设备的。根据信息交换方式的不同，把通道分成三种类型：

1) 选择通道。可以连接多台高速设备，由于它只含有一个分配型子通道，在一段时间内只能执行一个通道程序，控制一台设备进行数据传送，致使当某台设备占用了该通道后，便一直由它独占，直至该设备传送完毕后释放该通道。

数据选择通道虽然有很高的传输速率，但它每次只允许一个设备传输设备，这种通道利用率很低。

2) 字节多路通道。通常都含有许多非分配型子通道，其数量可以从几十到数百个，每一个子通道连接一台输入输出设备，这些子通道按时间片轮转方式共享主通道。

字节多路通道连接低速或中速设备时，不会丢失信息。

3) 数组多路通道。将数据选择通道传输速率高和字节多路通道能使各子通道（设备）分时并行操作的优点相结合，而形成的一种新通道。它含有多个非分配子通道，因而这种通道既具有很高的数据传输速率，又能获得令人满意的通道利用率。

该通道被广泛地用于连接多台高、中速的外围设备，其数据传送是按数组方式进行的。

3.1 I/O软件的特点及结构

设计I/O最关键的目标是设备独立性。

I/O设备管理软件的结构，基本思想是分层构造，使底层与硬件相关，把硬件与较高层次的软件隔离。

I/O软件分为四层：

中断处理程序

设备驱动程序

与设备无关的系统软件

用户空间的I/O软件

设备驱动程序的功能

balabala...

设计I/O最关键的目标是设备独立性。

I/O设备管理软件的结构，基本思想是分层构造，使底层与硬件相关，把硬件与较高层次的软件隔离。

I/O软件分为四层：

中断处理程序

设备驱动程序

与设备无关的系统软件

用户空间的I/O软件

设备驱动程序的特点：

- 1) 驱动程序主要是在请求输入输出的进程与设备控制器之间的一个通信程序。
- 2) 驱动程序与输入输出设备的特性密切相关
- 3) 驱动程序与输入输出控制方式紧密相关
- 4) 驱动程序与硬件紧密相关，其部分被固化在ROM中

与设备无关的系统软件

除了一些I/O软件与设备相关外，大部分软件是与设备无关的。一般情况，所有设备都需要的I/O功能可以在于设备独立的软件中实现。

设备无关软件层实现的一些功能：

统一命名

设备保护

提供与设备无关的逻辑块

缓冲

存储设备的块分配

独占设备的分配与释放

出错处理

用户空间的I/O软件

大部分I/O软件都包含在操作系统中，但是用户程序可以和库函数连接在一起

标准的I/O库包含了许多设计I/O的过程，他们都是作为用户程序的一部分运行的

4. 缓冲技术

为了提高输入输出设备的速度和利用率，在输入输出设备与处理器交换数据时引入了缓冲技术。缓冲技术是输入输出设备在与主存交换数据时使用缓冲区的技术。缓冲管理的主要功能是组织好缓冲区，并提供获得和释放缓冲区的手段。

4.1 缓冲的引入

- 1) 缓和CPU与输入输出设备间速度不匹配的矛盾
- 2) 减少对CPU的中断频率，放款对中断响应时间的限制
- 3) 提高CPU与输入输出设备间的并行性

4.2 单缓冲

单缓冲是指在设备和处理器之间设置一个缓冲区，用于数据的传输

4.3 双缓冲

双缓冲是指在设备和处理器之间设置两个缓冲区

4.4 多缓冲

在设备和处理器之间设置多个缓冲区

4.5 缓冲池

当系统较大时，可以利用多个进程共享的缓冲池来提高缓冲区的利用率

5.I/O性能问题及解决方案小结

I/O性能常常成为系统性能的瓶颈，因此提高I/O性能十分重要

- 1) 通过缓冲技术，减少或缓解不同设备之间传输速度的差距
- 2) 通过异步I/O技术，使CPU计算不必等待I/O操作结果
- 3) 通过应用DMA和通信部件，使CPU摆脱I/O操作，与这些部件并行执行
- 4) 通过虚拟设备技术，提高独占设备的利用率

第八章 死锁

活锁概念

数据资源释放时间不确定，导致某些事务长时间等待，得不到封锁的机会

饥饿的概念

系统没有发生死锁，但某些进程可能会长时间等待，当等待时间给进程推进和响应带来明显影响时，称进程饥饿

死锁的原因

- 1) 竞争资源
- 2) 进程推进顺序不合理

产生死锁的必要条件

死锁并不一定都会出现，如果一旦产生死锁，一定会满足下述4个必要条件

- 1) 互斥条件
- 2) 不剥夺条件
- 3) 请求和保护条件
- 4) 循环等待条件

解决死锁的方法分为四种

- 1) 预防死锁。

预防死锁是在进行资源分配之前，通过设置某些资源分配的限制条件，来破坏产生死锁的四个必要条件的一个或几个。预防死锁是一种较简单的方法，容易使用，但是，由于施加了限制条件，会导致系统资源利用率和吞吐量的下降。

2) 避免死锁。

避免死锁是在资源分配前不设置限制条件，在进行资源分配的过程中，用某种方法对每次资源分配进行管理，以避免某次分配使系统进入不安全状态，以致产生死锁。这种方法限制较小，可以获得较好的系统资源利用率和吞吐量。目前比较完善的系统中，通常采用此种方法。

3) 检测死锁

允许系统产生死锁，但在系统中设置检测机构，一旦发生死锁，采取措施解除死锁

4) 解除死锁

通过撤销或挂起一些死锁中的进程，回收相应的资源，进行资源的再次分配，从而将进程死锁状态解除。这种方法没有限制，可以获得较好的系统资源利用率

死锁的预防

通过对资源分配的原则进行限制，而使产生死锁的四个必要条件中的₁个或多个条件不能成立，来预防产生死锁。

破坏“互斥条件”条件

进制进程独占资源，如果资源不被一个进程独占，你们死锁肯定不会发生

此条件实现比较困难，限制比较多

破坏“不剥夺”条件

一个已经占有某些资源的进程，当它再提出新的资源需求而不能立即得到满足时，必须释放它已经占有的所有资源，待以后需要时再重新申请。这意味着进程已经拥有的资源，在运行过程中可能会暂时被迫释放，即被系统剥夺，从而摒弃了“不剥夺条件”。

这种预防死锁的方法，实现起来比较复杂，并付出较大的代价，会使前段时间的工作实效等。此外这种方法还会因为反复地申请和释放资源，延长进程的周转时间，增加系统开销，降低系统吞吐量。

破坏“请求和保持”条件

系统要求进程必须一次性地申请其在整个运行期间所需要的全部资源。若系统有足够的资源，便一次性将其所需要的所有资源分配给该进程，这样一来，该进程在整个运行过程中，便不会再提出资源请求，从而摒弃了“请求”条件。而在分配时，只要有一个资源要求不能满足，系统将不分配给该进程任何资源，此时进程没有占有任何资源，因而也摒弃了“保持”条件，所以，可以预防死锁的产生。

这种预防死锁的方法，简答方便、易于实现，但是，因为进程将一次性获得所有资源，并且独占使用，其中可能有些资源在该进程运行期间很少使用，造成资源严重浪费；同时有些进程因不能一次性获得所需要的资源，导致长时间不能投入运行。

破坏“循环等待”条件

在这种方法中规定，系统将所有的资源按照类型进行线性排序，赋予不同的资源序号。并且所有进程对资源的请求和分配必须严格的按照资源序号有小到大地进行，即只有先申请和分配到序号小的资源，才能申请和分配序号大的资源。这样在最后形成的资源分配图中，将不可能再出现环路，从而摒弃了“环路等待”条件。

这种预防死锁的方法与前两种相比，其资源利用率和系统吞吐量，都有明显的改善。但是，这种方法

涉及对各类资源的排序编号，考虑到实际的使用，其排序的合理性将受到很大的挑战。

破坏“循环等待”条件

在这种方法中规定，系统将所有的资源按照类型进行线性排序，赋予不同的资源序号。并且所有进程对资源的请求和分配必须严格的按照资源序号有小到大地进行，即只有先申请和分配到序号小的资源，才能再申请和分配序号大的资源。这样在最后形成的资源分配图中，将不可能再出现环路，从而摒弃了“环路等待”条件。

死锁的避免

在死锁的避免中，所施加的限制较弱，可以获得较好的系统性能。该方法的系统状态分为安全状态和不安全状态，只要能使系统时钟处于安全状态，便可以避免死锁发生。

安全状态和不安全状态

安全状态是指系统能够按照某种顺序为每个进程分配需要资源，直到最大需求，使每一个进程都可以顺利完成。反之，如果系统不存在这样的安全序列，则称系统处于不安全状态。

当系统进入不安全状态后，很有可能进入死锁状态；反之，只要系统处于安全状态，便可避免进入死锁状态。因此，避免死锁的实质就是如何使系统不进入不安全状态。

利用银行家算法避免死锁

最具代表性的避免死锁的算法是Dijkstra和Habermann的银行家算法。这是由于该算法能用于银行系统现金贷款的发放而得名。

死锁的检测与接触

当系统为进程分配资源时，如果没有采取任何限制措施，系统必须提供死锁的检测与解除机制

死锁的检测

在进行死锁的检测时，系统必须能保存有关资源的请求和分配的信息，并提供一种算法，以便利用这些信息来检测系统是否进入死锁状态。

死锁的解除

当检测到系统发生死锁时，就必须立即把死锁状态解除，常用的方法是：

1) 剥夺资源法。从其他进程剥夺足够数量的资源给死锁进程，使其得到足够的资源，然后继续执行，以解除死锁状态。

2) 撤销进程法。系统采用强制手段将死锁进程撤销。最简单的方法是将全部死锁进程一次性撤销，但是，代价较大；另一种方法是按照一定的算法，从死锁进程中一个一个地选择进行撤销，并同时剥夺这些进程的资源，直到死锁状态解除为止。

死锁定理

1) 如果资源分配图中没有环路，则系统没有死锁

2) 如果资源分配图中出现了环路，则系统可能出现死锁

如果环路中每个资源的数目只有一个，则意味着死锁的存在，此时，环路是死锁的中分必要条件。

如果处于环路的每个资源数目不为1，则是死锁产生的必要而不是充分条件

[前一篇](#) [黑幕！黑色的明天等待着我们](#)

[后一篇](#) [用你的魔法和勇气，与小伙伴们一起勇闯《西方世界》](#)



smallyu

Hello World

??

??

????

2017080 (6)

2017070 (2)

2017060 (3)

2017050 (2)

2017040 (2)

2017030 (3)

2017020 (3)

2017010 (4)

20160120 (1)

黑幕！黑色的明天等待着我们

□ smallyu □ 2017-07-25 □ 黑幕

考试前一天晚上，所有人都知道了考试题目

大学，期末考试，某门课，考试前一天晚上，很多（几乎所有）人，都知道了第二天考试要考的，80%（以上）的试题，

一字不差的那种。

试题的流传过程是这样的。

背景，学生_A和学生_B都是宿舍_A的。

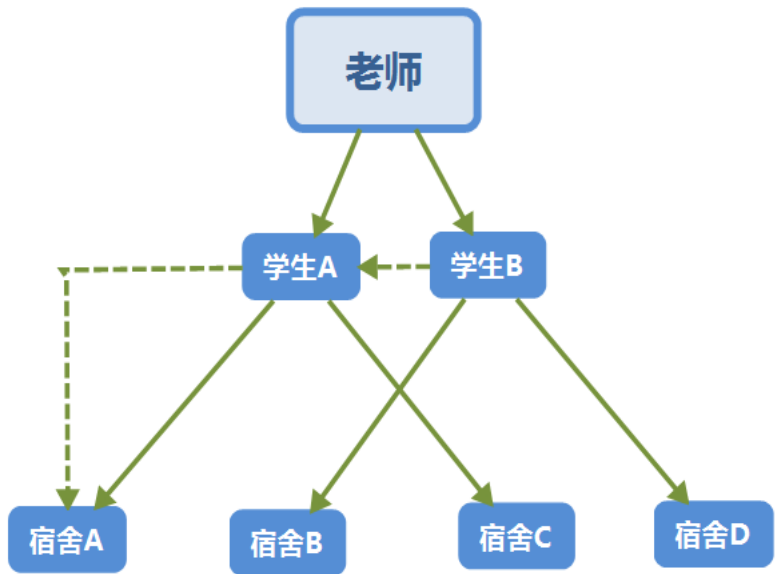
背景，学生_A和学生_B对试题的传播几乎没有过交流。

老师同时把题给了学生_A和学生_B，然后人情社会，利益相关，

学生_A把题告诉了宿舍_A（也就是自己宿舍）和宿舍_C（平时玩的比较好），

学生_B把题告诉了宿舍_B（可能玩的比较好）和宿舍_D（其实这里稍有误）。

20160111 (2)
20160100 (3)
20160090 (1)
20160080 (6)
20160070 (1)
20160060 (4)
20160050 (4)



图中虚线表示学生_A和学生_B都是宿舍_A的，实线表示试题的流传过程。

泄题本身很难判断对错。

学生_A的成绩差，老师一片好心。

学生_B是班长，（脑补原因）。

噢对，学生_A是老师的助理。（所以老师才会关心他的成绩差，而不是别人。成绩差也是能当助理的）

对于成绩差的学生_A，我们丝毫不介意甚至迫切希望老师泄题给他，因为我们也不希望他挂科。（好吧一个

宿舍的)

然后从结果上看，老师泄题，我们几乎所有人（起码我）得知了第二天要考的题目。

我们平时还盼着老师给划个重点，现在真题都有了，难不成还站在道德的制高点说泄题的行为是错误的？

当然不可以，但直接出来那么多真题确实有点怀疑人生。

原来人情，连这种违反规定的事情，都可以解决。

但是同样从人情的角度看，老师背着违反规定的风险泄题给你，想让你不挂科，给你个机会，这是老师在做好。

对我们有利，所以我们不能（也不会）说它是错的。

不过如果我们（我）不是受益者，可能我就会改变说法。

泄题的过程才让人感慨。（泄不泄题和我没太大关系，对成绩影响不大）

全班几十个学生，老师只告诉了两个人，其中没有你，你伤不伤？

学生_A只告诉了宿舍_A和宿舍_C，那么宿舍_B和宿舍_D的，你伤不伤？

同理，学生_B只告诉了宿舍_B和宿舍_D的，尤其，连它自己的宿舍_A都没告诉，你伤不伤？

问题是，资源在人家手里，想给谁给谁。

尴尬的是，最后，几乎所有人都知道了。

宿舍_B的人跑到宿舍_A，咦，你这题哪儿来的？

宿舍_A黑人问号脸，咦，怎么，你知道？

哈哈哈哈哈哈

谁都能理解，谁都不想理解。

面对资源，部分人性可以显露。

所以石油才会引发大国之间的战争？《文明₆》里面加成资源算是很值钱的东西，像牛羊马，鹿，鱼，或者战略资源，石油，铀，硝石。

问：好人是从什么时候开始变坏的？

答：觉得不公平的时候。

可是 (公平!=公正)==true

社会我社会

我等身与名俱灭，不废长江万古流啊

前一篇: [2017年上半年年度总结，2017年下半年年度展望](#)

后一篇: [NCRE4·网络工程师《操作系统原理》概要](#)

Copyright Hower'Blog Rights Reserved. . Powered By [Z-BlogPHP](#), theme by [????](#)



smallyu

Hello World

??

??

????

2017080 (6)

2017070 (2)

2017060 (3)

2017050 (2)

2017040 (2)

2017030 (3)

2017020 (3)

2017010 (4)

20160120 (1)

2017年上半年年度总结，2017年下半年年度展望

□ smallyu □ 2017-07-13 □ 2017

没啥好总结的，没啥好展望的。

(不忍心总结过去的半年，没有勇气开始新的半年)

上一篇: [SPL 中的常用迭代器和函数](#)

下一篇: [黑幕！黑色的明天等待着我们](#)

Copyright Hower'Blog Rights Reserved. . Powered By [Z-BlogPHP](#). theme by [????](#)

20161111 (2)

20161101 (3)

20161091 (1)

20161081 (6)

20161071 (1)

20161061 (4)

20161051 (4)



smallyu

Hello World

??

??

????

2017080 (6)

2017070 (2)

2017060 (3)

2017050 (2)

2017040 (2)

2017030 (3)

2017020 (3)

2017010 (4)

20160120 (1)

SPL中的常用迭代器和函数

□ smallyu □ 2017-06-16 □ SPL

有一天，我去世了，恨我的人，翩翩起舞，爱我的人，眼泪如露。

第二天，我的尸体头朝西埋在地下深处，恨我的人，看着我的坟墓，一脸笑意，爱我的人，不敢回头看那么一眼。

一年后，我的尸骨已经腐烂，我的坟堆雨打风吹，恨我的人，偶尔在茶余饭后提到我时，仍然一脸恼怒，爱我的人，夜深人静时，无声的眼泪向谁哭诉。

十年后，我没有了尸体，只剩一些残骨。恨我的人，只隐约记得我的名字，已经忘了我的面目，爱我至深的人啊，想起我时，有短暂的沉默，生活把一切都渐渐模糊。

几十年后，我的坟堆雨打风吹去，唯有一片荒芜，恨我的人，把我遗忘，爱我至深的人，也跟着进入了坟墓。对这个世界来说，我彻底变成了虚无。我奋斗一生，带不走一草一木。我一生执着，带不走一分虚荣爱慕。

今生，无论贵贱贫富，总有一天都要走到这最后一步。

到了后世，霍然回首，我的这一生，形同虚度！我想痛哭，却发不出一点声音，我想忏悔，却已迟暮！用心去生活，别以他人的眼光为尺度。爱恨情仇其实都只是对自身活着的，每一天幸福就好。

珍惜内心最想要珍惜的，三千繁华，弹指刹那，百年之后，不过一捧黄沙。

20160110 (2)

20160110 (3)

2016090 (1)

2016080 (6)

2016070 (1)

2016060 (4)

2016050 (4)

SPL的常用迭代器,通过某种统一的方式遍历链表或者数组中的元素的过程叫做迭代遍历,而这种统一的遍历工具我们叫做迭代器。

ArrayIterator

```
<?php
$fruits = [
    "apple" => 'apple value', // position = 0
    "orange" => 'orange value', // position = 1
    "grape" => 'grape value',
    "plum" => 'plum value'
];
// 一般方式
print_r($fruits);
foreach($fruits as $key => $value){
    echo $key.$value;
}

// 使用ArrayIterator遍历数组
$obj = new ArrayObject($fruits);
$it = $obj->getIterator();
// foreach方式
foreach($it as $key => $value){
    echo $key.$value;
}
/* foreach方式和普通的foreach方式没有区别,因为我们在使用foreach的过程中程序默认使用了迭代器,也就是底层实现的方式,先实例化数组为一个对象,然后调用迭代器的方法,进行遍历。 */
// while方式
$it->rewind(); // 调用current之前一定要rewind
while($it->valid()){
```

```

        echo $it->key().$it->current();
        $it->next();
    }
    /* while遍历的时候一定要记得rewind重置指针的位置。和普通foreach实现遍历相比，迭代器方式可以实现跳过某些元素再进行遍历，比如从位置1开始，而不是位置0。 */
    // 跳过某些元素进行打印
    $it->rewind();
    if($it->valid()){
        $it->seek(1); // 从position = 1开始
        while($it->valid()){
            echo $it->key().$it->current();
            $it->next();
        }
    }
    /* ksort方式使得遍历结果按照key的字典序排序，如果想要value按照字典序排序，则使用asort。 */
    $it->ksort();
    foreach($it as $key => $value){
        echo $key.$value;
    }

```

AppendIterator

```

<?php
$array_a = new ArrayIterator(['a', 'b', 'c']);
$array_b = new ArrayIterator(['d', 'e', 'f']);
$it->append($array_a);
$it->append($array_b);
// 通过append方法把迭代器对象添加到AppendIterator对象中
foreach($it as $key => $value){
    echo $value;
}
// 只用一个foreach就可以实现多次遍历

```

MultipleIterator 用于把多个 Iterator 里面的数据组合成为一个整体来访问

```
<?php
$idIter = new ArrayIterator(['01', '02', '03']);
$nameIter = new ArrayIterator(['张三', '李四', '王五']);
$ageIter = new ArrayIterator(['22', '23', '25']);
$mit = new MultipleIterator(MultipleIterator::MIT_KEYS_ASSOC);
$mit->attachIterator($idIter, "ID");
$mit->attachIterator($nameIter, "NAME");
$mit->attachIterator($ageIter, "AGE");
foreach ($mit as $value) {
    print_r($value);
}
```

FileSystemIterator 能遍历文件系统

```
<?php
$it = new FileSystemIterator('.');
foreach ($it as $finfo) {
    printf("%s\t%s\t%8s\t%s\n", date("Y-m-d H:i:s", $finfo->getMTime()),
        $finfo->isDir() ? "<DIR>" : "",
        number_format($finfo->getSize()),
        $finfo->getFileName()
    );
}
```

SPL基础接口

SPL 的基础接口里面定义了最常用的接口

-Countable

继承了接口的类可以直接调用 `count()` 得到元素个数

OuterIterator

如果想对迭代器进行一定的处理之后再返回，可以用这个接口

RecursiveIterator

可以对多层结构的迭代器进行迭代，比如遍历一棵树

SeekableIterator

可以通过 `seek` 方法定位到集合里面的某个特定元素

Countable，在代码里面经常可以直接用 `count($obj)` 方法获得对象里面的元素的个数，对于自定义的类可以 implements

```
<?php
$array = [
    [1, 2],
    [3, 4],
    [5, 6]
];
echo count($array);
echo count($array[1]);

<?php
class CountMe implements Countable {

    protected $_myCount = 3;
    public function count(){
```

```

        return $this->_myCount;
    }
}
$obj = new CountMe();
echo count($obj);

```

OuterIterator

如果想对迭代器进行一定的处理之后再返回，可以用这个接口

IteratorIterator 类是 OuterIterator 的实现，扩展的时候可以直接继承 IteratorIterator

```

<?php
$array = ['1', '2', '3', '4'];
$outterObj = new OuterImpl(new ArrayIterator($array));
foreach ($outterObj as $key => $value) {
    echo $key.$value;
}

class OuterImpl extends Iterator {
    public function current() {
        return parent::current()."_tail";
    }
    public function key() {
        return "Pre_".parent::key();
    }
}

```

RecursiveIterator

可以对多层结构的迭代器进行迭代，比如遍历一棵树

所有具有层次结构特点的数据都可以用这个接口遍历，如文件夹

关键方法

-hasChildren 方法用于判断当前节点是否存在子节点

-getChildren 方法用于得到当前节点子节点的迭代器

SPL 实现该接口的类

-RecursiveArrayIterator, RecursiveCachingIterator 等以 Recursive 开头的类都能够进行多层次结构化的遍历

SeekableIterator

-可以通过 seek 方法定位到集合里面的某个元素

-seek 方法的参数是元素的位置，以 0 开始计算

SPL 函数的使用

```
<?php
spl_autoload_extensions('.class.php', '.php');
// 设置autoload寻找php定义的类文件的扩展名，多个扩展名用逗号分隔，前面的扩展名优先被匹配
set_include_path(get_include_path().PATH_SEPARATOR."libs/");
// 设置sutooload寻找php定义的类文件的目录，多个目录用PATH_SEPARATOR进行分隔
spl_sutoload_register();
// 提示php使用Autoload机制查找类定义
new ClassName();
```

改进

```
<?php
function __autoload($class_name){ // 定义__autoload函数，可以不调用spl_autoload_register函数的情况下完成类的装载
    require_once("libs/".$class_name.".php"); // 装载类
}
new ClassName();
```

自定义装载

```
<?php
function classLoader($class_name){ // 定义一个替换__autoload函数的类文件装载函数
    require_once("libs/".$class_name.".php"); // 装载类
}
spl_autoload_register('classLoader'); //传入定义好的装载类的函数的名称替换__autoload函数
new ClassName();
```

The End.

[前一篇](#) [今天二十岁](#)

[后一篇](#) [2017年上半年年度总结，2017年下半年年度展望](#)



smallyu

Hello World

??

??

????

2017080 (6)

2017070 (2)

2017060 (3)

2017050 (2)

2017040 (2)

2017030 (3)

2017020 (3)

2017010 (4)

20160120 (1)

今天二十岁

□ smallyu □ 2017-06-13 □ smallyu

莫名想哭，偶尔想笑

前一篇: [比起努力，我实在太喜欢睡觉了](#)

后一篇: [SPL 中的常用迭代器和函数](#)

Copyright Hower'Blog Rights Reserved. . Powered By [Z-BlogPHP](#). theme by [????](#)

2016111 (2)

2016101 (3)

201691 (1)

201681 (6)

201671 (1)

201661 (4)

201651 (4)



smallyu

Hello World

??

??

????

2017080 (6)

2017070 (2)

2017060 (3)

2017050 (2)

2017040 (2)

2017030 (3)

2017020 (3)

2017010 (4)

20160120 (1)

比起努力，我实在太喜欢睡觉了

□ smallyu □ 2017-06-04

尤其是面对电脑不知所措的时候。

B站排行榜的视频刷完了。老友记越看越没劲了。知乎怎么看都看不完了。头条推送的内容没什么新鲜的了。优酷的UI越来越反人类了。虎绿林和_{zz}是早该放弃的社区了。

慕课网上的学习时长增长进度远远低于期望值，去年的日志（[一万小时定律实践计划](#)）Duang~任务进度到1%啦！post时间到现在将近一年减两个月，也就是十个月，学习时长的统计时间还不到1W小时的5%。然后PHP分类下的视频几乎全是对号。

IM工具，QQ空间，朋友圈，好友动态的更新频率可能比这个博客还要慢。

所以总的来说，就是无聊。无聊的时候，就睡一会儿。睡不着的时候，就再无聊一会儿。

听说无聊是对欲望的欲望。没有欲望，才对欲望有欲望。人怎么会没有欲望呢，也就是另一种情况，对欲望无能为力的时候，对欲望的欲望。

这个意思感觉就像，要理解递归，首先要理解递归。

20160111 (2)

20160110 (3)

20160109 (1)

20160108 (6)

20160107 (1)

20160106 (4)

20160105 (4)

最近没什么事情，但又想post点内容，最后可选的标题（和内容）有三个：

1. 利用WebView开发一个简单的安卓APP
2. 五月乱象：总有几个莫名奇妙的总结
3. 超星泛雅《创业管理实战》、《大学语文》答案分享

然后都放弃了。

第一个先留着，可能有心情再折腾一下安卓开发的时候再总结。之前没怎么接触过APP开发，有一个比赛的时候试着封装了一个用WebView加载指定页面的很简单的应用。也是踩了一点可能不少新手都会踩的坑。

第二个是一些看视频的笔记。单个拿出来太少，一起放出来有点乱。有这么几部分：

1. Java IO流 -> 字节流（概念）
2. Java工厂模式（简单案例）
3. PHP SPL常用迭代器（部分）
4. 测试用例（概念）

设计模式和SPL在之前的日志中提到过，只是很不全。Java IO流的概念和测试用例的概念也就是一些文字。就贴在后面。（刚发现SPL的内容比较多，也先留着）

第三个是最近刷视频课用到的答案。网上重复内容太多，搜索引擎应该不会收录。即使收录了权重也应该低

的吓人。而且不能复制粘贴，内容太多了，影响页面加载，只能附件`doc`。不过那样更没戏，搜索引擎认都不认。

Java IO 流（输入流、输出流）

1. 字节流

1) InputStream、OutputStream

InputStream 抽象了应用程序读取数据的方式

OutputStream 抽象了应用程序写出数据的方式

2) EOF = End 读到 -1 就读到结尾

3) 输入流基本方法

`int b = in.read();` 读取一个字节无符号填充到 `int` 低八位，-1 是 EOF

`in.read(byte[] buf)` 读取数据填充到字节数组 `buf`

`in.read(byte[] buf, int start, int size)` 读取数据到字节数组 `buf`，从 `buf` 的 `start` 位置开始存放 `size` 长度的数据

4) 输出流基本方法

`out.write(int b)` 写出一个 `byte` 到流，`b` 的低 8 位

```
out.write(byte[] buf) 将 buf 字节数组都写入到流
```

```
out.write(byte[] buf, int start, int size)
```

5) FileInputStream ---> 具体实现了在文件上读取数据

其实有字节流和字符流两部分，只看了字节流。内容很少，一些流操作用到的方法。

Java工厂模式

使用工厂模式简单实现一下理发店的逻辑。

HairInterface.java

```
/**
 * 发型接口
 */
public interface HairInterface {
    // 实现了发型
    public void draw();
}
```

LeftHair.java

```
/**
 * 左偏分发型
 */
public class LeftHair implements HairInterface {
```



```
// 画了一个左偏分发型
public void draw() {
    System.out.println("左偏分发型");
}
}
```

RightHair.java

```
/**
 * 右偏分发型
 */
public class RightHair implements HairInterface {
    // 画了一个左偏分发型
    public void draw() {
        System.out.println("右偏分发型");
    }
}
```

发型工厂是实现工厂模式的核心类。

HairFactory.java

```
/**
 * 发型工厂
 */
public class hairFactory {
    // 根据类型来创建对象
    public HairInterface getHair(Strign key) {
        if("left".equals(key)) {
            return new LeftHair();
        } else if("right".equals(key)) {
            return new RightHair();
        }
    }
}
```

```

    }
    return null;
}
// 根据类的名称生成对象
public HairInterface getHairByClass(String className) {
    HairInterface hair = Class.forName(className).newInstance();
    return hair;
}
}

```

最后模拟客户端实现。

SunntTest.java

```

/**
 * 模拟客户端实现
 */
public class SunnyTest {
    public static void main(String[] args) {
        // 原生方法
        HairInterface left = new LeftHair();
        left.draw();
        // 工厂方法
        HairFactory factory = new HairFactory();
        HairInterface left = factory.getHair("left");
        left.draw();
        // 优化后的工厂方法，根据类名获取对象
        HairInterface left = factory.getHairByClass("com.package.LeftHair");
        left.draw();
    }
}

```

模拟客户端的类在运行时会调用工厂方法，工厂类根据传入的参数返回对应的对象。

测试用例

测试用例是什么

测试工作的核心

一组在测试时输入输出的标准

软件需求的具体对照

测试用例有什么作用

检验软件是否满足客户需求

体现一个测试人员的工作量

展现测试用例的设计思路

测试用例包含哪些内容

用例编号

用例名臣

测试背景

前置条件

优先级

重要级

测试数据

测试步骤

预期结果

实际结果

备注

测试用例编写流程

需求分析

提取测试点

测试用例编写

测试用例评审

需求分析

业务需求，关注系统是否满足业务

用户需求，关注系统是否满足用户习惯

功能需求，关注系统是否满足功能要求

超星泛雅《创业管理实战》、《大学语文》答案分享

[大学语文.zip](#)

[创业管理实战.zip](#)

一入侯门深似海，从此萧郎是路人。

[前一篇](#) [大误：被人嫌弃的感觉是这样的](#)

[后一篇](#) [今天二十岁](#)



smallyu

Hello World

??

??

????

2017080 (6)

2017070 (2)

2017060 (3)

2017050 (2)

2017040 (2)

2017030 (3)

2017020 (3)

2017010 (4)

20160120 (1)

大误：被人嫌弃的感觉是这样的

□ smallyu □ 2017-05-20 □ 大误

遇到过自己讨厌的人，也遇到过讨厌自己的人。如果一个人不怎么搭理你，应该不再自找没趣。即使自找没趣了，接下来，不应该在内心里潜意识地抵触它才对么？原本以为我会那样。

事件₁：我发“作业借我抄一下”，没回。（IM）

事件₂：我说“Hi”，立回“你走吧”。（现实）

事件₃：我说“Hi”，没回。（IM）

事件₁和事件₃是在IM工具上，事件₂是现实世界中。这三件事情，让我好多种感觉交集。不但没有想要远远躲开它，保持距离，反倒感觉事情有点意思。人也是。毕竟没遇到过。

这些都基于事件之间以及直到目前为止还没有过任何其他联系的基础上。

时间顺序

事件₁分析。

是故意还是无意？那个时候完全不清楚什么情况。

如果是故意的，问题就可怕了。此前完全没有意识到它对我的排斥，所以有可能是一直对我心存不满，瞬间

20160111(2)

20160101(3)

20160901(1)

20160801(6)

20160701(1)

20160601(4)

20160501(4)

爆发了？（其实这个也挺有意思的，什么事情效果这么明显。坏笑脸。后补）

如果是瞬间爆发，那我到底做了什么事情，或者身上有哪些让它讨厌的地方，才会造成这样的情况？

如果不是瞬间爆发，说明它一直都有表现出对我的不满。为什么我没有发觉？是我愚钝了？

如果是我愚钝了，那我就好好想想愚钝在何处。

如果不是我愚钝，那就是它对我的不满表现的不够充分和合理。它为什么没能将不满表现好？

如果不是故意的，又为什么不回？是因为当时没看到，觉得过去一点时间了，没必要再回了？

```
事件1分析函数 ()
{
    if (故意的)
    {
        if (瞬间爆发)
        {
            对我不满在什么地方？
        }
        else if (不是瞬间爆发)
        {
            if (是我愚钝了)
            {
                愚钝在何处？
            }
            else if (不是我愚钝)
            {
                它为什么没能很好表现出不满？
            }
        }
    }
}
```

```
    }  
    else if (不是故意的)  
    {  
        if (觉的不需要回了)  
        {  
            不需要回心态分析函数();  
        }  
        else  
        {  
            事件1超越了我的智商。  
        }  
    }  
}
```

事件₂分析。

事件₂是在现实生活中。我在说出“Hi”的0.5s后，它不假思索的立即说出了“你走吧”。

???

即使没遇到它，我停在那里也不是没有理由啊。

从礼貌的角度讲，不管是否、多么不待见我，都不应该这么说话。不然以后还怎么混社会，走向人生巅峰？

同时，对我不满在什么地方？

或者其实本来想要表达的是其他意思。

```
事件2分析函数 ()  
{  
    if (故意的)  
    {
```



```
        我要教它做人。
    }
    else if (不是故意的)
    {
        不需要回心态分析函数();
    }
}
```

事件₃分析。

事件₁的分析忽略了事件₁的不回是因为抄作业本身的原因。因为事件₃没说是抄作业。虽然事件₃本来确实就是想抄作业。

内容和事件₁类似。事件₃的存在，就是为了从侧面证实事件₁，是故意的，还是无意的？

事件₁和事件₃相隔时间₁₀天左右。事件₃之后，感觉终于放心了。不需要再有疑问，是故意的还是无意的？不回是事实，是故意的。但也有可能进入事件₁分析不是故意的流程里的觉得不需要回函数。

如果是故意的，就是真的厌恶我到打招呼都不想回的程度。那么问题，我到底做了什么我自己没有意识到但却别人看起来惊世骇俗的厌恶至极的事情？

如果不是故意的，嗯。

```
事件3分析函数 ()
{
    if (故意的)
    {
        对我不满在什么地方？
    }
    else if (不是故意的)
```

```
{  
    if (觉的不需要回了)  
    {  
        不需要回心态分析函数();  
    }  
    else  
    {  
        事件3超越了我的智商。  
    }  
}
```

不需要回心态分析。

我以前有过不回别人消息的经历，但没有过第一句话就不回的，也没有过连续两次不回的。

以前没有过被别人不回的经历，可能也就是对这点事情和对那个人感兴趣的原因。

因为并不熟悉它的性格和行事态度，先从我自己的角度分析一下。

我不回别人的情况。

按人，比较熟的人，和相当不熟的人。

我和它，不生不熟。

如果是因为觉得熟，没话说。

如果是因为觉得不熟，好伤。

如果是因为半生不熟，接前，怎么也应该回的才对，从礼貌的角度，从做人的角度。

按事，不想做的。

它不想给我抄作业。

这是一个很扯的分支。

```
不需要回心态分析函数 ()
{
    if (针对人)
    {
        if (熟)
        {
            不理解。
        }
        else if (半生不熟)
        {
            我要教它做人。
        }
        else if (不熟)
        {
            不理解。
        }
    }
    else if (针对事)
    {
        不需要回心态超越了我的智商。
    }
}
```

逻辑顺序

逻辑顺序本是按照针对人、针对事的角度。突然不想写了。该写的都写了。

综上，感觉是否有趣并没有取决于它的态度。也许其他感觉也是。

当然，上面的所有内容全都忽略了最后一种可能：想多了。

切莫交浅言深。——共勉

[前一篇](#): [HTML5 音乐可视化](#)

[后一篇](#): [比起努力，我实在太喜欢睡觉了](#)



smallyu

Hello World

??

??

????

2017080 (6)

2017070 (2)

2017060 (3)

2017050 (2)

2017040 (2)

2017030 (3)

2017020 (3)

2017010 (4)

20160120 (1)

HTML5音乐可视化

□ smallyu □ 2017-05-17 □ HTML5

在线demo: <http://oldblog.smallyu.net/web/passionate/>

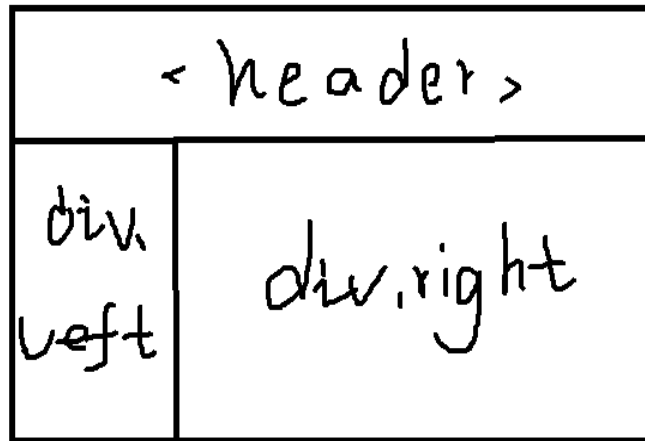
????20170514204448.png

下面梳理一下HTML5实现音乐可视化的流程。

搭建静态页面

2016111(2)
2016101(3)
201691(1)
201681(6)
201671(1)
201661(4)
201651(4)

静态页面的结构分三部分，header，left，right。header 为语义化标签，left 和 right 都用 div 来实现。



类似这样，然后把背景颜色改为黑色，字体改成白色，加上或不加边框线，给一定padding或margin，静态页面就搭建好啦。

不过重点不是页面布局。可以在左侧栏有一个曲目列表，点击切歌。这里就不那么麻烦，直接后台加载指定的歌曲。歌曲是许嵩的烟花笑，左侧栏显示部分歌词。

创建音频文件对象

AudioContext对象用于获取音频文件源。

（节点）AudioContext：包含各个AudioNode对象以及它们的联系的对象，可以理解为上下文对象。
绝大多数情况下，一个document中只有一个AudioContext。

（方法）createBufferSource()：创建audioBufferSourceNode对象。

可以这样写：

```
var ac = new window.AudioContext();    // 实例化ac为一个音频对象
var bs = ac.createBufferSource();      // 用bs来保存音频文件资源
```

有了音频资源，是否就可以播放音频了呢？当然不可以，因为还没有指定文件，告诉对象播放哪一个音频文件。

获取音频数据

创建`bufferSource`后，到了`AudioBufferSourceNode`这个节点。这个节点有一个属性值`buffer`，用来指定文件资源。这个属性值需要一个`buffer`类型的数据。

（节点）`AudioBufferSourceNode`：表示内存中的一段音频资源，其音频数据存在于`AudioBuffer`中（其`buffer`属性）。

（属性）`buffer`：`AudioBuffer`对象，表示要播放的音频资源数据。

获取`buffer`有两种方式，一种是`ajax`设置返回值类型为`arraybuffer`，然后解码，一种是用`FileReader`读取文件，获得`blob`值。后一种方式多用`input`上传文件后获得文件，再进行解析。前面一种好用一点。（留坑）

`ajax`的原生写法：

```
var url="...";
var xhr = new XMLHttpRequest();
xhr.abort();
xhr.open("GET", url);
xhr.responseType = "arraybuffer";
```

```
xhr.onload = function(){
    return xhr.response;
}
xhr.send();
```

onload触发的函数中xhr.response就是我们想要的值。

（方法）decodeAudioData(arrayBuffer, succ(buffer), err)：异步解码包含在arrayBuffer中的音频数据。

将arraybuffer解码：

```
ac.decodeAudioData(arraybuffer, function(buffer){ ... },function(err){ ... });
bs.buffer = buffer;    // 回调函数中的buffer就是想要的buffer
```

播放音乐

（方法）start：开始播放音频

这个时候调用start方法：

```
bs.start(0);
```

打开页面，音乐开始播放。

音量控制

（节点）GainNode：改变音频音量的对象，会改变通过它的音频数据所有的sample frame的信号强度。

(方法) createGain(): 创建GainNode对象。

(属性) value: 可以改变音频信号的强弱, 默认为1, 最小值为0。

音乐播放还不行, 要添加一个音量控制控件, 音量控制用GainNode节点:

```
var gainNode = accreateGain();    // 创建GainNode对象
gainNode.connect(ac.destination); // 将gainNode连接到AudioDestinationNode节点
```

这样就有了一个音量控制的对象。

```
gainNode.gain.value= ... ;
```

gain.value 用于控制输入信号的强弱, 也就是音量的大小。HTML 中用 type 为 range 的 input, 把值传递给对象, 就可以实时控制音量了。

得到音频解析数据

(节点) AnalyserNode: 音频分析对象, 他能实时的分析音频资源的频域和时域信息, 但不会对音频流做任何处理。

(方法) createAnalyser(): 创建AnalyserNode对象。

(属性) fftsize: 设置FFT (是离散傅里叶变换的快速算法, 用于将一个信号变换到频域) 值的大小, 用于分析得到频域, 为 $32 \sim 2048$ 之间2的整数次倍, 默认为2048, 实时得到的音频频域的数据个数为fftSize的一半。

(属性) frequencyBinCount, FFT值的一半, 即实时得到的音频频域的数据个数

（方法）`getByteFrequencyData(Uint8Array)`，复制音频当前的频域数据（数量是`frequencyBinCount`）到`Uint8Array`（8位无符号整型类型化数组）中。

先创建`AnalyserNode`对象，然后设置`fftSize`的值。FFT用于数字信号的处理，会把音频文件流输入的信号变换到频域，用`getByteFrequencyData()`方法得到经系统处理之后的频域数据。

```
var analyser = ac.createAnalyser(); // 实例化对象
analyser.fftSize = 32; // 设置fftSize
var arr = new Uint8Array(analyser.frequencyBinCount); // getByteFrequencyData(
) 方法要求参数为8位无符号整型类型化数组
analyser.getByteFrequencyData(arr); // arr为所需频域数据
```

这样得到的数组`arr`就是用于可视化效果绘制的数据。

（方法）`window.requestAnimationFrame()`：告诉浏览器您希望执行动画，并请求浏览器调用指定的函数在下次重绘之前更新动画。该方法将在重绘之前调用的回调作为参数。

一次解析只能得到一组数据，所以需要`requestAnimationFrame`不断解析，不断得到`arr`。

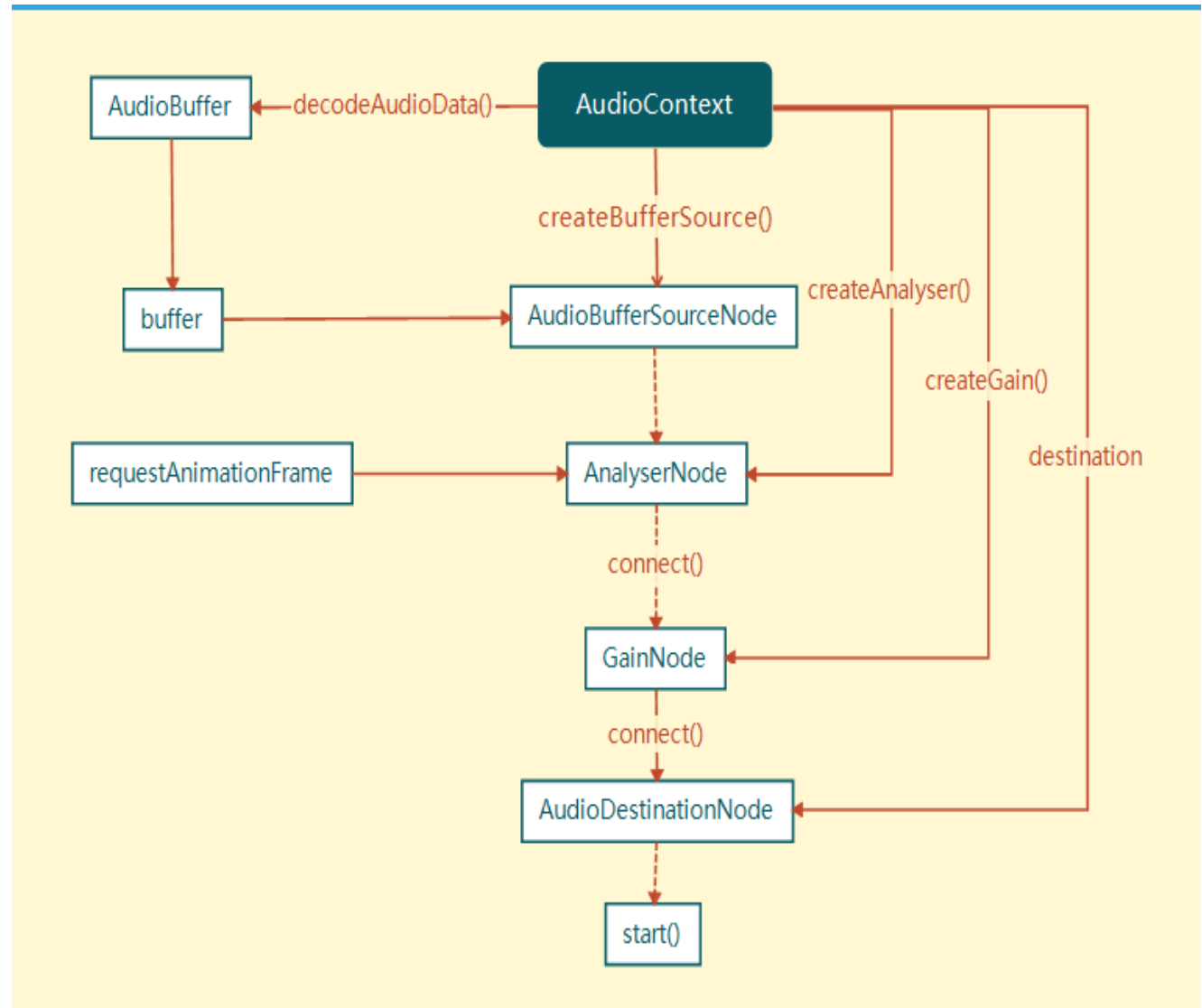
前端界面绘制

前端界面使用`canvas`绘制，实现音乐可视化的效果。

```
var box = document.getElementById("right"); // 获取right区域的dom
box.appendChild(canvas); // 创建canvas节点
var ctx = canvas.getContext("2d"); // 实例化canvas画笔
```

接下来就是使用`for`循环和`ctx`画矩形、圆圈，填充渐变的操作了。（留坑吧，估计不填了）

案例用到web Audio API的关系：



虚线是可以跳过的节点，直接播放音频文件。好吧有错。

[前一篇](#) 活着那么累，和我一起去死吧

[后一篇](#) 大误：被人嫌弃的感觉是这样的



smallyu

Hello World

??

??

????

2017080 (6)

2017070 (2)

2017060 (3)

2017050 (2)

2017040 (2)

2017030 (3)

2017020 (3)

2017010 (4)

20160120 (1)

活着那么累，和我一起去死吧

□ smallyu □ 2017-04-26

明天又要回家了。

这个月最后一次_{post}内容。

真的，这样就好。

.....内容分割线.....

参加那什么什么比赛很开心。

没想过会有这么一天。

没想到会有这么一个比赛。

想不到以后会怎样.....

20160110 (2)

20160109 (3)

20160909 (1)

20160809 (6)

20160709 (1)

20160609 (4)

20160509 (4)

这样就好，就这样就好。

不快不慢。

这样就好。

.....内容分割线.....

论bootstrap党参赛。

中国“软件杯”软件设计大赛和互联网+创新创业大赛，在4.8之后开始准备，4.15报名了校内选拔赛。

原先准备软件设计大赛，有一个题目是无微不至的阅读，做一个基于微信的图书借阅平台。

后来组队了。

队伍人数超出规则，就选择了创新创业大赛。

互联网+餐饮。

如果要真的实现，项目难度大于软件设计赛的题目。

比赛要求创意组只需要提交项目说明书和商业策划书就好了。

嗯。

首页: <http://oldblog.smallyu.net/g2025/index.html> (主移动)

后台: <http://oldblog.smallyu.net/g2025/admin.html>

登录: http://oldblog.smallyu.net/g2025/bis_log.html

安卓APP: <http://oldblog.smallyu.net/g2025/g2025.apk>

是不是有点拿不出手?

嗯。

反正大赛不要求做。

.....内容分割线.....

《全职高手》出动画了!

它承载着, 我的童年啊。

.....内容分割线.....

清明来后，情绪还算消极，想写点东西，这是大纲：

```Quote

大学回家，邻座路人

高中回家，出租车司机

1. 想做就去做

清明节回家

评论说说

2. 学会堕落

玩整天游戏

3. 偶尔沉默

终有一天会释然

#### 4. 为自己而活

晚上跑步

为什么要迷茫

...

过去一段时间，当时想要表达什么内容已经，差不多忘了。

.....时间分割线（4.9） .....

每到放假不知所措，清明小假也一样。纠结半天，回家了。

大巴邻座是一位大叔。

一开始沉默不语，后来突然问了一句，“你回哪儿？”

接着又问我在什么学校读书，什么专业，就业前景怎么样。

他的儿子高考复读一年，超过二本线个位数，现在进了一所理所当然的学学校读书，文科，历史。

地球人都知道，大学生工作不好找，普通二本的工作更不好找，文科专业地理、生物、历史、政治，除了师范，研究生都不好找。

所以大叔很担心他儿子。

想起来高中有一次拼车回家，开出租车的师傅也问过类似的问题。

他的孩子那年要考高中了，你们学校的分数是多少，你当时多少分考进的。

我问，害怕考不上吗？

他说，当然怕，要不然进去得花钱（五位数）。

然后我当时想，为什么一定要读书？

[前一篇](#): [Friends.S09E17](#)

[后一篇](#): [HTML5音乐可视化](#)



smallyu

Hello World

??

??

????

2017080 (6)

2017070 (2)

2017060 (3)

2017050 (2)

2017040 (2)

2017030 (3)

2017020 (3)

2017010 (4)

20160120 (1)

## Friends.S09E17

□ smallyu □ 2017-04-15 □ Friends

You ready to go?

Let me just finish this.

Hey, Ross. Check this out. Yeah, I can't do that.

What are you doing?

Have you seen this? It's our new alumni website for college. You can post messages for people let everyone know what you're up to.

Oh, great. A faster way to tell people that I'm unemployed and childless.

It's actually interesting to find out what people are doing. Remember Andrea Rich?

That tall girl that wouldn't sleep with you?

Well, her Internet company went under, and she lost an ear in a boating accident.

Bet she'd sleep with you now.

No, I already e-mailed her.

20160111 (2)

20160110 (3)

2016090 (1)

2016080 (6)

2016070 (1)

2016060 (4)

2016050 (4)

Let me see what you wrote about yourself. Doctor of paleontology, two kids. Wait a second. You split with Carol because you had different interests? I think you split with Carol because you had one very similar interest.

I'm gonna finish this later, okay? Let me just grab my coat.

Okay!

What, did you think you learned how to do it in the two minutes?

Maybe we finish this for him. Also, I cloned a dinosaur in my lab. She is now my girlfriend. I don't care what society says, it's the best sex I've ever had. And send.

No, no-- You can't do that to him.

All right, let's go.

Dude!

I think you made it clear you can't be trusted with the ball inside the house.

And send.

Hey. How was basketball?

It was fun. Right up until Chandler got a finger in the eye.

Oh, no! Who did that?

Chandler. Yeah.

Hey!

Rach?

Yeah?

What's Huggsy doing in the crib with Emma?

She was just crawling around and found him, so I just let her sleep with him.

That's all right, isn't it?

Of course. Yeah.

It's a stuffed animal, you know? It's for kids. Not for adults. I know that.

Joey, you're sure? I mean, I knew how much you love him.

Rachel, let's be clear on this, okay?

I do not love Huggsy. I like him a normal amount.

All right. Well, Emma loves him.

Yeah. Well, why wouldn't she? He's a wonderful person!

Hi.

Hey, Phoebe.

How you doing?

You felling better?

Breaking up sucks.

Oh, I really miss Mike.

Oh, I'm so sorry.

I've tried everything to feel better. I even tried writing a song about it... but I can't think of anything that rhymes with:

Oh, I so know how you feel. Breaking up with Richard was the hardest thing I've ever done in my life. All right, fine, I'll put another dollar in the Richard jar.

Hey, Monica, I'm gonna need your help getting through this.

You're saying you're not gonna need my help?

No. When I get to the point where I'm ready... to hear cruel, mocking jokes about Mike, I'm gonna come to you.

Oh, good, because I've already thought of three. Four! I just thought of a fourth!

Okay. I mean, I know I did the right thing. Mike never wants to get married... and I shouldn't be a relationshi

p that has no future. But pretty soon, I'm gonna miss him so much... I'm gonna want to see him again, and you have to stop me.

Okay, you got it.

Unless... Am I being too crazy about this? All right, so there's no future. But that doesn't mean we still can't have fun. You know what? Forget what I said.

Really? All right, if that's what you want.

That was a test, and you just failed.

Damn it! Rookie mistake!

I have sex with dinosaurs?

I believe I read that somewhere.

Not only is it not funny, it's physically impossible! Okay? Depending on the species, I'd have to have a 6-foot-long-- It's not funny!

I respectfully disagree.

I can't believe you put that on my alumni page.

Who cares? Nobody reads those.

You better hope not... because I just read what you put on your page today.



I don't have a page.

Oh, I respectfully disagree!

Huggsy?

Huggsy?

Huggsy.

I want my penguin.

Look at you. All sweet and innocent, sleeping like an angel.

With Emma's chubby little hands wrapped around you.

It's okay, Emma. You stay asleep.

Step away from the crib! I have a weapon!

It's okay! It's okay, Rach! It's me! Put down the scrunchie.

What are you doing?

Well, I heard Emma string... so I came in to make sure she could reach Huggsy.

Oh, thanks. All right, well, now that I'm up, I'm gonna go to the bathroom.

Okay, there you go, sweetie.

This isn't over!

Wait a second. Why would Ross tell everyone on your class that you are "as gay as the day is long"?

Because I told everyone he alept with dinosaurs.

Well, that's clearly a joke.

This could easily be true.

Would you get that, please?

People have been calling to congratulate me all day.

Hello?

No, he's not here. Yeah, this is his wife.

Yeah. Well, it came as quite a shock to me too!

Well, I guess I should've known. He just kept making me watch Molin Rouge.

Hang up! Hang up! And that was a great movie.

I'm so gonna get back at Ross.

Oh, yeah, this'll show him. Here we go.

What are you doing?

Oh, you'll see, my friend!

I'm dead?

And so young.

Posting that I died? That really isn't funny.

Well, how you died was funny.

Oh, please. Hit by a blimp?

It kills over one Americans every year.

Unbelievable. My classmates are gonna think I'm dead. My professors. My parents are gonna get phone calls! You're messing with people's feelings here.

You wanna talk about people's feelings? You should have heard how hurt Professor Stern was when I told him I wouldn't be to go to Key West!

You've really crossed a line here. But that's okay. Because I'm on my way to buy some Photoshop software... and a stack of gay porn. That's right. Your coming-out is about to get real graphic!

God, I wish Mike were here.

Okay, if Mike were here, what should the two of you be doing?

What are you, animates? It's 4:00 in the afternoon.

I gotta call him. Just talk to him. There's no harm in that.

That's how it starts. "I don't need to eat the cake, I'll just smell the icing. Why not just a little sliver?" Or, "Okay, just a slice or two." The next thing you know, you're 210 pounds... and you get wedged in going down the tunnel slide!

That's not true, is it?

They had to cut me out... With the Jaws of Life.

Phoebe, honey, I know this is hard. But look, if you talk to him, then you're gonna want to see him. And if you see him, you're gonna wanna get back together. And I know that's not what you want.

All right, so give me your phone.

Here.

- And now your cell. - Okay.

There you go.

- This is your cell phone? - Yes.

This is your current cell phone?

Yes. It reminds me of a simpler time.

Phoebe? Where's your purse?

- Hey! No! - No!

- Give it to me! - You can't have it!

- Give it to me! - No

- I'll go in there! - Oh, yeah.

- Phoebe, come here! - No!

- Give it to me! - You're crazy!

Damn you, Monica Geller hyphen Bing!

Don't even think of sneaking out and using a pay phone... because I am everywhere, baby!

Hey, look who's here.

It's Joey. And he brought home a friend.

Joey! Emma's right here.

You promised not to bring girls home in the middle of the day!

No, no, no, it's not a girl. It's... a brand-new Huggsy!

Oh! Oh, that's so great! Now Emma has two Huggsies!

No, no. Oh, no. No, Emma has one Huggsy. The new Huggsy. The other Huggsy, I don't know, I guess I'll j

ust take him back.

You know what? When I was a little girl, I had a little pink pony named Cotton. Oh, I loved her so much. I took her everywhere. I would braid her tail--

Make the transfer!

Should I be concerned that a button fell off the old Huggsy and I can't find it?

No, don't worry about it. I swallowed that years ago. Hey!

Oh, what? I don't think she likes the new Huggsy.

But he's the same.

I think she wants the old one back.

Yeah, but he's the same.

Joey, come on.

But he's the same!

You're not the same.

You know, it's a lot less surprising to do that after I've buzzed you into the building.

So, Phoebe, why are there men's shoes by the door?

Those are my shoes.

Well, when you get over this breakup, we need to go shopping.

Look, Monica, I really appreciate you checking in on me. But I'm actually feeling a lot better.

You are?

Yeah, I just kind of want to be alone right now.

- Who's that? - I ordered Chinese food.

What are the chances?! One billion Chinese people, and they send Mike!

What are you doing here?

Phoebe called me.

- Phoebe1 - I'm sorry. I broke down.

I wanted to see him.

Well, not on my watch.

Damn it, Phoebe, how did you even call him?

There's a speakerphone on the base unit.

Base unit. Think, Monica, think!

- Can I come in? - No! No.

Phoebe, isn't seeing Mike exactly what you wanted me to stop you from doing?

Look, if I want to see Phoebe, and she--

This doesn't concern you!

Sorry. I guess I was thrown off by the mention of my name.

Look, guys, you can't do this.

It's gonna make getting over each other harder.

Not if nothing happens. Why can't we just hang out as friends?

Sure. If you're just gonna hang out as friends, then maybe I'll join you. You know, I'm your friend and Mike's friend.

Sit down.

Make room for your friend.

So how have you been?

I've been pretty good.

You look really beautiful.



Thinks. You look good too.

No, no, no. This is dangerous territory. Keep it clean.

So how's the piano playing going?

Actually, I've been playing a lot of love songs lately. I've missed you.

I've missed you too.

You know, on the way over here, I saw this drunk guy throw up. And then a pigeon ate it.

Hey. Ross.

Look, I just wanted to apologize. Don't tell me you actually made those gay pictures of me.

Check this out.

So that's what I'd look like if I worked out. And was being serviced by a policeman.

You're not gonna actually send these out, are you?

Actually, I don't need to, because your little "Ross is dead" joke didn't work. Okay? There were no responses. Nobody posted anything on the website, nobody called my parents, so the joke is on you. Nobody called, nobody wrote anything, nobody cares that I'm dead.

Oh, my God. Nobody cares that I'm dead?

Oh, come on, you know that's not true.

What are you talking about? You get 60 responses... just for coming out of the closet. I didn't get one response, and I'm dead!

Well, the gay community is a lot more vocal than the dead community.

I can't believe this. Not even my geology lab partner. And I carried that guy!

Look, let's think about this, okay?

You think people are gonna disturb your family as this tragic time? That people are gonna post condolences on a website? This isn't about people not caring. This is about people not having a decent outlet for their grief.

You're right. There isn't a decent outlet.

Right. I'm sure if you had a funeral or memorial service, tons of people would come.

Exactly!

Ross, what are you doing?

You're having a memorial service for yourself?

No, that would be stupid. You're having it for me.

Ross, don't press send. Don't press--

No, too late. Too late. It's sent. Sorry, so is that picture of you and the policeman.

That's all right. I'm tan, limber... And it looks like I talked my way out of that ticket.

The surgery was not a success. And then you say: "What are we gonna do, Drake?"

Come in.

I'm trying to put Emma down for a nap. Have you seen Huggsy?

Original or crappy?

Original.

No.Sorry. Haven't seen him.

Then what's that big lump under your covers?

It's Monica, okay?

That's not Monical!

All right, fine! It's original Huggsy! I know Emma wants him, but he's mine and I need him. She's being unreasonable.

Joey, there is a reason that Emma loves that stupid penguin so much! Oh, don't cover its ears! It's because it reminds her of her Uncle Joey.

- It does? - Yeah.

And I think she really likes him because he smells like you. And she-- She loves that. Go figure. And she's co

mforted by him because she loves her Uncle Joey so much.

- Really? She loves me? -Oh, yeah.

But you know what? If you need Huggsy, don't worry. Emma will totally understand. I won't, but whatever/

Okay, wait a minute, Rach. I mean, if... If Huggsy means that much to Emma, then... Well, she can have him.

Oh, sweet, I knew Uncle Joey would step up. Look, Emma. Look who's back.

No, forget it! Forget it! I can't do it!

Are you--? You're gonna take Huggsy away from a little child?

How do you think I got him in the first place?

And then it just gobbled up that puke. I mean... That was one gross pigeon.

Oh, man. Why didn't I ask Joey to keep us apart? You throw a meatball in the hallway, we'd be doing it like rabbits right now.

All right, you two. I'm gonna go to the bathroom. I don't want anything going on while I'm gone. Here's a few things you can discuss: Mucus, fungus, and the idea of me and Ross doing it.

I've missed you so much. I won't ask you to get back together. I know we want different things, but... Just to be with you one more night.

I want that too. But is that gonna make it too hard?

It can't be any harder than this. I mean, if I had known the last time I saw you would be the last time, I-- I would have stopped to memorize your face, the way you moved. Everything about you. If I had known the last time I kissed you would be the last time... I never would have stopped.

Kiss him, you fool!

What?

Didn't you hear that speech? If you don't kiss him, then I will.

I missed you so much.

I knew you'd be here!

Crap.

Who is this?

My friend Manny.

I asked him to keep me away from you.

Hey, that's what I'm doing for Phoebe.

Well, you're not doing a very good job.

Excuse me?

What's with the kissing?

Hey, at least I knew where my guy was.

Yeah, think God you were here to oversee all the kissing.

You didn't hear the speech.

I've heard the speech. If he know it was gonna be the last time--

Hey, it was vary moving. You're just heartless!

You're weak!

You're weird!

Your pants are undone.

Where'd they go?

Damn it.

Well, we blew it. I blame myself.

And I blame you too.

It's been an hour, and not one of my classmates has shown up? I tell you, when I actually die, some people are gonna get seriously haunted.

There you go. Someone came.

Okay. Okay. I'm gonna go hide. This is so exciting. My first mourner!

Hi. Glad you could come.

Please come in.

Hi. You're Chandler Bing, right? I'm Tom Gordon. I was in your class.

Oh, yes, let me take your coat.

Thanks. I'm so sorry about Ross, it's...

No, at least he died doing what he loved. Watching blimps.

Who is it?

You know some guy, Tom Gordon?

I don't remember him. But then again, I touched so many lives.

So did you know Ross well?

Actually, I barely knew him. I came because I heard Chandler's news. Do you know if he's seeing anyone?

Yes, he is. Me.

What? You mean--?

Can I ask you a personal question? How do you shave your beard so close?

Okay, Tommy! That's enough mourning for you. Here we go. Bye-bye. All right.

Hey, listen. Call me.

Okay.

I'm dead and no one cares?

I look like a man?

Please, one ridiculous problem at a time. This isn't ridiculous. Look around, no one's here.

You gave them one day's notice. Not everyone checks the website every day. And Monica, it's probably the way you stand!

I guess you're right. Still, somebody must have seen it. I went to that school for four years and I didn't have an impact on anyone?

Well, that's not true. You had an impact on me. I mean, it's 15 years later and we're still best friends. Doesn't that count for something?

Oh, great, more party bony for Chandler.

I'm sure it's somebody for you. Now go hide.

Hi. I'm here for Ross Geller's memorial service.

Kori? Kori Weston?



Yeah.

You look amazing.

And you are?

Chandler. Chandler Bing. And I'm not gay. I'm not gay at all.

You are married, though.

Don't listen to him. He's in a really bad mood.

I can't believe that Ross is gone. It is just so sad.

I didn't know Ross and you were so close.

Oh, we weren't. But we had one class together. He was such a great guy. And he talked so passionately about science. I always remembered him.

I'm sure that would mean a lot to him. And if heaven has a door... I'm sure he's pressing his ear up against it and listening intently.

I thought so many times about calling him and asking him out. I guess I really missed my chance.

No, you didn't! I'm still alive!

Who did we bury?

Kori, I know this is a big surprise for you. It's a long story. But the things you just said really made my day. Th

e fact that you're here means more to me... than if this room were filled with people.

You sick freak! Who does that? I can't believe I had a crush on you.

Did you hear that? Kori Weston had a crush on me!

No, Mom, I am not dead. I know it's not something to kid about. It was just a practical joke between Chandler and me, but it's over, okay? Actually, no. Even if I had died, you would not be left childless. Monica?

====2017.6.8补=====

浑浑噩噩的当时 想要借助这样的方式来让英语不要淡出自己的视野 就把这一集的台词全打了一遍。2个小时+ 很用时间。

[前一篇: 风决定要走 云怎么挽留](#)

[后一篇: 活着那么累 和我一起去死吧](#)



smallyu

Hello World

??

??

????

2017080 (6)

2017070 (2)

2017060 (3)

2017050 (2)

2017040 (2)

2017030 (3)

2017020 (3)

2017010 (4)

20160120 (1)

## 风决定要走，云怎么挽留

□ smallyu    □ 2017-03-11    □ 精神错乱

昨天晚上，一位大学同学精神错乱（意识紊乱），没有清醒的自主意识，做出种种奇怪的行为。直到今天上午，在N多人熬夜的陪同下，等来了他的家人，短暂了解情况后，带走，休学。

这是一位奇怪又不奇怪的同学，这是一件可怕又不可怕的事情。

有生以来第一次亲眼见到精神错乱的人，可怕的不是他现在什么状态，而是他以前，再正常不过的一个普通人。

“普通人”只是我对他的定位，可能他自己并不这么认为。昨天晚上，事情刚刚发生但还没有完全发生的时候，他很大声地说（吼）了一句话：“宿舍楼里没有睡觉的人都听我指挥，我是宿舍部副部长！”。

他行为异常早有体现。他舍友说，最近几天，就在补考结束之后，他的动作变得奇怪起来。听歌，只单曲循环同一首歌；上课，只坐最后一排；上课的时候，唱歌；晚上，不睡觉；中午喝水，站在阳台，喝一口，咳一下，喝一口，咳一下，喝完一杯，再来一杯，喝完这杯，还有三杯……最奇怪的就是最近两三天尤其是昨天，不停到各个宿舍串门。

说奇怪是因为他的这些行为，都超过了该有的“度”。

- 听歌单曲循环不算是异常行为，但如果循环到让你发毛。你可以打开网易云设置单曲循环，然后打王者荣耀，写作业，聊天，做事情，或者这首歌真的让你喜欢到欲罢不能。好像他听的是，《没有你陪

20160110(2)

20160109(3)

20160909(1)

20160809(6)

20160709(1)

20160609(4)

20160509(4)

伴真的好孤单》。

- 上课坐最后一排是很多同学的惯例，他不是。以前积极进取，严谨认真，两次班里选积极分子都有他的份。两次积极分子是因为，第一次积极分子考试的时候，就是那种关于党，爱党信党忠党之类内容的考试，他被认为是作弊。现在进考场不能带通讯设备（手机）是惯例也是常识。也许他担心考试过不了。
- 上课的时候，唱歌。想象如果一个真正的精神病人坐在教室里面，旁若无人的“小”声唱《成都》。已经违反公共场合礼仪规范了，不是对不对，而是合不合适。他是一个大学生。据说其实离开宿舍后在上期间还是很正常的。
- 晚上，好几天没睡觉了，可能这个和意识紊乱的关系会比较大。而且每天早上6点就会起床并把其他人叫醒。
- 中午喝水，没亲眼见过，被同寝室的人认为不正常。
- 逛宿舍，整个事件的始作俑者。昨天中午宿舍挨着逛一遍，中午睡觉醒来准备去上课的时候发现他就在我们宿舍。当时我们宿舍没有醒着的人，都在睡觉，并没有人和他聊天。然后他就那样坐在我（二级）床下的凳子上。舍友和他打招呼，没感觉到异常。

直到昨天晚上，他待在了我们一个兄弟班的宿舍。11点熄灯，人家要睡觉了，他赖着不走。并且之前，他在人家宿舍随便把别人的东西搬来搬去，之前，在逛我们班的某一个宿舍的时候，不声不响的喝掉了某位同学三瓶牛奶。

赖在别人宿舍不走，说他要玩个游戏。进去一个人，他就会让那个人陪他玩个游戏，发布“关上门”，“面对

着门”，“把凳子搬到这里”，“把桌子收拾一下”，等等之类的“命令”。

然后只能惊动宿管楼姨、班主任、系辅导员、父母。担心他当时的精神状态会有什么危险，男生四个宿舍30个人几乎一晚上没睡（当然也有例外，我就睡得可好了）轮流值班看着他，不让他到阳台，不让他拿什么太锋利尖锐的东西。

综合来看，他突然间意识紊乱的客观因素大概有：

- 挂科
- 人际关系差

主观因素大概有：

- 现实与梦想的落差
- 迷茫

客观因素第一个是挂科，这是一个很重要的原因。前两个学期他都没有挂科并且成绩还不错。上个学期考试之前，他还来我们宿舍并不炫耀的问我们有什么题不会（毕竟我也是考的了前几名的好不好），充分说明了他当时的自信程度。事与愿违，上个学期的成绩下来后，他挂科了，总成绩排名倒数。

挂科对应的就是主观因素中现实与梦想的落差。原先他是一个成绩并不差的好学生，是一个被选为入党积极分子，马上就可以成为党员的优秀学生，是班里的常委，是宿舍部副部长。他有多重视在学校中的职位从那

一句“宿舍楼里没有睡觉的人都听我指挥，我是宿舍部副部长！”就看得出来。也许他的梦想是当官，做一个有权的人。

对这种梦想挂科带来的是毁灭性的打击。挂一门课总成绩一落千丈，同时只要挂科就不能入党。学生会那边的事情没人清楚，据我所知我们这种三流学校的学生会，部长和副部长差的就很多，如果他在学生会那边受点打击也很惨。

第二个客观因素的人际关系差。

有一次他舍友在商店借他一块钱，他舍友正好没有零钱。他说，我不借一块钱，只借十块钱。然后借给了他舍友十块钱。潜台词就是借一块钱我不好意思找你要！那是舍友啊舍友。像我们宿舍不管关系好不好什么时候计较过几块钱？这不是家庭经济情况好坏的原因好么。

有一次，我们三个人一组打扫教室。我和另外一个人给忘了，他打电话叫我们，当时已经晚了一会儿。然后我们去了之后，看见了啥？他坐在教室里，的椅子上，坐着，玩手机。当时心里一万头草泥马呼啸奔腾而过。我们没来你特喵就不能自己先开始么。后来查教室卫生打扫情况的人还打着官腔说我们不该迟到怎么怎么地。

还听说，他和他们宿舍的人打过架。他嫌他的下铺打游戏影响他睡觉，就直接睡在了他的下铺的床上。当时是夏天，他经常在外面打球，没有洗澡，就直接睡在了人家的床上。两个人看着不爽，就打了起来。可能他只是没有合理的办法表达出自己的需求。

人际关系差不是我的判断，是他的舍友们总结出来的，因为他的舍友们并不是很友好的看待他。

学业崩溃加上人际关系无望。上课上的很不爽，自己平时的努力毁于一旦，学习都会挂科那还学它干吗？（重要的是同学们会不会小看我？）人际关系不是很好，回到宿舍连个说话的人都没有，在宿舍待着岂不是很

郁闷？然而对于我们这种业余生活不丰富的学校和生活风气，整天的行程就两个地方，教室，宿舍。于是导致整个人会迷茫，不知所措，看不到希望，压力大，紧张。（迷茫只是我的判断，毕竟漫无目的的逛宿舍不就是迷茫的体现？）

以上是对那位同学仅有的一些了解和目前已知的一些情况。

我和他的关系没有多好，也并没有在他情绪最并不稳定的时候和他接触过，只是今天早上起床后到他的宿舍看了几分钟。他喘着粗气，偶尔会有一下不寻常的动静，那种只会在电影里看到的不寻常的动静。离得很远就能感受到屋子里面咄咄逼人和不寒而栗的气氛，尽管同在的还有好几个人。

问题是我能确切的想象到他的感受。

我不会说上个学期放寒假回家我差点没哭。

我想象不到复习一个月考5名开外是什么体验。

我不明白做很多题CET4不到400分是什么感受。

我不知道Hello World能给我带来什么。

我不知道我还要单身多久。

难道我离那样也不远了？

[前一篇: 站在巨人的肩膀上写代码——Spl中的三种数据结构解析](#)

[后一篇: Friends.S09E17](#)







smallyu

Hello World

??

??

????

2017080 (6)

2017070 (2)

2017060 (3)

2017050 (2)

2017040 (2)

2017030 (3)

2017020 (3)

2017010 (4)

20160120 (1)

## 站在巨人的肩膀上写代码--Spl中的三种数据结构解析

□ smallyu □ 2017-03-05 □ urlencode

今天身体莫名`get trouble`，心情莫名`bad`。想说什么又没有什么要说的。I want to get a new object?（她不喜欢我怎么办？）嗯? `%e9%9d%b3%e8%8d%a3`

来简单说说Spl中三种数据结构的基本操作，这三种数据结构分别为双向链表（`SplDoubleLinkedList`）、堆栈（`SplStack`）和队列（`SplQueue`）。

### 双向链表（`SplDoubleLinkedList`）

首先实例化对象为`$obj`

```
$obj = new SplDoublyLinkedList();
```

然后我们添加三条数据到双向链表中

```
$obj->push(1);
$obj->push(2);
$obj->push(3);
```

再来添加一条这样的数据

20160111 (2)

20160110 (3)

2016090 (1)

2016080 (6)

2016070 (1)

2016060 (4)

2016050 (4)

```
$obj->unshift(10);
```

这个时候打印一下\$obj

```
print_r($obj);
```

输出的结果会是这样

```
SplDoublyLinkedList Object
(
 [flags:SplDoublyLinkedList:private] => 0
 [dlist:SplDoublyLinkedList:private] => Array
 (
 [0] => 10
 [1] => 1
 [2] => 2
 [3] => 3
)
)
```

对我们看到，1、2和3分别按照次序被添加进了链表对象的数组中，而底部（Bottom）位置的数据则是10，即使用unshift方法添加的数据，这里得知unshift的作用就是添加数据到链表底部。

再调用这个方法

```
$obj->rewind(); // 10
```

输出一下当前指针指向的节点

```
echo 'current: ' . $obj->current();
```

输出的是这个节点

```
current: 10
```

说明双向链表的`rewind`会将指针移动到底部。

我们再试试`next`方法

```
$obj->next(); // 1
```

输出当前指针指向的节点

```
echo 'next node: ' . $obj->current();
```

结果是

```
next node: 1
```

结果明了，会将`current`向顶部移动一个位置。

再来几个测试

```
$obj->next(); // 2
$obj->next(); // 3
$obj->prev(); // 2
echo 'next node: ' . $obj->current(); // 2
$obj->next(); // 3
$obj->next(); //
```

```
echo 'next node: ' . $obj->current(); //
```

代码中反复使用了`next`方法和`prev`方法，每次指针指向的节点在代码中都有注释，并输出了两次来验证正确性，输出结果是正确的

```
next node: 2
```

```
next node:
```

并且我们知道指针超出链表的范围后会返回空值，因此接下来我们可以判断一下当前节点是否有效

```
if($obj->current()){
 echo "Current node valid
";
}else{
 echo "Current node invalid
";
}
```

返回值为

```
Current node invalid
```

或者可以用这样的方式来验证当前节点是否可用

```
if($obj->valid()){
 echo "valid list
";
}else{
 echo "invalid list
";
}
```

返回内容为

valid list

接下来是删除节点的操作，pop方法用于删除当前指针指向的节点，并返回该节点的内容

```
$obj->rewind();

echo "<hr /> Pop value: " . $obj->pop();

print_r($obj);

echo 'current: ' . $obj->current();
```

输出的内容为

Pop value: 3

```
SplDoublyLinkedList Object
(
 [flags:SplDoublyLinkedList:private] => 0
 [dllist:SplDoublyLinkedList:private] => Array
 (
 [0] => 10
 [1] => 1
 [2] => 2
)
)
```

current: 10

删除节点之前使用rewind重置了指针的位置到Bottom，同时pop删除了Top位置的节点。那么如果current正好指向Top位置会怎样？

```
$obj->next(); // 1
$obj->next(); // 2

// 把Top位置的节点从链表中删除，并返回
echo "<hr /> Pop value: " . $obj->pop() . "
";
echo 'current: ' . $obj->current() . '
'; // 10

print_r($obj);
```

输出

Pop value: 2

current:

```
SplDoublyLinkedList Object
(
 [flags:SplDoublyLinkedList:private] => 0
 [dllist:SplDoublyLinkedList:private] => Array
 (
 [0] => 10
 [1] => 1
)
)
```

如果current正好指向Top位置，那么调用pop之后current会失效。

最后一个要试的方法是shift，用于把Bottom位置的节点从链表中删除，并返回，执行

```
$obj->shift();
print_r($obj);
```

会只输出

```
SplDoublyLinkedList Object
(
 [flags:SplDoublyLinkedList:private] => 0
 [dllist:SplDoublyLinkedList:private] => Array
 (
 [0] => 1
)
)
```

另外两种数据结构的操作类似，啊，好懒，直接贴一下，要不然得很长时间。

## 堆栈 (SplStack)

测试代码为

```
<?php

$stack = new SplStack();

$stack->push('a'); // push操作箱堆栈里面放入一个节点到top位置
$stack->push('b');
$stack->push('c');
```

```
echo '<pre>';
print_r($stack);
echo '</pre>';

echo "Bottom: " . $stack->bottom() . "
";
echo "Top: " . $stack->top() . "
";

$stack->offsetSet(0, 'C'); // 堆栈的offset=0是Top所在的位置, offset=1是top位置节点靠近
bottom位置的相邻节点, 以此类推

echo '<pre>';
print_r($stack);
echo '</pre>';

$stack->rewind(); // 双向链表的rewind和堆栈的rewind相反, 堆栈的rewind是的当前指针指向To
p所在的位置, 而双向链表调用之后指向bottom所在位置
echo 'current: ' . $stack->current() . '
';

$stack->next(); // 堆栈的next操作使指针靠近Bottom位置的下一个节点, 而双向链表是靠近top的下
一个节点
echo 'current: ' . $stack->current() . '
';

echo '<hr />';

// 遍历堆栈
$stack->rewind();
while($stack->valid()){
 echo $stack->key().">". $stack->current() . "
";
 $stack->next(); // next操作不从链表中删除元素
}

// 删除堆栈数据
$popObj = $stack->pop(); // pop操作从堆栈里面提取出最后一个元素 (top位置), 同时在堆栈
里面删除该节点
```



```
echo "Popped object: " . $popObj . "
";

echo '<pre>';
print_r($stack);
echo '</pre>';
```

打印内容为

```
SplStack Object
(
 [flags:SplDoublyLinkedList:private] => 6
 [dllist:SplDoublyLinkedList:private] => Array
 (
 [0] => a
 [1] => b
 [2] => c
)
)
```

Bottom: a

Top: c

```
SplStack Object
(
 [flags:SplDoublyLinkedList:private] => 6
 [dllist:SplDoublyLinkedList:private] => Array
 (
 [0] => a
 [1] => b
 [2] => C
)
)
```

```
)

)
```

current: C

current: b

////////////////////////////////////

2=>C

1=>b

0=>a

Poped object: C

```
SplStack Object
(
 [flags:SplDoublyLinkedList:private] => 6
 [dllist:SplDoublyLinkedList:private] => Array
 (
 [0] => a
 [1] => b
)
)
```

## 队列 (SplQueue)

测试代码为

```
<?php

$obj = new SplQueue();

$obj->enqueue('a'); // 插入一个节点到队列里面的Top位置
$obj->enqueue('b');
$obj->enqueue('c');

echo '<pre>';
print_r($obj);
echo '</pre>';

echo "Bottom: " . $obj->bottom() . "
";
echo "Top: " . $obj->top() . "
";

$obj->offsetSet(0, 'A'); // 队列里面offset=0Bottom所在位置, offset=1是Top所在位置, 以此类推

echo '<pre>';
print_r($obj);
echo '</pre>';

$obj->rewind(); // 队列里面rewind操作是指针指向Bottom所在位置的节点
echo 'current: ' . $obj->current() . '
';

$obj->rewind();
while($obj->valid()){
 echo $obj->key().">". $obj->current() . "
";
 $obj->next(); // next操作是是当前指针指向top方向的下一个节点
}

echo "dequeue obj: " . $obj->dequeue() . "\n"; // dequeue操作从队列中提取bottom位置的节点, 并返回, 同时从队列里面删除该元素
```

```
echo '<pre>';
print_r($obj);
echo '</pre>';
```

打印内容为

```
SplQueue Object
(
 [flags:SplDoublyLinkedList:private] => 4
 [dllist:SplDoublyLinkedList:private] => Array
 (
 [0] => a
 [1] => b
 [2] => c
)
)
```

Bottom: a

Top: c

```
SplQueue Object
(
 [flags:SplDoublyLinkedList:private] => 4
 [dllist:SplDoublyLinkedList:private] => Array
 (
 [0] => A
 [1] => b
 [2] => c
)
)
```

```
)
```

current: A

0=>A

1=>b

2=>c

dequeue obj: A

```
SplQueue Object
(
 [flags:SplDoublyLinkedList:private] => 4
 [dllist:SplDoublyLinkedList:private] => Array
 (
 [0] => b
 [1] => c
)
)
```

The End.

[前一篇](#): 一个简单的对于Mysql的PDO操作类（PHP）

[后一篇](#): 风决定要走，云怎么挽留



smallyu

Hello World

??

??

????

2017080 (6)

2017070 (2)

2017060 (3)

2017050 (2)

2017040 (2)

2017030 (3)

2017020 (3)

2017010 (4)

20160120 (1)

## 一个简单的对于Mysql的PDO操作类 PHP

□ smallyu □ 2017-03-03 □ PHP PDO

类结构本身稍微混乱 方法没有规章 这是缺点。

PDOMysql.class.php

```
<?php
```

```
define("DB_HOST", "localhost");
define("DB_USER", 'root');
define("DB_PWD", 'root');
define("DB_NAME", 'minyy');
define("DB_PORT", '3306');
define("DB_TYPE", 'mysql');
define("DB_CHARSET", 'utf8');
```

```
class PDOMySQL{
```

```
 public static $config = array(); // 设置连接参数 配置信息
 public static $link = null; // 保存连接标志符
 public static $pconnect = false; // 是否开启长连接
 public static $dbVersion = null; // 保存数据库版本
 public static $connected = false; // 是否连接成功
 public static $PDOStatement = null; // 保存PDOStatement对象
 public static $queryStr = null; // 保存最后执行的操作
 public static $error = null; // 报错错误信息
 public static $lastInsertId = 0; // 保存上一步插入操作产生AUTO_INCREMENT
 public static $numRows = 0; // 上一步操作产生受影响的记录条数
```

20160110 (2)

20160109 (3)

20160901 (1)

20160801 (6)

20160701 (1)

20160601 (4)

20160501 (4)

```
/**
 * 构造函数实现连接数据库
 * @param array $dbConfig 配置项参数
 */
public function __construct($dbConfig = ''){
 if(!class_exists("PDO")){
 self::throw_exception('不支持PDO 请先开启');
 }
 if(!$dbConfig || !is_array($dbConfig)){
 $dbConfig = array(
 'hostname' => DB_HOST,
 'username' => DB_USER,
 'password' => DB_PWD,
 'database' => DB_NAME,
 'hostport' => DB_PORT,
 'dbms' => DB_TYPE,
 'dsn' => DB_TYPE.':host="'.DB_HOST.'";dbname="'.DB_NAME,
);
 }
 if(empty($dbConfig['hostname'])) self::throw_exception('没有定义数据库配置')
;

 self::$config = $dbConfig;
 if(empty(self::$config['params'])) self::$config['params'] = array();
 if(!isset(self::$link)){
 $configs = self::$config;
 if(self::$pconnect){
 // 开启长连接 添加到配置数组中
 $configs['params'][constant("PDO::ATTR_PERSISTENT")] = true;
 }
 try{
 self::$link = new PDO($configs['dsn'], $configs['username'], $con
figs['password'], $configs['params']);
```

```

 } catch (PDOException $e) {
 self::throw_exception($e->getMessage());
 }
 if (!self::$link) {
 self::throw_exception('PDO连接错误');
 return false;
 }
 self::$link->exec('SET NAMES '.DB_CHARSET);
 self::$dbVersion = self::$link->getAttribute(constant("PDO::ATTR_SERVER_VERSION"));
 self::$connected = true;
 unset($configs);
 }
}

/**
 * 得到所有记录
 * @param string $sql SQL语句
 * @return array 得到所有数据
 */
public static function getAll($sql = null) {
 if ($sql) {
 self::query($sql);
 }
 $result = self::$PDOStatement->fetchAll(constant("PDO::FETCH_ASSOC"));
 return $result;
}

/**
 * 得到结果集中的一条记录
 * @param string $sql sql语句
 * @return array 返回一条数据
 */
public static function getRow($sql = null) {

```



```

 if($sql != null){
 self::query($sql);
 }
 $result = self::$PDOStatement->fetch(constant("PDO::FETCH_ASSOC"));
 return $result;
 }

 /**
 * 根据主键查找记录
 * @param string $tableName 表名
 * @param string $priId 主键id
 * @param string $field 查询项
 * @return array 一条记录
 */
 public static function findById($tableName, $priId, $field=''){
 $sql = "SELECT * FROM %s WHERE id=%d";
 return self::getRow(sprintf($sql, self::parseField($field), $tableName, $priId));
 }

 /**
 * 执行普通查询
 * @param string $tables 查询表名
 * @param string $where where
 * @param string $fields 要查询字段
 * @param string $group group by
 * @param string $having having
 * @param string $order order
 * @param string $limit limit
 * @return array [description]
 */
 public static function find($tables, $where = null, $fields = '*', $group = null, $having = null, $order = null, $limit = null){
 $sql = 'SELECT ' . self::parseField($fields) . ' FROM ' . $tables

```

```

 .self::parseWhere($where)
 .self::parseGroup($group)
 .self::parseHaving($having)
 .self::parseOrder($order)
 .self::parseLimit($limit);
 $dataAll = self::getAll($sql);

 return count($dataAll)==1 ? $dataAll[0] : $dataAll;
}

/**
 * 添加记录的操作
 * @param array $data 要添加的内容
 * @param int $table 受影响的条数
 */
public static function add($data, $table){
 $keys = array_keys($data);
 array_walk($keys,array('PDOMySQL','addSpecilChar'));
 $fieldsStr=join(',',$keys);
 $values="".join("','",array_values($data))."";
 $sql = "INSERT {$table}({$fieldsStr}) VALUES({$values})";
 return self::execute($sql);
}

/**
 * 更新记录
 * @param array $data 要更新内容
 * @param string $table 表名
 * @param string $where 条件
 * @param string $order 排序
 * @param string $limit 限制
 * @return int 受影响条数
 */
public static function update($data, $table, $where = null, $order = null, $l

```

```

 limit = null){
 $sets = '';
 foreach ($data as $key => $val) {
 $sets .= $key."='".$val."'";
 }
 $sets = rtrim($sets,',');
 $sql = "UPDATE {$table} SET {$sets} ".self::parseWhere($where).self::parseOrder($order).self::parseLimit($limit);
 return self::execute($sql);
 }

 /**
 * 解析字段
 * @param string $fields 字段
 * @return string 解析后的字段
 */
 public static function parseField($fields){
 if(is_array($fields)){
 array_walk($fields, array('PDOMySQL', 'addSpecilChar'));
 $fieldsStr = implode(',',$fields);
 }elseif(is_string($fields) && !empty($fields)){
 if(strpos($fields, '`') === false){
 $fields = explode(',', $fields);
 array_walk($fields, array('PDOMySQL', 'addSpecilChar'));
 $fieldsStr = implode(',',$fields);
 }else{
 $fieldsStr = $fields;
 }
 }else{
 $fieldsStr = '*';
 }
 return $fieldsStr;
 }
 }

```

```

/**
 * 删除操作
 * @param string $table 表名
 * @param string $where 条件
 * @param string $order 排序
 * @param integer $limit 限制
 * @return integer 受影响条数
 */
public static function delete($table, $where=null, $order=null, $limit=0){
 $sql = "DELETE FROM {$table} ".self::parseWhere($where).self::parseOrder(
$order).self::parseLimit($limit);
 return self::execute($sql);
}

/**
 * 得到最后执行的sql语句
 * @return string sql语句
 */
public static function getLastSql(){
 $link=self::$link;
 if(!$link) return false;
 return self::$queryStr;
}

/**
 * 得到插入操作产生的AUTO_INCREMENT
 * @return int 被操作id
 */
public static function getLastInsertId(){
 $link=self::$link;
 if(!$link) return false;
 return self::$lastInsertId;
}

```

```

/**
 * 得到数据库的版本
 * @return float 数据库版本
 */
public static function getDbVersion(){
 $link=self::$link;
 if(!$link) return false;
 return self::$dbVersion;
}

/**
 * 得到数据库中的数据表
 * @return array 所有数据表
 */
public static function showTbales(){
 $tables = [];
 if(self::query("SHOW TABLES")){
 $result = self::getAll();
 foreach ($result as $key => $value) {
 $tables[$key]=current($value);
 }
 }
 return $tables;
}

/**
 * 解析where字段
 * @param string $where 字段
 * @return string 解析后的字段
 */
public static function parseWhere($where){
 $whereStr = '';
 if(is_string($where) && !empty($where)){
 $whereStr = $where;
 }
}

```

```

 }
 return empty($whereStr) ? '' : ' WHERE '.$whereStr;
}

/**
 * 解析分组字段group by
 * @param string $group 字段
 * @return string 解析后的字段
 */
public static function parseGroup($group){
 $groupStr = '';
 if(is_array($group)){
 $groupStr .= ' GROUP BY '.implode(',', $group);
 }elseif(is_string($group) && !empty($group)){
 $groupStr .= ' GROUP BY '.$group;
 }
 return empty($groupStr) ? '' : $groupStr;
}

/**
 * 对分组进行二次筛选
 * @param string $having 字段
 * @return string 解析后的字段
 */
public static function parseHaving($having){
 $havingStr = '';
 if(is_string($having) && !empty($having)){
 $havingStr = ' HAVING '.$having;
 }
 return $havingStr;
}

/**
 * 解析order by

```

```

* @param string $order 字段
* @return string 解析后的字段
*/
public static function parseOrder($order){
 $orderStr = '';
 if(is_array($order)){
 $orderStr .= ' ORDER BY '.join(',',$order);
 }elseif(is_string($order) && !empty($order)){
 $orderStr .= ' ORDER BY '.$order;
 }
 return $orderStr;
}

/**
* 解析limit
* @param string $limit 字段
* @return string 解析后的字段
*/
public static function parseLimit($limit){
 $limitStr = '';
 if(is_array($limit)){
 if(count($limit) > 1){
 $limitStr = ' LIMIT '.$limit[0].','.$limit[1];
 }else{
 $limitStr = ' LIMIT '.$limit[0];
 }
 }elseif(!empty($limit)){
 $limitStr .= ' LIMIT '.$limit;
 }
 return $limitStr;
}

/**
* 通过反引号引用字段

```

```

 * @param string &$value 不同类型字段
 */
 public static function addSpecilChar(&$value){
 if($value === '*' || strpos($value, '.') != false || strpos($value, '`')
!= false){
 // 不做处理
 }elseif(strpos($value, '`') === false){
 $value = '`'.trim($value).`.`;
 }
 return $value;
 }

 /**
 * 执行增删改操作 返回受影响的记录条数
 * @param string $sql sql语句
 * @return int 受影响条数
 */
 public static function execute($sql = null){
 $link = self::$link;
 if(!$link) return false;
 self::$queryStr = $sql;
 if(!empty(self::$PDOStatement)) self::free();
 $result = $link->exec(self::$queryStr);
 self::haveErrorThrowException();
 if($result){
 self::$lastInsertId = $link->lastInsertId();
 self::$numRows = $result;
 return self::$numRows;
 }else{
 return false;
 }
 }

 /**

```



```

* 释放结果集
* @return null null
*/
public static function free(){
 self::$PDOStatement = null;
}

public static function query($sql = ''){
 $link = self::$link;
 if(!$link) return false;
 // 判断之前是否有结果集 如果有 释放结果集
 if(!empty(self::$PDOStatement)) self::free();
 self::$queryStr = $sql;
 self::$PDOStatement = $link->prepare(self::$queryStr);
 $res = self::$PDOStatement->execute();
 self::haveErrorThrowException();
 return $res;
}

public static function haveErrorThrowException(){
 $obj = empty(self::$PDOStatement) ? self::$link : self::$PDOStatement;
 $arrError = $obj->errorInfo();
 if($arrError[0] != '00000'){
 self::$error = 'SQLStatus: '.$arrError[0].'
>SQLError: '.$arrError[2].'
>Error SQL: '.self::$queryStr;
 self::throw_exception(self::$error);
 return false;
 }
 if(self::$queryStr == ''){
 self::throw_exception('没有执行的SQL语句');
 return false;
 }
}

```

```

/**
 * 自定义错误处理
 * @param string $errMsg 错误信息
 * @return null null
 */
public static function throw_exception($errMsg){
 echo $errMsg;
}

/**
 * 销毁连接对象 关闭数据库
 * @return null null
 */
public static function close(){
 self::$link = null;
}
}

```

在场景中这样来使用 注释后的内容分别用来测试不同方法

index.php

```

<?php

$PDOMySQL = new PDOMySQL();

// 查询全部记录
$sql = 'SELECT * FROM cheat_exam';
$res = $PDOMySQL->getAll($sql);

// 查询一条记录
$sql = 'SELECT * FROM cheat_exam WHERE id=1';
$res = $PDOMySQL->getRow($sql);

```

```
// 插入一条记录
$sql = 'INSERT cheat_exam(name) VALUES (1)';
$res = $PDOMySQL->execute($sql);
echo $PDOMySQL::$lastInsertId;

// 删除一条记录
$sql = 'DELETE FROM cheat_exam WHERE id=115';
$res = $PDOMySQL->execute($sql);

// 更新一条记录
$sql = 'UPDATE arr SET name="1" WHERE id=1';
$res = $PDOMySQL->execute($sql);

// 根据主键查询一条记录
$tabName = 'cheat_exam';
$priId = 1;
$fields='subject,name';
$fields = array('subject','name');
$res = $PDOMySQL->findById($tabName, $priId, $fields);

// 执行普通查询
$tables = 'cheat_exam';
$res = $PDOMySQL->find($tables);
$res = $PDOMySQL->find($tables, 'id>=30');
$res = $PDOMySQL->find($tables, 'id>=30', 'subject,name');
$res = $PDOMySQL->find($tables, 'id>=30', '*', 'subject');
$res = $PDOMySQL->find($tables, 'id>=30', '*', 'subject', 'count(*)>=8');
$res = $PDOMySQL->find($tables, 'id>=30', '*', '', '', 'id desc');
$res = $PDOMySQL->find($tables, 'id>=30', '*', '', '', 'id desc', 5);

// 更新一条记录
$data = [
 'subject' => 'imooc',
```

```
 'name' => 'name'
];
 $table = "cheat_exam";
 $res = $PDOMySQL->add($data, $table);
 // 也是更新操作
 $data = [
 'subject' => 'imooc',
 'name' => 'name'
];
 $table = "cheat_exam";
 $res = $PDOMySQL->update($data, $table, 'id<=3', 'id desc', 5);

 // 删除操作
 $table = "cheat_exam";
 $res = $PDOMySQL->delete($table, 'id<3');

 // 得到数据表
 $res = $PDOMySQL->showTbales();

 print_r($res);
```

会打印出不同结果。

The End.

[前一篇](#): [PHP-GTK初体验](#) [用最简单的方式运行环境](#) [并实现Hello World](#)

[后一篇](#): [站在巨人的肩膀上写代码——Spl中的三种数据结构解析](#)



smallyu

Hello World

??

??

????

2017080 (6)

2017070 (2)

2017060 (3)

2017050 (2)

2017040 (2)

2017030 (3)

2017020 (3)

2017010 (4)

20160120 (1)

## PHP-GTK初体验：用最简单的方式运行环境，并实现Hello World

□ smallyu   □ 2017-02-22   □ GTK

首先用最简单的方式把php-gtk运行起来，在gtk扩展官网（<http://gtk.php.net/>）下载Binaries版本：

点击链接直接下载：<http://gtk.php.net/distributions/PHP55-GTK2.zip>（PHP-GTK-2.0.1 PHP 5.5）

官方文档提供了三种安装扩展的方式，

1. 使用Gnope
2. 手动安装
3. 使用CVS编译

事实上我们在下载.zip压缩包后解压到任意文件夹下，不需要任何配置或者编译行为。（如果只是先体验一下GTK实现的GUI窗口）

一段简单的Hello World脚本

在任意目录下新建一个文件，后缀名为.phpw，比如hello.phpw，编辑文件，写入以下代码：

```
<?php
if (!class_exists('gtk')) {
```

|              |
|--------------|
| 20160111 (2) |
| 20160110 (3) |
| 20160909 (1) |
| 20160808 (6) |
| 20160707 (1) |
| 20160606 (4) |
| 20160505 (4) |

```

 die("Please load the php-gtk2 module in your php.ini\r\n");
 }

 $wnd = new GtkWindow();
 $wnd->set_title('Hello world');
 $wnd->connect_simple('destroy', array('gtk', 'main_quit'));

 $lblHello = new GtkLabel("Just wanted to say\r\n'Hello world!'");
 $wnd->add($lblHello);

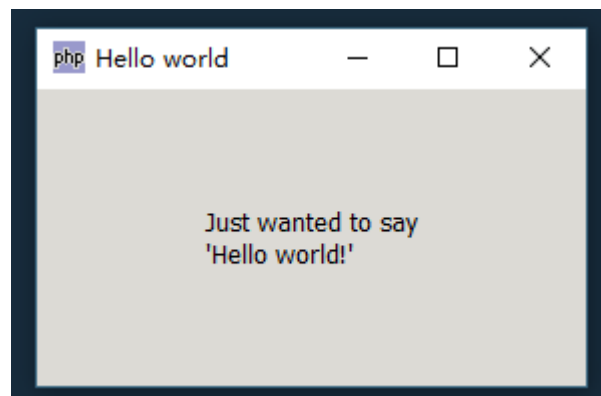
 $wnd->show_all();
 Gtk::main();
 ?>

```

保存后，右击文件>选择打开方式>其它应用>在这台电脑上浏览应用，选择我们下载并解压好的文件夹下的 php-win.exe 来执行文件，

|                                                                                                            |                 |                  |          |
|------------------------------------------------------------------------------------------------------------|-----------------|------------------|----------|
|  php.exe                  | 2014/3/6 0:32   | 应用程序             | 64 KB    |
|  php.gif                  | 2014/3/6 0:32   | GIF 文件           | 3 KB     |
|  php.ini                  | 2014/4/6 7:40   | 配置设置             | 72 KB    |
|  php.ini-development      | 2014/3/6 0:32   | INI-DEVELOPME... | 70 KB    |
|  php5.dll               | 2014/3/6 0:32   | 应用程序扩展           | 5,919 KB |
|  php-cgi.exe            | 2014/3/6 0:32   | 应用程序             | 51 KB    |
|  <b>php-win.exe</b>     | 2014/3/6 0:32   | 应用程序             | 27 KB    |
|  readme-redist-bins.txt | 2014/3/6 0:32   | TXT 文件           | 21 KB    |
|  snapshot.txt           | 2014/3/6 0:32   | TXT 文件           | 4 KB     |
|  ssleay32.dll           | 2014/3/6 0:32   | 应用程序扩展           | 264 KB   |
|  unins000.dat           | 2015/5/25 14:30 | DAT - MPEG 电...  | 76 KB    |
|  unins000.exe           | 2015/5/25 13:49 | 应用程序             | 705 KB   |

就可以得到这样的运行结果：



检查**PHP-GTK**是否可用

```
if (!class_exists('gtk')) {
 die("Please load the php-gtk2 module in your php.ini\r\n");
}
```

这里我们通过检查`gtk`类是否存在来确定**PHP-GTK**是否可用。不像**PHP 4**和**PHP-GTK 1**，你不应该通过`d1()`尝试加载**PHP-GTK**模块，因为这在**PHP 5**中是过时的，不应该用于新代码的编写。所以我们打印一个信息，如果模块不可用，就由用户来启用它。

创建一个窗口

```
$wnd = new GtkWindow();
```

创建一个窗口就像实例化一个新的对象一样简单：变量`$wnd`得到了新对象的引用。如果你已经使用**PHP 4**和**PHP-GTK 1**，你可以在关键词`new`之前不使用`&`符号，因为在**PHP 5**中它会被自动创建。

设置窗口标题

```
$wnd->set_title('Hello world');
```

为了使窗口在任务栏中更容易被认出，我们设置了标题，仅仅调用了对象的一个方法。

使关闭（操作）变得干净

```
$wnd->connect_simple('destroy', array('gtk', 'main_quit'));
```

这是代码中第一段稍有意思的部分：参数`'destroy'`传递到了静态方法`Gtk::main_quit`中。它告诉GTK当窗口被关闭或者销毁的时候退出主循环。（后面会提到关于主循环的内容）

创建一个展示文本的标签

```
$lblHello = new GtkLabel("Just wanted to say\r\n'Hello world!'");
```

和上面一样，我们创建了一个新的部件。这次我们想要展示一小段文本，`GtkLabel`类可以完美的实现这个任务。

添加标签到窗口

```
$wnd->add($lblHello);
```

我们直接把标签添加到了窗口。

设置窗口为可见的

```
$wnd->show_all();
```



到现在，我们在屏幕上看不见任何内容。为了使窗口可见，必须调用`show()`方法，它将会使窗口可见，但不是标签——我们在标签之上（也就是窗口对象上）调用`show()`。如果你有多个控件比如按钮、复选框或者其它窗口部件不方便一一调用，可以使用`show_all()`方法来显示所有可见的部件。

开启主循环

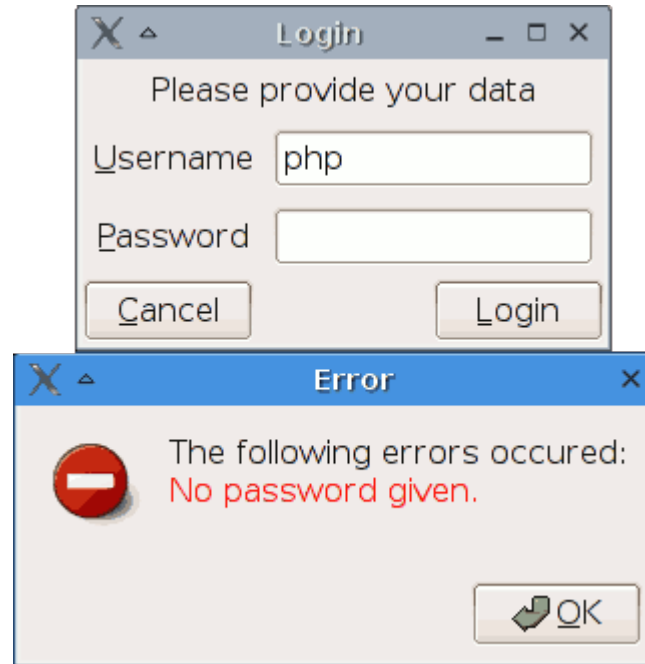
```
Gtk::main();
```

在设置完所有配置项之后，我们需要确定窗口停留在打开状态并且会对用户的行为进行反馈。一段普通的PHP脚本在文件执行结束后会停止，因此需要启动GTK主循环。它会保持应用处于运行状态并且会回应用户事件。如果事件发生，它会执行相应的回调。

（原内容：<http://gtk.php.net/manual/en/html/tutorials/tutorials.helloworld.html>）

更加先进的**Hello World**脚本

下面的参考内容会比上面的hello world脚本实现更多的功能。这里我们创建了一个登陆框，可以由用户来输入用户名和密码。当用户点击登陆，如果用户名或者密码为空，文本域可以检查错误并显示。



首先我们先创建一个窗口，设置标题并设置当窗口销毁停止主循环（上面已经实现过）：

```
$wnd = new GtkWindow();
$wnd->set_title('Login');
$wnd->connect_simple('destroy', array('gtk', 'main_quit'));
```

接下来我们的任务是创建所有要显示的部件，先是两个文本域用户名和密码：

```
$txtUsername = new GtkEntry();
$txtPassword = new GtkEntry();
```

我们要分别给两个文本域一个描述的标签：

```
// 第二个参数意思是把下划线解析为下划线
$lblUsername = new GtkLabel('_Username', true);
$lblPassword = new GtkLabel('_Password', true);
```

最后我们需要两个按钮，登录和取消：

```
$btnLogin = new GtkButton('_Login');
$btnCancel = new GtkButton('_Cancel');
```

你可能注意到`GtkLabel`类的构造函数中通过第二个参数设置解析下划线\_为一个字符，但是`GtkButton`类中并没有，因为它会自动设置，这一点是需要记忆的。

现在标签需要知道哪个行为被激活时使用哪个相应的内容，这样来做：

```
$lblUsername->set_mnemonic_widget($txtUsername);
$lblPassword->set_mnemonic_widget($txtPassword);
```

然后我们要决定哪个按钮被点击对应哪个事件。取消按钮会关闭窗口，结束程序，登录按钮会检查文本域是否为空等一系列操作：

```
$btnCancel->connect_simple('clicked', array($wnd, 'destroy'));
$btnLogin ->connect_simple('clicked', 'login', $wnd, $txtUsername, $txtPassword);
```

我们把三个参数`$wnd`、`$txtUsername`和`$txtPassword`传递给了登录函数，接下来的操作会用它们从文本域获取值，并且在所有工作结束后关闭窗口。

现在我们创建了我们需要的所有按钮，它们被添加到了窗口。`GtkWindow`库是功能容器，它可以捆绑为一个单独的部件，所以我们需要这个容器来更好的展示我们的其它元素。这里决定使用`GtkTable`，它可以保

持标签在一列，紧随其后的是确定域：

```
$tbl = new GtkTable(3, 2);
$tbl->attach($lblCredit, 0, 2, 0, 1);
$tbl->attach($lblUsername, 0, 1, 1, 2);
$tbl->attach($txtUsername, 1, 2, 1, 2);
$tbl->attach($lblPassword, 0, 1, 2, 3);
$tbl->attach($txtPassword, 1, 2, 2, 3);
```

（\$lblCredit 仅仅是一个消息标签），按钮有它们自己的 `GtkHButtonBox` 类，因为这个允许它们更美观的布局多个按钮：

```
$bbox = new GtkHButtonBox();
$bbox->set_layout(Gtk::BUTTONBOX_EDGE);
$bbox->add($btnCancel);
$bbox->add($btnLogin);
```

对于布局最后要做的事情是把表格和按钮添加到窗口。没有直接的方法，因为 `GtkWindow` 是一个 `GtkBin`（类库？），只能包含一个部件。所以我们需要另外的容器，这次用 `GtkVBox`：

```
$vbox = new GtkVBox();
$vbox->pack_start($tbl);
$vbox->pack_start($bbox);
```

它们所有都需要显示：

```
$wnd->add($vbox);
$wnd->show_all();
Gtk::main();
```

最后要做的事情，是当用户点击登陆按钮时候的回调事件。它需要检查用户名和密码是否为空，如果它们长度为0，则显示错误信息。如果一切ok，窗口将会关闭并且程序会继续运行来加载主窗口。

从部件获取文本很简单，我们用GtkEntry的get\_text()方法就能实现：

```
$strUsername = $txtUsername->get_text();
$strPassword = $txtPassword->get_text();
```

检查工作可以用平常的php函数。如果发生错误就显示一个带有错误消息的信息。GtkMessageDialog来完成这些工作是理想的，它可以自动创建图标和按钮（确认，是或否）：

```
$dialog = new GtkMessageDialog($wnd, Gtk::DIALOG_MODAL, Gtk::MESSAGE_ERROR, Gtk::
BUTTONS_OK, $errors);
$dialog->set_markup("The following errors occurred:\r\n" .
$errors . "");
$dialog->run();
$dialog->destroy();
```

现在可以确定你把登录功能函数放到了主循环中并使其运行。

完整代码

```
<?php
/**
 * Here we create a login window.
 * It has a username and a password field, and a
 * Cancel and Login button. Some error checking
 * is being done when the user clicks "Login".
 */
```

```

if (!class_exists('gtk')) {
 die("Please load the php-gtk2 module in your php.ini\r\n");
}

/**
 * This function gets called as soon as the user
 * clicks on the Login button.
 *
 * @param GtkWindow $wnd The login window, needed to close it
 * when all is ok
 * @param GtkEntry $txtUsername The username text field, used to get
 * the username
 * @param GtkEntry $txtPassword The password widget to retrieve the
 * password
 */
function login(GtkWindow $wnd, GtkEntry $txtUsername, GtkEntry $txtPassword)
{
 //fetch the values from the widgets into variables
 $strUsername = $txtUsername->get_text();
 $strPassword = $txtPassword->get_text();

 //Do some error checking
 $errors = null;
 if (strlen($strUsername) == 0) {
 $errors .= "Username is missing.\r\n";
 }
 if (strlen($strPassword) == 0) {
 $errors .= "No password given.\r\n";
 }

 if ($errors != null) {
 //There was at least one error.
 //We show a message box with the errors
 $dialog = new GtkMessageDialog($wnd, Gtk::DIALOG_MODAL,

```

```

 Gtk::MESSAGE_ERROR, Gtk::BUTTONS_OK, $errors);
 $dialog->set_markup(
 "The following errors occurred:\r\n"
 . "" . $errors . ""
);
 $dialog->run();
 $dialog->destroy();
 } else {
 //No error. You would need to hide the dialog now
 //instead of destroying it (because when you destroy it,
 //Gtk::main_quit() gets called) and show the main window
 $wnd->destroy();
 }
}

//Create the login window
$wnd = new GtkWindow();
$wnd->set_title('Login');
//Close the main loop when the window is destroyed
$wnd->connect_simple('destroy', array('gtk', 'main_quit'));

//Set up all the widgets we need
$lblCredit = new GtkLabel('Please provide your data');
//The second parameter says that the underscore should be parsed as underline
$lblUsername = new GtkLabel('_Username', true);
$lblPassword = new GtkLabel('_Password', true);
$txtUsername = new GtkEntry();
$txtPassword = new GtkEntry();
$btnLogin = new GtkButton('_Login');
$btnCancel = new GtkButton('_Cancel');

//Which widget should be activated when the

```

```
// mnemonic (Alt+U or Alt+P) is pressed?
$lblUsername->set_mnemonic_widget($txtUsername);
$lblPassword->set_mnemonic_widget($txtPassword);
//Hide the password
//$txtPassword->set_invisible_char('*');

//Destroy the window when the user clicks Cancel
$btnCancel->connect_simple('clicked', array($wnd, 'destroy'));
//Call the login function when the user clicks on Login
$btnLogin->connect_simple('clicked', 'login', $wnd, $txtUsername, $txtPassword);

//Lay out all the widgets in the table
$tbl = new GtkTable(3, 2);
$tbl->attach($lblCredit, 0, 2, 0, 1);
$tbl->attach($lblUsername, 0, 1, 1, 2);
$tbl->attach($txtUsername, 1, 2, 1, 2);
$tbl->attach($lblPassword, 0, 1, 2, 3);
$tbl->attach($txtPassword, 1, 2, 2, 3);

//Add the buttons to a button box
$bbox = new GtkHButtonBox();
$bbox->set_layout(Gtk::BUTTONBOX_EDGE);
$bbox->add($btnCancel);
$bbox->add($btnLogin);

//Add the table and the button box to a vbox
$vbox = new GtkVBox();
$vbox->pack_start($tbl);
$vbox->pack_start($bbox);

//Add the vbox to the window
```



```
$wnd->add($vbox);
//Show all widgets
$wnd->show_all();
//Start the main loop
Gtk::main();
?>
```

(原内容: <http://gtk.php.net/manual/en/html/tutorials/tutorials.helloadvanced.html>)

The End.

上一篇: [PHP写简单爬虫抓取太工教务处通告中的, 考试作弊学生名单](#)

下一篇: [一个简单的对于MySQL的PDO操作类 \(PHP\)](#)

Copyright Hower'Blog Rights Reserved. . Powered By [Z-BlogPHP](#). theme by [????](#)



smallyu

Hello World

??

??

????

2017080 (6)

2017070 (2)

2017060 (3)

2017050 (2)

2017040 (2)

2017030 (3)

2017020 (3)

2017010 (4)

20160120 (1)

## PHP写简单爬虫抓取太工教务处通告中的，考试作弊学生名单

□ smallyu    □ 2017-02-19    □ PHP 爬虫

目标是太原工业学院（tit）教务处（<http://jwc.tit.edu.cn>）通告栏信息中考试作弊学生的名单。

过程中涉及到的内容有：

- curl获取页面内容
- 正则表达式匹配目标内容
- 相关字符串处理函数整理数据
- mysql数据库基本操作

首先打开教务处最新一条考试作弊通告，查看页面源代码，内容类似于：



可以获取到两个信息：

2. 页面内容太乱了。

第二个信息是页面内容比较乱，主要是太多html标签+样式覆盖了内容。（吐槽一下教务处系统的开发者？）

接下来开始写代码了。

一、curl请求一共需要四个步骤：

1. 初始化会话请求
2. 设置请求选项，包括具体的url
3. 执行一个curl会话并且获得相关回复
4. 释放curl句柄，关闭一个curl会话

涉及到的四个函数分别为：

```
curl_init();
curl_setopt();
curl_exec();
curl_close();
```

略过一些内容后就可以拿到以字符串形式返回的包含页面内容的变量\$response。

二、然后写正则表达式结合一些函数来分离出所需信息，涉及到的函数有：

```
preg_replace();
```

好像就这一个，功能很强大。关于正则表达式可以用这个在线工具：<http://regexpal.isbadguy.com/>

三、再然后整理数据，也就使用这个函数：

```
substr();
```

以数组形式返回整齐的数据。

四、最后就要把数据储存在mysql，表的结构是：（资源小浪费）

| Field   | Type            | Null | Key | Default | Extra          |
|---------|-----------------|------|-----|---------|----------------|
| id      | int(6) unsigned | NO   | PRI | NULL    | auto_increment |
| stuId   | int(11)         | YES  |     | NULL    |                |
| date    | date            | YES  |     | NULL    |                |
| subject | tinytext        | YES  |     | NULL    |                |
| name    | tinytext        | YES  |     | NULL    |                |

基本步骤之后可以拿到包含一个页面所需数据的数组。最终执行需要重复执行这些基本步骤很多（3013）次。

过程中用ob缓存来实时输出内容反馈到浏览器：

```
ob_flush();
flush();
```

完整代码

```
<?php

set_time_limit(0); // 设置PHP最长执行时间

$times = 3013;
while($times > 0){
 $url = "http://jwc.tit.edu.cn/info/news/content/".$times.".htm";
 echo $url."
";
 $res = curl($url);
 if($res){
```

```

 $data = getData($res);
 if($data){
 $data = infoArr($data);
 saveData($data);
 }
 }else{
 $times++;
 }
 ob_flush(); // 从缓存中输出内容
 flush(); // 刷出缓存内容到浏览器
 sleep(1); // 延时1s执行
 $times--;
}

function curl($url = "http://www.baidu.com"){
 // 1.初始化会话请求
 $ch = curl_init();
 // 2.设置请求选项, 包括具体的url
 curl_setopt($ch, CURLOPT_URL, $url); // 需要获取的 URL 地址, 也可以在curl_init
 ()初始化会话的时候
 curl_setopt($ch, CURLOPT_RETURNTRANSFER, 1); // TRUE 将curl_exec()获取的信息
 以字符串返回, 而不是直接输出
 curl_setopt($ch, CURLOPT_HEADER, 0); // 启用时会将头文件的信息作为数据流输出 (到
 页面)
 curl_setopt($ch, CURLOPT_CONNECTTIMEOUT, 30); // 连接超时时长
 curl_setopt($ch, CURLOPT_TIMEOUT, 30); // 执行超时时长
 curl_setopt($ch, CURLOPT_HTTP_VERSION, CURL_HTTP_VERSION_1_0); //强制协议为1.0
 curl_setopt($ch, CURLOPT_HTTPHEADER, array("Expect: ")); //头部要送出'Expect: '
 curl_setopt($ch, CURLOPT_IPRESOLVE, CURL_IPRESOLVE_V4); //强制使用IPV4协议解析
 域名
 // 3.执行一个curl会话并且获得相关回复
 $response = curl_exec($ch);
 if($response === false){
 return 0;
 }
}

```

```
}
// 4.释放curl句柄, 关闭一个curl会话
curl_close($ch);

return $response;
}

function getData($response = ''){

 // 过滤html标签
 $replace_pattern = '/<[^>]*>/x';
 $str_replace = preg_replace($replace_pattern, '', $response);

 // 获取日期
 $dates = [];
 $date_pattern = '/\d{1,4}年\d{1,2}月\d{1,2}日/u';
 preg_match_all($date_pattern, $str_replace, $dates);

 if(empty($dates[0])){
 return 0;
 }

 foreach ($dates as $value) {
 foreach ($value as $v) {
 $v = str_replace(array('年','月','日'), array('-', '-', ''), $v);
 $date[] = $v;
 }
 }

 // 获取科目
 $subjects = [];
 $subject_pattern = '/《[^》|太)]*》/u';
 preg_match_all($subject_pattern, $str_replace, $subjects);
```

```
if(empty($subjects[0])){
 return 0;
}

foreach ($subjects as $value) {
 foreach ($value as $v) {
 $subject[] = $v;
 }
}

// 获取学号姓名
$students = [];
$students_pattern = '/\d{7,9}(班)?\d{0,2}(号)?[^\d\w\s,)]{6,9}/x';
preg_match_all($students_pattern, $str_replace, $students);

if(empty($students[0])){
 return 0;
}

$students=$students[0];

foreach($students as $info){
 $info = str_replace(array('班','号','在','\、'), '', $info);
 $stuId[] = substr($info, 0, 9);
 $name[] = substr($info, 9, 11);
}

$data['date'] = $date;
$data['subject'] = $subject;
$data['stuId'] = $stuId;
$data['name'] = $name;

return $data;
}
```



```
function infoArr($data = []){
 $dateNum = count($data['date']);
 $subNum = count($data['subject']);
 $idNum = count($data['stuId']);
 $nameNum = count($data['name']);

 // 如果日期数目为1, 其它不为1
 if($dateNum == 1){
 for($i = 0; $i < $idNum - 1; $i++){
 $data['date'][$i+1] = $data['date'][$i];
 }
 }

 // 如果课程数为1, 人数不为1
 if($subNum == 1 && $idNum != 1){
 for($i = 0; $i < $idNum - 1; $i++){
 $data['date'][$i+1] = $data['date'][$i];
 $data['subject'][$i+1] = $data['subject'][$i];
 }
 }

 // 如果课程数大于1, 人数大于课程数
 if($subNum > 1 && $idNum > $subNum){
 for($i = 0; $i < $idNum-1; $i++){
 if(substr($data['stuId'][$i], 0, 5) == substr($data['stuId'][$i+1], 0
, 5)){
 for($j = $idNum-1; $j > $i; $j--){
 $data['subject'][$j] = $data['subject'][$j-1];
 $data['date'][$j] = $data['date'][$j-1];
 }
 }
 }
 }

 // 暂时忽略其它情况
```

```

 return $data;
 }

function saveData($data = []){

 $mysql_host = 'localhost';
 $mysql_username = 'root';
 $mysql_password = 'root';
 $mysql_db = 'minyy';
 $mysql_table = 'cheat_exam';

 $link = mysql_connect($mysql_host, $mysql_username, $mysql_password);
 mysql_select_db($mysql_db, $link);
 mysql_query("set names utf8");
 $sql = "CREATE TABLE IF NOT EXISTS $mysql_table(
 id int(6) unsigned NOT NULL AUTO_INCREMENT,
 PRIMARY KEY(id),
 stuId int(11),
 date date,
 subject tinytext,
 name tinytext
) ENGINE=MyISAM DEFAULT CHARSET=utf8 AUTO_INCREMENT=1";
 mysql_query($sql);

 $times = count($data['stuId']);

 for($i = 0; $i < 1; $i++){

 $stuId = $data['stuId'][$i];
 $date = ''. $data['date'][$i].'';
 $subject = ''. $data['subject'][$i].'';
 $name = ''. $data['name'][$i].'';
 }
}

```

```
$sql = "SELECT * FROM $mysql_table WHERE stuId={$data['stuId'][$i]}";
$res = mysql_fetch_array(mysql_query($sql));
if(!$res){
 $sql = "INSERT INTO $mysql_table (stuId, date, subject, name) VALUES
($stuId, $date, $subject, $name)";
 mysql_query($sql);
}
}
return 1;
}
```

事实上是比较简单的一段小程序。

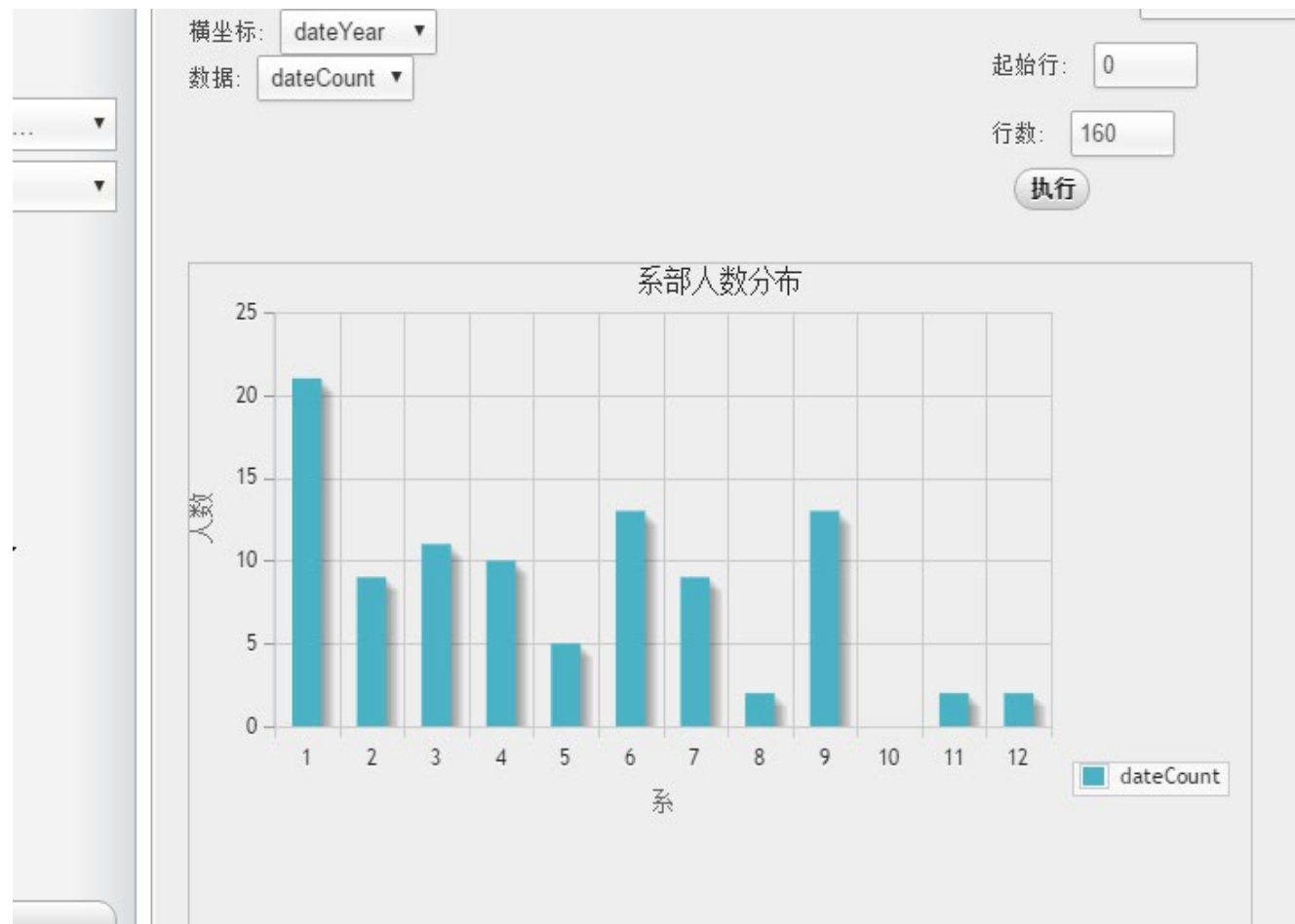
执行后的页面会不断显示出一个一个网址。（忘了截图）

（由于网速慢）程序跑了很多（10? ）个小时。

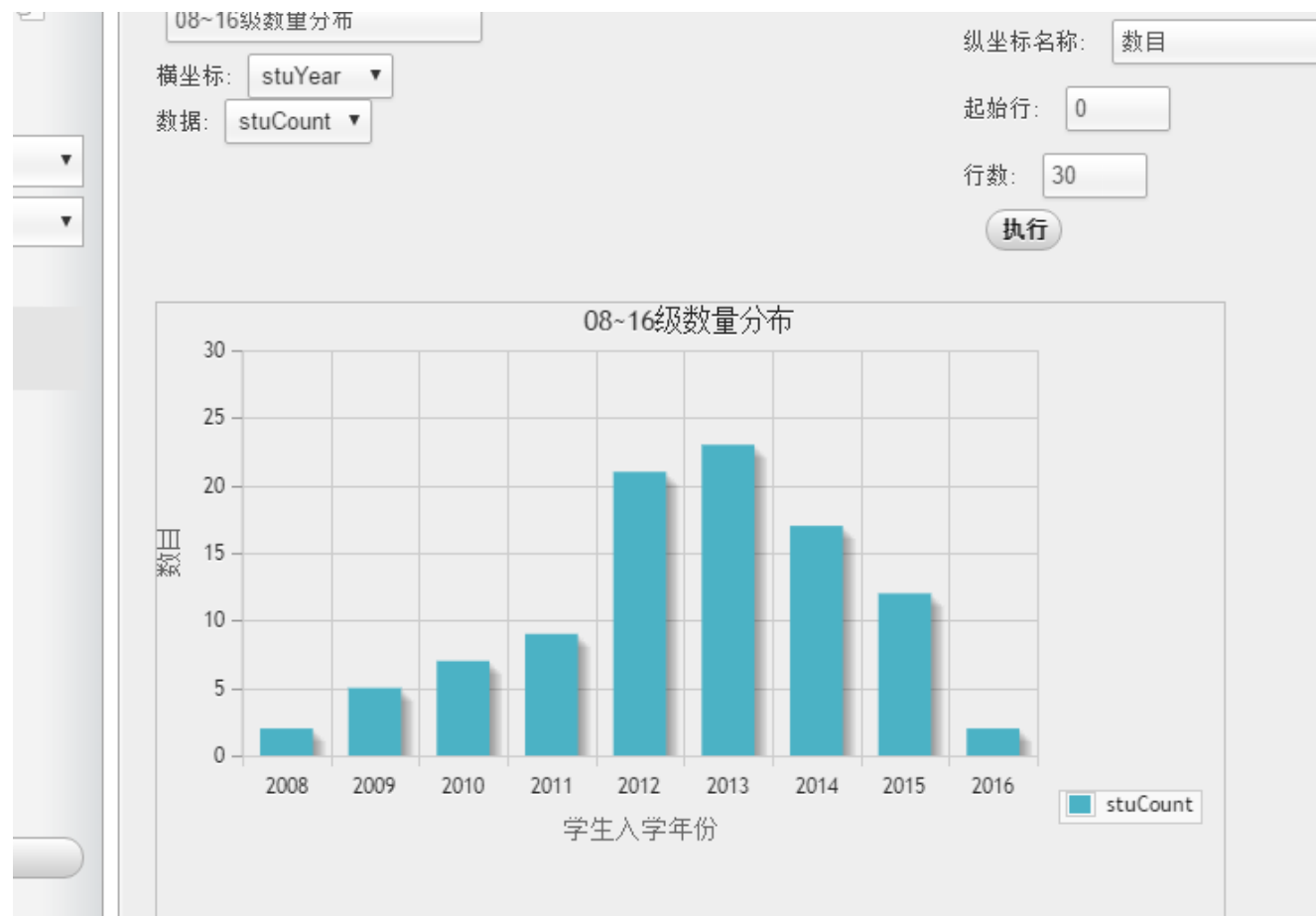
一共抓到106条数据，手工删掉几条非法数据后还剩98条。数据样本有点小啊。

| 浏览                       | 结构 | SQL | 搜索 | 插入 | 导出        | 导入         | 操作               | 触发器 |        |
|--------------------------|----|-----|----|----|-----------|------------|------------------|-----|--------|
| <input type="checkbox"/> | 编辑 | 复制  | 删除 | 1  | 142074241 | 2017-01-11 | 《电工技术》           |     | 王嘉伟    |
| <input type="checkbox"/> | 编辑 | 复制  | 删除 | 2  | 152074106 | 2017-01-10 | 《化工原理》           |     | 刘金宗    |
| <input type="checkbox"/> | 编辑 | 复制  | 删除 | 3  | 162074409 | 2017-01-07 | 《基础化学》           |     | 刘志伟    |
| <input type="checkbox"/> | 编辑 | 复制  | 删除 | 4  | 132112208 | 2017-01-07 | 《中外设计史》          |     | 冯治博    |
| <input type="checkbox"/> | 编辑 | 复制  | 删除 | 5  | 152012312 | 2017-01-07 | 《线性代数B》          |     | 辛振华    |
| <input type="checkbox"/> | 编辑 | 复制  | 删除 | 6  | 152066239 | 2017-01-06 | 《概率论与数理统计》       |     | 马宇鑫    |
| <input type="checkbox"/> | 编辑 | 复制  | 删除 | 7  | 152062232 | 2017-01-04 | 《大学英语3(1)》       |     | 钟潇抄    |
| <input type="checkbox"/> | 编辑 | 复制  | 删除 | 8  | 162056208 | 2017-01-04 | 《高等数学A1》         |     | 麻成龙    |
| <input type="checkbox"/> | 编辑 | 复制  | 删除 | 9  | 152011101 | 2017-01-16 | 《尔雅网络课程学生平台操作方法》 |     | )，选    |
| <input type="checkbox"/> | 编辑 | 复制  | 删除 | 10 | 132065233 | 2016-11-30 | 《细胞工程学》          |     | 林恒     |
| <input type="checkbox"/> | 编辑 | 复制  | 删除 | 11 | 186368336 | 2016-09-26 | 《军事理论》           |     | 13进行咨询 |
| <input type="checkbox"/> | 编辑 | 复制  | 删除 | 12 | 153066303 | 2016-09-03 | 《高等数学D2》         |     | 周清玉    |
| <input type="checkbox"/> | 编辑 | 复制  | 删除 | 13 | 132027222 | 2016-07-07 | 《微波技术基础》         |     | 王勇     |
| <input type="checkbox"/> | 编辑 | 复制  | 删除 | 14 | 132062134 | 2016-07-06 | 《安全人机工程学》        |     | 杨宏堃    |
| <input type="checkbox"/> | 编辑 | 复制  | 删除 | 15 | 152125128 | 2016-07-05 | 《中国法制史》          |     | 李精英    |
| <input type="checkbox"/> | 编辑 | 复制  | 删除 | 16 | 152211218 | 2016-07-04 | 《运动生理学》          |     | 张影     |
| <input type="checkbox"/> | 编辑 | 复制  | 删除 | 17 | 142027117 | 2016-07-04 | 《信号与系统》          |     | 王玉珠    |
| <input type="checkbox"/> | 编辑 | 复制  | 删除 | 18 | 132047318 | 2016-07-01 | 《化学反应工程》         |     | 张芝     |
| <input type="checkbox"/> | 编辑 | 复制  | 删除 | 19 | 142011139 | 2016-07-01 | 《电工技术》           |     | 田忠金    |
| <input type="checkbox"/> | 编辑 | 复制  | 删除 | 20 | 152093208 | 2016-06-30 | 《C程序设计》          |     | 宋振鹏    |

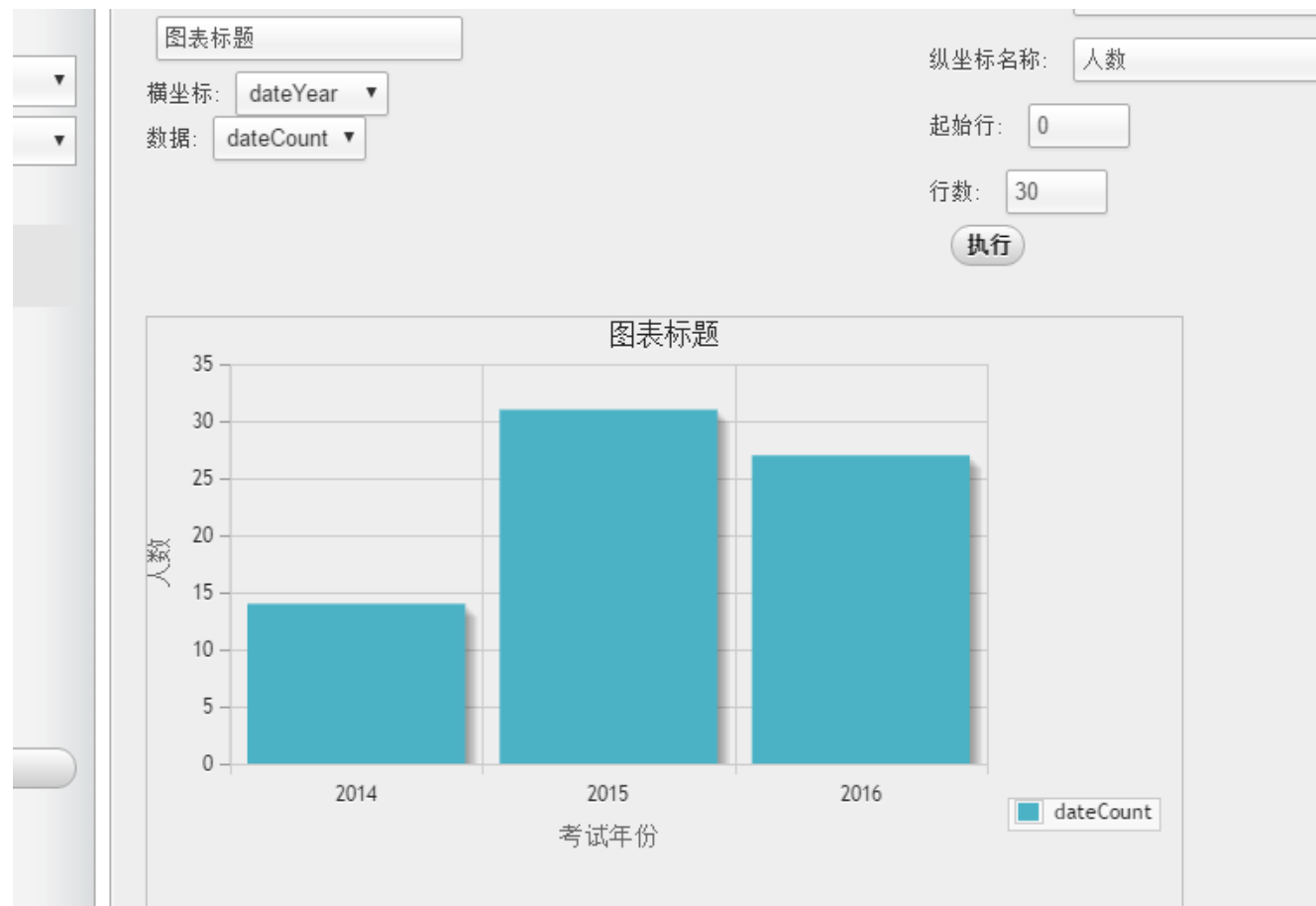
唯一（三）能得到的结果图：



不同系部的人数分布如上。



不同入学年份的学生人数分布如上。



不同年份考试的学生人数如上。（〇\_〇 "...）

程序比较简单懒得详细写了，注释很全。不具有普遍性也就不封装成类了，得加很多判断好麻烦。

The End.

[前一篇](#)

[中的几种设计模式](#) [后一篇](#)

[初体验：用最简单的方式运行环境，并实现](#)

: JAVA

: PHP-GTK

Hello World

Copyright Hower'Blog Rights Reserved. . Powered By [Z-BlogPHP](#), theme by [????](#)





smallyu

Hello World

??

??

????

2017080 (6)

2017070 (2)

2017060 (3)

2017050 (2)

2017040 (2)

2017030 (3)

2017020 (3)

2017010 (4)

20160120 (1)

## JAVA中的几种设计模式

□ smallyu □ 2017-02-11 □ JAVA 设计模式

单例模式

饿汉模式

Singleton.java

```
package com.imooc
public class Singleton {
 // 1.将构造方法私有化,不允许外部直接创建对象
 private Singleton() {

 }
 // 2.创建类的唯一示例,使用private static修饰
 private static Singleton instance = new Singleton();
 // 3.提供一个用于获取示例的方法
 public static Singleton getInstance() {
 return instance;
 }
}
```

饿汉模式指第二步时,加载类的同时生成了实例化的对象。特点是加载类时比较慢,但运行时获取对象的速度比较快,线程安全。

Test.java

20160110 (2)

20160100 (3)

2016090 (1)

2016080 (6)

2016070 (1)

2016060 (4)

2016050 (4)

```
package com.imooc;

public class Test {
 private static void main(String[] args) {
 Singleton s1 = Singleton.getInstance();
 Singleton s2 = Singleton.getInstance();
 }
}
```

懒汉模式

Singleton2.java

```
package com.imooc

public class Singleton2 {
 // 1.将构造方法私有化,不允许外部直接创建对象
 private Singleton2() {

 }

 // 2.创建类的唯一示例,使用private static修饰
 private static Singleton2 instance;

 // 3.提供一个用于获取示例的方法
 public static Singleton2 getInstance() {
 if(instance==null){
 instance = new Singleton2();
 }
 return instance;
 }
}
```

懒汉模式指第二步没有实例化对象，而是在第三步，当调用方法，才会判断并进行实例化，特点是加载类时比较快，但运行时获取对象的速度比较慢，线程不安全。

## 模板方法模式

应用场景为模拟制备饮料。制备饮料一共有四个步骤，即为RefreshBeverage.java中注释中写到的四步。泡制饮料和加入调味料在抽象基类中没有提供具体的实现方法，只是定义接口方法，在子类中应用模板方法模式分别实现具体内容。

第四步加入调味料用到了钩子方法，可以提高代码灵活性，判断子类要泡制的饮料中要不要加入调味料。

RefreshBeverage.java

```
package com.imooc.pattern.template;

/**
 * 抽象基类，为所有子类提供一个算法框架
 *
 * 提神饮料
 */
public abstract class RefreshBeverage {

 /**
 * 制备饮料的模板方法
 * 封装了所有子类共同遵循的算法框架
 */
 public final void prepareBeverageTemplate() {
 // 步骤1 将水煮沸
 boilWater();
 // 步骤2 泡制饮料
```

```
 brew();
 // 步骤3 将饮料倒入杯中
 pourInCup();
 if (isCustomerWantsCondiments()) {
 // 步骤4 加入调味料
 addCondiments();
 }
 }

 /**
 * Hook, 钩子函数, 提供一个默认或空的实现
 * 具体的子类可以自行决定是否挂钩或者如何挂钩
 * 询问用户是否加入调料
 */
 protected boolean isCustomerWantsCondiments() {
 return true;
 }

 /**
 * 基本方法, 将水煮沸
 */
 private void boilWater() {
 System.out.println("将水煮沸");
 }

 /**
 * 抽象的基本方法, 泡制饮料
 */
 protected abstract void brew();

 /**
 * 基本方法, 将饮料倒入杯中
 */
 private void pourInCup() {
```

```
 System.out.println("将饮料倒入杯中");
 }

 /**
 * 抽象的基本方法，加入调味料
 */
 protected abstract void addCondiments();
}
```

#### Coffee.java

```
package com.imooc.pattern.template;

/**
 * 具体子类，提供了咖啡制备的具体实现
 */
public class Coffee extends RefreshBeverage {

 protected void brew() {
 System.out.println("用沸水冲泡咖啡");
 }

 protected void addCondiments() {
 System.out.println("加入糖和牛奶");
 }
}
```

#### Tea.java

```
package com.imooc.pattern.template;

/**
```

```

 * 具体子类，提供了制备茶的具体实现
 */
 public class Tea extends RefreshBeverage {

 protected void brew() {
 System.out.println("用80度的热水浸泡茶叶5分钟");
 }

 protected void addCondiments() {
 System.out.println("加入柠檬");
 }

 /**
 * 子类通过覆盖的方式选择挂载钩子函数
 */
 protected boolean isCustomerWantsCondiments() {
 return false;
 }
 }
}

```

RefreshBeverageTest.java

```

package com.imooc.pattern.template;

public class RefreshBeverageTest {

 public static void main(String[] args) {

 System.out.println("制备咖啡...");
 RefreshBeverage b1 = new Coffee();
 b1.prepareBeverageTemplate();
 System.out.println("咖啡好了! ");
 }
}

```

```

 System.out.println("*****");

 System.out.println("制备茶...");
 RefreshBeverage b2 = new Tea();
 b2.prepareBeverageTemplate();
 System.out.println("茶好了! ");
 }

}

```

运行结果：

```

制备咖啡...
将水煮沸
用沸水冲泡咖啡
将饮料倒入杯中
加入糖和牛奶
咖啡好了!

制备茶...
将水煮沸
用80度的热水浸泡茶叶5分钟
将饮料倒入杯中
茶好了!

```

其中用到的钩子方法会使子类更加灵活。

## 适配器模式

应用场景为模拟二相转三相插座适配器，NoteBook需要三相电流供电，但只有二相电流插座，TwoPlugAd  
中适配了二相电流。

apter.java

ThreePlugIf.java

```
package com.imooc.appttern.adapter;
/**
 * 三相插座接口
 */
public interface ThreePlugIf {

 // 使用三相电流供电
 public void powerWithThree() {

 }

}
```

GBTwoPlug.java

```
package com.imooc.appttern.adapter;

public class GBTwoPlug {

 // 使用二相电流供电
 public void powerWithTwo() {
 System.out.println("使用二相电流供电");
 }

}
```

NoteBook.java

```
package com.imooc.appttern.adapter;
```



```

public class Notebook {

 private ThreePlugIf plug;

 public Notebook(ThreePlugIf plug){
 this.plug = plug;
 }

 // 使用插座充电
 public void change(){
 plug.powerWithThree();
 }

 public static void main(String[] args){
 GBTwoPlug two = new GBTwoPlug();
 ThreePlugIf three = new TwoPlugAdapter(two);
 Notebook nb = new Notebook(three);
 nb.change();
 }
}

```

#### TwoPlugAdapter.java

```

package com.imooc.appttern.adapter;

/**
 * 二相转三相的插座适配器
 */
public class TwoPlugAdapter implements ThreePlugIf {

 private GBTwoPlug plug;

 public TwoPlugAdapter(GBTwoPlug plug){

```

```
 this.plug = plug;
 }

 public void powerWithThree() {
 System.out.println("通过转化");
 plug.powerWithTwo();
 }
}
```

运行结果：

```
通过转化
使用二相电流供电
```

## 策略模式

策略模式将可变部分从程序中抽象分离成算法接口，在该接口下分别封装一系列算法实现

What is composition ?

在类中增加一个私有域，引用另一个已有的类的实例，通过调用引用实例的方法从而获得新的功能，这种设计被称作组合（复合）。

策略模式的优点：

1. 使用了组合，是架构更加灵活
2. 富有弹性，可以较好的应对变化（开\_\_闭原则）

3. 更好的代码复用性（相对于继承）
4. 消除大量的条件语句

策略模式的缺点：

1. 客户代码需要了解每个策略的细节
2. 增加了对象的数目

策略模式的应用场景：

1. 许多相关的类仅仅是行为差异
2. 运行时选取不同的算法变体
3. 通过条件语句在多个分支中选取一

代码的应用场景模拟了鸭子的生产过程，鸭子一共有三个行为：展示、鸣叫、飞行。不同的鸭子有不同的飞行行为，使用到了策略模式。

Duck.java

```
package com.imooc.pattern.strategy;

/**
 * 超类，所有的鸭子都要继承此类
 * 抽象了鸭子的行为：显示和鸣叫
 */
public abstract class Duck {
```

```

 /**
 * 鸭子发出叫声
 * 通用行为，由超类实现
 */
 public void quack(){
 System.out.println("嘎嘎嘎");
 }

 /**
 * 显示鸭子的外观
 * 鸭子的外观不相同，声明为abstract，由子类实现
 */
 public abstract void display();

 private FlyingStrategy flyingStrategy;

 public void setFlyingStrategy(FlyingStrategy flyingStrategy){
 this.flyingStrategy = flyingStrategy;
 }

 public void fly(){
 flyingStrategy.performFly();
 }
}

```

FlyingStrategy.java

```

package com.imooc.pattern.strategy;

/**
 * 策略接口，实现鸭子的飞行行为
 */
public interface FlyingStrategy {

```

```
void performFly();
}
```

MallardDuck.java

```
package com.imooc.pattern.strategy;

import com.imooc.pattern.strategy.impl.FlyWithWin;

// 绿脖鸭
public class MallardDuck extends Duck {

 public MallardDuck() {
 super();
 super.setFlyingStrategy(new FlyWithWin());
 }

 @Override
 public void display() {
 // TODO Auto-generated method stub
 System.out.println("我的脖子是绿色的");
 }

}
```

RedheadDuck.java

```
package com.imooc.pattern.strategy;

import com.imooc.pattern.strategy.impl.FlyWithWin;

// 红头鸭
```

```
public class RedheadDuck extends Duck {

 public RedheadDuck(){
 super();
 super.setFlyingStrategy(new FlyWithWin());
 }

 public void display() {
 System.out.println("我的头是红色的");
 }

}
```

RubberDuck.java

```
package com.imooc.pattern.strategy;

import com.imooc.pattern.strategy.impl.FlyNoWay;

// 橡胶鸭
public class RubberDuck extends Duck {

 public RubberDuck(){
 super();
 super.setFlyingStrategy(new FlyNoWay());
 }

 @Override
 public void display() {
 // TODO Auto-generated method stub
 System.out.println("我全身发黄，嘴巴很红");
 }

}
```

```
 public void quack(){
 // 叫声稍有不同，复写了父类方法
 System.out.println("嘎~嘎~嘎~");
 }
}
```

不同的飞行策略类：

FlyWithWin.java

```
package com.imooc.pattern.strategy.impl;

import com.imooc.pattern.strategy.FlyingStrategy;

public class FlyWithWin implements FlyingStrategy {

 @Override
 public void performFly() {
 // TODO Auto-generated method stub
 System.out.println("振翅高飞");
 }

}
```

FlyNoWay.java

```
package com.imooc.pattern.strategy.impl;

import com.imooc.pattern.strategy.FlyingStrategy;

public class FlyNoWay implements FlyingStrategy {
```

```
@Override
public void performFly() {
 // TODO Auto-generated method stub
 System.out.println("我不会飞行!");
}

}
```

最后测试生产出的鸭子:

DuckTest.java

```
package com.imooc.pattern.strategy;

public class DuckTest {

 public static void main(String[] args){
 System.out.println("测试鸭子程序");
 Duck duck = null;
 // duck = new MallardDuck();
 // duck = new RedheadDuck();
 duck = new RubberDuck();

 duck.display();
 duck.quack();
 duck.fly();

 System.out.println("测试完毕");
 }
}
```

运行结果:



```
测试鸭子程序
我全身发黄，嘴巴很红
嘎~嘎~嘎~
我不会飞行！
测试完毕
```

这样就用JAVA实现了策略模式。

## 代理模式

远程代理：为不同地理的对象提供局域网代表对象

虚拟代理：根据需要将资源消耗很大的对象进行延迟，真正需要的时候进行创建

保护代理：控制用户的访问权限

智能引用代理：提供对目标对象额外服务

应用场景模拟了汽车行驶时间的显示和日志记录两个功能。

Moveable.java

```
package com.imooc.proxy;

// 接口文件
public interface Moveable {
 void move();
}
```

Car.java

```
package com.imooc.proxy;

import java.util.Random;

public class Car implements Moveable {

 public void move() {

 // 实现开车
 try{
 Thread.sleep(new Random().nextInt(1000));
 System.out.println("汽车行驶中.....");
 }catch (InterruptedException e){
 e.printStackTrace();
 }
 }

}
```

Car2.java

```
package com.imooc.proxy;

// 实现汽车行驶时间
public class Car2 extends Car {

 public void move(){
 long strattime = System.currentTimeMillis();
 System.out.println("汽车开始行驶.....");
 super.move();
 long endtime = System.currentTimeMillis();
 System.out.println("汽车结束行驶中..... 汽车行驶时间: "+(endtime-strattime)+"毫秒");
 }

}
```

```
! ");
 }
}
```

Car3.java

```
package com.imooc.proxy;

// 时间代理实现汽车行驶时间
public class Car3 implements Moveable {

 public Car3(Car car) {
 super();
 this.car = car;
 }

 private Car car;

 public void move(){
 long strattime = System.currentTimeMillis();
 System.out.println("汽车开始行驶.....");
 car.move();
 long endtime = System.currentTimeMillis();
 System.out.println("汽车结束行驶中..... 汽车行驶时间: "+(endtime-strattime)+"毫秒
! ");
 }

}
```

Car4.java

```
package com.imooc.proxy;
```

```
// 增加log记录代理
public class Car4 implements Moveable {

 public Car4(Moveable m) {
 super();
 this.m = m;
 }

 private Moveable m;

 public void move() {
 System.out.println("日志开始.....");
 m.move();
 System.out.println("日志结束.....");
 }

}
```

Client.java

```
package com.imooc.proxy;

/**
 * 测试类
 */
public class Client {

 public static void main(String[] args) {

 // 直接
 // Car car = new Car();
 // car.move();
 }
}
```

```

 // 使用集成方法
// Moveable m = new Car2();
// m.move();

 // 使用聚合方法
// Car car = new Car();
// Moveable m = new Car3(car);
// m.move();

 // 增加日志记录
 Car car = new Car();
 Car3 ctp = new Car3(car);
 Car4 clp = new Car4(ctp);
 clp.move();
 }
}

```

运行结果像这样：

```

日志开始.....
汽车开始行驶.....
汽车行驶中.....
汽车结束行驶中..... 汽车行驶时间：497毫秒！
日志结束.....

```

下面使用动态代理来是代码更灵活（有错误）。

TimeHandler.java

```

package com.imooc.jdkproxy;

import java.lang.reflect.InvocationHandler;

```

```
import java.lang.reflect.Method;

import com.imooc.proxy.Car;

/**
 * JDK动态代理
 * 只能代理实现了接口的类
 * 没有实现接口的类不能实现JDK的动态代理
 */
public class TimeHandler implements InvocationHandler {

 public TimeHandler(Object target){
 super();
 this.target = target;
 }

 private Object target;

 /**
 * 参数
 * proxy 被代理对象
 * method 被代理的方法
 * args 方法的参数
 *
 * 返回值
 * Object 方法的返回值
 */
 @Override
 public Object invoke(Object arg0, Method arg1, Object[] arg2) throws Throwable {

 // TODO Auto-generated method stub

 long strattime = System.currentTimeMillis();
 System.out.println("汽车开始行驶.....");
 }
}
```

```

 method.invoke(target);
 long endtime = System.currentTimeMillis();
 System.out.println("汽车结束行驶中..... 汽车行驶时间: "+(endtime-strattime)+"毫秒
! ");

 return null;
 }

}

```

Test.java

```

package com.imooc.jdkproxy;

import java.lang.reflect.InvocationHandler;
import java.lang.reflect.Proxy;

import com.imooc.proxy.Car;
import com.imooc.proxy.Moveable;

/**
 * JDK动态代理测试类
 */
public class Test {

 public static void main(String[] args){
 Car car = new Car();
 InvocationHandler h = new TimeHandler(car);
 Class<?> cls = car.getClass();

 /**
 * loader 类加载器
 * interface 实现节后

```

```
 * h InvocationHandler
 */
 Moveable m = (Moveable)Proxy.newProxyInstance(cls.getClassLoader(), cls.g
etInterfaces(), h);
 m.move();
 }
}
```

etc.

[前一篇: PHP中多种常用的设计模式举例](#)

[后一篇: PHP写简单爬虫抓取太工教务处通告中的，考试作弊学生名单](#)





smallyu

Hello World

??

??

????

2017080 (6)

2017070 (2)

2017060 (3)

2017050 (2)

2017040 (2)

2017030 (3)

2017020 (3)

2017010 (4)

20160120 (1)

## PHP中多种常用的设计模式举例

□ smallyu □ 2017-01-25 □ PHP 设计模式

### 适配器模式

1. 适配器模式，可以将截然不同的函数接口封装成统一的API
2. 实际应用举例，PHP的数据库操作有mysql,musqli,pdo三种，可以用适配器模式统一成一致。类似的场景还有cache适配器，将memcache，redis，file，apc等不同的缓存函数，统一成一致

Database.php

```
<?php
interface IDatabase{
 function connect($host, $user, $passwd, $dbname);
 function query($sql);
 function close();
}
```

在Database.php中定义了数据库操作的接口，接下来将MySQL、MySQLi、PDO三种数据库接口中定义的方法。

MySQL.php

```
<?php
namespace Database;
```

适配器模式  
策略模式  
数据对象映射模式  
观察者模式  
原型模式  
装饰器模式  
迭代器模式  
代理模式

20160110 (2)

20160109 (3)

2016090 (1)

2016080 (6)

2016070 (1)

2016060 (4)

2016050 (4)

```
class MySQL implements IDatabase{
 protected $conn;
 function connect($host, $user, $passwd, $dbname){
 $conn = mysql_connect($host, $user, $passwd);
 mysql_select_db($dbname, $conn);
 this->conn = $conn;
 }
 function query($sql){
 $res = mysql_query($sql, $this->conn);
 return $res;
 }
 function close(){
 mysql_close($this->conn);
 }
}
```

## MySQLi.php

```
<?php
namespace Database;
class MySQLi implements IDatabase{
 protected $conn;
 function connect($host, $user, $passwd, $dbname){
 $conn = mysqli_connect($host, $user, $passwd, $dbname);
 this->conn = $conn;
 }
 function query($sql){
 $res = mysqli_query($this->conn, $sql);
 return $res;
 }
 function close(){
 mysqli_close($this->conn);
 }
}
```

PDO.php

```
<?php
namespace Database;
class PDO implements IDatabase{
 protected $conn;
 function connect($host, $user, $passwd, $dbname){
 new \PDO("mysql:host=$host;dbname=$dbname", $user, $passwd);
 $this->conn = $conn;
 }
 function query($sql){
 return $this->conn->query($sql);
 }
 function close(){
 unset($this->conn);
 }
}
```

然后在应用中可以这样使用：

index.php

```
<?php
$db = new Database\MySQL();
$db->connect('localhost', 'root', 'root', 'test');
$db->query('show database');
$db->close();
```

或者实例化其他类型的数据库，只需要使用同一种标准。

策略模式

1. 策略模式，将一组特定行为和算法封装成类，以适应某些特定的上下文环境，这种模式就是策略模式
2. 实际应用举例，假如一个电商网站系统，针对男性女性用户要各自跳转到不同的商品类目，并且所有广告位展示不同的广告
3. 使用策略模式可以实现IOC，依赖倒置、控制反转

UserStrategy.php

```
<?php
interface UserStrategy {
 function showAd();
 function showCategory();
}
```

UserStrategy.php文件定义了用户策略的接口，面对不同的用户将分别实现相应的内容。

FemaleUserStrategy.php

```
<?php
class FemaleUserStrategy implements UserStrategy {
 function showAd() {
 echo "2017新款女装";
 }
 function showCategory() {
 echo "女装";
 }
}
```

MaleUserStrategy.php

```
<?php
class MaleUserStrategy implements UserStrategy {
 function showAd() {
 echo "IPhone7";
 }
 function showCategory() {
 echo "电子产品";
 }
}
```

在index.php中这样做就可以实现从硬编码到实现解耦，即不需要修改Page类的内容，只需要传递不同的参数。

index.php

```
<?php
class Page {
 function index() {
 echo "AD:";
 $this->strategy->showAd();
 echo "Category:";
 $this->strategy->showCategory();
 }
 function setStrategy(UserStrategy $strategy) {
 $this->strategy = $strategy;
 }
}
$page = new Page();
if (isset($_GET['female'])) {
 $strategy = new FemaleUserStrategy();
```

```
} else {
 $strategy = new MaleUserStrategy();
}
$page->setStrategy($strategy);
$page->index();
```

## 数据对象映射模式

数据对象映射模式，是将对象和数据存储映射起来，对一个对象的操作会映射为对数据存储的操作。

在下面的代码中实现数据对象映射模式，我们将实现一个ORM类，将复杂的SQL语句映射成对象属性的操作。

User.php

```
<?php
class User {
 public $id;
 public $mobile;
 public $regtime;

 protected $db;

 function __construct($id) {
 // 数据库查询操作
 $this->db = new MySQLi();
 $this->db->connect('localhost', 'root', 'root', 'test1');
 $res = $this->db->query("select * from user where id={$this->id} limit 1"
);

 $data = $res->fetch_assoc();
 $this->id = $data['id'];
 $this->name = $data['name'];
 }
```

```

 $this->mobile = $data['imobile'];
 $this->regtime = $data['regtime'];
 }

 function __destruct() {
 // 数据库写入操作
 $this->db->query("update user set name = '{$this->name}',
 mobile='{$this->mobile}', regtime='{$this->regtime}' where id = {$this->id} limit 1");
 }
}

```

这样来使用：

index.php

```

<?php
$user = new User(1);
$user->mobile = '18812345678';
$user->name = 'test';
$user->regtime = date('Y-m-d H:i:s');

```

下面结合使用数据对象映射模式，工厂模式，注册器模式：

index.php

```

<?php
class Page {
 function index() {
 $user = Factory::getUser(1); // 工厂模式的实现
 $user->name = 'rango';
 }
}

```

```

 $this->test();
 }

 function test() {
 $user = Factory::getUser(1); // 工厂模式的实现
 $user->mobile = '18844448888';
 }
}
$page = new Page();

```

Factory.php

```

<?php
class Factory {
 static function getUser($id) {
 // 注册器模式的实现
 $key = 'user_'. $id;
 $user = Register::get($key);
 if (!$user) {
 $user = new User($id);
 Register::set($key, $user);
 }
 return $user;
 }
}

```

## 观察者模式

1. 观察者模式（Observer），当一个对象状态发生改变时，依赖它的对象全部会收到通知，并自动更新
2. 场景：一个事件发生后，要执行一连串更新操作，传统的编程方式，就是在事件的代码之后直接加



入处理逻辑。当更新的逻辑增多之后，代码会变得难以维护。这种方式是耦合的，侵入式的，增加新的逻辑需要修改事件主体的代码

### 3. 观察者模式实现了低耦合，非侵入式的通知与更新机制

index.php

```
<?php
class Event extends EventGenerator {
 function trigger() {
 echo "Event
";
 $this->notify();
 }
}

class Observer1 implements Observer {
 function update($event_info = null) {
 echo "逻辑1
";
 }
}

class Observer2 implements Observer {
 function update($event_info = null) {
 echo "逻辑2
";
 }
}

$event = new Event;
$event->addObserver(new Observer1); // 观察者模式的实现
$event->addObserver(new Observer2); // 观察者模式的实现
$event->trigger();
```

## EventGenerator.php

```
<?php
abstract class EventGenerator {

 private $observers = array();

 function addObserver(Observer $observer) {
 $this->observers[] = $observer;
 }

 function notify() {
 foreach($this->observers as $ovserver) {
 $observer->update();
 }
 }
}
```

## Observer.php

```
<?php
interface Observer {
 function update($event_info = null);
}
```

## 原型模式

1. 与工厂模式作用类似，都是用来创建对象
2. 与工厂模式的实现不同，原型模式是先创建好一个原型对象，然后通过`clone`原型对象来创建新的对象。这样就免去了类创建时重复的初始化操作

3. 原型模式适用于大对象的创建。创建一个对象需要很大的开销，如果每次`new`就会消耗很大，原型模式内存拷贝即可

Canvas.php

```
<?php
class Canvas {
 ...
}
```

index.php 将实例化`Canvas`类以实现图像绘制，并使用关键字`clone`实现原型模式：

index.php

```
<?php
$prototype = new Canvas();
$prototype->init();

// 原型模式的实现
$canvas1 = clone $prototype;
$canvas2 = clone $prototype;

$canvas1->rect(1,2,3,4);
$canvas1->draw();
$canvas2->rect(5,6,7,8);
$canvas2->draw();
```

## 装饰器模式

1. 装饰器模式（Decorator），可以动态地添加修改类的功能

2. 一个类提供了一项功能，如果要在修改并添加额外的功能，传统的编程模式，需要写一个子类继承它，并重新实现类的方法
3. 使用装饰器模式，仅需要在运行时添加一个装饰器对象即可实现，可以实现最大的灵活性

DrawDecorator.php

```
<?php
interface DrawDecorator {
 function beforeDraw();
 function afterDraw();
}
```

DrawDecorator.php 定义了装饰器的接口，将会在装饰器类中用到。

Canvas.php

```
<?php
class Canvas {
 protected $decorator = array();

 function draw() {
 // 装饰器模式实现点
 $this->beforeDraw();
 ...
 $this->afterDraw();
 }

 function addDecorator(DrawDecorator $decorator) {
 $this->decorators[] = $decorator;
 }
}
```

```

 function beforeDraw() {
 foreach($this->decorators as $decorator) {
 $decorator->beforeDraw();
 }
 }

 fucntion afterDraw() {
 $decorators = array_reverse($this->decorators); // 反转，后进先出
 foreach($decorators as $decorator) {
 $decorator->afterDraw();
 }
 }
 }
}

```

ColorDrawDecorator.php

```

<?php
// 第一个装饰器：颜色装饰器
class ColorDrawDecortor implements DrawDecorator {
 function beforeDraw() {
 ...
 }

 function afterDraw() {
 ...
 }
}

```

## 迭代器模式

1. 迭代器模式，在不需要了解内部实现的前提下，遍历一个聚合对象的内部元素

2. 相比于传统的编程模式，迭代器模式可以隐藏遍历元素所需的操作

index.php

```
<?php
$users = new AllUser();
foreach($users as $user) {
 var_dump($user);
}
```

迭代器的实现：

AllUser.php

```
<?php
$canvas1 = new Canvas();
$canvas1->init();
$canvas1->addDecorator(new ColorDrawDecortor()); // 添加装饰器
$canvas1->draw();
```

这样来使用：

index.php

```
<?php
class AllUser implements \Iterator {

 protected $ids;
 protected $data = array();
 protected $index;
```

```

function __construct() {
 $db = Factory::getDatabase();
 $result = $db->query('select id from user');
 $this->ids = $result->fetch_all(MYSQLI_ASSOC);
}

function current() {
 $id = $this->ids[$this->index]['id'];
 return Factory::getUser($id);
}

function next() {
 $this->index++;
}

function valid() {
 return $this->id < count($this->ids);
}

function rewind() {
 $this->index = 0;
}

function key() {
 return $this->index;
}
}

```

## 代理模式

1. 在客户端与实体之间建立一个代理对象（`proxy`），客户端对实体进行操作全部委派给代理对象，隐藏实体的具体实现细节

2.Proxy 还可以与业务代码分离，部署到另外的服务器，业务代码中通过RPC来委派任务

index.php

```
$proxy = new Proxy();
$proxy->getUserName();
$proxy->setUserName($id, $proxy);
```

Proxy.php

```
<?php
class Procy implemets IUserProxy {
 function getUserName($id) {
 $db = Factory::getDatabase('slave');
 $db->query('select name from user where id = 1 limit 1');
 }

 function setUserName($id, $name) {
 $db1 = Factory::getDatabase('master');
 $db->query('update user name = $name from user where id = 1 limit 1');
 }
}
```

IUserProxy.php

```
<?php
interface IUserName {
 function getUserName($id);
 function setUserName($id, $name);
}
```



[前一篇: PHP面向对象编程的基本原则，以及三种基本设计模式](#)

[后一篇: JAVA中的几种设计模式](#)



smallyu

Hello World

??

??

????

2017080 (6)

2017070 (2)

2017060 (3)

2017050 (2)

2017040 (2)

2017030 (3)

2017020 (3)

2017010 (4)

20160120 (1)

# PHP面向对象编程的基本原则，以及三种基本设计模式

□ smallyu □ 2017-01-24 □ PHP 设计模式 面向对象

## 面向对象编程的基本原则

1. 单一职责：一个类，只需要做好一件事情
2. 开放封闭：一个类，应该是可扩展的，而不可修改的
3. 依赖倒置：一个类，不应该强依赖另一个类。每个类对于另外一个类都是可替换的
4. 配置化：尽可能地使用配置，而不是硬编码
5. 面向接口编程：只需要关心接口，不需要关心实现

## PSR-0命名规范

1. 命名空间必须与绝对路径一致
2. 类名首字母必须大写
3. 除入口文件外，其他".php"必须只有一个类

## 开发符合PSR-0规范的基础框架

1. 全部使用命名空间
2. 所有PHP文件必须自动载入，不能有include/require

20160110 (2)

20160109 (3)

2016090 (1)

2016080 (6)

2016070 (1)

2016060 (4)

2016050 (4)

### 3. 单一入口

#### 类的自动载入

Test1.php

```
<?php

class Test1{

 static function test(){

 echo __METHOD__;

 }

}
```

Test2.php

```
<?php

class Test2{

 static function test(){

 echo __METHOD__;

 }

}
```

test.php

```
<?php

spl_autoload_register('autoload1');

spl_autoload_register('autoload2');

function autoload1($class){

 require __DIR__.'/'.$class.'.php';

}

function autoload2($class){

 require __DIR__.'/'.$class.'.php';

}

Test1::test();

Test2::test();
```

运行test.php会得到这样的结果：

```
Test1::testTest2::test
```

`__autoload()`函数只可以存在一个，如果重复定义会报致命错误，所以目前一般使用`spl_autoload_register()`来实现类的自动加载。

## PHP链式调用的实现

Database.php

```
<?php

class Database{

 function where($where){

 ...

 }

 function order($order){

 ...

 }

 function limit($limit){

 ...

 }

}
```

如果Database.php是这样，那么在index.php中就需要这样来写以实现sql查询：

index.php

```
<?php

$db=new Database();

$db->where("id=2");

$db->where("name=2");

$db->order("id desc");

$db->limit(10);
```

要实现链式调用只需要这样做:

Database.php

```
<?php

class Database{

 function where($where){

 return $this;

 }

 function order($order){

 return $this;

 }

 function limit($limit){
```

```
 return $this;

 }

}
```

在每个方法的末尾加上'return \$this;', 然后就可以:

Data

```
<?php

$db->where("id=1")->where("name=2")->order("id desc")->limit(10);
```

三种基础的设计模式包括工厂模式、单例模式和注册器模式。

工厂模式

Factory.php

```
<?php
class Factory{
 static function createDatabase(){
 $db = new Database();
 return $db;
 }
}
$db = Factory::createDatabase();
```

工厂模式的作用在于不在类外实例化对象。

## 单例模式

Database.php

```
<?php
class Database{
 protected $db;
 // 第一步
 private function __construct(){
 // private可以屏蔽外部new这个类
 }
 // 第二步
 static function getInstance(){
 if(self::$db){
 return self::$db;
 }else{
 self::$db = new self();
 return self::$db;
 }
 }
}
$db = Database::getInstance();
```

单例模式用于数据库连接等只需要一个对象的场景，避免资源的浪费。

## 注册器模式

Register.php

```
<?php
class Register{
```



```

protected static $objects;
static function set($name, $object){
 self::$objects[$name] = $object;
}
static function get($name){
 return self::$objects[$name];
}
static function _unset($name){
 unset(self::$objects[$name]);
}
}
Register::set('db1',$db);
$db = Register::get('$db1');

```

## 适配器模式

1. 适配器模式，可以将截然不同的函数接口封装成统一的API
2. 实际应用举例，PHP的数据库操作有mysql,musqli,pdo三种，可以用适配器模式统一成一致。类似的场景还有cache适配器，将memcache，redis，file，apc等不同的缓存函数，统一成一致

Database.php

```

<?php
interface IDatabase{
 function connect($host, $user, $passwd, $dbname);
 function query($sql);
 function close();
}

```

在Database.php中定义了数据库操作的接口，接下来将MySQL、MySQLi、PDO三种数据库操作适配为接

口中定义的方法。

#### MySQL.php

```
<?php
namespace Database;
class MySQL implements IDatabase{
 protected $conn;
 function connect($host, $user, $passwd, $dbname){
 $conn = mysql_connect($host, $user, $passwd);
 mysql_select_db($dbname, $conn);
 this->conn = $conn;
 }
 function query($sql){
 $res = mysql_query($sql, $this->conn);
 return $res;
 }
 function close(){
 mysql_close($this->conn);
 }
}
```

#### MySQLi.php

```
<?php
namespace Database;
class MySQLi implements IDatabase{
 protected $conn;
 function connect($host, $user, $passwd, $dbname){
 $conn = mysqli_connect($host, $user, $passwd, $dbname);
 this->conn = $conn;
 }
}
```

```
function query($sql){
 $res = mysqli_query($this->conn, $sql);
 return $res;
}
function close(){
 mysqli_close($this->conn);
}
```

## PDO.php

```
<?php
namespace Database;
class PDO implements IDatabase{
 protected $conn;
 function connect($host, $user, $passwd, $dbname){
 new \PDO("mysql:host=$host;dbname=$dbname", $user, $passwd);
 $this->conn = $conn;
 }
 function query($sql){
 return $this->conn->query($sql);
 }
 function close(){
 unset($this->conn);
 }
}
```

然后在应用中可以这样使用：

## index.php

```
<?php
$db = new Database\MySQL();
$db->connect('localhost', 'root', 'root', 'test');
```

```
$db->query('show database');
$db->close();
```

或者实例化其他类型的数据库，只需要使用同一种标准。

[前一篇: 一个简单的验证码生成类 \(PHP\)](#)

[后一篇: PHP中多种常用的设计模式举例](#)



smallyu

Hello World

??

??

????

2017080 (6)

2017070 (2)

2017060 (3)

2017050 (2)

2017040 (2)

2017030 (3)

2017020 (3)

2017010 (4)

20160120 (1)

## 一个简单的验证码生成类 PHP

□ smallyu □ 2017-01-17 □ PHP 验证码

可配置项

```
/**
 * $_fontfile='' // 字体文件
 * $_size=20; // 字体大小
 * $_width=120; // 画布宽度
 * $_height=40; // 画布高度
 * $_length=4; // 验证码长度
 * $_image=null; // 画布资源
 * // 干扰元素
 * $_snow=0; // 雪花个数
 * $_pixel=0; // 像素个数
 * $_line=0; // 线段个数
 */
```

其中字体文件路径必须配置。

Captcha.class.php

```
Class Captcha{
 // 字体文件
 private $_fontfile='';
 // 字体大小
 private $_size=20;
 // 画布宽度
```

20160110(2)

20160109(3)

2016090(1)

2016080(6)

2016070(1)

2016060(4)

2016050(4)

```
private $_width=120;
// 画布高度
private $_height=40;
// 验证码长度
private $_length=4;
// 画布资源
private $_image=null;
// 干扰元素
// 雪花个数
private $_snow=0;
// 像素个数
private $_pixel=0;
// 线段个数
private $_line=0;
/**
 * 初始化数据
 * @param array $config [description]
 */
public function __construct($config=array()){
 if(is_array($config)&&count($config)>0){
 // 检测字体文件是否存在并且可读
 if(isset($config['fontfile'])&&is_file($config['fontfile'])&&is_readable($config['fontfile'])){
 $this->_fontfile=$config['fontfile'];
 }else{
 return false;
 }
 // 检测是否设置字体大小
 if(isset($config['size'])&&$config['size']>0){
 $this->_size=(int)$config['size'];
 }
 // 检测是否设置画布宽和高
 if(isset($config['width'])&&$config['width']>0){
 $this->_width=(int)$config['width'];
 }
 }
}
```

```

 }
 if(isset($config['height'])&&$config['height']>0){
 $this->_height=(int)$config['height'];
 }
 // 检测是否设置验证码长度
 if(isset($config['length'])&&$config['length']>0){
 $this->_length=(int)$config['length'];
 }
 // 配置干扰元素
 if(isset($config['snow'])&&$config['snow']>0){
 $this->_snow=(int)$config['snow'];
 }
 if(isset($config['pixel'])&&$config['pixel']>0){
 $this->_pixel=(int)$config['pixel'];
 }
 if(isset($config['line'])&&$config['line']>0){
 $this->_line=(int)$config['line'];
 }
 $this->_image=imagecreatetruecolor($this->_width, $this->_height);
 return $this->_image;
}
}
}
else{
 return false;
}
}
}
/**
 * 得到验证码
 * @return [type] [description]
 */
public function getCaptcha(){
 $white=imagecolorallocate($this->_image, 255, 255, 255);
 // 填充矩形
 imagefilledrectangle($this->_image, 0, 0, $this->_width, $this->_height,
 $white);
 // 生成验证码

```

```
$str=$this->_generateStr($this->_length);
if(false==$str){
 return false;
}
$fontfile=$this->_fontfile;
// 绘制验证码
for($i=0;$i<$this->_length;$i++){
 $size=$this->_size;
 $angle=mt_rand(-30,30);
 $x=ceil($this->_width/$this->_length)*$i+mt_rand(5,10);
 $y=ceil($this->_height/1.5);
 $color=$this->_getRandColor();
 // $text=mb_substr($str,$i,1,'utf-8');
 $text=$str{$i};
 imagettftext($this->_image, $size, $angle, $x, $y, $color, $fontfile,
$text);
}
if($this->_snow){
 // 使用雪花当做干扰元素
 $this->_getSnow();
}else{
 if($this->_pixel){
 $this->_getPixel();
 }
 if($this->_line){
 $this->_getLine();
 }
}
// 输出图像
ob_clean();
header('content-type:image/png');
imagepng($this->_image);
imagedestroy($this->_image);
return strtolower($str);
```



```
}
/**
 * 产生雪花
 * @return [type] [description]
 */
private function _getSnow(){
 for($i=1;$i<=$this->_snow;$i++){
 imagestring($this->_image, mt_rand(1,5), mt_rand(0,$this->_width), mt
_rand(0,$this->_height), '*', $this->_getRandColor());
 }
}
/**
 * 绘制像素
 * @return [type] [description]
 */
private function _getPixel(){
 for($i=1;$i<=$this->_pixel;$i++){
 imagesetpixel($this->_image, mt_rand(0,$this->_width), mt_rand(0,$thi
s->_height), $this->_getRandColor());
 }
}
/**
 * 绘制线段
 * @return [type] [description]
 */
private function _getLine(){
 for($i=1;$i<=$this->_line;$i++){
 imageline($this->_image, mt_rand(0,$this->_width), mt_rand(0,$this->_
height), mt_rand(0,$this->_width), mt_rand(0,$this->_height), $this->_getRandColo
r());
 }
}
/**
 * 产生验证码字符
```

```

 * @param integer $length 验证码长度
 * @return string 随机字符
 */
 private function _generateStr($length=4){
 if($length<1||$length>30){
 return false;
 }
 $chars=array(
 'a','b','c','d','e','f','g','h','k','m','n','p','x','y','z',
 'A','B','C','D','E','F','G','H','K','M','N','P','X','Y','Z',
 '2','3','4','5','6','7','8','9'
);
 $str=join('',array_rand(array_flip($chars),$length));
 return $str;
 }
 private function _getRandColor(){
 return imagecolorallocate($this->_image, mt_rand(0,255), mt_rand(0,255),
 mt_rand(0,255));
 }
 }
}

```

然后在应用中使用这个类

captcha.php

```

require 'Captcha.class.php';
ob_clean();
$config=array(
 'fontfile'=>'INFROMAN.TTF',
 // 'snow'=>10,
 'pixel'=>100,
 'line'=>5
);

```

```
$captcha=new Captcha($config);
$captcha->getCaptcha();
```

就可以生成类似这样的验证码



或者



getCaptcha()会以字符串的形式返回验证码的值。

比较简单的一种验证码 用了PHP的扩展GD库。

上一篇: [2016下半年年度总结](#) [2017上半年年度展望](#)

下一篇: [PHP面向对象编程的基本原则](#) [以及三种基本设计模式](#)



smallyu

Hello World

??

??

????

2017080 (6)

2017070 (2)

2017060 (3)

2017050 (2)

2017040 (2)

2017030 (3)

2017020 (3)

2017010 (4)

20160120 (1)

## 2016下半年年度总结，2017上半年年度展望

□ smallyu   □ 2017-01-08   □ 2017

为什么面朝大海，就站成了大海？因为有人在心里面浮沉啊。

2016下半年年度总结：

（某个人的名字）

（某个人的名字）

（某个人的名字）

.....

好吧总结可以结束了。

2017上半年年度展望：

注定平凡。

.....

2016111(2)

2016101(3)

201699(1)

201688(6)

201677(1)

201666(4)

201655(4)

好吧展望可以结束了。

<<<(2017.1.17)

水了好久都有点开始怀疑人生了。

2016年确实过得一塌糊涂。上半年过去都不知道过去了。下半年，四级四级考不过，期末期末没考好。我都干了些什么？

(2017.1.17);

<<<(2017.2.10)

明天是正月十五了，什么节日来着？

啊，最近在写singcms。

(2017.1.20);

<<<(2017.3.3)

时过境迁。

(2017.1.3.3);

[前一篇](#): [春宵苦短日高起，不来](#)

[后一篇](#): [一个简单的验证码生成类（PHP）](#)

Copyright Hower'Blog Rights Reserved. . Powered By [Z-BlogPHP](#), theme by [????](#)



smallyu

Hello World

??

??

????

2017080 (6)

2017070 (2)

2017060 (3)

2017050 (2)

2017040 (2)

2017030 (3)

2017020 (3)

2017010 (4)

20160120 (1)

## 春宵苦短日高起，不来

□ smallyu □ 2016-12-17 □ CET4

标题来自知乎。

这个东西，折磨了我几个月。



刷完，大概粗略算了一下正确率：

20160110(2)

20160100(3)

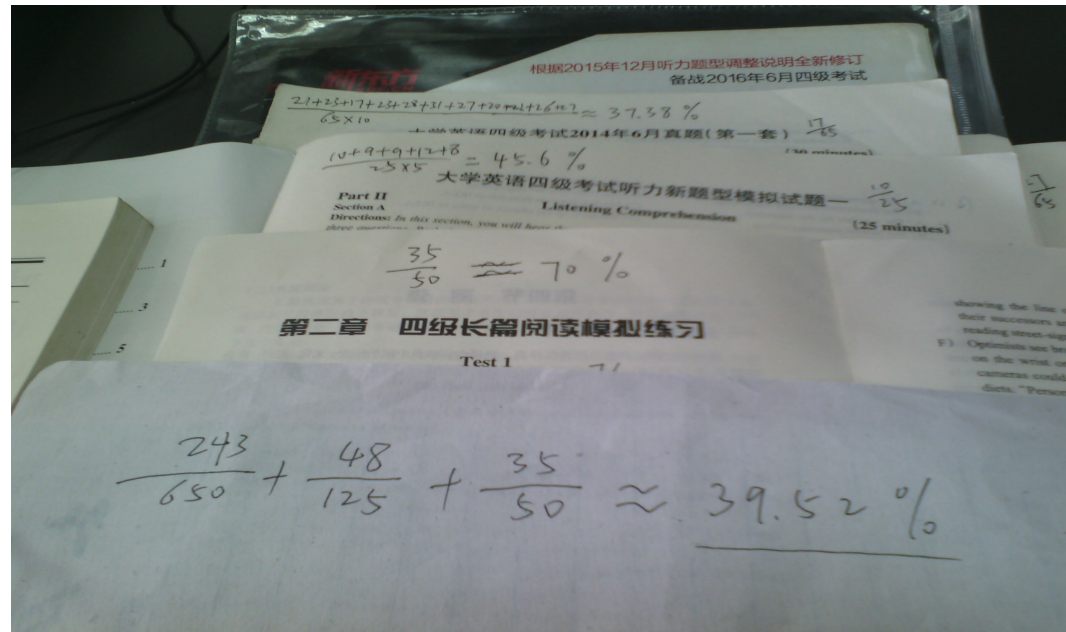
20160090(1)

20160080(6)

20160070(1)

20160060(4)

20160050(4)



拍照环境不好，也没必要对图片做什么高大上的处理。这种黑乎乎的照片，放在洁白的网页上真的真的会影响美观，但真的真的就不处理了，也可能我不会。图片就是事实，而已。

图片不清楚，并不影响看到图片上写着的大大的最终结果：**39.52%**

图片本身不小，有1M。确实像素低可能看不清。上面的所有数字都表示选择题的个数，除了旧型听力最后一题的十个听力填空。分母表示总数，分子表示正确的数量。总之，按照平均数的算法，正确率就是，残酷的39.52%。

其实用平均数来表示正确率是有问题的，万一一不小心就有了那么一点点的进步呢？事实证明我想多了。

数据分析的话，拍照来的总归感觉不是很好，于是，就真的无聊地统计了一下，一共有三部分，第一部分是十套真题，除掉写作和翻译的，第二部分是长篇阅读专项，第三部分是新型听力专项。



这个是十套真题的统计：

| 四级套题/10 |       |         |         |         |    |
|---------|-------|---------|---------|---------|----|
|         | 听力/35 | 选词填空/10 | 长篇阅读/10 | 仔细阅读/10 | 总数 |
| 1       | 8     | 1       | 5       | 3       | 17 |
| 2       | 9     | 6       | 5       | 3       | 23 |
| 3       | 10    | 5       | 6       | 7       | 28 |
| 4       | 15    | 4       | 7       | 5       | 31 |
| 5       | 10    | 7       | 5       | 5       | 27 |
| 6       | 10    | 1       | 1       | 8       | 20 |
| 7       | 9     | 4       | 6       | 7       | 26 |
| 8       | 10    | 6       | 4       | 6       | 26 |
| 9       | 10    | 4       | 3       | 4       | 21 |
| 10      | 12    | 1       | 4       | 6       | 23 |

这个是长篇阅读专项的统计：

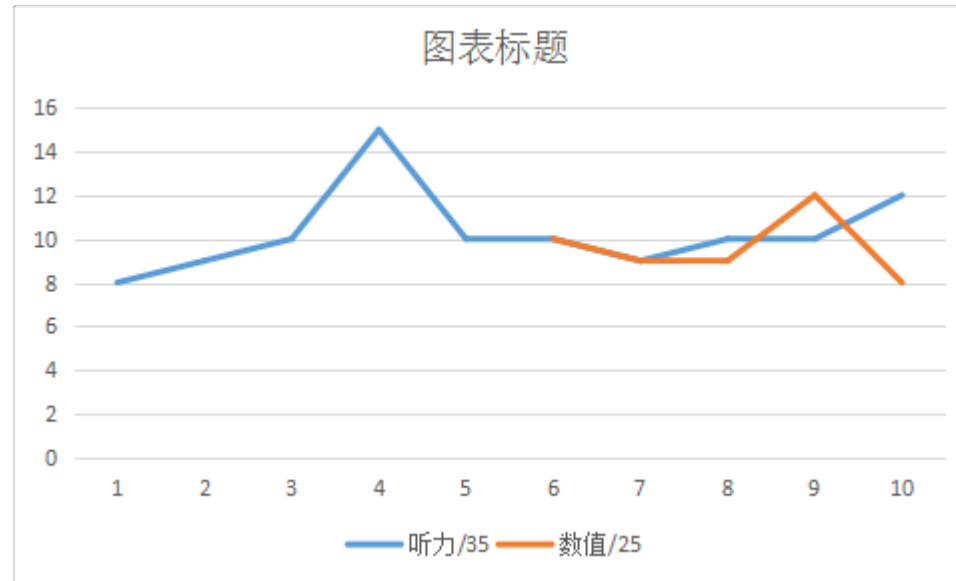
|  |
|--|
|  |
|--|

| 长篇阅读专项 |   |   |   |   |   |
|--------|---|---|---|---|---|
|        | 1 | 2 | 3 | 4 | 5 |
| 数值/10  | 7 | 8 | 7 | 7 | 6 |

这个是新型听力的专项：

| 新型听力专项 |    |   |   |    |   |
|--------|----|---|---|----|---|
|        | 1  | 2 | 3 | 4  | 5 |
| 数值/25  | 10 | 9 | 9 | 12 | 8 |

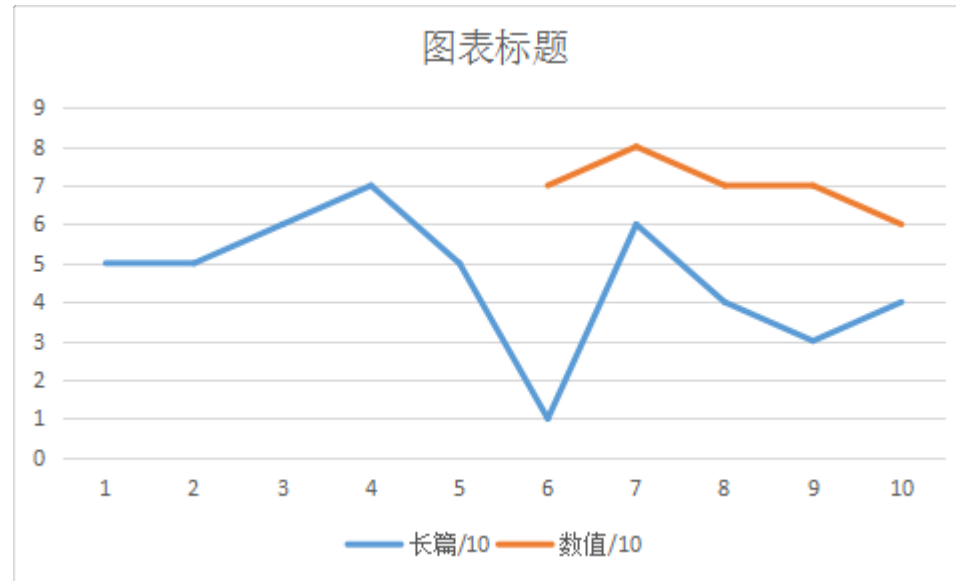
只看表格好像没什么感觉，那就把它变成折线图。两个专项是在四级考前的后半段时间做的，时间上和套题的后半段比较接近。



先是听力。默认没有图表标题，因为有两条线，就懒得手动改了。新型听力比旧型听力少十道题，所以可以看到，有没有那十道题正确的总量是差不多的，说明那十道题本来就几乎对不了几个啊。



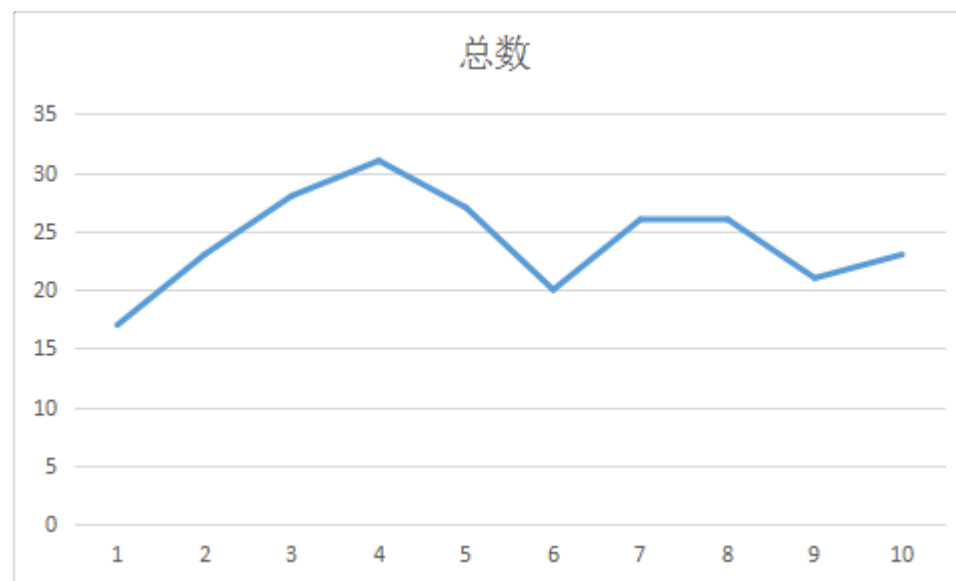
然后是选词填空，就是十五选十，波动比较大，应该是看懂了就对得多，看不懂就对的少。所以这个图表没有任何价值。



这个也没有图表标题，这个是长篇阅读。可以看到，专项的正确率远超套题的，然而做它们的时间差不多，说明，我没认真做啊。专项用的时间比较多一点。还有可能专项的题简单一点，毕竟不是真题。



仔细阅读，精读，总体来说还是稍微有那么一点点进步的看样子。



最后这个是正确量的总数。整体来看，最后一次做的和第一次做的没什么差别。没有有体现出上升或者下降。也就是说，这两个半月来，我只是简单地做到了没有退步.....

至于四级考试。

对啊，今天刚考的，上午9点。考前心情异常沉重。估计有生以来还没有哪次考试是明知道自己不会过但还必须得认真复习认真考试的。

然后就炸了。

按39.52%的正确率来算， $710 * 0.3952 = 280.592$ 。

严肃地说，根据我多年考试的经验以及对我自己的了解，按照最好的情况来算，710的总分，我会考差不多400分。详情如下：

短文写作  $106.5 * 0.5 = 53.25$

听力  $248.5 * 0.5 = 124.25$

选词填空  $35.5 * 0.8 = 28.4$

长篇阅读  $71 * 0.6 = 42.6$

仔细阅读  $142 * 0.7 = 99.4$

翻译  $106.5 * 0.5 = 53.25$

$53.25 + 124.25 + 28.4 + 42.6 + 99.4 + 53.25 = 401.15$

401.15，差不多400。但是，这估计已经是最最好的情况下的数字。当然，425才会pass，差一大截。

计算的时候的比例都要超过平时练习时候正确率的比例，一方面因为考试会比较认真，另一方面。

实在不能接受不到300分的英语四级成绩啊。

嗯，过的希望是很渺茫的。

嗯。

一个月以后，看我估计的对不对。

然后对四级想说的也真的不只是这些啊。

<<<(2016.2.21)

cet4成绩明天出，等待一个会令人失望但仍然抱有一丝丝点侥幸希望的心情，还是不要不要的~

(2016.2.21);

<<<(2016.2.22)

估的，高了.....

(2016.2.22);



[前一篇: 生命不息，折腾不止——hexo 并不好玩](#)

[后一篇: 2016 下半年年度总结，2017 上半年年度展望](#)



smallyu

Hello World

??

??

????

2017080 (6)

2017070 (2)

2017060 (3)

2017050 (2)

2017040 (2)

2017030 (3)

2017020 (3)

2017010 (4)

20160120 (1)

## 生命不息，折腾不止--hexo并不好玩

□ smallyu □ 2016-11-18 □ hexo

双十一妥妥地过去了。

对于一只无聊的野生单身汪来说，这样的节日确实，无聊。于是尝试了一下看起来<sup>13</sup>格很高的github+hexo博客。

可能时间隔得稍微有点长，双十一不是早就过去了么。。。

阮一峰在博客里这么写：

喜欢写Blog的人，会经历三个阶段。

第一阶段，刚接触Blog，觉得很新鲜，试着选择一个免费空间来写。

第二阶段，发现免费空间限制太多，就自己购买域名和空间，搭建独立博客。

第三阶段，觉得独立博客的管理太麻烦，最好在保留控制权的前提下，让别人来管，自己只负责写文章。

我就高高兴兴地奔着第三个阶段去了。

后面留坑，如果可能的话就分享一下在用github搭建博客过程中遇到的问题以及解决方法。

<<<(2016.11.19)-1

20160111 (2)

20160110 (3)

2016090 (1)

2016080 (6)

2016070 (1)

2016060 (4)

2016050 (4)

hexo是用node.js写的，github要用到git，所以首先要在电脑上安装这两样东西。

node.js: <https://nodejs.org/>

git: <https://git-scm.com/>

装好之后，多半需要配置环境变量

|           |                                                        |
|-----------|--------------------------------------------------------|
| NODE_PATH | D:\application\Git\bin                                 |
| path      | D:\application\Git\mingw32\libexec\git-core            |
| TEMP      | C:\Users\天天雨\AppData\Roaming\npm\node_modules\hexo\bin |

差不多这样，要配置git的和hexo的。

应用的安装顺序是：

1. 安装node.js
2. 安装git
3. 在git命令行中安装hexo

环境变量的配置顺序是：

1. git的bin和git-core
2. （安装hexo）

### 3. hexo

hexo 是全局安装的，git 命令行要用到的 npm 工具其实是 node.js 的，所以可以在 node.js 或者 git 的 CMD 中执行 hexo 的安装命令，都行（建议在 node.js 中，因为能成功）

```
npm install -g hexo-cli
```

这里的意思是，node.js 安装成功后就有 CMD 了，在这个 CMD 中始终可以使用 npm。如果环境变量配置有问题，会发生 git 的 CMD 里 npm 出错的情况。

安装 hexo 后要配置 hexo 的环境变量，所以说上面环境变量的配置不是一步完成的。

然后，，，

坑还是留着吧。

hexo: <https://hexo.io/> 有完整的文档，英文的

(2016.11.19)-1;

后来发现对于在 github 上部署的博客，git 的使用难于站点的搭建。

因为 hexo 要配置域名才能正常使用，图方便前几天就把 blog.how.gs 这个域名解析过去了。不过最终，现在还是回来了。

同样原因，留坑 .....

<<<(2016.11.19)-2

gthub搭建博客的缺点：

1. 每次发布文章都需要重新部署站点
2. 本地文件备份难
3. 百度不收录
4. Markdown真的好用？
5. 附件管理

第一个，我们常用的博客，每次发布文章只需要添加一条新的记录到数据库就好了。就像web开发中的Ajax。hexo是静态的博客框架。每一次新建文章，都需要执行clean命令清空数据库、缓存，然后执行g命令重新编译文件，生成博客内容，最后将public目录中的内容推到gtithub上。

这个过程稍微繁琐。如果拿ajax作类比，有异步数据交互的方法不用，非要每次都完全刷（部）新（署）网（网）页（站）吗？

如果博客内容非常多。百兆以下还将就，如果以上，每发布一篇文章都需要推那么数据上去呢。



所以每次发布文章都需要部署站点是缺点。

第二个是本地文件备份难。

比如多云转晴<sup>hower</sup>用的<sup>zblog</sup>，需要备份的数据有两部分，<sup>mysql</sup>中的配置数据和文件中的附件。

<sup>hexo</sup>的话需要备份的是当时选定的本地目录。这里的问题是原作者本意希望将<sup>hexo</sup>目录下的文件也都推送到<sup>git</sup>上面。可以前几天在尝试的时候，（由于能力不足）遇到一些不好解决的问题。在<sup>hexo</sup>推送文件的过程中会在本地生成隐藏的<sup>git</sup>目录，包含一些<sup>git</sup>信息。然后在<sup>github</sup>提交<sup>hexo</sup>模板目录时，会因为<sup>git</sup>已经被管理提交失败。

每次都手动删掉？（麻烦。。）

或许可以尝试用<sup>onedrive</sup>把目录挂载到网盘上。

第三个，百度不收录。知乎上可以找到很好的解决办法，于是，我还是觉得独立博客好用。

第四个，<sup>MarkDown</sup>会将内容解析成<sup>HTML</sup>，目的是提高可读性。这本身没什么问题，可猿们很多用<sup>sublime</sup>，换行不是像段落一样换行，是折行。当然这不是什么大问题。

第五个有点和第一个重了，好吧其实本来就三个。<sup>hexo</sup>的附件管理需要像管理网页资源一样放到<sup>sources</sup>文件夹。怎么用都感觉不如<sup>zblog</sup>。

<sup>github</sup>搭建博客的优点：

1. 速度理想，流量无限，空间很大
2. 13格高

hexo自然是有优点的，很多人都是冲着github的可靠性去的。

不过我相信大多数人更倾向于第二点。

(2016.11.19)-2;

[前一篇](#): [CET4 · 不屈的Writing路](#) (2016.12.8)

[后一篇](#): [春宵苦短日高起，不来](#)



smallyu

Hello World

??

??

????

2017080 (6)

2017070 (2)

2017060 (3)

2017050 (2)

2017040 (2)

2017030 (3)

2017020 (3)

2017010 (4)

20160120 (1)

## CET4 · 不屈的Writing路（2016.12.8）

□ smallyu □ 2016-11-02 □ CET4

(2016.12.8)

英语老师并不给口语不好的人机会。😞

原本有 Unit 1 ~ Unit 6 一共六个单元的内容需要背。是的这里列出了两篇，默写的。

时间隔了好久，从1.9的最后一篇四级作文到12.7的第一篇课文翻译。英语确实很难啊！

四级作文记了两篇坚持不下去，就搁置了。当时两篇作文用了两个星期的时间。然后就到最近，老师要求被课文翻译，一共六个单元。反正快考四级了，英语怎么学都是学，就决定背一下，期末还会考课文，背了也不亏。两天时间背了两篇。课文翻译比四级作文要简单很多，句子大多是简单句，虽然也有生词，不过还不算是问题。

问题是老师说只能背，不能背写。平时成绩的问题。

这种要求真的可以完全把我难住。如果可以背写的话还能试一下，但是背，嗯，挂科无憾。

所以就决定可以放心地旷课了，几乎没有故意旷课过，感谢英语课给了我旷课的勇气。😏

有的人英语本来就很好，不用做任何准备裸考就能过。然而有的人T\_T。

以我三个月断断续续的复习、做了很多篇模拟题以及很多份真题的经验来看，我敢很肯定的说，今年的四



20161110 (2)

20161101 (3)

20161091 (1)

20161081 (6)

20161071 (1)

20161061 (4)

20161051 (4)

级考试，肯定过不了。。。

难不成为了听好听力，就从小学的字母发音开始恶补基础吗，应该为了写好作文，就从小学的遣词造句开始学习语法吗。你努力，拼命在追赶的，是别人的起点。为什么非要走这条路？如果可以在其它路上超越别人，可以凭借其它技能找到工作，挣到钱，还有必要依靠这个吗？

可是连这个都搞不定，不甘心啊。

(Unit 2) Realizing the great national rejuvenation, which we define as the Chinese Dream, has been the greatest Chinese expectation since modern times. It basically means achieving prosperity for the country, renewal of the nation and happiness for the people, thus ensuring that every enterprising Chinese carries, generation after generation, the firm conviction that a better life is accomplished through persistent effort. People should achieve their prosperity through diligence, courage, creativity and determination instead of aid from society or other people. Each individual is a participant and a designer in the casue of realizing the Chinese Dream, it is a dream not only for the entire nation, but also for every Chinese.

(2016.12.7)

(Unit 1) Nowedays, many young people no longer choose "stable" jobs. Instead, they prefer to start their own businesses and realize their self-value through their own wisdom and efforts. Young entrepreneurship is the source of national economic vitality in the future. The success of entrepreneurs not only creates fortune, increases job opportunities, improves people's life, but it is also good for the country in long term. Entrepreneurs are a driving force in upgrading China's economy. Especially for the time being, our country is encouraging people to start their own businesses and make innovations and giving pocily support for medium

and small businesses. This further arouses young people's enthusiasm to start their own businesses.

(2016.11.9)

Many cultural and sporting events are held on our campus each year, and I often participate actively in them. I find it beneficial to be involved in various campus activities. The speech contest held last month has benefited me most.

The speech contest was held by the Students Union. The moment I was informed I would compete with more than 100 competitors, I became exceedingly nervous. But my roommates encouraged me and cheered me on. They are all willing to be my audience whenever I wanted to practice speaking, giving me much courage. They also offered me constructive advice to help me improve my presentation skills. Thanks to their support, I ranked third in this speech contest. The speech contest not only gave me an opportunity to practice eloquence and strengthen my confidence, but also deepened the friendship between my roommates and me.

I will take part in such campus activities as a speech contest. I would contend they should be organized regularly to make our college life more colorful and meaningful.

(2016.11.2)

If a foreign friend is coming to visit our campus, the most interesting place I'd like to take him/her to see is the school art gallery. There are many exhibitions held here periodically, including calligraphy, painting, phot

ography and sculpture, etc. Most of exhibits are created by our university students, but sometimes, there are some authentic works of well-known artists, like the famous painting Shrimp by Qi Baishi.

The reason why I take him/her to the school art gallery is that I hope he/she can know more about the campus life of Chinese university students. I'd like him/her to know that our campus life is not defined to study; on the contrary, our campus life is rich and colorful. In addition, compared with other places, the school art gallery is more representative, because it collects excellent works of our university students and it shows our intelligence and wisdom intensively.

I'm glad that our school has this art gallery and I believe that everyone comes to see it will like it.

[前一篇](#) · [课程手记](#) · [Hello，移动web](#)

[后一篇](#) · [生命不息，折腾不止——hexo并不好玩](#)



smallyu

Hello World

??

??

????

2017080 (6)

2017070 (2)

2017060 (3)

2017050 (2)

2017040 (2)

2017030 (3)

2017020 (3)

2017010 (4)

20160120 (1)

## 课程手记 · Hello，移动web

□ smallyu □ 2016-10-18 □ 移动web

内容来自慕课网WebApp开发课程的内容，看完之后回想起来，确实有必要记录一下，便于温故

课程地址：<http://www.imoooc.com/learn/494>

### Pixel移动开发像素基础

px: CSS pixels 逻辑像素，浏览器使用的抽象单位

dp/pt: device independent pixels 设备无关像素

dpr: devicePixelRatio 设备像素缩放比

计算公式： $1px = (dpr)^2 * dp$

DPI: 打印机每英寸可以喷的墨汁点（印刷行业）

PPI: 屏幕每英寸的像素数量，即单位英寸内的像素密度

目前，在计算机显示设备参数描述上，二者意思表达的是一样的（单位为硬件像素，非<sub>px</sub>）

PPI越高，像素数越高，图像越清晰，但可视度越低（小），系统默认设置缩放比越大

20160111 (2)

20160110 (3)

20160109 (1)

20160108 (6)

20160107 (1)

20160106 (4)

20160105 (4)

|            | <i>ldpi</i> | <i>mdpi</i> | <i>hdpi</i> | <i>xhdpi</i> |
|------------|-------------|-------------|-------------|--------------|
| <i>ppi</i> | <b>120</b>  | <b>160</b>  | <b>240</b>  | <b>320</b>   |
| 默认缩放比      | <b>0.75</b> | <b>1.0</b>  | <b>1.5</b>  | <b>2.0</b>   |

Retina屏（高清屏） $dpr$ 都是大于等于2

## Viewport视图

思考下面一个问题：



手机浏览器默认认为我们做了两件事情：

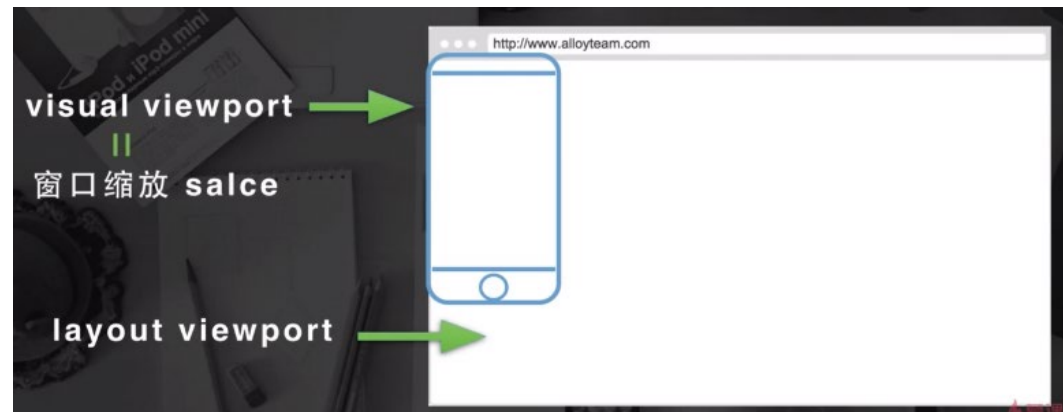
一：页面渲染在移动设备分辨率大小的viewport

二：缩放

为什么渲染时，要有Viewport？

为了排版正确。

viewport 有两个，visual viewport 和 layout viewport



Viewport Meta 标签

```
<meta name="viewport" content="name1=value1,name2=value2">
```

width: 设置布局 viewport 的特定值 ("device-width")

initial-scale: 设置页面的初始缩放

minimum-scale: 最少缩放

maximum-scale: 最大缩放

user-scalable: 用户能否缩放

常用写法:

```
<meta name="viewport" content="width=device-width,initial-scale=1,user-scalable=no">
```

## Flex弹性盒子布局

```
.nav{display:-webkit-flex;}/父元素*/
```

```
.item{flex:1;}/子元素*/
```

display:-webkit-flex 标示使用弹性布局

flex:num 占容器的比例



## Flex不定宽高的水平垂直居中

```
.parent{
```

```
justify-content:center;}/子元素水平居中*/
```

```
align-items:center;/*子元素垂直居中*/

display:-webkit-flex;

}
```

## Web的Flex弹性盒模型

[http://oldblog.smallyu.net/web/web\\_flex.html](http://oldblog.smallyu.net/web/web_flex.html)

访问这个链接查看flex的所有属性案例

## 响应式设计

### 兼容性

·iOS可以使用最新的flex布局

·android4.4以下，只能兼容旧版的flexbox布局

·android4.4及以上，可以使用最新的flex布局

| 新flex布局                  | 旧flexbox布局                 |
|--------------------------|----------------------------|
| display: -webkit-flex;   | display: -webkit-flex-box; |
| -webkit-flex: 1;         | -webki-flex-box: 1;        |
| justify-content: center; | box-pack:center;           |
| align-items: center;     | box-align: center;         |



## 媒体类型

screen（屏幕）

print（打印机）

handheld（手持设备）

all（通用）

## 常用该媒体查询参数：

width—视口宽高

height—视口宽高

device-width—设备的宽度

device-height—设备的高度

orientation：检查设备处于横向（landscape）还是竖屏（portrait）

## 设计点一：百分比布局

仅仅使用媒体查询来适应不同的固定宽高设计，只会从一组<sub>CSS</sub>到另一组<sub>CSS</sub>的切换。两种之间没有任何平滑渐变。当没有命中媒体查询时，表现就会变得不可控制（滚动，换行）。

## 设计点二：弹性图片

很简单，如同多一点布局使用百分比，图片也使用百分比。其背后的思路是：无论何时，都全包在图片的元素宽度范围内，以最大的宽度同比完整的显示图片。

### 设计点三：重新布局，显示与隐藏

当页面达到屏幕手机宽度的时候，很多时候就要放弃一些传统的页面设计思想。力求页面简单，简介。所以我们必须要做出以下处理：

1. 同比例缩减元素尺寸
2. 调整页面结构布局
3. 隐藏冗余的元素

### 移动web特别样式处理

#### 高清图片

在移动web也页面上渲染图片，为了避免图片产生模糊，图片的宽高应该用物理像素单位渲染，即是100\*100的图片，应该使用100dp\*100dp

```
width:(w_value/dpr)px;
```

```
height:(h_value/dpr)px;
```

#### 一像素边框

同样是retina屏幕下的问题，根本原因：1px使用2dp渲染

border:0.5px (FALSE) 仅仅ios8可以用

sacleY(.5)

### 相对单位rem

为了是一个各大屏幕的手机，px略显固定，不能根据尺寸的大小而改变，使用相对单位更能体验页面兼容性

em：根据父节点的font-size为相对单位

rem：是根据html的font-size为相对单位

em在多层嵌套下，变得非常难以为何

rem更加能作为全局统一设置的度量

rem的基值设置为多少好？

为了使用各大手机屏幕，rem=screen.width/20

不使用rem的情况：font-size

### 多行文本溢出

单行文本溢出，对title类的使用非常多，而多行文本溢出，在详情介绍则使用的比较多

### 终端交互优化

#### 基础事件

## Tap

### 300毫秒的故事

移动web 页面上的click 时间响应都要慢上300 毫秒。移动设备访问的web 页面都是pc 上的页面。在默认的情况下，viewport（980px）的页面往往都是需要“双击”或“捏开”放大页面，来看高清页面。而正是为了确认用户是“双击”还是“单击”，safari 需要300ms 的延迟来判断。

### 300ms延迟怎么破？

使用Tap 时间来代替click 事件

自定义Tap事件原理：

在touchstart、touchend 时记录时间、手指位置，在touchend 时进行比较，如果手指位置为同一位置（或允许移动一个非常小的位移值）且时间间隔比较短（一般认为是200ms），且过程中未曾触发过touchmove，即可认为触发了手持设备上的“click”，一般称它为“tap”。（移动框架库Zepto.js）

### Tap“点透”的bug



看过这个过程应该可以秒懂“点透”

### Tap透传的解决方案

1. 使用缓动动画，过度300ms的延迟
2. 中间层dom元素的假如，让中间层接受这个“穿透”
3. “上下”都使用tap时间，原理上解决tap透传事件（但不可避免原生标签click事件）
4. 改用Fastclick的库（听过最新的zepto已经fixed掉这个bug）

## 触摸Touch

### Touch基础事件

触摸才是移动设备的交互的核心事件

touchstart：手指触摸屏幕触发（已经有手指放屏幕上不会触发）

touchmove：手指在屏幕滑动，连续触发

touchend：手指离开屏幕时触发

touchcancel：系统取消touch时候触发（不常用）

除常见的事件属性外，触摸事件包含专有的触摸属性

touches：跟踪触摸操作的touch对象数组

targetTouches：特定时间目标的touch对象数组

changeTouches：上次触摸改变的touch对象数组

每个touch对象包含属性：

clientX：触摸目标在视口中的<sub>x</sub>坐标

clientY：触摸目标在视口中的<sub>y</sub>坐标

identifier：表示触摸的唯一ID。

pageX：触摸目标在页面中的<sub>x</sub>坐标（包含滚动）。

pageY：触摸目标在页面中的<sub>y</sub>坐标（包含滚动）。

screenX：触摸目标在屏幕中的<sub>x</sub>坐标。

screenY：触摸目标在屏幕中的<sub>y</sub>坐标。

target：触摸的DOM节点目标。

根据touch对象跟踪触摸具体位置，从而判断触摸交互或手势事件。

BUG

Android只会触发一次touchstart，一次touchmove，touchend不触发

解决方案

在touchmove中加入： `event.preventDefault()`。

但注意

`event.preventDefault()`会导致默认行为不发生，如`scroll`，导致页面不滚动。

弹性滚动

当客户端的页面滚动到顶部或底部的时候，滚动条会收缩并让我们多滑动一定距离。通过缓冲反弹的效果，带给用户良好的体验。

移动web页面也是拥有这样能力的，但滚动有几种情况需要考虑：

body层滚动：（系统特殊化处理）

再带弹性滚动， `overflow: hidden`失效， `GI`F和定时器暂停

局部滚动开启弹性滚动：

```
body{

 overflow:scroll;

 -webkit-over-scrolling:touch;

}
```

注意Android不支持原生的弹性滚动！但可以借助三方库*iScroll*来实现

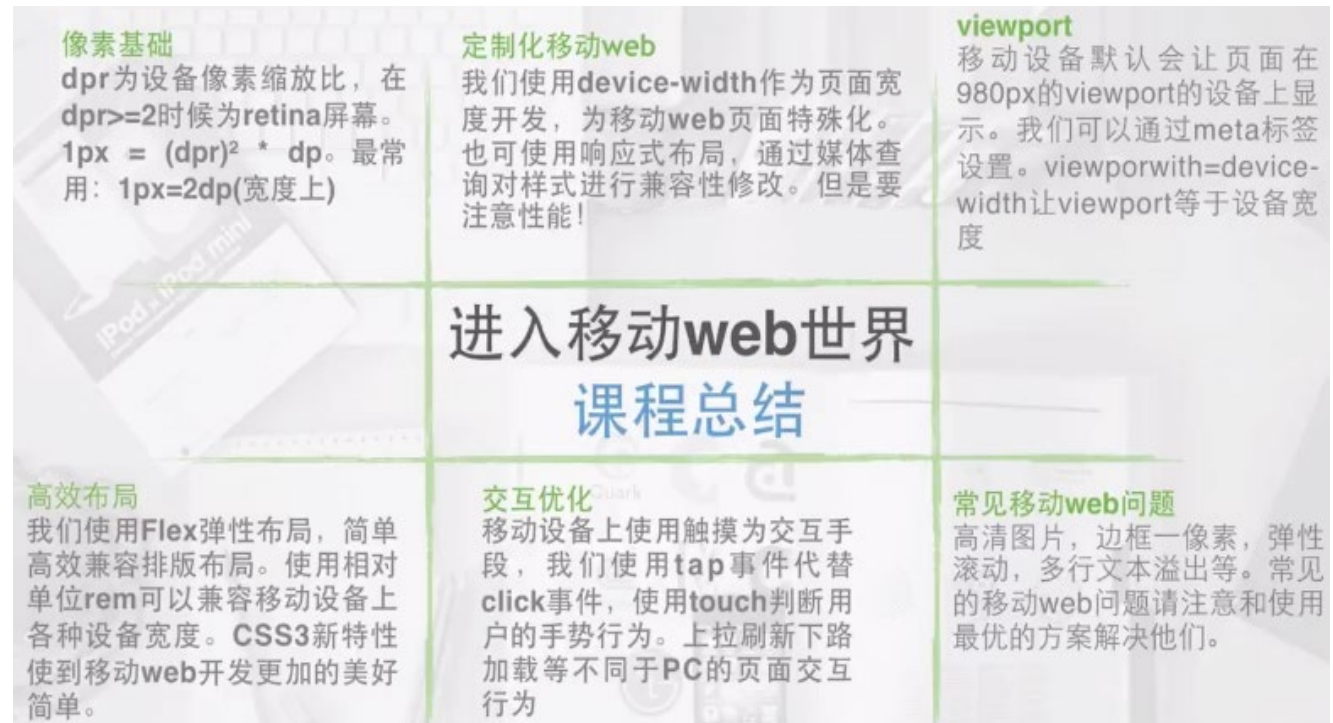
下拉刷新

顶端下拉一小点距离，页面弹性滚动向下

上拉加载

使用scroll事件而不是touch时间（android有bug）

总结



|                                                                                                                |                                                                                                     |                                                                                                                    |
|----------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| <p><b>像素基础</b><br/>dpr为设备像素缩放比，在dpr&gt;=2时候为retina屏幕。<br/><math>1px = (dpr)^2 * dp</math>。最常用：1px=2dp(宽度上)</p> | <p><b>定制化移动web</b><br/>我们使用device-width作为页面宽度开发，为移动web页面特殊化。也可使用响应式布局，通过媒体查询对样式进行兼容性修改。但是要注意性能！</p> | <p><b>viewport</b><br/>移动设备默认会让页面在980px的viewport的设备上显示。我们可以通过meta标签设置。viewportwith=device-width让viewport等于设备宽度</p> |
| <p><b>进入移动web世界<br/>课程总结</b></p>                                                                               |                                                                                                     |                                                                                                                    |
| <p><b>高效布局</b><br/>我们使用Flex弹性布局，简单高效兼容排版布局。使用相对单位rem可以兼容移动设备上各种设备宽度。CSS3新特性使到移动web开发更加的美好简单。</p>               | <p><b>交互优化</b><br/>移动设备上使用触摸为交互手段，我们使用tap事件代替click事件，使用touch判断用户的手势行为。上拉刷新下路加载等不同于PC的页面交互行为</p>     | <p><b>常见移动web问题</b><br/>高清图片，边框一像素，弹性滚动，多行文本溢出等。常见的移动web问题请注意和使用最优的方案解决他们。</p>                                     |

前一篇 [实习报告](#) 软件实验：Protel99（Altium designer 6.9）



后一篇: [CET4 · 不屈的Writing路](#) (2016.12.8)



smallyu

Hello World

??

??

????

2017080 (6)

2017070 (2)

2017060 (3)

2017050 (2)

2017040 (2)

2017030 (3)

2017020 (3)

2017010 (4)

20160120 (1)

# 实习报告·软件实验：Protel99（Altium designer 6.9）

□ smallyu   □ 2016-10-16   □ Protel99

## 一、实验目的

- 1.掌握电路原理图的设计步骤
- 2.掌握原理图报表文件的生成方法与作用
- 3.熟悉元器件库编辑器及其原理
- 4.学会使用印制电路板编辑器，达到自动布线画电路板比较熟悉的程度

## 二、实验内容

### 1.电路原理图的绘制

实验在仿真软件Altium designer 6.9上进行。绘制电路原理图大体分为四个步骤，新建原理图文件、放置元件、设置元件参数、画线。

#### 1) 新建原理图文件

进入AD主界面，点击菜单栏File选项，在出现的菜单选项中选择New>Schematic，软件自动生成的Free Documents工程以及Source Document文件夹和Sheet1.SchDoc原理图文件均会出现在左侧File View操作面板中

|              |
|--------------|
| 20160110 (2) |
| 20160109 (3) |
| 20160909 (1) |
| 20160809 (6) |
| 20160709 (1) |
| 20160609 (4) |
| 20160509 (4) |

。

## 2) 放置元件

明确并在元件库中寻找电路中需要放置的元件。寻找元件有两条思路。

1>点击菜单栏Design选项，在出现的菜单选项中选择Browse Library...，主界面右侧会弹出Libraries面板。点击Libraries...按钮，出现Available Libraries窗口，点击Install...按钮加载常用的两个杂项集成库Miscellaneous Connectors.IntLib和Miscellaneous Devices.IntLib，然后点击close按钮关闭窗口。

加载元件库后就可以在元件库操作面板的下拉选项中选择相应的库，并用上下键选择、预览元件，逐个寻找所需元件。

2>点击search...按钮，出现Libraries Search窗口，在输入框中输入想要搜索的元件名，选择Libraries on path选项并设置Path的值就可以在遍历本地目录下的所有元件库搜索所需元件。

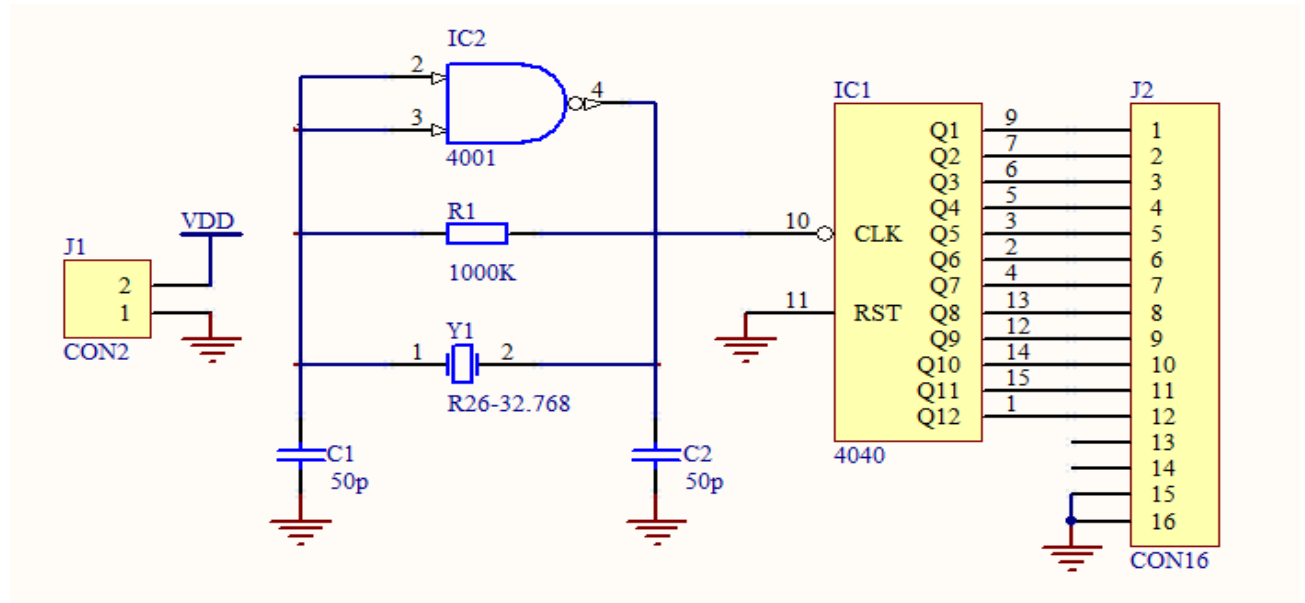
如果出现元件库中没有所需元件，可以点击File，选择New>Library>Schematic Library，左侧File View面板中会出现Schlib1.SchLib文件，单击选定，菜单栏中选择Tool的New Component，就可以自定义元件，之后保存，直接作为库加载就可以用了。

## 3) 设置元件参数

元件需要设置的参数一般有三个，Designator（元件名）、Comment（元件注释）、Footprint（元件封装）。按照需要分别设置就可以了。

## 4) 画线

原理图操作界面中，点击菜单栏中Place，选择Wire，鼠标光标会变成十字形，然后就可以根据需求按照网格放置线条了。



## 2.网络表的生成与作用

1) 网络表的生成：画好原理图后，点击菜单栏中的Design，选择Netlist for document>Protel，在原理图同目录就会生成Sheet1.NET文件。

2) 网络表的作用：网络表包含了原理图中的所有元件以及元件参数，包括值的大小和封装信息等，用于在PCB电路的设计中导入元件以及校验元件信息。

### 3.PCB（印制电路板）的绘制

绘制PCB可以分为两个步骤，导入元件和布线。

#### 1) 导入元件

点击菜单栏中的Project，选择Show Differences...，会弹出Choose Documents To Compare窗口，在下角勾选Advanced Mode选项，窗口会变成两栏。左边一栏选中已经保存过的PCB文件，右边一栏选择刚才生成的网络表文件，点击OK，就会出现工程差异的一个比对窗口，右击任一列表中的文件，选择Update All >> PCB Document [PCB1.PcbDoc]选项，然后点击窗口左下角Create Engineering Change Order...按钮，就出现了Engineering Charge Order窗口。

在出现的窗口中点击左下角的Execte Charges执行更改，之后就会出现一个执行情况统计的窗口。如果窗口中有错误项，就需要关闭窗口，根据错误提示更改、调整出现问题的元件。如果没有错误项，那么所有元件就成功地导入PCB文件中了。

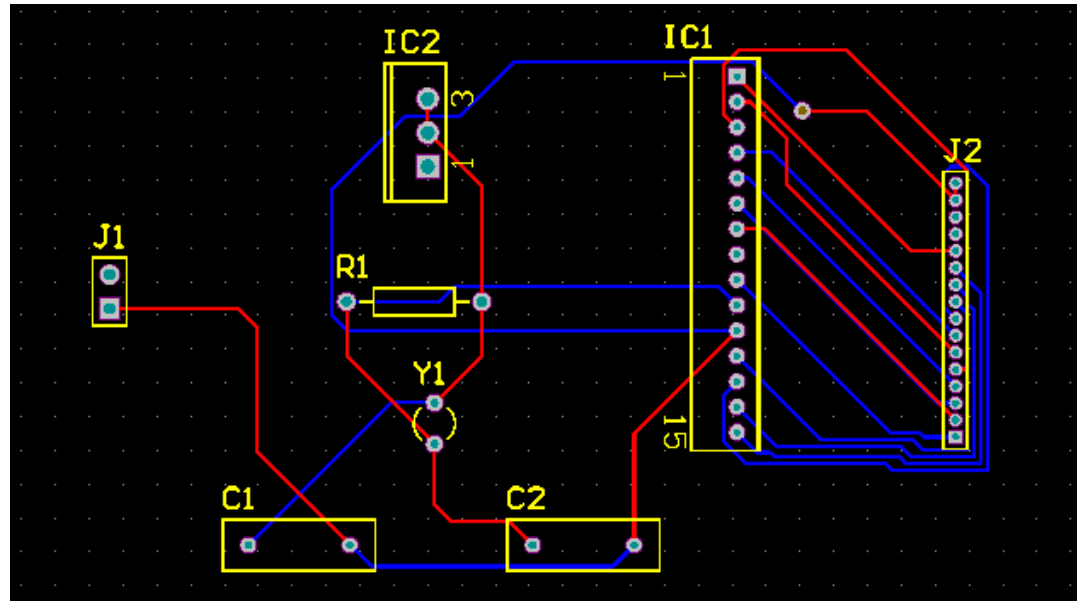
自定义放置元件的位置，然后点击Place，选择Line，画置印刷版的边界，限制电路板的大小，不然在自动布线的时候软件会卡死。

#### 2) 布线

点击菜单栏的Auto Route，选择All，弹出窗口中保持默认选项选择OK，软件就可以在5秒之内自动为PCB印制板布线。

AD在自动布线的时候不需要切换到Keep-Out Layer画布也可以实现双面板的布线，因为在弹出窗口中会选择布线格式。

最后检查PCB是否有错误。



应付作业的内容，先在电脑上写的，就顺手发上来。对软件的学习没有借鉴意义。

[前一篇](#) [博观而约取，厚积而薄发](#)

[后一篇](#) [课程手记](#) [Hello, 移动web](#)



smallyu

Hello World

??

??

????

2017080 (6)

2017070 (2)

2017060 (3)

2017050 (2)

2017040 (2)

2017030 (3)

2017020 (3)

2017010 (4)

20160120 (1)

## 博观而约取，厚积而薄发

□ smallyu □ 2016-10-06 □ 王者荣耀

博观而约取，厚积而薄发。

可能在说这句话的时候，我也不知道自己想要表达什么。（真的想写个技术类的，分享一下自己的知识，顺便提升一下自己的格调。想想而已。）听着陈旧的许嵩的歌，在图书馆信息阅览室里，用着极不舒服的键盘，在XP系统的word上，写这篇日志（的时候心情是沉重的）。

许嵩的歌不新，却异常喜欢。可能仅仅是耳朵适应了他的歌的节奏和频率，不想再猎新奇。虽然我还年轻。天下雨了，打开窗户就可以感受到雨水带来的降温，和寒意。博观而约取，厚积而薄发，是刚才在意林杂志上看到的一句话。我们不止一次看到过类似这样的话，“我们总是对博学能够侃侃而谈的人感到敬佩，可谁又看到他们十年寒窗门前苦读的光景。”不鸣则已，一鸣惊人？我不屑一笑。

总是希望自己以后可以变成厉害的人，希望现在没有做的一些事情，不会给自己留下遗憾，希望自己不要一无是处，然而。国庆节的假期，回家的回家，旅游的旅游。我不想回家，也没人和我一起旅游，所以，只剩下宅。但是又不甘愿浪费时间。所以，就有一段时间呆在图书馆了。短短几天时间，根本不足够我学习到多少东西。正是有这样的想法，让我对未来倍感迷茫。

我知道自己可以不在乎游玩，没兴趣聚会。可以像暑假一样一下子呆在电脑前两个月。可是每当看到朋友圈的各种动态，就稍微有点按耐不住了。是羡慕吗？是羡慕吧？或者是攀比心理？或者是担心被别人“小看”的心理？（看，我们的生活丰富多彩，你呢？）

20160110 (2)

20160109 (3)

20160909 (1)

20160809 (6)

20160709 (1)

20160609 (4)

20160509 (4)

看到NCRE三级网络技术的教材，才接触到线程、分页、作业、UPS等概念。看到软考的网络工程师考题，才亲身接触到，有那么多我连名字都看不懂的东西。网上N多人说，NCRE完全没用，软考除了评职称也没什么用。可为什么我却觉得，那些考试的内容都不简单？着实不是突击就可以把证书考下来的程度。

在这个博客网站，不知道可以写些什么内容。写这句话的同时，发现自己终究还是怕出丑。是的，担心博客文章的质量太低，像其他独立博客一样，发发牢骚，写写XXX的配置，贴点福利内容，科学上网方法等。每当看到那样的文章内容，总会觉得一下子就可以知道站长的水平了。什么样的文章，由什么样水平的作者来写。（所以你也应该知道多云转晴hower是个彻头彻尾的萌新小白了吧？）网络上的大牛很多，牛到难以想象。

这段时间，一直以学习web开发自居。学到了什么？每当想，“自己写个小东西出来吧”的时候，会有一个始终回答不了的问题，写什么？入门容易学精难也就这么回事吧，高不成低不就的。想到写一个在线文件管理器，那么，写出来的东西给谁用呢？现在已经不是几年前独立网站的概念盛行，文件管理器还会用爱特的年代了。vps越来越流行，也许未来的某一天都装windows系统了，阿里云现在不就有这样的主机在卖吗。

程序方面，目前就有很多优秀的在线文件管理系统，不管是中文的还是英文的。这些程序有什么功能不完善的地方需要自己改进吗？完全没有。那么，如果自己开始写这样一个程序，除了基本的copy、rename、unzip，都是PHP内置函数的使用，难以扩展它的功能了。界面方面，有好用的jquery插件，配置上就可以用。说的轻描淡写了一点。

站点功能的话，还是按需开发比较好，有时候我们不需要乱七八糟的功能。<http://oldblog.smallyu.net/demo/uplaod>只有简单的上传功能，不上KB的代码，却够我用用了。（没有安全措施，牛们不要黑我。）其他类型的内容也差不多是这样，在没有实施之前想法就被自己推翻了。





最近迷上一款游戏，王者荣耀。

和LOL比起来，节奏快，打一场时间短，也算是变相的节省时间了。和英魂之刃比起来，画面好，技能炫。当然，王者荣耀是手游，不过我一般在电脑上用模拟器玩。为什么说是迷上呢？以前是不玩网游的。玩过，一点兴趣都没有。（如果保卫萝卜不算网游）后来和一个朋友一起玩王者荣耀。5V5种类的游戏很多。玩了一段时间后，发现这种竞技游戏确实有不少吸引人的地方。

自由度非常高。单机游戏和RPG网游是没法比的。队友和对方分别有5个英雄，一队间不可重复。假如荣耀有50个可选英雄，那么用排列来算，一队间可能的情况有：

$$C_{50}^1 + C_{49}^1 + C_{48}^1 + C_{47}^1 + C_{46}^1$$

这个数是多少？懒得算了。实际上不会有这么多种情况，不可能每个人都有这么多英雄可选，但有这么多种

可能的队伍组合。那么，自己一队，对方一队，对战的组合就要二次方了。玩一款游戏，常常因为“玩腻了”不想再玩，单机游戏会找秘籍输命令尽可能看看这个游戏到底“有什么”。但是这个……

合作的精神。个人技术可以秀一时，秀不了一场。合作的好能虐杀对方，不懂合作各打各的人家一个一个也能团死你。那么，其实我还是比较喜欢辅助。不用带兵线，不用打野，工作也就是在队友身边放放技能就好了，还可以心安理得的全场乱跑，辅助队友嘛！被人杀了，我是辅助，血薄！杀的人少？我是辅助！只是排位就不敢那么随便了。

竞技的精神。这本身就是竞技游戏。排位。赢了，还想再赢。输了，不服，想赢回来。感觉和赌博差不多。最怕的也就是挂机了。只要有人挂机，那基本就是输了。暂时没输也就垂死挣扎，只要对方不菜。于是，每当因为有人挂机输了的时候，总觉得特别不爽，不行，还得再来！总不会一直有人挂机吧？对了，还真遇到连3场有人挂机的。

亚瑟是小短手，攻击距离近，除了第三个技能能跳一下，跑路是硬伤，一般带个闪现就挺好。最怕狄仁杰，远距攻击还带减速，见了就跑吧。如果有三技能见了后羿还是能拼一拼的。妲己的减速也厉害，不过战士不应该怕她。

孙悟空据说最强。定位属于刺客和战士。血薄，靠暴击，死得快，打一下就跑的那种。

后羿是热门英雄，经常看见他的身影。身为射手主要输出，远远地打就行了，三技能用处不详，什么时候都可以放，但不知道什么时候能够发挥好作用。

蔡文姬比较讨人喜欢，（我单身久了？）因为可爱。

孙斌看样子是S4的主角了。（现在是正S4呢吧？）辅助很强，一技能眩晕，二技能是2s的加速，三技能就

挺酷的，时光炸弹，到后期冷却也不长，伤害不高，减速能力很强，团的时候丢对位置能把对方\*死。

.....

另一款也不错的游戏，贪吃蛇大作战。

这要不是单机游戏，估计它的火爆程度不会低于荣耀。规则很简单，生存。不是长大，是生存。前几天玩slither.io的时候，不断有一个想法冒出来：“蛇蛇大作战杀气太重了，我不敢再玩。”暴走蛇蛇也好，supersnake.io也好，总会有人不断想要杀掉你，你自己在玩的时候也会不断想要杀掉其它的蛇，尤其是大的，因为长得快嘛，积分很容易就上去。如果抢银行变成一种合法的行为，谁不愿意去抢？

就这么乱乱地写写吧，没毛病。

前几天杂志上看到一篇文章，论题是“你相信男女之间有纯洁的友谊吗？”我不信。

我不只不相信男女之间没有友谊，还有男男之间，女女之间，所以说，我不相信友谊。没有永恒的友谊，只有永恒的利益，不是空穴来风。同时，我还看到一篇文章，论题是，“世界上的感情，都是一种仪式。”大概意思是说，仪式其实很重要。有仪式不一定有感情，但要有感情就应该有仪式，以表达，“我在乎你，我需要你”这类的含义。因为对方需要这样的仪式来相信你。

看到这样的文章，会引起我一些思考，也会让我想起一些事情，所以有时候需要看看杂志。比如，上面的话题，我不得不想起一个大学同学。

我很喜欢的一个朋友。

[前一篇: 叙述一次不愉快的拼车](#)

[后一篇: 实习报告 . 软件实验: Protel99 \(Altium designer 6.9\)](#)



smallyu

Hello World

??

??

????

2017080 (6)

2017070 (2)

2017060 (3)

2017050 (2)

2017040 (2)

2017030 (3)

2017020 (3)

2017010 (4)

20160120 (1)

# 叙述一次不愉快的拼车

□ smallyu    □ 2016-09-16    □ 拼车



过去几天时间了，总是心有不甘。就当一次教训。事情很简单。

家里有事，正好周末，就在周六早上早早地回家了。周日整整一天在处理该处理的事情。周一早上，出发回到了这座城市。

坐大巴来的，停在了汽车站。差不多十点半。汽车站离目的地还有一段不短的距离，挤公交车得两个小时。其实本来两站路就能到，最多也就一个半小时，因为万恶的修路，修了老长时间了，公交车不能过。一般到断路部分是坐摩的过去的，不太熟悉路。

于是有点嫌麻烦，天正热，还要奔波两个小时，就想到了拼车。上次拼车正赶上开学，人多，三十块钱不到一个小时就到了。原想快十一点了，最好能尽快去，下午还有事，而且三十块钱就当用来买时间了，不算太

20160110 (2)

20160109 (3)

20160108 (1)

20160107 (6)

20160106 (1)

20160105 (4)

20160104 (4)

亏，还图个省心省事，就搭了一个汽车站出口附近拉人的出租车，说拼车。

上次是开学季，所以三十就能搞定，这次没什么人，我也知道路不近，司机要五十，随口对付了几句，就答应了。当时车里只有我一个人，在等。先付了钱的。

等啊等。

特喵的，想想都来气。等了一个小时。是的，硬是到了十一点半，那个司机没拉到人。

怎么办？

等了这么长时间，难不成中途放弃？可是时间都出去了，又快到中午，比刚来更热了。这时候，出租车司机回来了，说要我稍等，让另一辆车拉我去吧。以前也遇到过这样的情况，反正我给钱了，剩下的应该交给他处理，不管坐哪个车只要我能去就行。我是这么想的。

另一辆车来了之后，他在找人。

我又等了二十分钟。同时，接到了朋友的电话，说中午稍微有点事。手机信号不好，正好那天我的手机欠费了，只能接电话，不能打电话，没太听清楚说的是什么。总之能早点就早点吧。心想。

人找到了，还没完。上车后，还要去接两个人。打车的另外一个人的同伴。让人抓狂的是，没找到，找了半个小时。重点是那个人的同伴第一次来这个城市，不知道他们自己现在身处哪儿。问他们在哪条路？他们看了看路标后就说了。对，指示哪个方向是哪条路的那种，导向路标。

快十二点半了。

如果我当时坐公交车，省好多钱不说，现在也就到了。可是现在？难不成弃车？去得更晚。

于是我耐了一下心。半个小时后，拼车的人到目的地，下车了。他们给了司机七十。本来说好的五十，因为他的同伴耽误了，加了点钱。一点了。

再然后，司机问我去哪儿？我说了。

再一次特喵的。原来他本来不知道我去的是个什么地方，是我原先拼的那个车得司机忽悠了他。他拿出手机导航，一查，十几公里。于是，你猜他会干嘛？

加钱。

我说那是你和前一辆车那个司机的事。钱我已经付过了，说好了的。

他说，要么加钱，要么你下车，自己坐公交车回去。

当时是在有点偏远的地方，我的目的地本来就是郊区，哪有那么多方便的公交车让你坐。问题是就算有车，我也不清楚该坐哪路车，手机没网想查也查不了，同时万一要是还遇上修路那段，不还得折腾一下么，修路那段好点十块，差点十五。

我加钱了。

人在汽车里，不得不低头。

火大，没办法。

加了二十。

二十分钟后，到达目的地。

他说，小兄弟，真不是想坑你钱，你自己看导航，真挺远的，要是两三公里我肯定就顺你了。看嘛，一共走

了三十二公里。

我也不知道三十二公里是什么概念。

百度地图提示，单就我的路程，按驾车的最短路程算，38元。

其实还好，也没什么，以后注意点，小心点。面对这种事情，暂时想不出什么好的解决办法。毕竟回家也是真的有事。

我回家干嘛去呢，当孝子。



祭奠我已故的伯父。

[前一篇](#) [（失败2016.9.16）](#) [（程序猿养成计划）](#) [PHP参考手册实例遍历计划](#)

[后一篇](#) [博观而约取，厚积而薄发](#)







smallyu

Hello World

??

??

????

2017080 (6)

2017070 (2)

2017060 (3)

2017050 (2)

2017040 (2)

2017030 (3)

2017020 (3)

2017010 (4)

20160120 (1)

## （失败2016.9.16）（程序猿养成计划）PHP参考手册实例遍历计划

□ smallyu   □ 2016-08-29   □ PHP 程序猿养成计划

就是把参考手册里面的实例代码打一遍。

参考手册涵盖了该语言的任何内容（用法不包括理解）。不要笑我做无用的事情，这就是我的学习方法。

决定一个小节就用一篇文章来承载。有的小节内容会很短，但多云转晴hower的文章列表里面不会跳票。虽然这样做有可能会拉低站点文章的质量（本来就不高），但有的很长，如果采用另外的处理方法，比如把短的小节整合，文章名就难取了，和PHP参考手册上面的内容也难以对应。

同时，会建立Python参考手册和PHP参考手册的分类。

Just do it!

PS: 参考手册就是目前PHP官方文档最新的版本

参考手册在线版: <http://php.net/manual/zh/>

参考手册下载地址: [http://cn2.php.net/distributions/manual/php\\_manual\\_zh.chm](http://cn2.php.net/distributions/manual/php_manual_zh.chm)

（失败2016.9.16）

20161110 (2)

20161101 (3)

2016109 (1)

2016108 (6)

2016107 (1)

2016106 (4)

2016105 (4)

这样做确实没有意义。

发布文章后的几天我翻出文章再看时，看不懂。

（同时）

删除实例文章。

撤销分类。

前一篇: [（失败2016.9.16）（程序猿养成计划）Python参考手册翻译计划](#) 后一篇: [叙述一次不愉快的拼车](#)



smallyu

Hello World

??

??

????

2017080 (6)

2017070 (2)

2017060 (3)

2017050 (2)

2017040 (2)

2017030 (3)

2017020 (3)

2017010 (4)

20160120 (1)

## （失败2016.9.16）（程序猿养成计划）Python参考手册翻译计划

□ smallyu   □ 2016-08-24   □ Python 程序猿养成计划

这可能是无稽之谈。

为什么目前流传的高质量的Python中文参考手册少之又少呢？

1. 热度。不论。
2. 难度。篇幅比PHP的长很多。

目的？

学习。

能坚持多久？

谁知道。

20160110 (2)

20160100 (3)

2016090 (1)

2016080 (6)

2016070 (1)

2016060 (4)

2016050 (4)

质量？

比机器翻译差一点。

翻译目标：

Python 版本： 2.7.12

原版手册： [Python Manuals.zip](#)

翻译计划？

应该会是个漫长的过程。篇幅真的很长。

不断修改。

不讲废话。

还有什么要说的？

不足之处，敬请指点。

(失败2016.9.16)

好难。

(同时)

删除文章。

撤销分类。

////////////////////////////////////  
(2016.12.8) 偶然翻出以前的文件，看到了这些。

Python2.7.12参考手册.txt

Python2.7.12参考手册

欢迎你！这是Python2.7.12版本的中文参考手册，（原版本）最后的更新时间是2016年7月27日。

参考手册目录：

Python2.7有什么新内容？ 或者说所有2.0版本以来所有的新内容

使用说明 在这儿开始

库参考 把这个放到你的枕头下

语言参考 描述语法和语言描述

Python的配置和用法 如何在不同的平台使用Python

Python用户手册 具体主题的深入描述

安装Python模块 从Python的包索引或者其他来源安装

Python的模块分类 用于安装的其他模块

继承和植入 对C/C++程序员的教程

Python对C语言的API 给C/C++程序员的参考

常见问题解答 经常被问到的问题及其答案

索引和表：

全局模块索引 快速进入所有的模块

[总索引](#) 所有的函数、类、条款

[词汇表](#) 最重要的条款解释

[搜索页面](#) 搜索这个文档

[完整的内容表](#) 列出所有的章节和小小节

元信息：

[错误报告](#)

[关于文档](#)

[历史许可](#)

[版权](#)

[Python模块索引.txt](#)

Python模块索引

[\\_](#)[a](#)[b](#)[c](#)[d](#)[e](#)[f](#)[g](#)[h](#)[i](#)[j](#)[k](#)[l](#)[m](#)[n](#)[o](#)[p](#)[q](#)[r](#)[s](#)[t](#)[u](#)[v](#)[w](#)[x](#)[y](#)[z](#)



\_\_builtin\_\_ 提供了内置命名空间的模块

\_\_future\_\_ 本声明的定义

\_\_main\_\_ 顶层脚本运行的环境

\_winreg (Windows) 操作Windows注册表的程序和对象

a

abc 基于PEP 3119的抽象基类

aepack (Mac) 不赞同的: Python和苹果事件之间会话的容器

aetools (Mac) 不赞同的: 对发送苹果事件的基础支持

aetypes (Mac) 不赞同的: Python显示苹果的事件对象模型。读和写AIFF和AIFC格式的音频文件

aifc 读和写AIFF或AIFC格式的音频

al (IRIX) 不赞同的: SGI上的音频功能

AL (IRIX) 不赞同的: 与al模块结合使用的常量

anydbm DBM 型数据库模块的通用接口

applesingle (Mac) AppleSingle 格式文件基本的解码器

argparse 命令行选项和参数解析库

array 节省空间的类型一致的数字阵列

ast 抽象语法树的类和操作。

asynchat 对异步命令/响应协议的支持

asyncore 一个异步套接字处理服务的基类

atexit 注册和执行清楚功能

audioop 操作未经加工的音频数据

autoGIL (Mac) 不赞同的：在时间循环中的全局解释器锁处理

## b

base64 标准 6548: Base16,Base32,Base64 Data Encodings

BaseHTTPServer 基础的 HTTP 服务 (对 SimpleHTTPServer and CGIHTTPServer)

Bastion 不赞同的：提供受限制的对象访问

bdb 调试器框架

binascii 二进制码和ASCII二进制编码之间转换的工具

binhex 加解密binhex4格式的文件

bisect 对二进制搜索的数组分割算法

bsddb 对Berkeley DB数据库的接口

buildtools (Mac) 不赞同的：对BuildApplet,BuildApplication和macfreeze的辅助模块

bz2 兼容bzip2的压缩和解压缩程序接口

## c

calendar 日历工作的功能，包含一些Unix计算的仿真程序

Carbon

Carbon.AE (Mac) 不赞同的：苹果事件工具箱的接口

Carbon.AH (Mac) 不赞同的：苹果帮助管理器的接口

Carbon.App (Mac) 不赞同的：外观管理器的接口

Carbon.Appearance (Mac) 不赞同的：外观管理器定义常量的接口

Carbon.CarbonEvents (Mac) 不赞同的: Carbon 事件管理器接口的常量

Carbon.CarbonEvt (Mac) 不赞同的: Carbon 事件管理器的接口

Carbon.CF (Mac) 不赞同的: 核心基础的接口

Carbon.CG (Mac) 不赞同的: 绘图核心的接口

Carbon.Cm (Mac) 不赞同的: 组件管理器的接口

Carbon.Components (Mac) 不赞同的: 组件管理器的界面常数

Carbon.ControlAccessor (Mac) 不赞同的: 控制器接口的访问函数

Carbon.Controls (Mac) 不赞同的: 控制器界面常数

Carbon.CoreGraphics (Mac) 不赞同的: 核心绘图界面常数

Carbon.Ctl (Mac) 不赞同的: 控制器的接口

Carbon.Dialogs (Mac) 不赞同的: 对话框管理器界面常数

Carbon.Dlg (Mac) 不赞同的: 对话框管理器的接口

Carbon.Drag (Mac) 不赞同的: 拖放管理器的接口

Carbon.Dragconst (Mac) 不赞同的: 拖放管理器的界面常数

Carbon.Events (Mac) 不赞同的: 经典事件管理器的界面常量

Carbon.Evt (Mac) 不赞同的：经典事件管理器的接口

Carbon.File (Mac) 不赞同的：文件管理器的接口

Carbon.Files (Mac) 不赞同的：文件管理器的界面常量

Carbon.Fm (Mac) 不赞同的：字体管理器的接口

Carbon.Folder (Mac) 不赞同的：文件夹管理器的接口

Carbon.Folders (Mac) 不赞同的：文件夹管理器的界面常量

Carbon.Fonts (Mac) 不赞同的：字体管理器的界面常量

Carbon.Help (Mac) 不赞同的：Carbon 帮助管理器的接口

Carbon.IBCarbon (Mac) 不赞同的：Carbon 接口建立支持库的接口

Carbon.IBCarbonRuntime (Mac) 不赞同的：Carbon 接口建立支持库的界面常量

Carbon.Icns (Mac) 不赞同的：Carbon 图标管理器的接口

Carbon.Icons (Mac) 不赞同的：Carbon 图标管理器的界面常量

Carbon.Launch (Mac) 不赞同的：Carbon 运行服务的接口

Carbon.LaunchServices (Mac) 不赞同的：Carbon 运行服务的界面常量

Carbon.List (Mac) 不赞同的：列表管理器的接口

Carbon.Lists (Mac) 不赞同的：列表管理器的界面常量

Carbon.MacHelp (Mac) 不赞同的：Carbon 帮助管理器的接口

Carbon.MediaDescr (Mac) 不赞同的：QuickTime 媒体描述符解析器和发生器

Carbon.Menu (Mac) 不赞同的：菜单管理器的接口

Carbon.Menus (Mac) 不赞同的：菜单管理器的界面常量

Carbon.Mlte (Mac) 不赞同的：多语言文本编辑器的接口

Carbon.OSA (Mac) 不赞同的：Carbon OSA 库的接口

Carbon.OSAconst (Mac) 不赞同的：Carbon OSA 库的界面常量

Carbon.Qd (Mac) 不赞同的：QuickDraw 工具箱的接口

Carbon.Qdoffs (Mac) 不赞同的：QuickTime 工具箱的接口

Carbon.QDOffscreen (Mac) 不赞同的：QuickDraw Offscreen APIs 的界面常量

Carbon.Qt (Mac) 不赞同的：QuickTime 工具箱的接口

Carbon.QuickDraw (Mac) 不赞同的：QuickDraw 工具箱的界面常量

Carbon.QuickTime (Mac) 不赞同的：QuickTime 工具箱的界面常量

Carbon.Res (Mac) 不赞同的：资源管理器和句柄的接口

Carbon.Resources (Mac) 不赞同的：资源管理器和句柄的界面常量

Carbon.Scrap (Mac) 不赞同的：Scrap 管理器提供了剪切、粘贴和剪贴板的操作

Carbon.Snd (Mac) 不赞同的：声音管理器的接口

Carbon.Sound (Mac) 不赞同的：声音管理器的界面常量

Carbon.TE (Mac) 不赞同的：文本编辑器的接口

Carbon.TextEdit (Mac) 不赞同的：文本编辑器的界面常量

Carbon.Win (Mac) 不赞同的：窗口管理器的接口

Carbon.Windows (Mac) 不赞同的：窗口管理器的界面常量

cd (IRIX) Silicon 图形系统上光盘的接口

cfmfile (Mac) 不赞同的：代码片段资源模块

cgi 不赞同的：运行Python脚本通过公共网关接口的助手

CGIHTTPServer 这个模块提供了当运行CGI脚本的时候HTTP服务的处理程序

cgitb CGI脚本配置的追踪处理

chunk 读取IFF块的模块

cmath 对于复杂数学运算的功能

cmd 构建面向行的命令解释器

code 实行读-执行循环的工具

codecs 编码和解码数据和流

codeop 编译（可能不完全）的Python代码

collections 高性能的数据类型

ColorPicker (Mac) 不赞同的：标准颜色编辑对话框的接口

colorsys RGB和其它颜色系统的绘画功能

commands (Unix) 不赞同的：运行外部命令的实用功能

compileall 字节编译目录树中所有的Python源文件的工具

compiler 不赞同的：写在Python里的Python代码编译

compiler.ast

compiler.visitor

ConfigParser 配置文件分析器

contextlib 实用的声明上下文

Cookie HTTP数据(cookie)管理的支持



cookiec lib 对于 HTTP cookies 的自动处理类

copy 浅层和深部复制操作

copy\_reg 寄存器 pickle 支持功能

cPickle 快捷版本的 pickle，但不能产生子类

cProfile

crypt (Unix) crypt() 功能用来检查 Unix 密码

csv 从分隔的文件中写入和读取表格数据

ctypes Python 中一个外部的库

curses (Unix) 一个 curses 库的接口，支持便携式终端处理

curses.ascii 对 ASCII 字符的常量和关系函数

curses.panel 一个面板堆栈的扩展，对 curses 窗口增加了深度

curses.textpad Emacs 一样在 curses 窗口输入和编辑

d

datetime 基本日期和时间类型

dbhash BSD数据库DBM风格的接口

dbm (Unix) 基于ndbm的标准的数据库接口

decimal 通用十进制算术规范的实现

DEVICE (IRIX) 不赞同的：和gl模块的常量一起使用

difflib 辅助计算对象之间差异

dircache 不赞同的：返回目录列表，具有缓存机制

dis Python字节码反汇编程序

distutils 对组建和安装Python的支持

distutils.archive\_util 对创建归档文件是有用的功能（如原始码文件或zip文件等等）

distutils.bcppcompiler

distutils.ccompiler CCompiler类的摘要

distutils.cmd 这个模块提供了基类命令的摘要。这个类是distutils命令包模块的子类

distutils.command 这个包包含了一个模块，涵盖每一条标准Distutils命令

distutils.command.bdist 为程序组建一个二进制安装包

distutils.command.bdist\_dumb 组建一个"dumb"安装包——一个简单的存档文件

distutils.command.bdist\_msi 组建二进制程序为一个Windows MSI文件

distutils.command.bdist\_packager 包的基类

distutils.command.bdist\_rpm 组建一个二进制程序为一个Redhat的RPM和SRPM

distutils.command.bdist\_wininst 组建一个Windows安装包

distutils.command.build 组件所有文件为一个包

distutils.command.build\_clib 组建所有C函数库在一个包里

distutils.command.build\_ext 组建任何扩展程序在一个包里

distutils.command.build\_py 组建一个包中的.py和.pyc文件

distutils.command.build\_scripts 组建一个包中的脚本文件

distutils.command.check 检查一个包中的测试数据

distutils.command.config 执行包的配置

distutils.command.insatll 安装一个包

distutils.command.install\_data 从一个包中安装数据文件

distutils.command.install\_headers 从一个包中安装C或C++头文件

distutils.command.install\_lib 从一个包中安装数据库文件

distutils.command.install\_scripts 从一个包中安装脚本文件

distutils.command.register 注册一个有Python包索引的模块

distutils.command.sdist 资源分配

distutils.core Distutils 功能核心

distutils.cygwinccompiler

distutils.debug 支持distutils 错误标记

distutils.dep\_util 有用的功能对于简单的从属检查

distutils.dir\_util 有用的功能对于目录和目录树的操作

distutils.dist 提供了配置类，可以代替该模块组建、安装、设置

distutils.emxccompiler 对OS/2 EMX编译器的支持

distutils.errors 提供标准的distutils 异常

distutils.extension 提供了扩展类，被用于在脚本中描述C和C++扩展模块

distutils.fancy\_getopt 额外的getopt 功能

distutils.file\_util 对单个文件操作是有用的功能

distutils.filelist 这个FileList类被用于解除关于文件系统和组建文件列表

distutils.log 一个简单的记录机制，282 风格

distutils.msvccompiler 微软软件编译器

distutils.spawn 提供 spawn() 函数

distutils.sysconfig Python 解释器配置信息低等级的入口

distutils.text\_file 一个简单的文件系统接口，提供文本文件类

distutils.unixccompiler Unix 系统下的 C 语言编译器

distutils.util 混杂着其他工具功能

distutils.version 表示模板版本数字的工具类

dl (Unix) 不赞同的：在一个共享的对象中调用 C 语言的函数

doctest 文档中测试部分的代码

DocXMLRPCServer 文档自身的 XML-RPC 服务工具

dumdbbm DBM 接口的一个便携工具

dummy\_thread 进入代替交织的模块

dummy\_threading 进入代替正在交织的模块

e

EasyDialogs (Mac) 不赞同的：基本的Macintosh对话框

email 一个提供了解析、操作和生成email信息功能的包，包括MIME文档

email.charset 字符设置

email.encoders email信息封装编码器

email.errors 使用email包的异常类

email.generator 从一个email结构中生成单调的email文本消息

email.header 描述非正规的ASCII头部

email.iterators 对一个信息对象树的迭代

email.message 描述email信息的基类

email.mime 组建MIME消息

email.parser 解析浅显的email文本信息去生成一个消息模板

email.utils 混杂着email包工具

encodings

encodings.idna 国际化域名绑定

encodings.utf\_8\_sig UTF-8编码无BOM格式

ensurepip pip安装助手的引导程序对于已存在的Python的安装或程序环境

errno 标准的errno系统符号

exceptions 标准的异常类

f

fcntl (Unix) fcntl()和ioctl()系统响应

filecmp 有效地比较文件

fileinput 标准的输入或文件列表循环

findertools (Mac) 探测苹果事件接口周围的包装

FL (IRIX) 不赞同的：和fl模块一起使用的常量

fl (IRIX) 不赞同的：表单函数库，生动的用户接口应用

flp (IRIX) 不赞同的：加载储存的表单设计功能

fm (IRIX) 不赞同的：SGI终端智能的字体管理器接口

fnmatch Unix shell风格文件名样式匹配

formatter 普通输出格式化程序和配置接口

fpformat 不赞同的：普通的浮点数格式化功能

fractions 合理的数字

FrameWork (Mac) 不赞同的：交互式的应用框架

ftplib FTP协议客户端（需要sockets）

functools 更高的处理和操作功能在可响应的对象中

future\_builtins

g

gc 循环检测垃圾回收的接口

gdbm (Unix) dbm 下GNU的重新编译

gensuitemodule (Mac) 从一个OSA字典中创建一个存根包

getopt 便携的命令行选择分析工具，提供了长短两种选择

getpass 便携的用户名和密码检索工具

gl (IRIX) 不赞同的：绘图功能



GL (IRIX) 不赞同的：和`gl`模块一起使用的常量

`glob` Unix shell风格的路径名样式扩展

`grp` (Unix) 数据库组 (`getgrnam()`和`friends`)

`gzip` 压缩和解压文件对象的接口

## h

`hashlib` 安全散列和消息摘要算法

`heapq` 堆排序的算法（即优先级队列）

`hmac` 散列消息鉴别码（HMAC）的实现

`hotshot` 高性能日志分析器，主要是用C语言编写的

[前一篇：换主题啦！告别Metro，迎来百度efe](#)

[后一篇：（失败2016.9.16）（程序猿养成计划）PHP参考手册实例遍历计划](#)



smallyu

Hello World

??

??

????

2017080 (6)

2017070 (2)

2017060 (3)

2017050 (2)

2017040 (2)

2017030 (3)

2017020 (3)

2017010 (4)

20160120 (1)

## 换主题啦！告别Metro，迎来百度efe

□ smallyu □ 2016-08-24 □ [主题](#)

免不了灌水。刚刚发了牢骚，咋么P大点事也要发一篇文章？

此话有理。不过转念一想，谁看呐！况且独立个人博客的意义何在？

博客标题。

一开始是“多云转晴<sub>hower</sub>”，后来是“Hower's Blog”，现在又改回了“多云转晴<sub>hower</sub>”。

Hower<sub>hower</sub>只有多云转晴<sub>hower</sub>一个人是事实。

Hower<sub>how.gs</sub>称呼何来？这个域名作怪。这确实是一个不错的域名，但又是一个有风险的域名，因为它是国别域名，谁都无法保证它有多长时间的可用性，以及续费价格是否可以稳定。指不定哪天，就不存在了。聊以情怀。

索引问题。不精通SEO，但是如果用“Hower' Blog”这么一个全英文的标题，是不会受本土居民以及搜索引擎待见的。

友情链接。不必要更名。

|              |
|--------------|
| 20160110 (2) |
| 20160109 (3) |
| 2016009 (1)  |
| 2016008 (6)  |
| 2016007 (1)  |
| 2016006 (4)  |
| 2016005 (4)  |

博客主题。

由原先系统自带的“新鲜人”的“Metro”主题，换成“老白”的“ydefe”仿百度<http://efe.baidu.com>主题。

Metro确实有很多难以容忍的缺点。固定的像素布局感觉老套落后于潮流，颜色搭配也很难调剂，包括背景图，虽然它也有优点。

efe既然是仿百度的，从某些审美角度就可以忽略N多问题，并且实现一开始没有实现的响应式。虽然它也有缺点。

内容展示。

由主题更改带来一些展示内容的更改。

目前没有文章阅读量、评论数目和网站访问量统计，因为模板本身没有。Metro模板文章相关的统计是自带的，站点访问量是后加的。至于现在的efe，没有添加这些功能的打算。没有辣眼睛的数据统计何尝不是一件好事？

后来：保留了网站页面加载数目统计，在每个页面的最下面一行。因为我喜欢大数据。至于单个文章阅读量，还在考虑。

后来：稍微折腾了一下下Font awesome，把该有的统计都加上了。虽然辣眼睛，不过把数据明明白白摆出来，总好过什么都不知道，想看看不上。

efe的列表展示效果不尽人意，没有Metro条理、清晰。暂时忽略。还在考虑以前的文章要不要重新排版？

1313 这么些，说什么优点缺点？

并没有褒贬的成份在里面。谁又能说PHP不是世界上最好的语言？

那你怎么不自己开发个模板？

不会。

主题版权。

可能“不道德”地去除了版权。个人观点，影响版面内容。不是版面问题，多这么一句外部内容总感觉不放心。也看的出来作者对于这句版权稍上心，文件里面用了不少缩进。原版权内容：

Powered By [Z-BlogPHP](#). theme by [letterpress](#)

如果侵权（应该没那么严重），留言，即刻更改。



[前一篇](#) [没有尼禄的本事还想泡茱莉亚一样的妞？](#)

[后一篇](#) [（失败2016.9.16）（程序猿养成计划）Python参考手册翻译计划](#)



smallyu

Hello World

??

??

????

2017080 (6)

2017070 (2)

2017060 (3)

2017050 (2)

2017040 (2)

2017030 (3)

2017020 (3)

2017010 (4)

20160120 (1)

## 没有尼禄的本事还想泡菜莉亚一样的妞？

□ smallyu □ 2016-08-22 □ 鬼泣4

知道我写这个东西用的是什么编辑器吗？对，sublime

不善言辞，不喜表达，却又不想无趣。博客应该有什么内容？在这个博客早已落寞的年代。现在互联网上最热门的是什么内容？只知道贴吧人多，天涯、猫扑有人灌水，头条访问量高，慕课的人气也经久不衰。还有呢？互联网辣么庞大，除去所谓的暗网，也有数以万计的站点散布在网络上，我们能够看到的网站入口却寥寥无几。

优酷？爱奇艺？还是英雄联盟？人生似乎有点boring。网络世界很繁华，却不止一次感受到它的冷漠。

敲键盘的手承载不起按吉他和弦的强度。

这段时间唯一的长进也就打字速度稍微能看了一点，却在打这段汉字的时候，发现根本毫无暖用。也就一两个月没怎么碰吉他，十分钟就开始手疼了。sublime不折不扣是一个灰常给力的编辑器，小巧轻便，功能丰富，界面上也可以甩notepad几条街，边角圆滑，还有平滑的过度动画。可惜是用Python开发的。

为什么可惜呢？因为不会啊！有人比喻，Java是重剑，PHP是轻剑，Python是匕首。要怎么用，看用来干什么。java除了完全面向对象，语法上和js还是稍微有点像的，都用+作为连接符。也就是说，能看得懂。不熟练，但也不能说生疏。就像在学过C之后看了看C++的基础课程，发现并没有多么变态。可Python的dict和map就有点无解了，想学，就要花时间去弄清楚一些新概念。

对于新生猿来说，花点时间确实没啥？是的。曾经以为echo和include就可以做很多事情，直到后来用了GD库，听到了tpl，还有MVC构架，和smarty模板、tp框架、yii框架，7.0的新特性、mysql，设计模式，微信开发，等等等等N多内容需要学习甚至精通。Python？我真的还想再活五百年。（H5和CSS3呢？Js呢？

20160110 (2)

20160109 (3)

2016090 (1)

2016080 (6)

2016070 (1)

2016060 (4)

2016050 (4)

提都不提，你多伤它们的心)



人生就像保卫萝卜，没钱，你的萝卜就会被怪吃掉。不管是跑得飞快体积娇小的小黄怪，还是身高体宽，虎背熊腰的大黑怪，都能咬你几口。

就萝卜本身而言，<sub>3</sub>的难度比<sub>2</sub>高了不少一个难度。高难度的情况下有哪些方法可以通关呢？

1. 提高自身实力。萝卜的生命也好，技能也好，炮塔也好，肯定越厉害就真的越厉害。如果一个人就是一根萝卜，炮塔就是萝卜的各项技能，炮塔越多，炮塔等级越高，怪就越渺小。多，指知识范围广，等级高，指学术精。只多不行，伤害低人家怪防得住。只有一个炮塔也不行，攻击再高人家躲过那一段就没事。单个炮塔放满屏幕？一招吃遍天下的伎俩，够你一时猖狂，不够你一生保命。

2. 换装。萝卜也是有等级的，也是有服装的。等级？就是身价咯，得用角色碎片来进阶，不止是金钱就可以买到的。服装？就是装<sub>13</sub>呗。装的好，就能把小怪给唬住喽。

3. 好友助阵。可以提高炮塔的攻击力，不同等级的好友增益不同，不言而喻，看你朋友的身价。不过当然，次数多了就要花金币才能让人家帮助你，等级高的还得用钻石。

4. 没了，就这么没了。真的没有其他方法了吗？你还有颠覆你人生更好的方法吗？

( )

<<< 2016.9.8

保卫萝卜告诉我们，

1. 不要怕输，输了可以再来，怕输连开始都难。面对不进则退的晋升关卡，勇敢地打。
2. 后期新出的炮塔不一定就越好，集市主题第一关的船锚炸弹也就如此，用起来可能还不如飞机，所以告诉我什么是爱情？
3. 顶级炮塔的攻击速度和攻击力远超降一级的炮塔，性价比更高。学艺要学精？低级炮塔多不见得比少数几个顶级炮塔有气场。不过在糖果赛中炮塔等级高，损失的也就多。学PHP再精通，人家企业就要Python的，奈何？此题无解。

(2016.9.8) ;



想写博客不容易。毕竟看多了水到发慌的数不胜数的个人博客，或者为盈利而生的商业博客。

技术博客？啥也不会啊！技术圈有大波大波的大神在写博客，技术到什么程度？到很多人看不懂的程度。甚至出现的名词都没听过。

生活博客？生活比看时间简史还要枯燥。



没有尼禄的本事还想泡茉莉亚一样的妞？是不是稍微有点跳。对，Devil May Cry 4，就是鬼泣4...  
是我老了吗？接受不了新鲜事物了吗？玩的游戏还是08年的鬼泣4，和孢子，听的歌，也是前几年，很多年前的歌，筷子兄弟的你一定会成功算不算？  
不会写小说，写不下去，时间成本大。其实我还想再活六百年。



单纯闲的发发牢骚。你想不想再多活几年？

前一篇: [\(一万小时定律实践计划\)](#) Duang~任务进度到1%啦！

后一篇: [换主题啦！告别Metro，迎来百度efe](#)



smallyu

Hello World

??

??

????

2017080 (6)

2017070 (2)

2017060 (3)

2017050 (2)

2017040 (2)

2017030 (3)

2017020 (3)

2017010 (4)

20160120 (1)

## (一万小时定律实践计划) Duang~任务进度到1%啦!

□ smallyu □ 2016-08-02 □ 一万小时定律实践计划

作家格拉德威尔在《异类》一书中指出:

“人们眼中的天才之所以卓越非凡，并非天资超人一等，而是付出了持续不断的努力。1万小时的锤炼是任何人从平凡变成超凡的必要条件。”

他将此称为“一万小时定律”。要成为某个领域的专家，需要10000小时，按比例计算就是：如果每天工作八个小时，一周工作五天，那么成为一个领域的专家至少需要五年。这就是一万小时定律。

那么这一万小时按照什么标准算呢？我也不知道O\_O



20160110 (2)

20160107 (3)

20160101 (1)

20160101 (6)

20160101 (1)

20160101 (4)

20160101 (4)

就当做里程碑式的纪念，100个小时，1%！

[前一篇](#)：(程序猿养成计划) Linux 手记 (1)

[后一篇](#)：没有尼禄的本事还想泡茉莉亚一样的妞？

Copyright Hower'Blog Rights Reserved. . Powered By [Z-BlogPHP](#), theme by [00000](#)



smallyu

Hello World

??

??

????

2017080 (6)

2017070 (2)

2017060 (3)

2017050 (2)

2017040 (2)

2017030 (3)

2017020 (3)

2017010 (4)

20160120 (1)

## (程序猿养成计划) Linux手记 (I)

□ smallyu □ 2016-08-02 □ 程序猿养成计划 快捷键 Linux手记

跳了好多内容呢，所以手记内容并不全，真是一段艰辛的历程，然而还没有结束。

课程来源: <http://www.imooc.com/learn/175>

### Linux基本操作

查询目录中内容: ls

ls [选项] [文件或目录]

选项:

-a 显示所有文件，包括隐藏文件

-l 显示详细信息

-d 查看目录属性

-h 人性化显示文件大小

-i 显示inode

20160110 (2)

20160109 (3)

2016090 (1)

2016080 (6)

2016070 (1)

2016060 (4)

2016050 (4)

举例: `ls -lh`

建立目录(make directories): `mkdir`

`mkdir -p [目录名]`

`-p` 递归创建

切换所在目录(change directory): `cd`

简化操作

`cd` 进入当前用户的家目录

`cd ~` 进入当前用户的家目录

`cd -` 进入上次所在目录

`cd ..` 进入上一级目录

`cd cd.` 进入当前目录

查询所在目录位置(print working directory): `pwd`

pwd

删除空目录(remove empty directors): rmdir

rmdir[目录名]

复制命令(copy): cp

cp [选项] [原文件或目录] [目标目录]

-r 复制目录

-p 连带文件属性复制

-d 若原文件是链接文件，则复制链接属性

-a 相当于 -pdr

剪切或改名命令(move): mv

mv [原文件或目录] [目标目录]

删除目录或文件(remove): rm

rm -rf [文件或目录]

-r 删除目录

-f 强制

链接命令(link): ln

ln -s [原文件] [目标文件]

-s 创建软链接

#文件搜索命令 locate

#命令搜索命令 whereis 与 which

#文件搜索命令 find

#字符串搜索命令 grep

locate 命令格式

locate 文件名 # 在后台数据库中按文件名搜索，搜索速度更快

/var/lib/mlocate # locate 命令所搜索的后台数据库

updatedb # 更新数据库

搜索命令的命令 whereis

whereis 命令名

-b 只查找可执行文件

-m 只查找帮助文件

find 命令

find [搜索范围] [搜索命令]

#find / -name install.log # 按文件名查找

#find /root -iname install.log # 不区分大小写

#find /root -user root # 按照所有者搜索

#find /root -nouser # 查找没有所有者的文件



#find /var/log/ -mtime +10 #查找10天前修改的文件

-10 10天内修改的文件

10 10天当天修改的文件

+10 10天前修改的文件

atime 文件访问时间

ctime 改变文件属性

mtime 修改文件内容

#find /etc -size +20k -a -size -50k #查找/etc/目录下，大于20KB并且小于50KB的文件

-a and 逻辑与

-o or 逻辑或

#find /etc -size +20k -a -size -50k -exec ls -lh {} \; #查找/etc/目录下，大于20KB并且小于50KB的文件，并且显示详细信息

#-exec/ok 命令 {} \; 对搜索结果执行操作

搜搜字符串命令 grep

选项 字符集 文件名

grep [ ]

-i 忽略大小写

-v 排除指定字符集

帮助命令 man

man 命令 # 获取指定命令的帮助

man ls # 查看ls的帮助

man -f 命令 # 相当于 whatis 命令

举例： man -5 password

按 q 键退出 man 命令

选项帮助 --help

举例： ls --help

shell 内部命令帮助

help shell 内部命令

举例: whereis cd # 确定是否是 shell 内部命令

举例: help cd # 获取内部命令帮助

详细命令帮助 info

info 命令

\_ 回车: 进入子帮助页面 (带有\*号标记)

-u: 进入上层页面

-n: 进入下一个帮助小节

-p: 进入上一个帮助小节

-q: 退出

.zip 格式压缩

zip 压缩文件名 原文件 # 压缩文件

zip -r 压缩文件名 源目录 # 压缩目录

.zip 格式解压缩

unzip 压缩文件 #解压缩文件

.gz 格式压缩

gzip 源文件 #压缩为 .gz 格式的压缩文件，源文件会消失

gzip -c 源文件 > 压缩文件 #压缩为 .gz 格式，源文件保留

例如: gzip -c cangls > cangls.gz

gzip -r 目录 #压缩目录下所有的子文件，但是不能压缩目录

gzip -d 压缩文件 #解压缩文件

gunzip 压缩文件 #解压缩文件

.bz2 格式压缩

bzip2 源文件 #压缩文件 .bz2 格式，不保留源文件

bzip2 -k 源文件 #压缩之后保留源文件

注意: bzip2命令不能压缩目录

打包命令tar

tar -cvf 打包文件名 源文件

-c 打包

-v 显示过程

-f 指定打包后的文件名

例如 tar -cvf longzls.tar longzls

.tar.gz 压缩格式

tar -zcvf 压缩包名.tar.gz 源文件

-z 压缩为.tar.gz格式

tar -zxvf 压缩包名.tar.gz

-x 解压缩.tar.gz格式

.tar.bz2 压缩格式

tar -jcvf 压缩包名.tar.bz2 源文件

-z 压缩为.tar.bz2 格式

tar -jxvf 压缩包名.tar.bz2

-x 解压缩.tar.bz2 格式

关机 shutdown 命令

[root@localhost ~]# shutdown [选项]时间

-c 取消前一个关机命令

-h 关机

-r 重启

其他关机命令

[root@localhost ~]# halt

```
[root@localhost ~]# poweroff
```

```
[root@localhost ~]# init 0
```

### 其他重启命令

```
[root@localhost ~]# reboot
```

```
[root@localhost ~]# init 6
```

### 系统运行级别

0 关机

1 单用户

2 不完全多用户，不含NFS服务

3 完全多用户

4 未分配

5 图形界面

6 重启

```
[root@localhost ~]# cat/etc/inittab
```

#修改系统默认运行级别

```
id:3:initddefault:
```

```
[root@localhost ~]# runlevel
```

#查询系统运行级别

退出登录命令

```
[root@localhost ~]# logout
```

1. 查询与自动挂载

```
[root@localhost ~]# mount
```

#查询系统中已经挂载的设备

```
[root@localhost ~]# mount -a
```



#依据配置文件/etc/fstab的内容，自动挂载

## 2. 挂载命令格式

```
[root@localhost ~]# mount [-t 文件系统] [-o 特殊选项] 设备 文件名 挂载点
```

-t 文件系统：指定挂载类型，可以ext3、ext4、iso9660等

-o 特殊选项：可以指定挂载的额外选项

## 3. 挂载光盘

```
[root@localhost ~]# mkdir /mnt/cdrom
```

#建立挂载点

```
[root@localhost ~]# mount -t iso9660 /dev/cdrom/mnt/cdrom
```

#挂载光盘

```
[root@localhost ~]# mount /dev/sr0/mnt/cdrom
```

## 4. 卸载命令

```
[root@localhost ~]# umount 设备文件名或挂载点
```

```
[root@localhost ~]# umount /mnt/cdrom
```

## 5. 挂载U盘

```
[root@localhost ~]# fdisk -l
```

#查看U盘设备文件名

```
[root@localhost ~]# mount -t vfat /dev/sdb1/mnt/usb/
```

注意：Linux默认不支持NTFS文件系统

查看登录用户信息

w 用户名

命令输出：登录的用户名

USER: 登录终端

TTY: 从哪个IP地址登录

FROM: 登录时间

LOGIN@: 用户闲置时间

IDLE: 用户限制时间

JCPU: 连接所有进程占用时间

PCPU: 当前进程占用时间

WHAT: 当前正在运行的命令

Linux中的通配符

\* 匹配任意内容

? 匹配任意一个字符

[] 匹配任意一个中括号内的字符

文件类型显示

第一位: `.`文件 `d`目录 `|`软链接

后三位: `u`所有者

后三位: `g`所属组

后三位：o 其他人

共十位：r 读 w 写 x 执行

常用目录作用

/ 根目录

/bin 命令保存目录（普通用户就可以读取的命令）

/boot 启动目录，启动相关文件

/dev 设备文件保存目录

/etc 配置文件保存目录

/home 普通用户的家目录

/lib 系统库保存目录

/mnt 系统挂载目录

/media 挂载目录

Shell 基础

## 1. echo 输出命令

echo [选项] [输出命令]

-e 支持反斜杠控制的字符转换

```
[root@localhost ~]# echo -e "hell\bo"
```

#删除左侧字符

```
[root@localhost ~]# echo -e "h\te\tl\n\to"
```

#制表符与换行符

```
[root@localhost ~]# echo -e \ "x68\t\x65\t\x6c\n\x6c\t\x6f\t"
```

#按照十六进制ASCII码也可以输出

```
[root@localhost ~]# echo -e "\e[1;31m 嫁人就要嫁凤姐 \e[0m"
```

#输出颜色

## 2. 第一个脚本

```
[root@localhost sh]# vi hello.sh
```

```
#!/bin/bash
```

```
#The first program
```

```
echo -e "\e[1;34m 天上掉下个林妹妹! \e[0m"
```

### 3. 脚本执行

赋予执行权限，直接运行

```
chmod 755 hello.sh
```

```
./hello.sh
```

通过 `Bash` 调用执行脚本

```
bash hello.sh
```

### 1. 查看与设定别名

```
alias # 查看系统中所有的命令别名
```

`alias 别名='原命令' # 设定命令别名`

别名永久生效与删除别名

`vi ~/.bashrc # 写入环境变量配置文件`

`unalias 别名 # 删除别名`

命令生效顺序

...

常用快捷键

`ctrl + c` 强制终止当前命令

`ctrl + l` 清屏

`ctrl + a` 光标移动到命令行首

`ctrl + e` 光标移动到命令行尾

`ctrl + u` 从光标所在位置删除到行首

ctrl + z 把命令放入后台

ctrl + r 在历史命令中搜索

## 2. 历史命令

history [历史命令] [历史命令保存文件]

-C: 清空历史命令

-w: 把缓存中的命令写入历史命令保存文件 ~/.bash\_history

历史命令默认保存1000条，可以在环境变量配置文件/etc/profile中进行修改

### 历史命令的调用

使用上下箭头调用以前的历史命令

使用"!n"重复执行第n条命令

使用"!!"重复执行上一条命令

使用"!字符串"重复执行最后一条以该字符串开头的命令



# 命令与文件补全

在Bash中，Tab自动补全

...

输入命令后 + 两次Tab 路径补全、命令补全

...

|| == |s -|

## 1. 标准输入输出

设备 设备文件名 文件描述符 类型

键盘 /dev/stdin 0 标准输入

显示器 /dev/sdtout 1 标准输入

显示器 /dev/sdterr 2 标准错误输出

## 2. 输出重定向

...

### 3. 输入重定向

```
[root@localhost ~]# wc [选项] [文件名]
```

-c 统计字节数

-w 统计单词数

-l 统计行数

### 1. 顺序命令执行

...

### 2. 管道符

```
[root@localhost ~]# 命令1 | 命令2
```

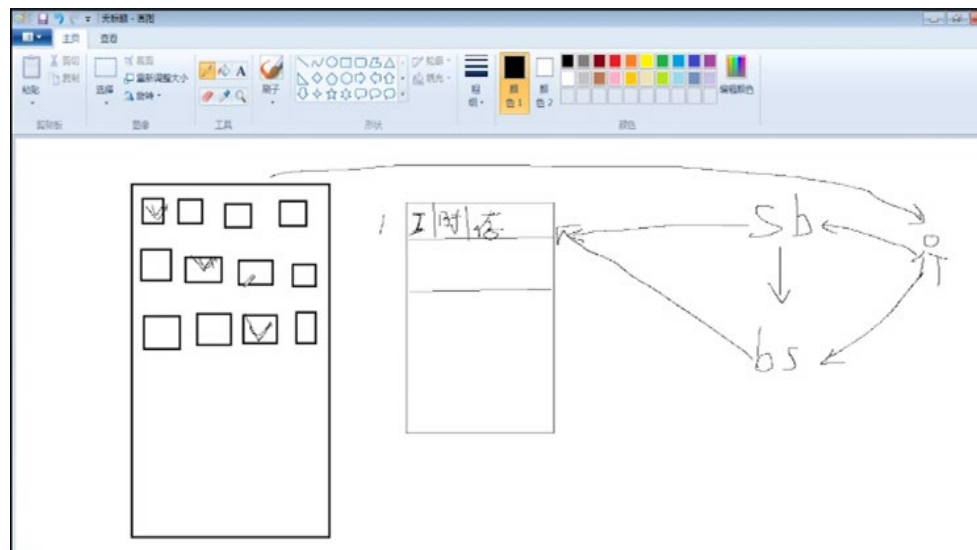
# 命令1的正确输出作为命令2的操作对象

### 1. 通配符

## 2.bash 中其他特殊符号

虽然用了引用格式，但内容并不是真的引用，只是为了排版舒服一点。

课程的主讲老师是@Tony 老师，Tony 老师不仅是一位Linux 大神，还是一个灵魂画手哦！上图：



最后由衷感谢这位坏坏的老师！

PS: 参考手册：[Linux 常用命令大全.chm](#)

[前一篇: 写代码过程中几个无足轻重的小习惯](#)

[后一篇: \(一万小时定律实践计划\)](#) Duang~任务进度到1%啦!



smallyu

Hello World

??

??

????

2017080 (6)

2017070 (2)

2017060 (3)

2017050 (2)

2017040 (2)

2017030 (3)

2017020 (3)

2017010 (4)

20160120 (1)

## 写代码过程中几个无足轻重的小习惯

□ smallyu □ 2016-07-13 □ 习惯

想哪写哪。

### 1、\$,"\$"和"?"

语言里""表示字符串，"表示字符，规则，乱了就会报错，PHP作为“松散”的语言，都能用来表示字符串。

```
$str = "aaa"; //定义变量为字符串
echo $str; //输出aaa
echo "$str"; //输出aaa
echo '$str'; //输入$str
```

这就能看出""和"的区别了，当然，这是基础。那么在使用的时候，是直接使用变量还是""?

效率：当然是少打个""了。

不过在Mysql里就有点问题，如果定义column1 varchar(arbitrary value)，然后执行

```
$str = $_POST['name'];
mysql_query("INSERT INTO table_name (column1) VALUES ($str)");
```

程序会报错（一般人会记住错误内容吗），因为\$str是不带""的字符串，PHP认识，Mysql不认识...

解决办法是，\$str = "\" . \$str . "\"; 是不是有点无聊，哈哈。

### 2、空格还是Tab?

20160110 (2)

20160109 (3)

20160909 (1)

20160809 (6)

20160709 (1)

20160609 (4)

20160509 (4)

老生常谈的问题，这种问题的答案我怎么会知道T\_T

习惯用四个空格，常用的文本开发工具会支持Tab转空格的，可以转写好的，也可以直接转Tab键输入的，比如Notepad++，缩进也方便，因为有快捷键，重点是网页显示友善。

补充：空格可以解决这种样式的排版问题，tab是做不到的：

```
<?php
 $abc = 1;
 $abcdef = 2;
 $a = 3;

?>
```

### 3、{\$}

```
<?php
 $link = abc.xyz;
 echo "google";

?>
```

挺有用的一种用法，可能是个人都知道、然而我后来才知道，后来，我终于学会了，如何.....

### 4、紧凑、空格还是换行？

比如function abc(){}, 大括号怎么安排位置？

CSS里，可以

```
body {

```

```
}
```

PHP里，可以

```
if (
{

}
```

毕竟if后面的条件可能会很长，css选择器一般很短，话说function也换行比较醒目，嗯，还是换行好。（补充：不提倡）

补充：

```
class AClass {

}
```

## 5、格式化

在一些代码编辑工具中，比如Visual Studio，会自动在运算符"="、">"、"<"等两边加上空格，以及在多参数的","后边加上空格，原先是

```
int a=3;
a(int,int){}
```

格式化之后

```
int a = 3;
a (int, int) {}
```

有的工具其中一个选项就是“格式化代码”，做的就是这样的事情，在写好的代码上进行这些操作。

习惯在写的时候就加上这些空格。

无伤大雅，哈，要不然怎么说是“无足轻重”的小习惯呢。

#### 6、elseif还是else if?

都差不多，凑合用吧。

#### 7、没了。





前一篇: [不得不说的科学上网浏览器: Tor Browser](#) (洋葱浏览器)

后一篇: [\(程序猿养成计划\) Linux手记](#) (|)

Copyright Hower'Blog Rights Reserved. . Powered By [Z-BlogPHP](#). theme by [\[?\]\[?\]\[?\]](#)



smallyu

Hello World

??

??

????

2017080 (6)

2017070 (2)

2017060 (3)

2017050 (2)

2017040 (2)

2017030 (3)

2017020 (3)

2017010 (4)

20160120 (1)

## 不得不说的科学上网浏览器：Tor Browser（洋葱浏览器）

□ smallyu □ 2016-06-19 □ Tor Browser



可以访问大陆屏蔽了的国外网站

Tro本是通过多层代理节点用于实现匿名访问网络的浏览器，被更多用于访问暗网，在实现多节点代理的同时，也就使得大陆网络用户可以访问国外的网站，因为Tro用的代理节点全是国外的：

|              |
|--------------|
|              |
|              |
| 20160111 (2) |
| 20160110 (3) |
| 20160109 (1) |
| 20160108 (6) |
| 20160107 (1) |
| 20160106 (4) |
| 20160105 (4) |

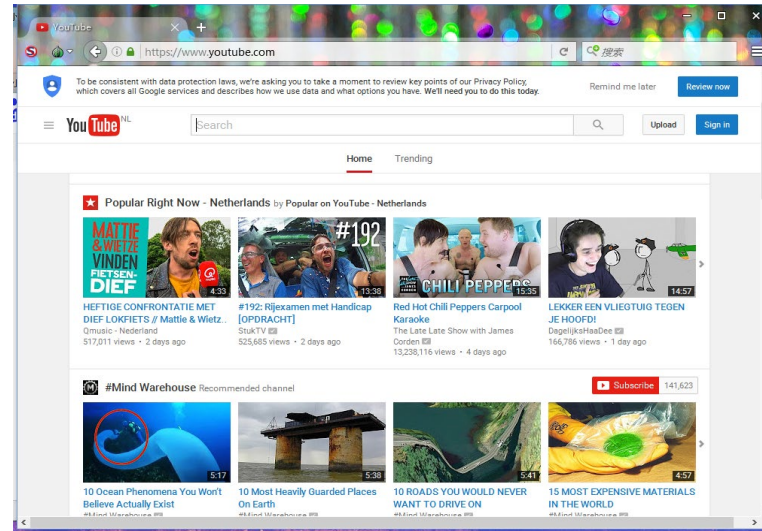


### Tro并不好用

Tro和VPN相比天壤之别，Tro网络的链接速度非常非常慢（可能由于使用多层代理节点需要足够的服务器响应时间），并且Tro并不稳定，以前出现过一直无法连接网络的情况（现在好转）。

不过，当ISP封锁了Tro网络连接的时候，用户可以通过建立网桥来连接互联网，还可以在Tro官网获得专属的网桥。

在大陆的网络环境下，可用的免费科学上网方法有限，Tro并不理想，但是喜欢折腾的友友可以试一试来玩，毕竟它是少有的匿名浏览器，黑客难以从用它浏览的站点上追踪到用户的访问信息。



Tro无法直接从官网下载（无法访问），只能从国内的软件下载站下载。

PS1：每天免费一小时的vpn赤兔（号称不限免费时长）：<http://www.getchitu.cc>

PS2：可用的可选择线路Chrome扩展：[Jumbo Proxy Switcher.zip](#)

PS3：二师兄（免费）VPN：<http://2-vpn5.com/home.action>（每三天登录激活）

前一篇：[这些常用的Windows10快捷键，你都会用了吗？](#) 后一篇：[写代码过程中几个无足轻重的小习惯](#)



smallyu

Hello World

??

??

????

2017080 (6)

2017070 (2)

2017060 (3)

2017050 (2)

2017040 (2)

2017030 (3)

2017020 (3)

2017010 (4)

20160120 (1)

## 这些常用的Windows10快捷键，你都会用了吗？

□ smallyu □ 2016-06-09 □ 快捷键 Windows10

快捷键这种东西，可以不会，但适度掌握一些还是有必要的，尤其是想要把计算机玩6的小伙伴们。下面引用了一些常用的win10快捷键：

Win键+Tab：激活多任务视图

Win键+A：激活操作中心

Win键+C：通过语音激活Cortana

Win键+D：显示桌面

Win键+E：打开文件管理器

Win键+G：打开Xbox游戏录制工具栏，供用户录制游戏视频或截屏

Win键+H：激活Windows 10应用的分享功能

Win键+I：打开Windows 10设置

Win键+K：激活无线显示器连接或音频设备连接

Win键+L：锁定屏幕

20160110 (2)

20160109 (3)

20160909 (1)

20160809 (6)

20160709 (1)

20160609 (4)

20160509 (4)

Win 键 + P: 投影屏幕

Win 键 + R: 运行

Win 键 + S: 激活 Cortana

Win 键 + X: 打开高级用户功能

Win 键 + 左/右/上/下: 移动应用窗口

Win 键 + Ctrl + D: 创建一个新的虚拟桌面

Win 键 + Ctrl + F4: 关闭最近使用的虚拟桌面

Win 键 + Ctrl + 左/右: 切换虚拟桌面

Win 键 + Shift + 左/右: 将应用从一个显示屏移至另一个显示屏

Win 键 + 1/2/3: 打开任务栏中固定的程序, 1 代表任务栏中第一个应用图标

Ctrl + V: 粘贴文本光标

Ctrl + C: 将选中的文本复制到剪贴板

Ctrl + A: 选择当前行中的所有文本

Shift + 左/右/上/下: 移动光标并选中文本

Ctrl + Shift + 左/右: 移动光标向左或向右

Shift + 向上翻页/下一页：移动光标向上或向下一页

Ctrl + Shift + Home /End：移动光标的开始或者结束“屏幕缓冲区”，“选择所有文本光标和命令提示符开头或结尾的输出。

Ctrl + 上/下：向上或向下移动一行命令提示符的历史

Alt + F4：关闭当前窗口

保存下来， 用到的时候可以require一下哦！

[前一篇：（程序猿养成计划）脚本语言实例（暂停更新）](#)

[后一篇：不得不说的科学上网浏览器：Tor Browser（洋葱浏览器）](#)



smallyu

Hello World

??

??

????

2017080 (6)

2017070 (2)

2017060 (3)

2017050 (2)

2017040 (2)

2017030 (3)

2017020 (3)

2017010 (4)

20160120 (1)

## (程序猿养成计划) 脚本语言实例 (暂停更新)

□ smallyu □ 2016-06-04 □ 程序猿养成计划 脚本语言实例

用一个页面来记录学习过程中编写的一些实例，包括原创和非原创，代码一定是经过多云转晴<sup>hower</sup>之手的。

页面地址：<http://oldblog.smallyu.net/demo/>

/\*\*以下引用内容用内联框架显示

# Not Found

The requested URL /demo/ was not found on this server.

*Apache/2.4.7 (Ubuntu) Server at oldblog.smallyu.net Port 80*



2016111 (2)

2016101 (3)

201691 (1)

201681 (6)

201671 (1)

201661 (4)

201651 (4)

\*/

欢迎来和多云转晴hower一起交流学习!

暂停更新（2016.8.11）：任重而道远。

上一篇: [屏幕录制软件：Bandicam](#)面前屏幕录制专家只是浮云（带注册机）

下一篇: [这些常用的Windows10快捷键，你都会用了吗？](#)



smallyu

Hello World

??

??

????

2017080 (6)

2017070 (2)

2017060 (3)

2017050 (2)

2017040 (2)

2017030 (3)

2017020 (3)

2017010 (4)

20160120 (1)

## 屏幕录制软件：Bandicam面前屏幕录制专家只是浮云（带注册机）

□ smallyu   □ 2016-06-03   □ Bandicam

当你使用屏幕录制专家录制的游戏视频卡成汪的时候，当屏幕录制专家提示丢失了50%以上帧的时候，不要太过疑虑你的机子是不是out了，也许只是你应该换一个更好用的屏幕录制软件。

Bandicam是国外的一款软件，支持中国简体语言，功能强大，可以访问官网下载未注册版本：

官网地址：<http://www.bandicam.com>

关于Bandicam的介绍官网上已经很齐全，就不贴图了，重点在于介绍中有这么一句：Bandicam是针对应用DirectX/OpenGL图形技术的程序进行视频录制的软件。也就是说在录制游戏方面的性能很优越，值得友友尝试一下。

下面引用官网对Bandicam的部分介绍：

### 主要特点及优点

- 可同时录制电脑屏幕、电脑声音、麦克风声音。
- 被录制的视频容量很小。(现有程序的1/5至1/20水平)
- 如果硬盘容量足够，可24小时长时间录制视频。

20160110 (2)

20160109 (3)

20160901 (1)

20160801 (6)

20160701 (1)

20160601 (4)

20160501 (4)

- 与其他软件不同，电脑上很少出现网速减慢现象(Lag)。
- 支持图像截图(BMP、PNG、JPG)及连续截图功能。
- 在录像对象屏幕上可添加网络摄像头(Webcam)视频。(可实时合并网络摄像头画面，PIP功能)
- 无需编码，即可将录制的视频上传至Youtube。(可上传720p及1080p)
- 可录制电脑全屏，或选择指定区域录制视频。
- 支持在预约时间开始录制视频，可使用自动结束的计时器功能。
- 可通过硬件加速并快速录制高清视频，也能实现高压缩。
- 支持CFR功能，能与Sony Vegas、Adobe Premiere等视频编辑软件兼容

当然，万能的网友还是提供了注册机的，可以通过下面的链接直接下载注册机以注册Bandicam： 下载附件

：  [Bandicam注册机.zip](#)

[前一篇](#) 每个人心中都有一个夏洛，尽管只是一场梦

[后一篇](#) (程序猿养成计划) 脚本语言实例 (暂停更新)



smallyu

Hello World

??

??

????

2017080 (6)

2017070 (2)

2017060 (3)

2017050 (2)

2017040 (2)

2017030 (3)

2017020 (3)

2017010 (4)

20160120 (1)

## 每个人心中都有一个夏洛，尽管只是一场梦

□ smallyu   □ 2016-05-30   □ 夏洛特烦恼

时隔N久，还是对《夏洛特烦恼》不能忘怀，很少有电影可以像它，让人翻出来一看再看。

文章标题前半句来自豆瓣某一影评，这不重要。我不写影评，也不会写，只是夏洛着实让我笑了，开怀大笑，并且，让我想了，想想人生，想想理想。本质来说，夏洛只是一场梦，他所经历的一切，只是自己的空想而已，电影将梦的内容真实华丽地展现了出来，然而事实上，如果夏洛真实存在于世间，那么，他经历了许多许多，经历了另一次人生，大彻大悟，改变自己对马冬梅的心意，是好事，可对别人来说，他仅仅只是做了一场梦，就是这样，夏洛还是夏洛，爱马冬梅了，很爱很爱，但还是一事无成、一无是处的夏洛。

写到这儿，我突然发现需要改变一下自己的观点了。原本以为夏洛只是改变了对马冬梅的情绪，而已，毕竟电影没有关于夏洛事业相关的内容，他会奋起吗？还是想像先前马冬梅说的，“整天躺在床上，什么都不做，四肢都快躺退化了”？电影没有说明，其实还是留给了我们一些遐想的。我以为夏洛没有事业上的变化，想要稍稍吐槽一下下，刚才却意识到，或许不是，改变了对马冬梅态度的同时，万一一不小心改变了对生活的态度，积极向上地生活，迎着新时代的光芒，走向人生的巅峰？

每个人心中都有一个夏洛。

一方面，《夏洛》思路“新颖”，给我们很强的代入感。以前，有一句话让我兴奋不已，“如果有一天你从睡梦中醒来，发现自己趴在小学课堂的课桌上，身旁的同桌在认真的发呆，看看窗外，熟悉的操场，原来自己所经历的一切，只是一场梦，一切都才要开始，一切都还充满希望……”夏洛，则将这个场景还原。它满足

20160110 (2)

20160108 (3)

20160901 (1)

20160801 (6)

20160701 (1)

20160601 (4)

20160501 (4)

了我的这个心愿，也可能是很多人的心愿，饱了眼福。在看电影的时候就会想，如果自己是夏洛，遇上这样的情况，会发生些什么？拥有现代的记忆，过着以往的生活。回到中学，我要首先要做的事情和夏洛一样，一定一样，那就是拼命追求自己喜欢的女孩，不会给自己的过去留下遗憾。

虽然自己的中学时代并没有这样的人，如果有，那么一定会。即使没有，也可以有嘛（。。。）女神不都是想出来的……好吧，换个话题，夏洛用别人的歌让自己成为了大明星。其实这里也可以省略了，毕竟自己连唱歌都不会。危机感又袭来，现在想想，即使自己回到了过去，也不见得能混得好，是的，真的是的T\_T。

另一方面，情节感人至深。马冬梅对夏洛无拘无束大大咧咧毫不遮掩的爱，多少人的梦，谁不想有一个这样爱自己的人。情感高潮，让夏洛情绪反转，推动电影大情节的，就要是马冬梅跟着坏混混进小树林了。看的时候曾经一度心疼啊，结果并没有发生想象中的事情，只是被电影的这种手法吊起了胃口。不得不说，我认为，马冬梅戴着彩色狗狗帽对着人流广告挥手的那个笑容，是整部片子马冬梅最美的镜头。当然，某人在某年某月某日某个时刻戴着彩色狗狗帽的笑容也是我认为曾经的最美最温暖的的笑容，可惜已经成为过去了。



自从前几天多云转晴hower博客建立后，突然觉得自己拥有了一个可以发言的阵地，这也是这篇文章会出现的原因。以前也有过几个博客，去年夏天的神天天雨博客，没几天就没落然后放弃了。想想原因，也许是因为无知，然后和某人有关，也确实没有什么存在的意义。仅仅是给自己一个展现自己的平台，写一些无所谓的话，同时技术也不过关，界面连自己的眼睛都满足不了，放弃在情理之中。对啊，时间也就只过了一年，发生了好多事情呢。

回到《夏洛》，片中涉及到的有三首歌非常好，重点是配上夏洛的相关情节，感人。



《一次就好》



《有个爱你的人不容易》



## 《夏洛特烦恼》

我也想做一场梦。

前一篇: [XAMPP错误: Apache shutdown unexpectedly](#) 解决办法: [更改端口](#)

后一篇: [屏幕录制软件: Bandicam](#) [面前屏幕录制专家只是浮云 \(带注册机\)](#)





sallyu

Hello World

??

??

????

2017080 (6)

2017070 (2)

2017060 (3)

2017050 (2)

2017040 (2)

2017030 (3)

2017020 (3)

2017010 (4)

20160120 (1)

## XAMPP错误：Apache shutdown unexpectedly解决办法： 更改端口

□ sallyu □ 2016-05-27 □ XAMPP

如果你新安装的XAMPP遇到apache不能启动的情况，提示如下错误信息：

[Apache] Error: Apache shutdown unexpectedly.

[Apache] This may be due to a blocked port, missing dependencies,

[Apache] improper privileges, a crash, or a shutdown by another method.

[Apache] Press the Logs button to view error logs and check

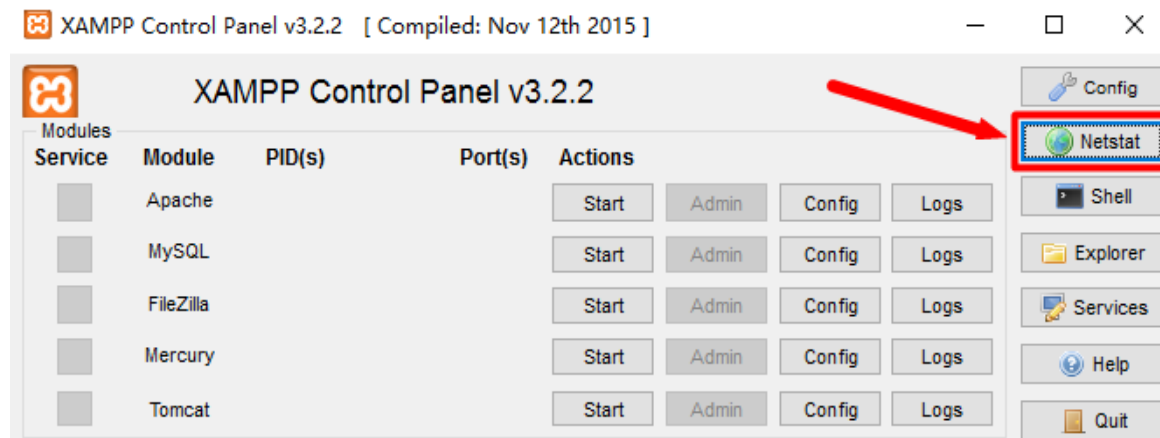
[Apache] the Windows Event Viewer for more clues

[Apache] If you need more help, copy and post this

[Apache] entire log window on the forums

关键信息在第二句：这可能是一个堵塞的端口，缺少了依赖关系。apache启动需要80端口和443端口，如果被占用将发生错误，可以通过XAMPP控制面板右侧栏的Netstat按钮查看主机的端口使用情况。

|              |
|--------------|
|              |
|              |
|              |
| 20160111 (2) |
| 20160110 (3) |
| 20160109 (1) |
| 20160108 (6) |
| 20160107 (1) |
| 20160106 (4) |
| 20160105 (4) |



想要解决这个问题的方法是简单的。只要更改apache的配置文件所需使用的被占用的端口80或者443为其它端口，80端口的配置文件为httpd.conf，443端口的配置文件为httpd-ssl.conf，找到语句Listen，后面的数字即为对应端口号。（这里省略相应配置文件路径的内容，网络上相关内容很丰富了）

注：

- 1.可能在配置文件中会找到多个80或者443的字样，只需更改Listen后面的数字就可以了。
- 2.端口号可以随意定义，但是要注意端口号最大值为65000，不要超过这个数字，比如将80端口改为8080是合法的，但把443改为443443就是非法的，可以使用444等端口数值。

前一篇: [浅思考：我为什么想要坚持写博客](#)

后一篇: [每个人心中都有一个夏洛，尽管只是一场梦](#)



smallyu

Hello World

??

??

????

2017080 (6)

2017070 (2)

2017060 (3)

2017050 (2)

2017040 (2)

2017030 (3)

2017020 (3)

2017010 (4)

20160120 (1)

## 浅思考：我为什么想要坚持写博客

□ smallyu □ 2016-05-24 □ 浅思考



一直想要得到一个足够说服自己坚持写博客的理由，未果。但现实征服了我自己，作为一个IT爱好者，网站builder，未来的程序猿，网络已然植根在我的生命中。

来看一下下面的引用内容：

作者：陈素封

链接：<https://zhuanlan.zhihu.com/p/19743861>

20160110 (2)

20160108 (3)

20160901 (1)

20160801 (6)

20160701 (1)

20160601 (4)

20160501 (4)

来源：知乎

著作权归作者所有。商业转载请联系作者获得授权，非商业转载请注明出处。

为什么你要写博客？

## 一个选择

我知道现在可能说这话有点不合时宜，毕竟博客时代都已经过去了，再号召大家用过就好像时的东西是不是有点逆流而上？

我曾经也问过自己这个问题，但是我觉得，博客时代过去跟我们要开博客是没有多大关系的，就好像你的读书时代已经过去你就不再读书一样。

判断一件事情值不值得去做有一个方法：在一张白纸的左边写不值得做的原因，然后在右边写值得做的原因，写完一比较，一权衡，自然能够得出结果。

大家都成年人了，你会觉得这样思考分析总结的过程才是正确的思考的方法吧？

所以，我在这里列出要写（独立）博客的原因，供大家去选择，然后填在你白纸的右边。

注意，我不是给你一个建议，而是提供一个选择，这个选择蕴藏着我也不知道的可能。

## 博客的内容

写博客不难，你可以当作是生活的记录，但是这样的记录没有任何的意义。写要对得住写本身，写出来的东西应该是思考的结果。我认为，如果你要开一个博客，博客的内容应该是这样的：

- 1. 不是生活杂记、不是流水账、不是牢骚、不是抱怨、不是心情琐记.....；
- 2. 有目的地写，要务实，追求质量；
- 3. 承认真实的自己，不要吹嘘，不要装逼，无需讨好读者；
- 4. 记录自己学习、思考、总结的过程；
- 5. 分享你的故事、所得、感想、经验；

## 值得写的原因

以下是一个清单，可以根据自己的情况匹配，然后选择。

## 重新认识自己

是不是很久没有跟自己对话了？

你可以尝试从回答一些问题开始，将你过去要回避的问题写下来，例如就可以从这个九个问题开始：

- 1、请你介绍一下你自己，你是个什么样的人？
- 2、你有什么理想吗？这个理想是怎么形成的？
- 3、你理想的伴侣关系是什么样的？你自己在这个伴侣关系中扮演什么样的角色？要承担什么样的责任？
- 4、你理想的事业是什么，你正在做的工作符合你的事业理想吗？这份工作对你的意义是什么？

- 5、你对亲子关系怎么看？对你来说，什么是一个理想的父亲（母亲），你期望自己成为这样一个理想父亲（母亲）吗？
- 6、你对钱怎么看，你认为赚到多少钱是足够的？如果你明天一早醒来，已经有足够的钱，你会如何继续安排自己的生活？
- 7、对你来说，什么是理想的性生活？什么是理想的性道德，在你的性道德观中，什么样的性生活是禁忌的，需要避免的，什么样的性生活是美好的，需要得到鼓励和发展的？
- 8、你的择友标准是什么？什么样的人你会愿意交往，什么样的人你会拒绝和他交往？
- 9、你对死亡怎么看？你希望自己活到多少岁，你准备怎么度过从现在到死亡的这段时间？如果你要立遗嘱，这份遗嘱会怎么写？

以上的这九个问题摘自《很少人能顺畅回答这9个问题——心理治疗刚开始医生常常会先问你的》by 李孟潮。

这些问题的答案你可以选择不发，但是我强烈地建议写下来，只有在写的时候你才可以慎重地思考这些问题，而不会回避跳过或者留下空白，这是接受自己的第一步。

## 提供持续学习的动力

例如，我为自己设限每天写一千字，信息的不断输出给我带来恐惧，我害怕有一天我写无可写，于是我不停地阅读，通过个人的知识管理促使自己不断学习，提高核心竞争力。

详细的知识管理可以看我的这篇文章：《个人知识管理的方法》，回复「知识」可见

## 积累更多的知识

写并不是单纯的写。

例如你写着写着，你突然忘记了一个概念，于是上网找，找回来这个概念的时候，你重温这个概念，可能还会顺便看了一下这个概念的其他东西。

例如你需要获取第一手的资料，寻找信息来源本身就是一个知识积累的过程，同时，你慢慢就学会了鉴别知识：什么是没有用的心灵鸡汤，什么是不值得关注的吐槽名人，还有，在这个过程中，你还养成你的心智。

## 提高将事情讲清楚的能力

很多东西你以为懂了，但当你在写下来的时候，你就觉得无从下手了。

如果一件事情你不能讲清楚，十有八九你还没有完全理解。

将事情写下来，慢慢就可以提高你的逻辑思维能力，分析能力，写会迫使你在你脑中搭建一个有条理的框架。例如我写这篇文章一样，我就将值得写博客的原因一点一点地罗列出来，事情就更加清晰，你也可以更好的思考问题。

## 分享带来的连锁反应

“通过分享，你获得了直接而快速的回报，你最终或许会发现你已将版权和“保留所有权利”抛诸脑后。新的经济学准则是：参与你作品的人越多，回报越高。在分享主义里，如果你愿意你可以保留所有权，但是我乐于分享。” by 毛向辉 《分享主义：一场思维革命》

互联网精神其中最重要的就是分享主义，基于分享主义，你可以享受到社会化及互联网给你带来的种种便利和好处，你分享了一个知识，你就成为了互联网中的一个点，这个点的大小由你自己来决定，互联网的大潮会将你的这个点推送到它所能触及的每个角落，让需要的人得到，同时，你的这个点也会继续扩大，连接到整个网络，这个点有可能连接成一张网，而你就是这张网的中心。

## 帮你找到志同道合的人

在微博，在朋友圈，你可能找不到跟你志同道合的人，而在博客，你可以通过看他的几篇文章就迅速地理解认同这个人，即使你没有见过这个人，但你也可以通过这种关联来相互学习。

如果你在一个领域有相当的了解，你将这些内容发在网络上，网络上跟你志趣相投的人也会被你吸引过来，根据吸引力法则，你是怎样的人你就会被怎样的人吸引，这就是博客所能赋予你的魅力。

即使博客没有被他人关注，我们依然可以找到同好，你可以自己将博文转载到其他站点，人们会通过搜索引擎找到你，有邮件、微博等工具，我们不乏与他人交流的途径。by Gabriel Weinberg 《Why I blog》

## 记录成长

隔一段时间，你再回头看你写的博客，你会发现自己正在通过这样的方式在不断的成长，这种成长在自己眼里是一种财富，在别人眼里是一张地图，你得到了收获，不断修正自己的错误，别人得到了指引，避免走弯路。

更多的情况是当你回望自己的时候你会发现自己是一个傻逼，so what，that is what I am!

## 培养持续做一件事情的能力



开始是坚持，后来是习惯，接着喜欢。以后当有人对你说，「你写那么多有用的东西，你真的很厉害啊！」你可以笑而不语，也可以大声说道：「你妹，你不知道我开始的时候多么痛苦！」

让你长久地去跑步，你可能做不到；让你每个月看一本书，你也可能做不到；但让你持续地写一个博客，你可以做得到。

你不相信？你不试试你怎么知道？

默默地持续做一件事是一种难得的能力，也是一种难得的品质。

## 讨论反思

每人都会有思维的盲点，就好像这篇文章一样，可能你觉得我可能说得不对，你可以反驳我，我欢迎这种讨论，因为讨论的过程中会产生各种的思维的碰撞，这种碰撞会让你反思，也会激发出你新的灵感，这种讨论反思给自己的带来巨大的受益。

互联网给你的反馈就是让你承受更多，接受更多，成为一个更好的人。

## 搜寻到你意想不到东西

世界不止是你的家，你的公司，你的朋友圈，你应该去发现一个更大的世界，通过写博客，你会知道世界上还有很多人像你一样在写博客，这些人和知识正在世界的某个角落在等着你。

例如，在写这篇文章的过程中，我才知道了Gabriel Weinberg，我才要将阳志平的博客重读一遍。写的过程会让你有很多新的发现，这些新的发现都值得你去再写下来，总结分享出去。

## 一个人在做一件属于自己的事

很多你认为自己很牛逼的事情都是自己一个人做出来。

别人在刷微博，你在看书，别人在看穿越剧，你在学英文，别人在去唱K，你在写个人总结。吃饭也要找同伴，出游要找同伴，看电影要找同伴，你上一次一个人在做一件属于自己的事是在什么时候？

如果你想要清晰地思考，就必须远离人群。但是走得越远，你的处境就会越困难，收到的阻力也会越大。因为你没有迎合社会习俗，而是一步步地与它背道而驰。如果自己就是潮水的一部分，怎么能看见潮流的方向呢？你只能永远保持质疑，问自己，什么话是我不能说的？为什么？——Paul Graham 《不能说的话》

## 互联网的身份识别：

一个长期的价值博客是一份很好的简历。这里的“简历”并非是狭义上的求职简历，毕竟现在还没有到价值博客的时代，很多人写博客都是到处转载或者干脆碎碎念，正因此面试官未必拿个人博客当成了解一个人的更可靠窗口。

这里的“简历”是指一个让别人了解自己的窗口，虽然我们未必做得到像罗永浩、Keso这样的博客，个人的影响力已经足以支撑出一份事业（牛博和5gme），但至少你会因此而结识更多的人，你的博客价值越高，你结识的人就越牛，跟牛人交流又会让你的眼界得到极大的开阔，打开一扇又一扇你原本不知道的窗，于是你就变得更牛……这是一个良性循环。by 刘未鹏

## 最后

你可能想不到在白纸的左边（不值得写博客的原因）写什么了，想不到写个「博客时代已经过去」或

者「我没有时间」也可以，但与此同时，你也可以用那些时间去思考一下「怎么做到长期写一个价值博客」。

如果你不想思考，也可以回复「价值」看看别人的建议。

推荐写作平台：

- 简书（推荐）
- 自己搭建一个独立博客
- 新浪博客

可能稍微有点长，讲述的很详细了，那么下面的引用是关于，我为什么选择独立博客：

第三方**BSP**永远不靠谱。只要你有钱，还是用独立域名和空间，可以保证你30年后还可以访问你的文件。blogcn不过六七年就没了，微软的Space、雅虎的360、地球村，都证明，公司再大，服务也不靠谱。

用微博的人多了，写博客的人少了，但这和博客式微是两码事。微博只是把原先并不适应写博客的人吸引走了，博客依然有自己的生存空间。为了推广你的博客，也可能用长微博之类的方式嘛。

但博客从来都不是专门写给别人看的，因为微博比博客更适合写给别人看。如果你觉得写博客得到的回音少了，或者别的什么，那你理所应当转去微博。否则，你选啥BSP都行，反正是写给自己看。

作者：李书航

链接：

<http://www.zhihu.com/question/21140875/answer/17314663>

来源：知乎

著作权归作者所有。商业转载请联系作者获得授权，非商业转载请注明出处。

当然，种种原因，还要归结于自己对建设站点的爱好。我不能保证可以坚持多长时间，但我由衷地希望可以坚持很长，很长的时间，就算是为了曾经的自己，现在的自己，未来的自己，以及，未出现的你。

最后，再来看一个引用，“作为技术人员为什么要写博客”：

## 一 我心中的博客

我所以指的写博客，不单只是写一篇文章出来这一结果。而应该是写的这一过程，写过技术文章的朋友应该跟我一样有这么一个过程。

- 1 自己了解学习，文章所涉及到的知识点，及知识点衍生出来的知识点。
- 2 对学习的知识点进行验证，以确保理论值与实践值保持一致
- 3 构思文章的大纲，哪些部分需要重点写，需要配合实例代码，图片等信息
- 4 动手写，写完后再次检查校正并排版，然后发表

## 5 针对网友的评论中提出的问题进行回复

我写文章一般都会经历以上5步，最终以上5步融合成一个结果那就是"一篇文章" 这一过程也是我心中对的"写博客"一词的诠释

---

## 二 为什么要写博客

---

### 2.1 为自己

写博客对自己的提升是很大的，可能写一篇体现不出来，但是只要你坚持写效果就很明显，好处人个认为有以下几点

**2.1.1 强化知识点：**在写一篇文章前，你必定是要把以文章中心为主的知识点及衍生的知识点都详细了解一篇，在这一过程中必须会涉及到自己以前所了解过的知识，人的记忆是存在记忆曲线的需要不断的重复记忆才能长久的记住某一事物，而每写一篇文章时都会查阅资料，在这一过程中必然会遇到以前记住了而现在渐渐淡忘的知识点，当你再次看到时瞬间就会回想起，此时以前的知识点就得到了强化。

**2.1.2 提升学习能力：**同一样的人，了解同一知识点，用不同的方法，产生的结果必然会不一样，找到最佳的学习方法，这也是一种能力，这种能力是经过多次实践探索之后总结出来的，以前我每次需了解某一种技术时都会先百度看各种搜索结果，发现没有想要的之后，再Google因为Googl

e的结果与百度的会有所不同，Google结果中国外的文章相对会多一点，而偶然点了一个链接进入了博客园，发现就是自己想要的东西，而且把概念，代码，及经验都写上去了，看完之后对我帮助很大。

渐渐的我便开始采这种方法了解新知识概念性的直接看百度百科，实质性的直接上博客园的找找看，群里的朋友还推荐了一种方法，比如我要学MVC园子里很多人都写了关于MVC的一系列文章，把那一系列的文章都看一遍，对于MVC就基本有了了解了，这便是学习能力的提升,对于某种技术用最短的时间做到了比较全面的了解

**2.1.3 提升文字组织能力：**这个就不用说了，写博客，既然是写，就必然会有大量的文字，而如何组织文字表达出自己想表达的意思，是长期练习的，而写博客正好帮助你提高了你的文字组织能力，

**2.1.4 提升逻辑思维能力：**不用说，技术性的东西从来就没的单独存在的，都一层层技术相结合，那在了解某种技术时，自己的思维也是要顺着这种关系逐渐深入的，比如MVC，你不能只知道M是什么V是什么C是什么就行了吧，你得知道MVC这三者关系是怎样的，又是怎样交互，而你了解之后再把它写出来时，需要清晰逻辑。

## **2.2 为他人**

**2.2.1 有意的：** 园子里有很多人都写过关于 MVC 框架 WCF 等等系列文章，目的就在于帮助新人快速上手，这个我深有体会，当初我开始学习MVC时就是看的T2噬菌体的MVC系列文章，整篇看完后再配合自己动手对于MVC就有了基本的了解了，在次感谢园子里无私献的大牛们。而以上行为就是有意的帮助

**2.2.2 无意的：** 很多时候在开发项目的过程中，遇到了技术问题，花了时间解决后，有人会写博客记录，并附上解决方法旨在当再次遇到问题时直接看下文章就知道如何处理了，

而碰巧的是，这种问题不止他一个人遇到了。很多人在开发时也遇到了这个问题，在网上找答案时，就找到了这篇文章，并根据文章提供的解决方法，顺利的解决了问题，这种帮助就是无意的帮助

---

### 三 一定要写博客吗

---

答案肯定是否定的，中国几百万的程序员，如果都写博客，那程序员的春天就来了，但是事实并非如此。而我所讲的 写博客 的产物并不只是一篇文章，更多的是，对自己能力的提升，自己对知识点的总结，而发表在博客上只是为了公开，还有很多人喜欢记录在云笔记里面。还有工作很 忙，没有空闲时间写出来，因为写技术性的文章，花的时间是很长的。写过的朋友都知道。

---

### 四 博客会给你带来哪些收获

---

古人云:一份耕耘,一份收获

以下这些是帮助他人而得到的一些认可，并非主观上去追求的

**MVP**: 微软每年都颁发MVP给那些经常与其他专业人士分享知识和专业技能，受人尊敬、信任，而且平易近人的专家。而这个称号则是对你写的博客质量的肯定

**知名度**:文章写得好的，技术水平肯定也很好，知道的人多了，知名度就有了，比如园子里排名前10的大家都知道，都看过他们的文章。

**尊敬**:对于技术界的大神，都是受人敬仰的，在园子里或者工作中也是一样的，在心里对大神们都是默默的佩服!至少我是这样啦，哈哈~也是我学习的榜样!

---

## 五 总结

---

我所认的写博客是对自己所了解知识的强化，分享，自身能力的提升。当然 写 博客只是一种方法而以，只要能达到提升自我的效果什么方法都是可以的

之前有看过一则新闻：即便没有读者，你也要写博客，文章也讲解了很多写博客的好处。

而我写博客是希望，能提升自己的综合能力，并把自己的知识与经验分享给大家，如果有幸我的分享帮助了一些人，那将使我更加欣慰



最后附上一句名言：有些事情你现在不去做，可能以后都不会有机会了！

注：我代表不了大家，所以以上观点只代表我个人。

本文作者：[Zery](#)

---

就是这样。

[前一篇: 欢迎使用Z-BlogPHP!](#)   [后一篇: XAMPP错误: Apache shutdown unexpectedly](#)   [解决办法: 更改端口](#)

Copyright Hower'Blog Rights Reserved. . Powered By [Z-BlogPHP](#). theme by [????](#)



smallyu

Hello World

??

??

????

2017080 (6)

2017070 (2)

2017060 (3)

2017050 (2)

2017040 (2)

2017030 (3)

2017020 (3)

2017010 (4)

20160120 (1)

## 欢迎使用Z-BlogPHP!

□ smallyu   □ 2016-05-23   □ Z-Blog

欢迎使用Z-Blog,这是程序自动生成的文章,您可以删除或是编辑它.)

系统总共生成了一个"留言本"页面,和一个"欢迎使用Z-BlogPHP!"文章,祝您使用愉快!

[后一篇: 浅思考：我为什么想要坚持写博客](#)

Copyright Hower'Blog Rights Reserved. . Powered By [Z-BlogPHP](#). theme by [????](#)

20161111 (2)

20161101 (3)

20161091 (1)

20161081 (6)

20161071 (1)

20161061 (4)

20161051 (4)



smallyu

Hello World

??

??

????

2017080 (6)

2017070 (2)

2017060 (3)

2017050 (2)

2017040 (2)

2017030 (3)

2017020 (3)

2017010 (4)

20160120 (1)

## 关于

smallyu很懒，什么都没有写~

上一篇: [换主题啦！告别Metro，迎来百度efe](#)

后一篇: [（失败2016.9.16）（程序猿养成计划）Python参考手册翻译计划](#)

Copyright Hower'Blog Rights Reserved. . Powered By [Z-BlogPHP](#). theme by [????](#)

2016111 (2)

2016101 (3)

201691 (1)

201681 (6)

201671 (1)

201661 (4)

201651 (4)